

LESSON 1

REVISION OF FUNCTIONS AND OOPS

LEARNING CONTENT

WHAT ARE FUNCTIONS?

- **Definition:** Functions are reusable blocks of code that perform a specific task.
- **Syntax:** Defined using the `def` keyword, followed by the function name and parentheses `()`.
- **Parameters:** Can accept input values, called parameters, to customise their operation.
- **Return Values:** Can return a value or multiple values using the `return` statement.
- **Modularity:** Help in breaking down complex problems into smaller, manageable parts.
- **Reusability:** Allow code to be reused without redundancy.
- **Scope:** Variables inside a function are local to that function unless explicitly declared as global.

- **Invocation:** Functions are called or invoked by their name followed by parentheses, optionally with arguments.

Example: Let's consider a pizza-making process. Imagine you have a pizza recipe that consists of several steps like preparing the dough, adding toppings, and baking it.

1. `prepare_dough()` – This function prepares the pizza dough
2. `add_toppings()` – This function adds various toppings to the pizza
3. `bake_pizza()` – This function bakes the pizza in the oven

```
# Function to prepare the dough
def prepare_dough():
    print("Preparing the dough...")

# Function to add toppings
def add_toppings():
    print("Adding toppings...")

# Function to bake the pizza
def bake_pizza():
    print("Baking the pizza...")

# Calling the functions
prepare_dough()
add_toppings()
bake_pizza()
```

```
Preparing the dough...
Adding toppings...
Baking the pizza...
```

RETURN STATEMENT IN A FUNCTION :

- In Python, the `return` statement is used in a function to exit the function and optionally pass an expression back to the caller.

Example:

```
def add_numbers(a,b):
    total = a+b
    return total

# now lets use the function
result = add_numbers(3,4)
print("The sum is:" , result)
```

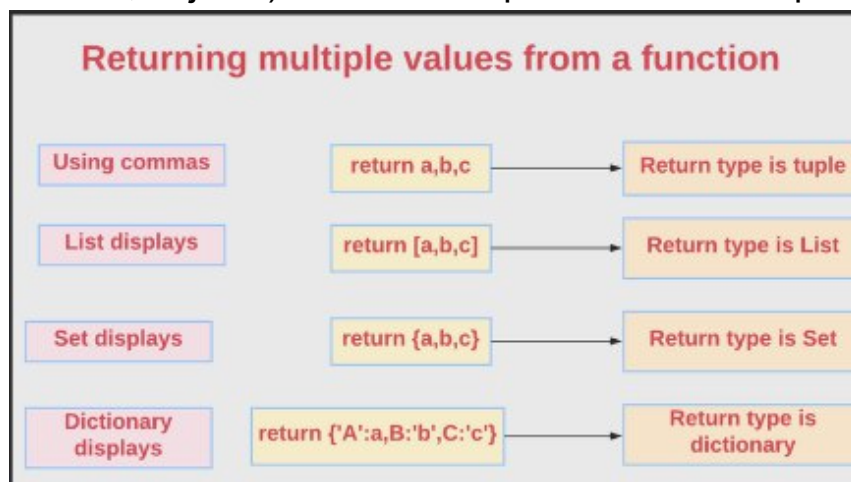
```
The sum is: 7
```

- When the `return` statement is executed, the function exits immediately, and no subsequent code in the function is run.

```
def example():
    print("Before return")
    return
    print("After return") # This will not be executed

example()
# Output: Before return
```

- You can return any type of value (e.g, integers, strings, lists, dictionaries, objects) or even multiple values as a tuple.



- Functions can also return other functions.

```
def outer_function():
    def inner_function():
        return "Hello from the inner function!"
    return inner_function

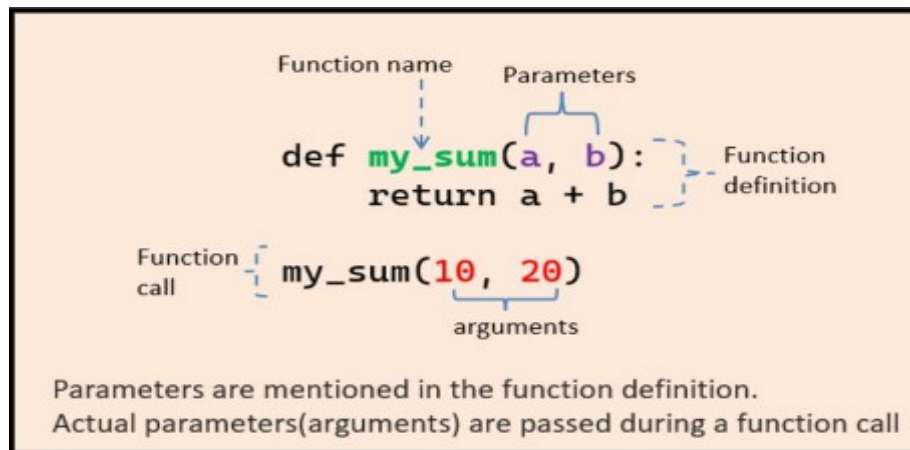
func = outer_function()
print(func()) # Output: Hello from the inner function!
```

WHAT ARE ARGUMENTS?

- Information sent to a function by passing values, are known as arguments or parameters.
- Multiple arguments can be passed by separating them with a comma.
- Failure to place a colon (:) after a function's parameter list is a

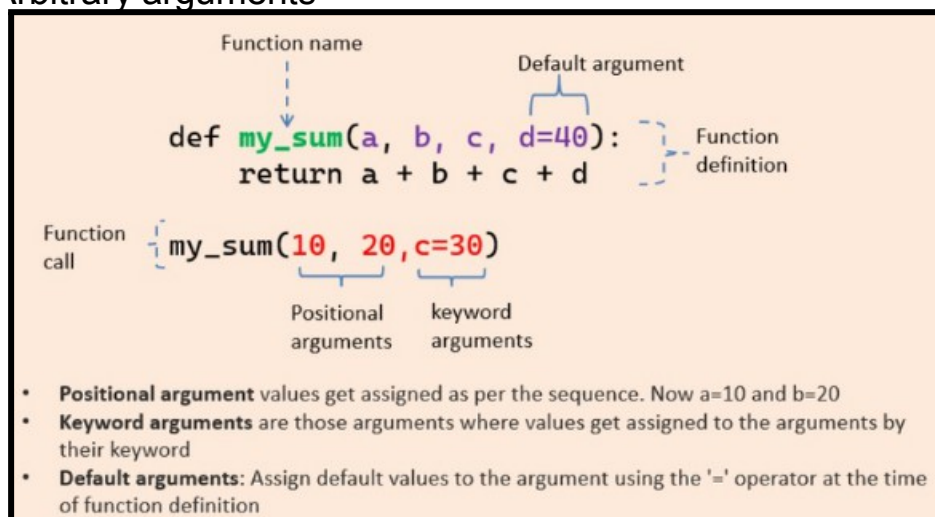
syntax error.

- The function must be called with the correct number of arguments.
- If your function has two arguments, it must be called with two arguments.



TYPES OF ARGUMENTS :

- There are various ways to use arguments in a function. In Python, we have the following 4 types of function arguments.
 1. Default argument
 2. Keyword arguments (named arguments)
 3. Positional arguments
 4. Arbitrary arguments



RECURSIVE FUNCTION :

- **Definition:** A recursive function is a function that repeats its behaviour until a specified condition is met.
- **Base Case:** The condition under which the function stops calling itself to avoid infinite recursion.
- **Recursive Case:** The part of the function where it calls itself with modified arguments, moving towards the base case.

Example:

```
def count_toys(box):
    # Check if the box is empty
    if not box:
        return 0 # No toys left, so return 0

    # Take out one toy and count it
    toy = box.pop()
    print("One toy!") # Say "One toy!"

    # Recursively count the remaining toys in the box
    remaining_toys = count_toys(box)

    # Return the total count
    return 1 + remaining_toys

# Let's play with our magical toy-counter friend Recursy!
toys_box = ["Teddy Bear", "LEGOs", "Doll", "Action Figure"]
total_toys = count_toys(toys_box)
print("Total toys in the box:", total_toys)
```

```
One toy!
One toy!
One toy!
One toy!
Zero toys! Done!
Total toys in the box: 4
```

OOPs CONCEPTS IN PYTHON :

Object-Oriented Programming (OOP) in Python is a paradigm that uses "objects" to design software. Here's a breakdown of the core concepts of OOP in Python:

- **Classes and Objects :**

Class : A blueprint for creating objects (a particular data structure). It defines a set of attributes that characterise any object of the class.

Object : An instance of a class. When a class is defined, no memory is allocated until an object of that class is created.

Example:

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

# Creating an object of the Dog class
my_dog = Dog("Buddy", 3)
```

- **Inheritance :**

Inheritance allows a class to inherit attributes and methods from another class. This helps to reuse code and build a hierarchy of classes.

Superclass: The class being inherited from (also called base class).

Subclass: The class that inherits from the superclass (also called derived class).

Example:

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def bark(self):
        return f"{self.name} says Woof!"

# Creating an object of the Dog class
my_dog = Dog("Buddy", 3)
print(my_dog.bark()) # Output: Buddy says Woof!
```

- **Polymorphism :**

It allows methods to do different things based on the object it is acting upon. It refers to the way in which different object classes can share the same method name but can act differently based on which object calls it.

Example:

```
class Cat(Animal):
    def speak(self):
        return f"{self.name} says Meow!"

animals = [Dog("Buddy"), Cat("Whiskers")]

for animal in animals:
    print(animal.speak())

# Output:
# Buddy says Woof!
# Whiskers says Meow!
```

- **Encapsulation :**

Encapsulation restricts direct access to some of an object's components and can prevent the accidental modification of data. This is achieved through public and private access specifiers.

Example:

```
class Person:
    def __init__(self, name, age):
        self._name = name # Private attribute
        self._age = age

    def get_age(self):
        return self._age

    def set_age(self, age):
        if age > 0:
            self._age = age

person = Person("Alice", 30)
print(person.get_age()) # Output: 30
```

IN CLASS CHALLENGES

CHALLENGE1- Guardian of Internet:

HOME TASK

TASK1-COOL GADGET:

Bob owns a super cool gadget store full of awesome stuff but he doesn't have an inventory for his store so, Let's make it happen with code magic! Imagine creating a Python program using the OOP concept where you can add different types of gadgets, set their prices, and keep track of how many are left.

TASK 2- TASK ORGANISER:

Hey Sam! Imagine you're helping a superhero organize their tasks to save the day! They need your coding powers to create a special list that can add tasks, remove tasks, mark tasks as completed, and show all the tasks. Can you use your coding superpowers to create this amazing to-do list for our superhero friend? Let's get ready to code and save the day!

WHAT'S NEXT?

Revision of Tkinter

LESSON 1

REVISION OF FUNCTIONS AND OOPS

LEARNING CONTENT

WHAT ARE FUNCTIONS?

- **Definition:** Functions are reusable blocks of code that perform a specific task.
- **Syntax:** Defined using the `def` keyword, followed by the function name and parentheses `()`.
- **Parameters:** Can accept input values, called parameters, to customise their operation.
- **Return Values:** Can return a value or multiple values using the `return` statement.
- **Modularity:** Help in breaking down complex problems into smaller, manageable parts.
- **Reusability:** Allow code to be reused without redundancy.
- **Scope:** Variables inside a function are local to that function unless explicitly declared as global.

- **Invocation:** Functions are called or invoked by their name followed by parentheses, optionally with arguments.

Example: Let's consider a pizza-making process. Imagine you have a pizza recipe that consists of several steps like preparing the dough, adding toppings, and baking it.

1. `prepare_dough()` – This function prepares the pizza dough
2. `add_toppings()` – This function adds various toppings to the pizza
3. `bake_pizza()` – This function bakes the pizza in the oven

```
# Function to prepare the dough
def prepare_dough():
    print("Preparing the dough...")

# Function to add toppings
def add_toppings():
    print("Adding toppings...")

# Function to bake the pizza
def bake_pizza():
    print("Baking the pizza...")

# Calling the functions
prepare_dough()
add_toppings()
bake_pizza()
```

```
Preparing the dough...
Adding toppings...
Baking the pizza...
```

RETURN STATEMENT IN A FUNCTION :

- In Python, the `return` statement is used in a function to exit the function and optionally pass an expression back to the caller.

Example:

```
def add_numbers(a,b):
    total = a+b
    return total

# now lets use the function
result = add_numbers(3,4)
print("The sum is:" , result)
```

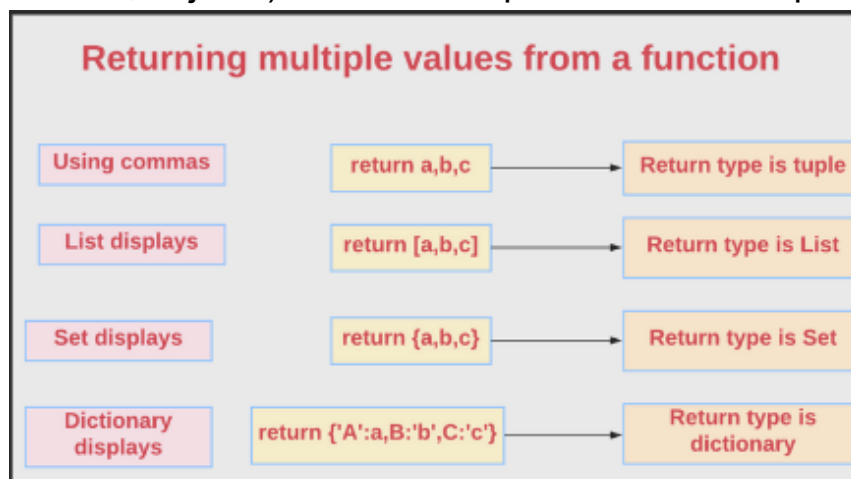
```
The sum is: 7
```


- When the `return` statement is executed, the function exits immediately, and no subsequent code in the function is run.

```
def example():
    print("Before return")
    return
    print("After return") # This will not be executed

example()
# Output: Before return
```

- You can return any type of value (e.g, integers, strings, lists, dictionaries, objects) or even multiple values as a tuple.



- Functions can also return other functions.

```
def outer_function():
    def inner_function():
        return "Hello from the inner function!"
    return inner_function

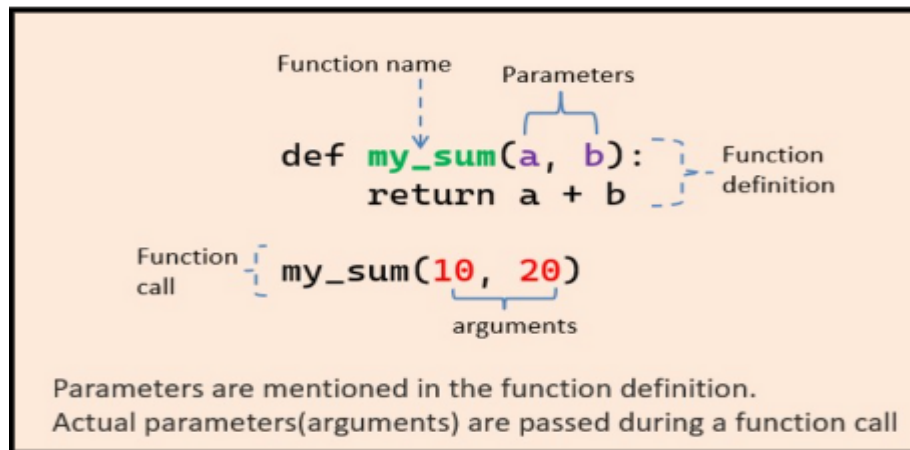
func = outer_function()
print(func()) # Output: Hello from the inner function!
```

WHAT ARE ARGUMENTS?

- Information sent to a function by passing values, are known as arguments or parameters.
- Multiple arguments can be passed by separating them with a comma.
- Failure to place a colon (:) after a function's parameter list is a

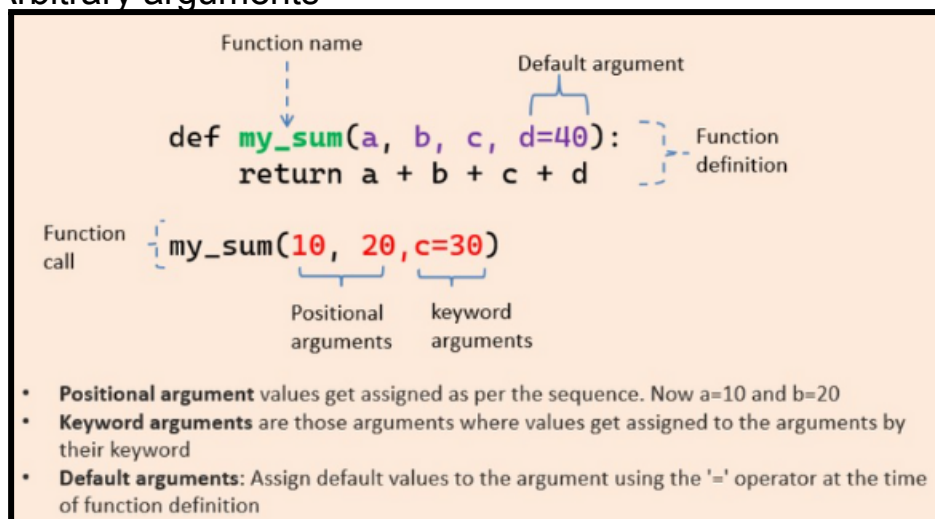
syntax error.

- The function must be called with the correct number of arguments.
- If your function has two arguments, it must be called with two arguments.



TYPES OF ARGUMENTS :

- There are various ways to use arguments in a function. In Python, we have the following 4 types of function arguments.
 1. Default argument
 2. Keyword arguments (named arguments)
 3. Positional arguments
 4. Arbitrary arguments



RECURSIVE FUNCTION :

- **Definition:** A recursive function is a function that repeats its behaviour until a specified condition is met.
- **Base Case:** The condition under which the function stops calling itself to avoid infinite recursion.
- **Recursive Case:** The part of the function where it calls itself with modified arguments, moving towards the base case.

Example:

```
def count_toys(box):  
    # Check if the box is empty  
    if not box:  
        return 0 # No toys left, so return 0  
  
    # Take out one toy and count it  
    toy = box.pop()  
    print("One toy!") # Say "One toy!"  
  
    # Recursively count the remaining toys in the box  
    remaining_toys = count_toys(box)  
  
    # Return the total count  
    return 1 + remaining_toys  
  
# Let's play with our magical toy-counter friend Recursy!  
toys_box = ["Teddy Bear", "LEGOs", "Doll", "Action Figure"]  
total_toys = count_toys(toys_box)  
print("Total toys in the box:", total_toys)
```

```
One toy!  
One toy!  
One toy!  
One toy!  
Zero toys! Done!  
Total toys in the box: 4
```

OOPs CONCEPTS IN PYTHON :

Object-Oriented Programming (OOP) in Python is a paradigm that uses "objects" to design software. Here's a breakdown of the core concepts of OOP in Python:

- **Classes and Objects :**

Class : A blueprint for creating objects (a particular data structure). It defines a set of attributes that characterise any object of the class.

Object : An instance of a class. When a class is defined, no memory is allocated until an object of that class is created.

Example:

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

# Creating an object of the Dog class
my_dog = Dog("Buddy", 3)
```

- **Inheritance :**

Inheritance allows a class to inherit attributes and methods from another class. This helps to reuse code and build a hierarchy of classes.

Superclass: The class being inherited from (also called base class).

Subclass: The class that inherits from the superclass (also called derived class).

Example:

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def bark(self):
        return f"{self.name} says Woof!"

# Creating an object of the Dog class
my_dog = Dog("Buddy", 3)
print(my_dog.bark()) # Output: Buddy says Woof!
```

- **Polymorphism :**

It allows methods to do different things based on the object it is acting upon. It refers to the way in which different object classes can share the same method name but can act differently based on which object calls it.

Example:

```
class Cat(Animal):
    def speak(self):
        return f"{self.name} says Meow!"

animals = [Dog("Buddy"), Cat("Whiskers")]

for animal in animals:
    print(animal.speak())

# Output:
# Buddy says Woof!
# Whiskers says Meow!
```

- **Encapsulation :**

Encapsulation restricts direct access to some of an object's components and can prevent the accidental modification of data. This is achieved through public and private access specifiers.

Example:

```
class Person:
    def __init__(self, name, age):
        self._name = name # Private attribute
        self._age = age

    def get_age(self):
        return self._age

    def set_age(self, age):
        if age > 0:
            self._age = age

person = Person("Alice", 30)
print(person.get_age()) # Output: 30
```

IN CLASS CHALLENGES

CHALLENGE1- Guardian of Internet:

Taylor, Think of yourself as the guardian of the Internet, protecting us from the dangers of unsafe links! So, here's your task : Create a function called 'is_valid_url' that takes a URL as input. Inside this function, check if the URL starts with

HOME TASK

TASK1-COOL GADGET:

Bob owns a super cool gadget store full of awesome stuff but he doesn't have an inventory for his store so, Let's make it happen with code magic! Imagine creating a Python program using the OOP concept where you can add different types of gadgets, set their prices, and keep track of how many are left.

TASK 2- TASK ORGANISER:

Hey Sam! Imagine you're helping a superhero organize their tasks to save the day! They need your coding powers to create a special list that can add tasks, remove tasks, mark tasks as completed, and show all the tasks. Can you use your coding superpowers to create this amazing to-do list for our superhero friend? Let's get ready to code and save the day!

WHAT'S NEXT?

Revision of Tkinter

LESSON 2

REVISION OF TKINTER

LEARNING CONTENT

ABOUT GUI :

- GUI stands for Graphical User Interface, which allows users to interact with electronic devices using graphical elements.
- Features icons, buttons, menus, and windows that are manipulated through direct interactions like clicking, dragging, and dropping.
- GUI is designed to be intuitive and user-friendly, reducing the need for text-based commands.
- Commonly used in operating systems, applications, and websites to enhance user experience.
- There are several Python libraries available for creating Graphical User Interfaces (GUI). Few of them are Tkinter, Kivy, Python QT, Wx python. Among all, Tkinter is mostly used.

INTRODUCTION TO TKINTER :

- **Tkinter** is the standard GUI (Graphical User Interface) library for Python. You don't need to worry about the installation of the Tkinter module separately as it comes with Python already.
- Easy for beginners to create simple applications with buttons, labels, and other widgets. Works on Windows, macOS, and Linux.
- Includes various widgets like buttons, labels, text boxes, menus, and more and responds to user actions like clicks, key presses, etc.

Example: Creating a window using Tkinter.

```
import tkinter as tk

# Create the main window
root = tk.Tk()
root.title("My Codeyoung Tkinter Window")

# Set the window size
root.geometry("400x300")

# Run the application
root.mainloop()
```



WIDGETS IN TKINTER :

- In Tkinter, a widget is a graphical component, such as a button, label, or entry field, that you can place on a window or frame to build user interfaces in Python.

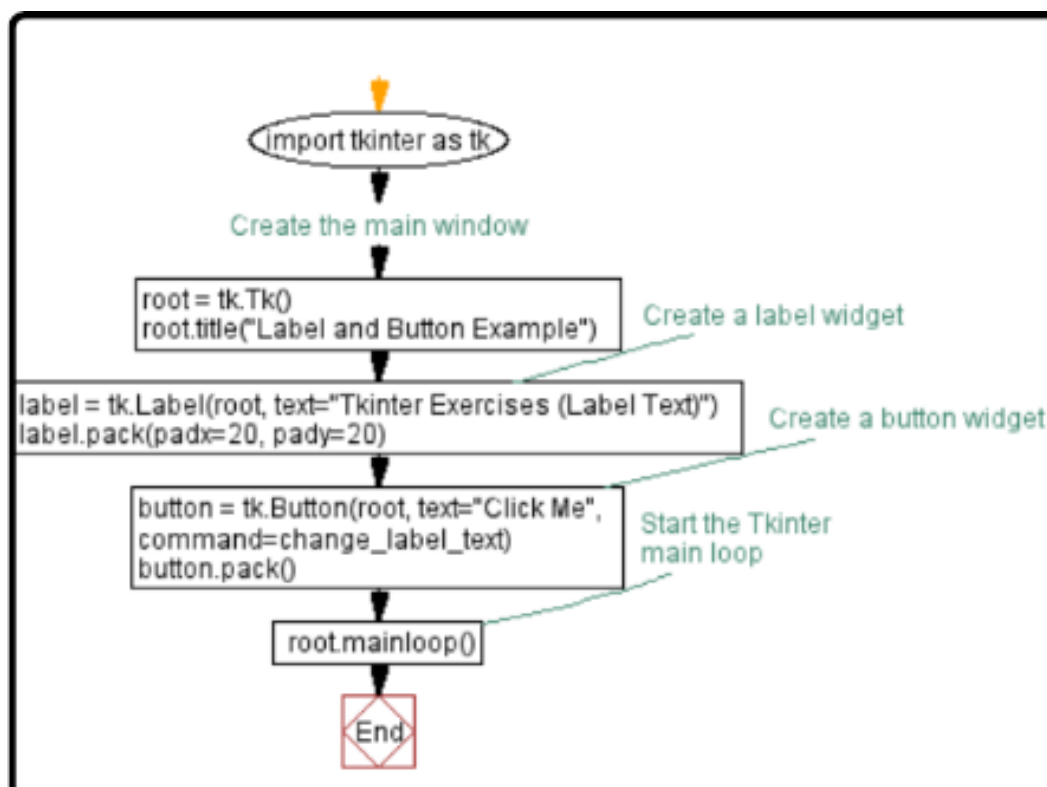
- Each widget serves a specific purpose and can be configured with various properties to control its appearance and behaviour.

Label Widget in Tkinter :

- **Purpose:** Display text or images on a window or frame.
- **Usage:** Used for static text that doesn't require user input.
- **Creation:** Created using `Label(parent, text='Sample')`.
- **Configuration:** Options include text, font, colour, and alignment.
- **Placement:** Positioned using grid, pack, or place managers in Tkinter.

Button Widget in Tkinter :

- **Purpose:** Triggers actions or commands when clicked by the user.
- **Creation:** Created using `Button(parent, text='Button text', command=callback_function)`.
- **Callback Function:** Specifies the function to call when the button is clicked.
- **Styling:** Options for customization include text, font, color, and background.
- **Event Handling:** Executes the specified function when clicked; often used for user interactions in GUI applications.



Example: Creating a Window, Label & Button Widget using Tkinter.

```
import tkinter as tk

def button_click():
    label.config(text="Button clicked!")

root = tk.Tk()
root.title("My Codeyoung Tkinter Window")
root.geometry("400x300")

label = tk.Label(root, text="Hello, Codeyoung!")
label.pack()

button = tk.Button(root, text="Click me", command=button_click)
button.pack()

root.mainloop()
```



TKINTER GEOMETRY :

- The Tkinter geometry specifies the method by which the widgets are represented on display.
- The python Tkinter provides the following geometry methods.
 1. pack()
 2. grid()
 3. place()

- **pack**: Organises widgets in blocks before placing them in the parent widget.

Example:

```
import tkinter as tk

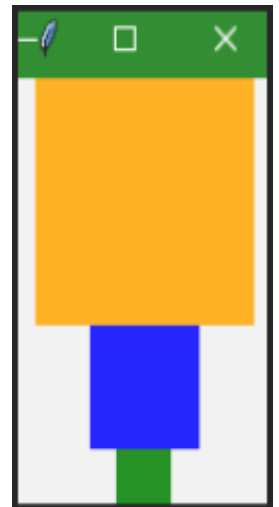
win = tk.Tk()

# add an orange frame
frame1 = tk.Frame(master=win, width=100, height=100, bg="orange")
frame1.pack()

# add blue frame
frame2 = tk.Frame(master=win, width=50, height=50, bg="blue")
frame2.pack()

# add green frame
frame3 = tk.Frame(master=win, width=25, height=25, bg="green")
frame3.pack()

win.mainloop()
```



- **grid**: Organises widgets in a table-like structure, using rows and columns.

Example:

```
import tkinter as tk

win = tk.Tk()

for i in range(5):
    for j in range(3):
        frame = tk.Frame(
            master = win,
            relief = tk.RAISED,
            borderwidth = 1
        )
        frame.grid(row=i, column=j)
        label = tk.Label(master=frame, text=f"Row {i}\nColumn {j}")
        label.pack()

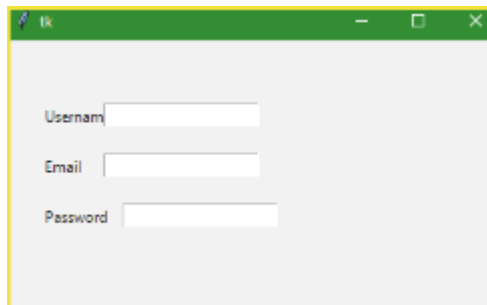
win.mainloop()
```

Row 0 Column 0	Row 0 Column 1	Row 0 Column 2
Row 1 Column 0	Row 1 Column 1	Row 1 Column 2
Row 2 Column 0	Row 2 Column 1	Row 2 Column 2
Row 3 Column 0	Row 3 Column 1	Row 3 Column 2
Row 4 Column 0	Row 4 Column 1	Row 4 Column 2

- **place**: Places widgets at an absolute position you specify.

Example:

```
from tkinter import *
top = Tk()
top.geometry("400x250")
Username = Label(top, text = "Username").place(x = 30, y = 50)
email = Label(top, text = "Email").place(x = 30, y = 90)
password = Label(top, text = "Password").place(x = 30, y = 130)
e1 = Entry(top).place(x = 80, y = 50)
e2 = Entry(top).place(x = 80, y = 90)
e3 = Entry(top).place(x = 95, y = 130)
top.mainloop()
```



IN CLASS CHALLENGES

CHALLENGE1- Quiz Game :

Hey there, future Code Wizard! How well do you know about quizzes and programming? Let's create a fun quiz game together! Imagine you're building a game where players answer multiple-choice questions. Each question has a prompt, options to choose from, and a correct answer. Can you write a program using Python's Tkinter library to make this game a reality? It should display questions one by one, allow players to select an answer, and show their score at the end. Ready to give it a try?

CHALLENGE 2- DIGITAL FIREWORKS :

Hey there! Imagine you're in charge of creating a digital fireworks show using your computer. Can you write a program that displays colourful fireworks on the computer screen whenever you click your mouse? You can use different colours and sizes for the fireworks bursts, and make them appear randomly around where you click. Let's bring some magic to the screen with your coding skills!

HOME TASK

TASK1-ENROLLMENT FORM :

Imagine you're Harry Potter, the most famous wizard in the wizarding world, and you're tasked with designing a mystical enrollment form for the legendary Hogwarts School of Witchcraft and Wizardry! How would you create a magical window with fields for young apprentices' names, ages, genders, and chosen magical subjects? Don't forget to add buttons for submitting your enrollment form, displaying your gathered information, and clearing the form for new entries!

TASK 2- DIGITAL CANVAS :

Hey there, budding artist! Ever dreamt of having a magical canvas where you just click and shapes appear? So let's dive into Python territory with Tkinter magic to create a digital canvas extravaganza! Picture this: left click for circles, right click for rectangles. Think about how we can capture the mouse clicks and translate them into shapes on our canvas. We'll need to use event handling to detect when the mouse is clicked and then use tkinter's drawing functions to create the shapes. Are you ready to sprinkle some coding fairy dust and make art come to life?

WHAT'S NEXT?

Build GUI Application

LESSON 3

BUILD GUI APPLICATION

LEARNING CONTENT

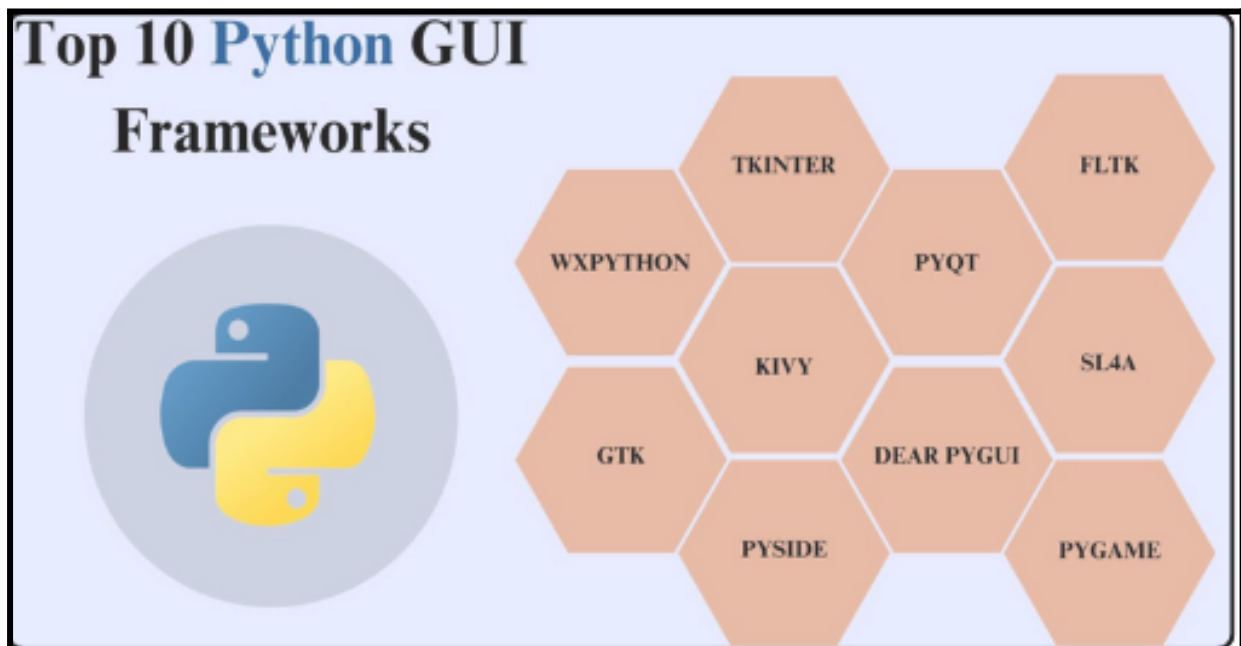
WHAT IS A GUI APPLICATION?

- GUI applications are software programs that utilise the graphical interfaces to enable users to interact with the application through graphical icons and visual indicators such as buttons, menus, windows, and dialog boxes.
- Commonly seen in software like web browsers, video games, and operating systems (e.g., Windows, macOS).

Example:

- Think of a GUI application like a digital playground where you don't need to memorise secret codes or text commands to have fun. Instead, you get colourful buttons, menus, icons that you can click, drag & interact with using your computer mouse or touchscreen.
- So, in simple terms, a GUI application is like a fun and colourful way to interact with your electronic devices, where you get to click and play instead of typing commands.

TOP 10 PYTHON GUI FRAMEWORKS :



- For creating a GUI app using Python, Tkinter is mostly used.
- Tkinter enables developers to create GUI applications efficiently and quickly.

CHECKBUTTON :

- **Purpose:** Allows the user to select or deselect an option.
- **State:** Can be either "on" or "off".
- **Variable:** Typically associated with a `BooleanVar`, `IntVar`, or `StringVar`.
- **Options:** Common options include
 - ☐ `text`: The label displayed next to the checkbox.
 - ☐ `variable`: The associated Tkinter variable.
 - ☐ `onvalue`: The value when the checkbox is checked.
 - ☐ `offvalue`: The value when the checkbox is unchecked.
 - ☐ `command`: A callback function triggered when the state changes.

RADIOBUTTON :

- **Purpose:** Allows the user to select one option from a set of mutually exclusive options.
- **Grouping:** Multiple `Radiobutton` widgets are grouped by associating them with the same variable.
- **Variable:** Typically associated with an `IntVar` or `StringVar`.
- **Options:** Common options include
 - ❑ `text`: The label displayed next to the radio button.
 - ❑ `variable`: The associated Tkinter variable.
 - ❑ `value`: The value assigned to the variable when the radio button is selected.
 - ❑ `command`: A callback function triggered when a radio button is selected.

Both widgets are commonly used in forms and settings where user input is required.

GUI APP WITH CHECKBUTTON & RADIOBUTTON :

```
import tkinter as tk
from tkinter import messagebox

class PicnicPlanner:
    def __init__(self, root):
        self.root = root
        self.root.title("Picnic Planner")

        self.activities = {
            "Hiking": tk.BooleanVar(),
            "Frisbee": tk.BooleanVar(),
            "Music": tk.BooleanVar(),
            "Picnic": tk.BooleanVar(),
        }

        self.selected_activity = tk.StringVar()

        self.create_widgets()

    def create_widgets(self):
        tk.Label(self.root, text="Select Activities:").pack()

        for activity, var in self.activities.items():
            tk.Checkbutton(self.root, text=activity, variable=var).pack()

        tk.Label(self.root, text="Select Activity to Do:").pack()

        for activity in self.activities:
            tk.Radiobutton(self.root, text=activity, variable=self.selected_activity,
                           value=activity).pack()

        tk.Button(self.root, text="Plan Picnic", command=self.plan_picnic).pack()
```

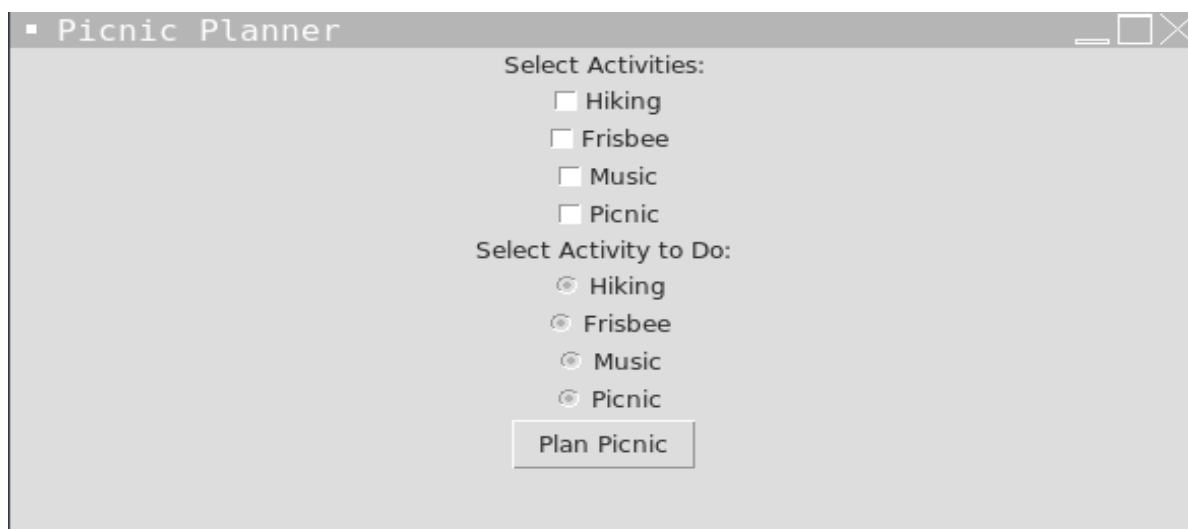
```

def plan_picnic(self):
    selected_activities = [activity for activity, var in self.activities.items() if var.get()]
    if not selected_activities:
        messagebox.showerror("Error", "Please select at least one activity.")
        return

    chosen_activity = self.selected_activity.get()
    messagebox.showinfo("Picnic Planned", f"Activities: {' '.join(selected_activities)}\nChosen Activity: {chosen_activity}")

if __name__ == "__main__":
    root = tk.Tk()
    app = PicnicPlanner(root)
    root.mainloop()

```



IN CLASS CHALLENGES

CHALLENGE1- CALENDAR GUI :

Hey, future coder! Have you ever thought about creating your own colourful calendar? Imagine if you could make a calendar where each day has its own special colour! Here's a hint: think about how we can use Python to create a window where you can input the year and month. Then, when you click a button, Python will generate a calendar for that month with each day having a different colour. Can you imagine how cool that would be? Let's brainstorm together on how we can make it happen!

CHALLENGE 2- LOGIN APP :

Alice: Hey Bob, check out this cool login app I made using Tkinter!

Bob: Wow, that looks neat! But why did you choose those specific colours for the labels and entries?

Alice: I wanted to make it visually appealing. The blue background gives it a modern look, don't you think?

Bob: Yeah, it does stand out. How about adding a feature to check the password?

Alice: That's a great idea! I'll add that in the next version. Thanks for the suggestion, Bob!

What colours did you choose for the labels and entry widgets in your Tkinter login app, and why did you select those specific colours?

HOME TASK

TASK1-VOCABULARY QUIZ :

Hey, Young coder! Can you create a fun vocabulary quiz game using Python? It should randomly select words from a list and ask the player to guess their meanings. To make it more fun, you could display a word and ask them to type in what they think it means. If they guess correctly, give them a thumbs up, and if not, give them a hint or show the correct answer. It's a great way to learn new words while having fun with Python programming! Ready to give it a try?

TASK 2- TO DO LIST APP :

Diana has a special notebook where she can write down all the things she needs to do, like homework, chores, or fun activities! Using your coding magic, can you help Diana create a virtual to-do list just like her notebook? She'll need an entry box where she can type in her tasks, buttons to add them to the list and remove them when she's done, and of course, a place to see all her tasks lined up neatly. Oh, and don't forget about the scrollbar, so Diana can scroll through her list if it gets too long! Ready to turn Diana's notebook into a cool computer program?

WHAT'S NEXT?

Recursion

LESSON 4

RECURSION

LEARNING CONTENT

ABOUT RECURSION :

- **Definition:** The process in which a function calls itself directly or indirectly is called recursion and the corresponding function is called a recursive function.
- **Base Case:** The condition under which the recursive function stops calling itself, preventing infinite loops.
- **Recursive Case:** The part of the function where it calls itself with a modified argument, moving towards the base case.
- **Stack Memory:** Each recursive call is stored in the call stack, which can lead to stack overflow if too many calls are made.
- **Use Cases:** Commonly used for problems like factorial calculation, Fibonacci sequence, tree traversals, and solving puzzles like the Towers of Hanoi.

Example: Factorial of a number using Recursion in python.

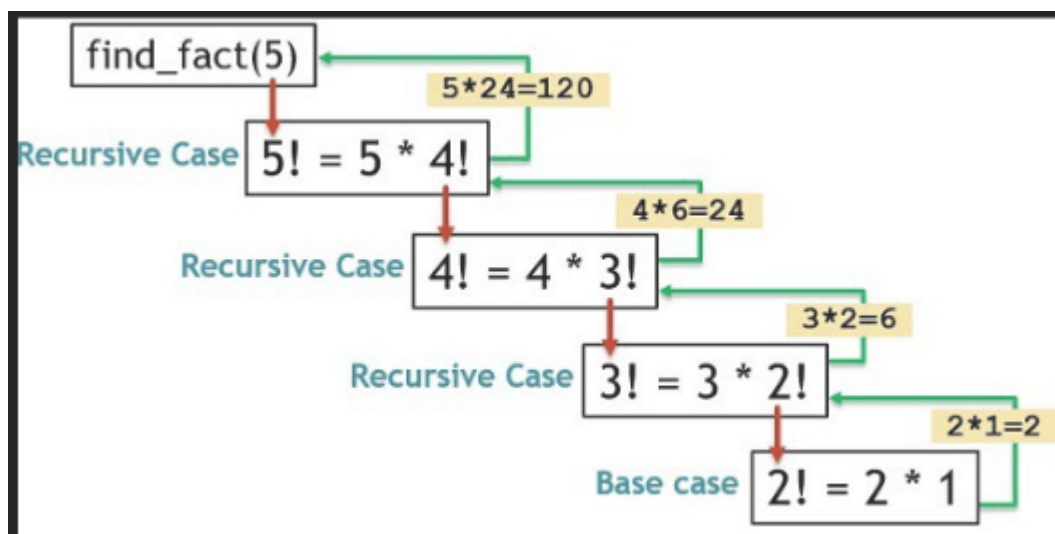
```
def factorial(n):  
    # Base case: if n is 0 or 1, return 1  
    if n == 0 or n == 1:  
        return 1  
    else:  
        # Recursive case: n * factorial of (n-1)  
        return n * factorial(n - 1)  
  
# Example usage  
print(factorial(5)) # Output: 120
```

Explanation:

1. **Base Case:** If n is 0 or 1, the function returns 1.
2. **Recursive Case:** The function calls itself with $n - 1$ and multiplies the result by n .

- $\text{factorial}(5)$ calculates $5 \times 4 \times 3 \times 2 \times 1$, which equals 120.

This recursive approach breaks down the factorial calculation into smaller subproblems until it reaches the base case, simplifying the overall computation.



ADVANTAGES OF RECURSION :

- **Simplicity:** Recursive solutions can be more intuitive and easier to understand for problems that can be divided into smaller, similar subproblems.
- **Clarity:** Recursive code often mirrors the mathematical induction used to solve problems, making it clearer to reason about.
- **Divide and Conquer:** Recursion naturally fits problems that exhibit a divide-and-conquer approach, such as tree traversals or sorting algorithms like merge sort.
- **Reduction in Code Size:** Recursive solutions can often be more concise than iterative solutions, reducing the amount of code needed.

DISADVANTAGES OF RECURSION :

- **Stack Overflow:** Recursive solutions rely on the call stack, which can lead to stack overflow errors if the recursion depth is too large or not properly managed.
- **Performance Overhead:** Recursive calls typically involve more overhead in terms of function call management compared to iterative solutions.
- **Difficulty in Debugging:** Recursive functions can be harder to debug due to their implicit control flow and potential for deep recursion.
- **Limited Use for Some Problems:** Some problems may not naturally lend themselves to recursive solutions or may be more efficiently solved using iterative approaches.

TYPES OF RECURSION :

Recursion can be categorised in two different types based on how it calls itself within the function:

1. Direct Recursion
2. Indirect Recursion

Direct Recursion :

- This occurs when a function calls itself directly. It's the most common type of recursion.
- Direct recursion can further be classified based on how the recursive calls are made within the function. Here are some common types of direct recursion:
 1. Tail Recursion
 2. Head Recursion
 3. Tree Recursion
 4. Nested Recursion

Tail Recursion : Tail recursion occurs when the recursive call is the last operation performed in the function, with no further computation after the call returns.

```
# Code Showing Tail Recursion

# Recursion function
def fun(n):
    if (n > 0):
        print(n, end=" ")
        # Last statement in the function
        fun(n - 1)

# Driver Code
x = 3
fun(x)
```

Output

```
3 2 1
```

Head Recursion : In head recursion, the recursive call is made at the beginning of the function, before any other operations. This means the function calls itself first, and then processes the results of the recursive call.

```
def fun(n):
    if n > 0:
        fun(n - 1)
        print(n, end=" ")

x = 3
fun(x)
```

Output

```
1 2 3
```


Tree Recursion : Tree recursion occurs when a function makes multiple recursive calls, leading to a branching structure similar to a tree. Each recursive call may further make additional recursive calls, resulting in an exponential growth of calls.

```
def fun(n):  
    if n > 0:  
        fun(n - 1)  
        fun(n - 1)  
        print(n, end=" ")  
  
x = 3  
fun(x)
```

1 1 2 1 1 2 3

Nested Recursion : Nested recursion occurs when a recursive function's argument itself is the result of another recursive call to the same function. This type of recursion often leads to a complex and deeply nested set of function calls.

```
# Python program to show Nested Recursion |  
def fun(n):  
  
    if (n > 100):  
        return n - 10  
  
    # A recursive function passing parameter  
    # as a recursive call or recursion inside  
    # the recursion  
    return fun(fun(n + 11))  
  
# Driver code  
r = fun(95)  
print("", r)
```

Output

91

Indirect Recursion :

- Indirect recursion occurs when two or more functions call each other in a circular manner.
- Debugging and tracing can be more complex compared to direct recursion, so careful planning and base conditions are crucial.

```

# Python program to show Indirect Recursion |
def funA(n):
    if (n > 0):
        print("", n, end='')

        # Fun(A) is calling fun(B)
        funB(n - 1)

def funB( n):
    if (n > 1):
        print("", n, end='')

        # Fun(B) is calling fun(A)
        funA(n // 2)

# Driver code
funA(20)

```

Output

```

20 19 9 8 4 3 1

```

IN CLASS CHALLENGES

CHALLENGE1- CHRIS FACTORIAL :

Chris decided to manually calculate the factorial of 20. However, his calculator switched to scientific notation due to the enormity of the result. Can you write a Python program to find the factorial of a number using recursion and display it in a readable format?"

"After calculating the factorial, can you also inform Chris and his friends about the size of the factorial, given that it's too large to be displayed in standard numerical form?"

CHALLENGE 2- TOWER OF HANOI :

Emily is learning about recursion and comes across the Tower of Hanoi problem, where she needs to move a stack of disks from one rod to another following certain rules. She decides to write a Python program to solve this problem using recursion.

HOME TASK

TASK1-FIBONACCI DAY :

Ayman recently came to know that November 23rd is Fibonacci day because when written in mm/dd format as 11/23, these four numbers form a Fibonacci sequence. He got more interested in the Fibonacci sequence and finally decided to create a function in a python program to print the entire Fibonacci sequence. Write the same program in python and check how many terms are possible for you to print on the screen.

TASK 2- CHOCOLATES OFFERS :

Chris loves chocolates and he came to know that a shop is giving a special offer on the chocolates. The price of chocolate is Rs. 2 and the shop gives 1 more chocolate on 2 wrappers of the same chocolate. Chris has Rs. 16 with him. So how many chocolates Chris can buy?

The shopkeeper informed Chris that he is going to make this kind of offer on a daily basis. So Chris decided to write a python program and create a function that can find the number of chocolates that can be brought in different scenarios.

Write a program and create a function to find the answer with the help of recursion.

WHAT'S NEXT?

Searching Algorithm

LESSON 5

SEARCHING ALGORITHM

LEARNING CONTENT

WHAT IS AN ALGORITHM?

- An algorithm is a systematic set of instructions or rules designed to perform a specific task or solve a problem.
- step-by-step set of instructions that transforms input into desired output.
- Algorithms typically take inputs and produce outputs.
- Algorithms form the basis of computer programs and software.

WHAT IS SEARCHING ALGORITHMS IN PYTHON?

- Searching algorithms in Python are systematic methods used to locate specific elements within collections of data, such as arrays, lists, or dictionaries.
- Python provides programmers with a range of searching methods to efficiently retrieve information from lists, arrays, dictionaries, and other data structures.

- some common searching algorithms in Python are:
 1. Linear Search
 2. Binary Search

LINEAR SEARCH :

- In this type of search, list or array is traversed serially and each element of list or array is examined.
- It is straightforward but less efficient compared to binary search, especially for large datasets or when the data is sorted.

```
def linear_search(arr, target):  
    for index, value in enumerate(arr):  
        if value == target:  
            return index  
    return -1  
  
my_list = [5, 2, 9, 1, 5, 6]  
target_value = 9  
result = linear_search(my_list, target_value)  
  
if result != -1:  
    print(f"Target {target_value} found at index {result}")  
else:  
    print("Target not found in the list")
```

Output:
Target 9 found at index 2

BINARY SEARCH :

- Binary search is used to quickly find a value in a sorted sequence (array) by repeatedly dividing the search interval in half.
- Binary search is much more efficient than Linear Search as they repeatedly target the centre of the search structure and divide the search space in half.

```
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2
        mid_value = arr[mid]

        if mid_value == target:
            return mid
        elif mid_value < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1

# Example usage
my_list = [1, 2, 5, 6, 9, 12, 15]
target_value = 9
result = binary_search(my_list, target_value)

if result != -1:
    print(f"Target {target_value} found at index {result}")
else:
    print("Target not found in the list")
```

Output: Target 9 found at index 4

LINEAR SEARCH vs BINARY SEARCH :

Feature	Linear search	Binary search
Search technique	Starts searching from the first element and compares each element with the searched element.	Search the position of the searched element by finding the middle element of the array.
Time complexity	$O(n)$	$O(\log n)$
Precondition	None (array need not be sorted).	Array must be sorted.
Algorithm Description	Sequentially checks each element.	Divides search interval in half.
Efficiency	Slower for large datasets.	Faster for large datasets.
Best case scenario	$O(1)$ (if the target is found at the beginning).	$O(1)$ (if the target is found in the middle).
Worst case scenario	$O(1)$ (if the target is at the end or not present).	$O(\log n)$
Memory usage	$O(1)$ constant	$O(1)$ constant
Use cases	Small datasets or unsorted data.	Large sorted datasets.
Advantage	Simple to implement.	Efficient for large datasets.
Disadvantage	Inefficient for large datasets.	Requires sorted data, more complex to implement.

These comparisons highlight the key differences between linear search and binary search in terms of time complexity, memory usage, efficiency, and suitability for different types of data and search requirements.

IN CLASS CHALLENGES

CHALLENGE1- DAVE PROJECT :

Dave is working on a project and she wants a `getIndex()` function. `getIndex()` function should return the index number of a value present inside a list of numbers and remove that value as well. She told her friend Arleen to create this function.

Note: Use binary search algorithm to search the element in the list.

She wants to use a binary search algorithm in the function. Arleen tried to create the function but she is not able to return the value. Write a program in python for the same to help Dave and Arleen.

CHALLENGE 2- EXAM TIME :

It's time to start preparing for examinations. Ali is worried about his maths examination. He started noting down the topics that he has already covered. Soon the total number of topics covered by Ali was more than 50. Now, whenever he wants to confirm whether he has covered any topic or not, it takes a lot of time to search it in a normal list. To save his time he thought of creating a program in python where he can give the input of covered topics that will get added to a list. And he also has one option in the program to search for the topic that will just say whether the entered topic is already covered or not. To search the topics he used a binary search algorithm. Write the same program to store the topics that you have covered in python then search the topics to check what you have covered and what is remaining.

Note: Use binary search algorithm to search the topic.

HOME TASK

TASK1-ROLL NUMBER :

In a class, the roll number of the students are assigned in alphabetical order. The class monitor requires the roll number of the students. To make his work easier he decided to write a program in python to store all the names in a list. He wanted to apply a linear search algorithm to find out the roll number of any student. Write a program according to the given condition. (Create a function that takes the name as a parameter and return the roll number of the given name)

Note: The roll number of the student is equal to index+1 in the given list.

TASK 2- BROWN CONFUSED?? :

Brown is confused about how a binary search algorithm is better than a linear search algorithm. To compare these two algorithms he created a list of 10000 numbers and a random element between 1 to 10000. He checked how many comparisons are made to find the number in both the algorithms. For a better result, he repeated this entire process 100 times. At the end, he compared the number of comparisons in both algorithms. Do the same thing and find the given ratio:

Number of comparisons in linear search: Number of comparisons in binary search

WHAT'S NEXT?

Sorting Algorithms

LESSON 6

SORTING ALGORITHMS

LEARNING CONTENT

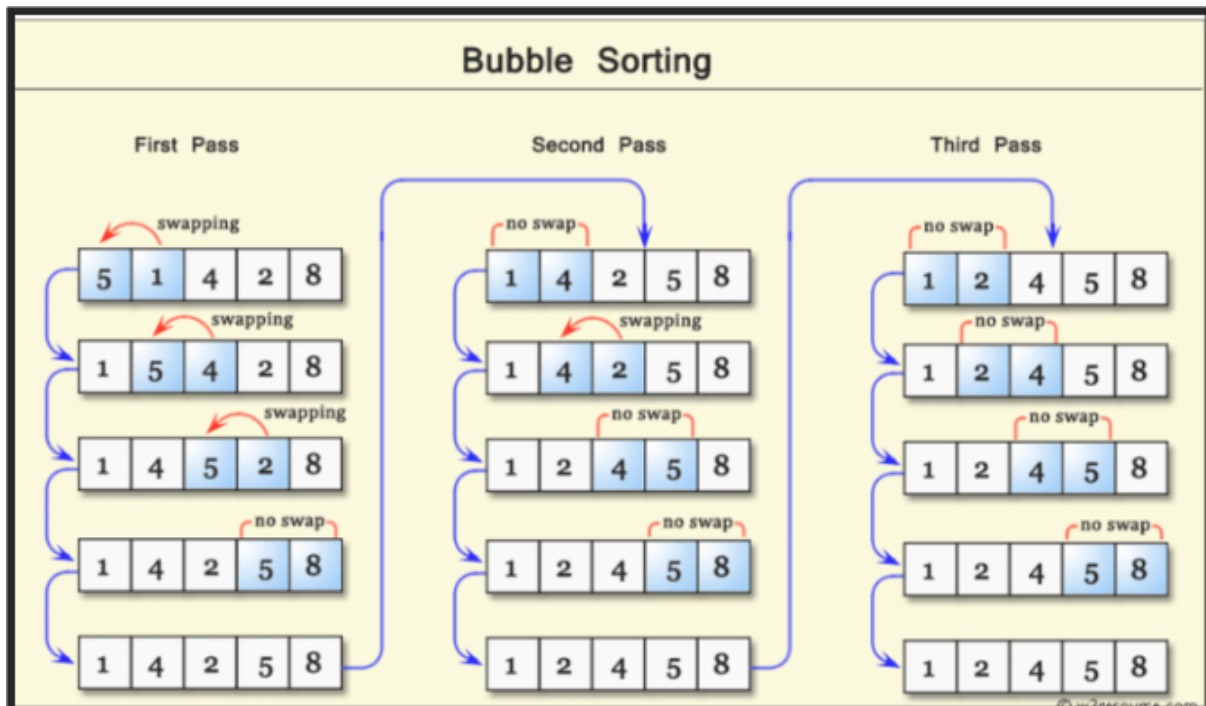
ABOUT SORTING ALGORITHM :

- A sorting algorithm is used to rearrange a given list or an array of elements in a defined order, either increasing or decreasing.
- Sorting algorithms are a fundamental concept in computer science, used to arrange elements in a particular order (usually ascending or descending).
- There are several sorting algorithms, each with its own advantages, disadvantages and use cases.

BUBBLE SORT :

- **Description:** Repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order.
- **Time Complexity:** $O(n^2)$ in the worst and average cases.
- **Space Complexity:** $O(1)$.

- **Use Case:** Simple to implement, useful for small datasets or educational purposes.



```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr

# Example usage
arr = [64, 34, 25, 12, 22, 11, 90]
sorted_arr = bubble_sort(arr)
print("Sorted array is:", sorted_arr)
```

Sorted array is: [11, 12, 22, 25, 34, 64, 90]

SELECTION SORT :

- **Description:** Divides the list into a sorted and an unsorted region, and repeatedly selects the smallest (or largest) element from the unsorted region and moves it to the sorted region.
- **Time Complexity:** $O(n^2)$ in the worst and average cases.
- **Space Complexity:** $O(1)$.

- **Use Case:** Simple to understand and implement, better than Bubble Sort for readability.

```
def selection_sort(numbers):
    n = len(numbers)
    total_comparisons = 0
    total_swaps = 0

    for i in range(n):
        min_index = i
        for j in range(i+1, n):
            total_comparisons += 1
            if numbers[j] < numbers[min_index]:
                min_index = j
        if min_index != i:
            numbers[i], numbers[min_index] = numbers[min_index], numbers[i]
            total_swaps += 1
        print(f"Iteration {i+1}: {numbers} (Comparisons: {i+1}, Swaps: {total_swaps})")

    print("\nSorted List:", numbers)
    print("Total Comparisons:", total_comparisons)
    print("Total Swaps:", total_swaps)

# Initial list
numbers = [29, 72, 98, 13, 87, 66, 52, 51, 36]

# Sorting the list
selection_sort(numbers)
```

```
Iteration 1: [13, 72, 98, 29, 87, 66, 52, 51, 36] (Comparisons: 1, Swaps: 1)
Iteration 2: [13, 29, 98, 72, 87, 66, 52, 51, 36] (Comparisons: 2, Swaps: 2)
Iteration 3: [13, 29, 36, 72, 87, 66, 52, 51, 98] (Comparisons: 3, Swaps: 3)
Iteration 4: [13, 29, 36, 51, 87, 66, 52, 72, 98] (Comparisons: 4, Swaps: 4)
Iteration 5: [13, 29, 36, 51, 52, 66, 87, 72, 98] (Comparisons: 5, Swaps: 5)
Iteration 6: [13, 29, 36, 51, 52, 66, 87, 72, 98] (Comparisons: 6, Swaps: 5)
Iteration 7: [13, 29, 36, 51, 52, 66, 72, 87, 98] (Comparisons: 7, Swaps: 6)
Iteration 8: [13, 29, 36, 51, 52, 66, 72, 87, 98] (Comparisons: 8, Swaps: 6)
Iteration 9: [13, 29, 36, 51, 52, 66, 72, 87, 98] (Comparisons: 9, Swaps: 6)

Sorted List: [13, 29, 36, 51, 52, 66, 72, 87, 98]
Total Comparisons: 36
Total Swaps: 6
```

QUICK SORT :

- **Description:** Picks an element as a pivot and partitions the array around the pivot, recursively sorting the sub-arrays.
- **Time Complexity:** $O(n \log n)$ on average, $O(n^2)$ in the worst case.
- **Space Complexity:** $O(\log n)$ due to the recursion stack.
- **Use Case:** Very efficient for large datasets, but performance depends on the pivot selection.

```
def quick_sort(arr):  
    if len(arr) <= 1:  
        return arr  
    else:  
        pivot = arr[0]  
        less_than_pivot = [x for x in arr[1:] if x <= pivot]  
        greater_than_pivot = [x for x in arr[1:] if x > pivot]  
        return quick_sort(less_than_pivot) + [pivot] +  
        quick_sort(greater_than_pivot)  
  
# Initial list  
numbers = [51, 95, 66, 72, 42, 38, 39, 41, 15]  
  
# Sorting the list  
sorted_numbers = quick_sort(numbers)  
print("Sorted list:", sorted_numbers)
```

```
Sorted list: [15, 38, 39, 41, 42, 51, 66, 72, 95]
```

IN CLASS CHALLENGES

CHALLENGE1- HEIGHT MEASURE :

Ajay and his friends were measuring their heights in random order. They created the list of heights of all the students. Ajay wanted to know the range of the height in his class. i.e lowest height, biggest height, and the difference between lowest height and biggest height. Write a python program to get the answer.

CHALLENGE 2- SPORTS :

Sachin is the sports captain of his school. He is selecting players for different sports. He made a list of players and also he had to assign the chest number to each player in alphabetical order of their names. To simplify the work of the Sachin .write a program in python where you can append the names of the players inside a list. Then sort the entire list using the bubble sort algorithm. Then by using the index number of the list, assign the chest number to each player.

HOME TASK

TASK1-SWAP :

Raj learned two new sorting algorithms, bubble sort, and selection sort. For him, bubble sort is easy as compared to selection sort. But still, he wants to know which of these two sorting algorithms is more efficient. He wants to compare the number of comparisons and the number of swaps made in sorting a list. Write a program to do the same thing and find which of these sorting algorithms is efficient, bubble sort and selection sort algorithms on a sample list [10, 9, 8, 7, 6, 5, 4, 3, 2, 1] and calculates the number of comparisons made by each sorting algorithm.

TASK 2- SORT :

Alina is aware of the inbuilt sorted() function. She is very interested to know how we can create our manual sorting function. To show the same thing to Alina, write a python program and create sortIt() function. Use any suitable sorting algorithm inside this function and return the sorted list.

```
sampleList = [1,3,5,7,9,0,2,4,6,8,10]
```

WHAT'S NEXT?

Sorting Algorithm-2

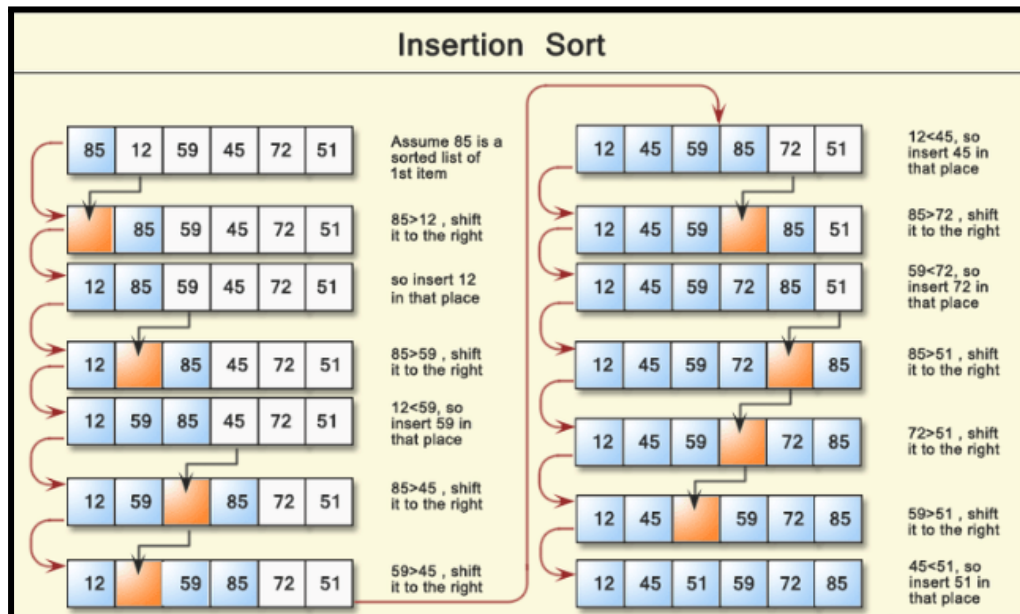
LESSON 7

SORTING ALGORITHM - 2

LEARNING CONTENT

INSERTION SORT :

- **Description:** Builds the sorted array one item at a time, It iterates over the input list and grows a sorted list behind it. At each iteration, the algorithm removes one element from the input data, finds the location it belongs to in the sorted list, and inserts it there. This process repeats until no input elements remain.
- **Time Complexity:** $O(n^2)$ in the worst case, $O(n)$ in the best case when the array is already sorted.
- **Space Complexity:** $O(1)$.
- **Use Case:** Efficient for small datasets and nearly sorted datasets.



Example :

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key

# Initial list
numbers = [85, 12, 59, 45, 72, 51]

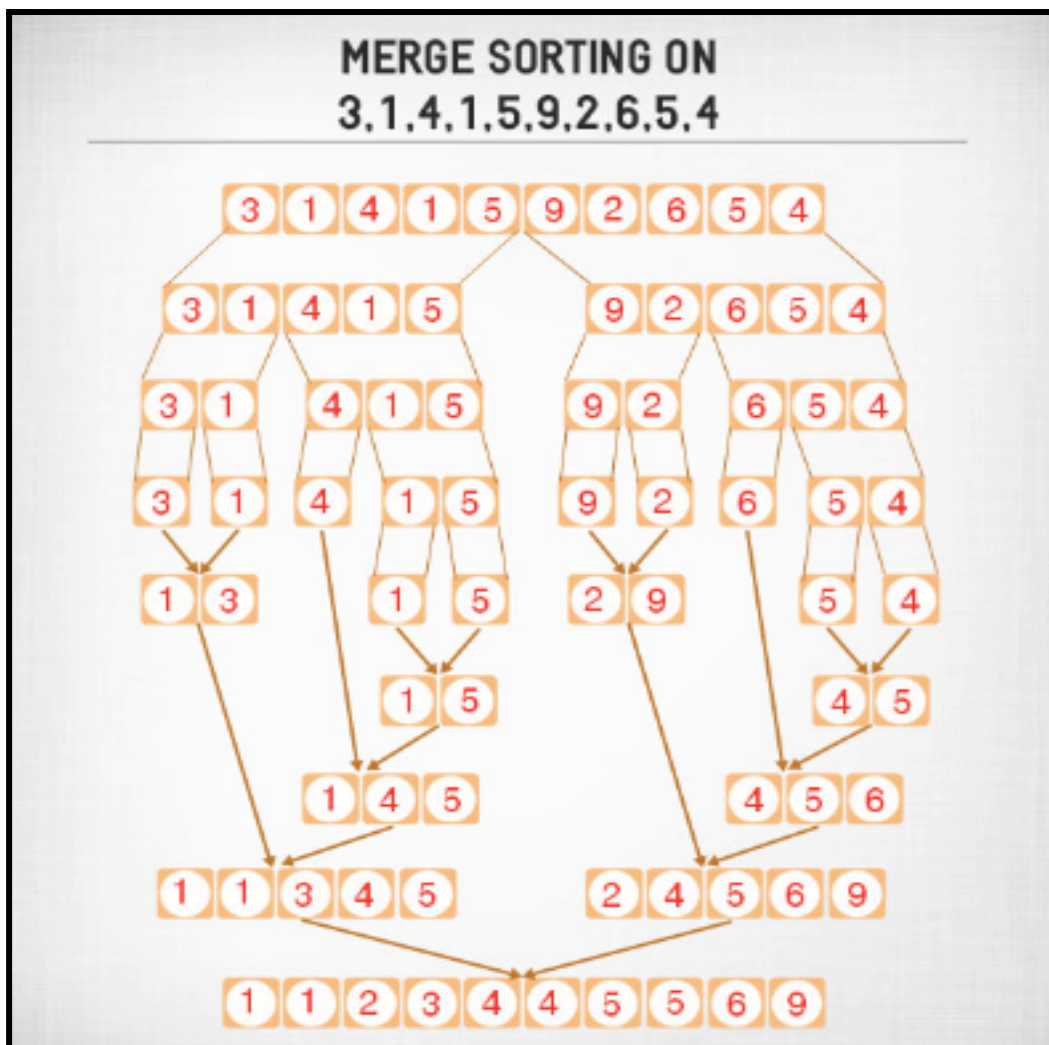
# Sorting the list
insertion_sort(numbers)

# Printing the sorted list
print("Sorted list:", numbers)
```

Sorted list: [12, 45, 51, 59, 72, 85]

MERGE SORT :

- **Description:** Divides the array into two halves, recursively sorts each half, and then merges the sorted halves.
- **Time Complexity:** $O(n \log n)$ in all cases.
- **Space Complexity:** $O(n)$ due to the auxiliary space used for merging.
- **Use Case:** Stable sort, useful for large datasets and linked lists.



Example :

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left_half = arr[:mid]
    right_half = arr[mid:]
    left_half = merge_sort(left_half)
    right_half = merge_sort(right_half)
    return merge(left_half, right_half)

def merge(left, right):
    result = []
    left_idx, right_idx = 0, 0
    while left_idx < len(left) and right_idx < len(right):
        if left[left_idx] < right[right_idx]:
            result.append(left[left_idx])
            left_idx += 1
        else:
            result.append(right[right_idx])
            right_idx += 1
    result.extend(left[left_idx:])
    result.extend(right[right_idx:])
    return result

# Initial list
numbers = [3, 1, 4, 1, 5, 9, 2, 6, 5, 4]

# Sorting the list
sorted_numbers = merge_sort(numbers)

# Printing the sorted list
print("Sorted list:", sorted_numbers)
```

```
Sorted list: [1, 1, 2, 3, 4, 4, 5, 5, 6, 9]
```

IN CLASS CHALLENGES

CHALLENGE1- JUMBLED LIST :

Hey buddy! Do you know how to sort a jumbled-up list of numbers? Imagine you have a magic wand that can organise numbers in a sequence, just like how a wizard arranges their spell ingredients. How about we create a game where we watch the magic happen? Can you write a program that takes a list of numbers and sorts them using your magical sorting wand? Let's add some colours, emojis, and maybe even a little countdown for some extra fun! Are you ready to bring some magic into our computer world?

CHALLENGE 2- SORTING SORCERER :

Picture yourself as a sorting sorcerer in the mystical realm of Numeria. The Numerian Forest is filled with enchanted numbers, but they're all jumbled up and causing quite a stir! Your task is to bring order to this lively scene using the ancient magic of **Merge Sort**. Can you conjure up a spell in the form of a Python program that will weave these numbers into a harmonious sequence? Get ready to unlock the secrets of the Numerian Forest and become the hero of the sorting saga!

HOME TASK

TASK1-CONTACTS :

Alex wants to save the contact number of his friends on his mobile phone so he created a list of his friends with numbers. He decided to sort the list in alphabetical order first then save the number. Write a program in Python to sort a list using insertion sort to help Alex organise his contacts.

TASK 2- DESCENDING ORDER :

Kevin believes that the insertion sort algorithm can only arrange a list in ascending order, whereas Sam believes it's versatile enough to sort the list in descending order as well. To settle the debate and help Sam, write a Python program showcasing how to sort a list of numbers in descending order using the insertion sort algorithm.

WHAT'S NEXT?

Problem Solving - 1

LESSON 8

PROBLEM SOLVING - 1

IN CLASS CHALLENGES

CHALLENGE1- PASSWORD GENERATION :

Olivia wants to create a password for his social media account. For her password, she wants to use only characters that occur in her name. She wants to use the characters O, L, I, V, I and A. She wondered how many possible passwords there are if the length of the password can be anything with no repetition of characters. Write a python program and use recursion to find the answer to the same question. Also, check how many passwords are possible with the characters of your name. Print all the passwords on the screen.

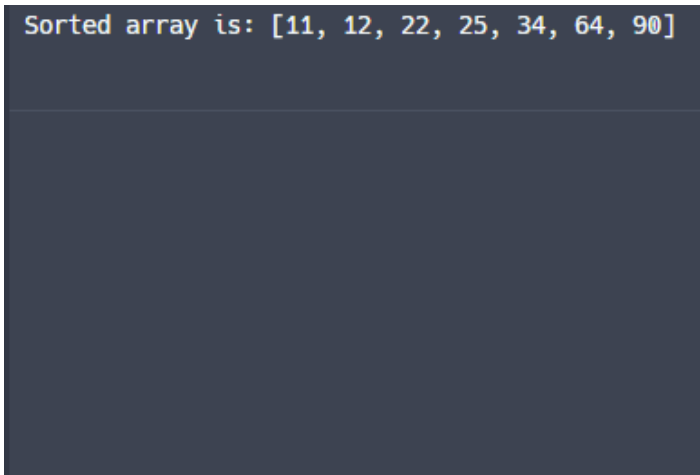
```
All possible passwords for 'OLIVIA':  
OLIVIA  
OLIVAI  
OLIIVA  
OLIIAV  
OLIAVI  
OLIAIV  
OLVIAI  
OLVIAI  
OLVIAI  
OLVIAI  
OLVAII  
OLVAII  
OLIIVA  
OLIIAV  
OLIVIA  
OLIVAI  
OLIAIV  
OLIAVI  
OLAIVI  
OLAIV  
OLAVII  
OLAVII  
OLAIV  
OLAIVI  
OLAIVI  
OILVIA  
OILVAI  
OILIVA  
OILIAV  
OILAVI  
OILAIV  
OIVLIA  
OIVLAI  
OIVILA  
OIVIAL
```

```
AVILIO  
AVIIOL  
AVIIL0  
AVIOLI  
AVIOIL  
AVILOI  
AVILIO  
AVIIOL  
AVIIL0  
AIOLIV  
AIOLVI  
AIOILV  
AIOIVL  
AIOVLI  
AIOVIL  
AILOIV  
AILOVI  
AILIOV  
AILIVO  
AILVOI  
AILVIO  
AITOLV  
AIIOVL  
AIILOV  
AIIILV0  
AIIVOL  
AIIVLO  
AIVOLI  
AIV0IL  
AIVLOI  
AIVLIO  
AIVIOI  
AIVILO  
AIVILO
```

Number of possible passwords for 'OLIVIA': 720

CHALLENGE 2- SORTING BUBBLES :

Sophia just learned the Bubble sort algorithm and recursion. As the bubble sort algorithm needs the repetitive process she thinks that the bubble sort algorithm can be developed using recursion instead of loop. She tried doing this but was not able to complete the program. Write a program in python for the bubble sort algorithm and use recursion to help Sophia in solving the problem.



```
Sorted array is: [11, 12, 22, 25, 34, 64, 90]
```

CHALLENGE 3- GUESSING GAME :

Himanshi and Arunima were returning from school. On the school bus, they were playing a game. Using the quicksort algorithm, sort the list of characters' ages: [Alice - 25, Bob - 30, Charlie - 22, David - 35, Eve - 28]. Also, participate in a guessing game to find the secret number between 1 and 100. You will receive hints if your guess is too high or too low.


```
Sorted list of characters' ages:
Charlie - 22
Alice - 25
Eve - 28
Bob - 30
David - 35

Welcome to the Guessing Game!
Try to guess the secret number between 1 and 100.
Enter your guess: 5
Too low! Try again.
Enter your guess: 9
Too low! Try again.
Enter your guess: 20
Too high! Try again.
Enter your guess: 12
Too low! Try again.
Enter your guess: 15
Too low! Try again.
Enter your guess: 17
Too high! Try again.
Enter your guess: 16
Congratulations! You guessed the secret number 16.
```

HOME TASK

TASK1 - UPSIDE DOWN :

Hana thought that if we can use recursion for the repetitive process then definitely we can create different shapes as well with the help of recursion instead of using nested loops. Hana's friend Maddy said it is impossible to print patterns with the help of recursion. To prove that printing the following patterns is possible with help of recursion write a python program.

```
*****
*****
*****
*****
***
*
*
```

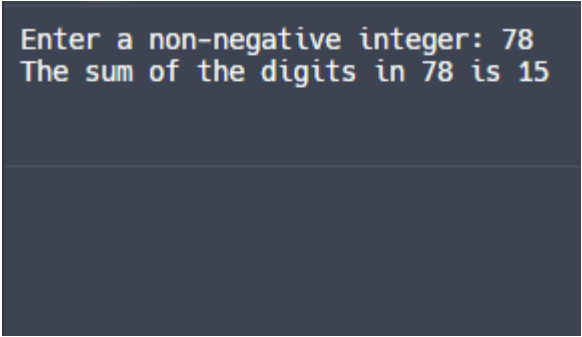
TASK 2- SUM OF NON NEGATIVE INTEGER :

Priya: Hey Priti, I learned a Python program that sums digits in a number using recursion.

Priti: Sounds interesting! How does it work?

Priya: It adds the last digit to the sum of the remaining digits by recursively calling itself.

Write a Python program that calculates the sum of a non-negative integer using recursion



```
Enter a non-negative integer: 78
The sum of the digits in 78 is 15
```

WHAT'S NEXT?

Mini Project - 1

LESSON 9

MINI PROJECT - 1

MINI PROJECT

MAGICAL SORTING GAME :

Captain Jack wants to create a game where he needs to sort a magical list of numbers in ascending order. Each time he successfully sorts the list, he moves to the next list of numbers. So, are you ready to help Captain Jack build this game? Write a program that creates this game using Python! You'll need to use the Tkinter library for creating the game window and buttons, and implement a sorting algorithm, such as bubble sort, selection sort, or insertion sort, to sort the numbers.



Sorting Algorithm Game

Enter numbers separated by commas:

Submit

Sort Numbers



Sorting Algorithm Game

Enter numbers separated by commas:

Submit

Sort Numbers

Sorted List: [2, 4, 5, 6, 9]

HOME TASK

TASK1 - ARTIFACTS :

The leader of a space expedition Sindhu exploring a distant galaxy. During the mission, Sindhu discovers a mysterious planet with artifacts of varying rarities scattered across its surface. To uncover the planet's secrets, sindhu must sort these artifacts in ascending order of rarity.

Can you create a program in Python to help sindhu sort these artifacts using insertion sort, merge sort, or quick sort based on the number of artifacts?

Note: This program generates a random list of artifacts with a random number of elements between 5 and 20. It then uses Insertion sort, merge sort, or quick sort based on the number of artifacts to sort them in ascending order.

Sorted artifacts: [11, 17, 18, 32, 49, 61, 83, 84, 89]

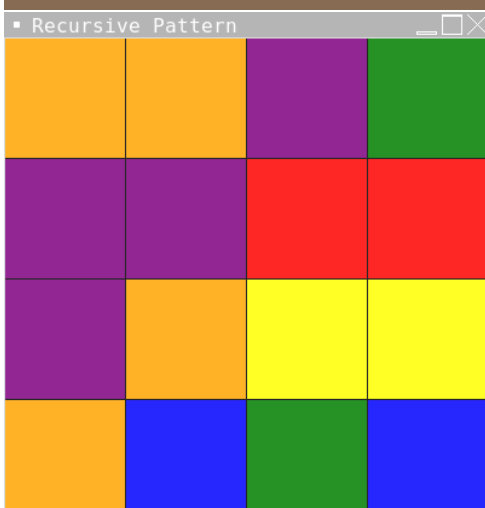
Sorted artifacts: [3, 5, 8, 15, 17, 23, 65, 66, 72, 79, 90, 92]

Sorted artifacts: [2, 24, 25, 30, 34, 37, 38, 51, 83, 90]

TASK 2- RECURSIVE PATTERN :

Emma is fascinated by patterns and wants to create a program that generates a unique pattern each time it runs. She wants the program to use recursion to create the patterns. Help Emma by writing a Python program that generates a random pattern using recursion. Use the Tkinter library to display the pattern in a window. The pattern should be visually interesting and different each time the program is run.

Note: Generates a random pattern and different each time the program is run.



WHAT'S NEXT?

Assessment - 1

LESSON 10

ASSESSMENT - 1

ASSESSMENT

SOCCKER :

Shriyan and his friends decided to play Soccer. They were not able to divide themselves into two groups perfectly. Shriyan thought that the team should be divided randomly into two parts. Write a program in python to help them in dividing the group randomly into two parts.

Note : Create a shuffle function to shuffle the elements of the list then divide it into two parts.

Hint :

Use sorting algorithm, random module, and recursion.

Create a list by taking the input from the users and then print two lists on the screen with team A and team B.

```
Run
Enter the names of players separated by commas: david,sanjay,rahul,afzal,prashanth,majid,vijay,mahi
Team A: ['afzal', 'majid', 'prashanth', 'mahi']
Team B: ['rahul', 'vijay', 'sanjay', 'david']
```

HOME TASK

TASK1 - SURPRISE TEST :

The teacher conducted a surprise class test and wanted two different lists of students. One of them should be sorted according to the names of the students and the second one should be sorted according to the marks they obtained.

NOTE: Take the input from the user to fill the dictionary and use bubble or selection sort for sorting.

```
Run
Enter the number of students: 5
Enter student's name: max
Enter student's marks: 78
Enter student's name: jill
Enter student's marks: 89
Enter student's name: harry
Enter student's marks: 67
Enter student's name: kairav
Enter student's marks: 90
Enter student's name: flex
Enter student's marks: 56
```

```
Students sorted by name:
flex 56.0
harry 67.0
jill 89.0
kairav 90.0
max 78.0
```

```
Students sorted by marks:
flex 56.0
harry 67.0
max 78.0
jill 89.0
kairav 90.0
```

TASK 2- BASKETBALL TEAM :

Kairav and his friends decided to play basketball. They were not able to divide themselves into two groups perfectly. Abhishek thought that the team should be divided randomly into two parts. Write a program in python to help them in dividing the group randomly into two parts.

NOTE: The program will randomly shuffle the list of players and divide them into two teams

```
Team 1: ['Player2', 'Player6', 'Player5', 'Player7']  
Team 2: ['Player4', 'Player8', 'Player1', 'Player3']
```

WHAT'S NEXT?

Data Structures - stack

LESSON 11

DATA STRUCTURES - STACKS

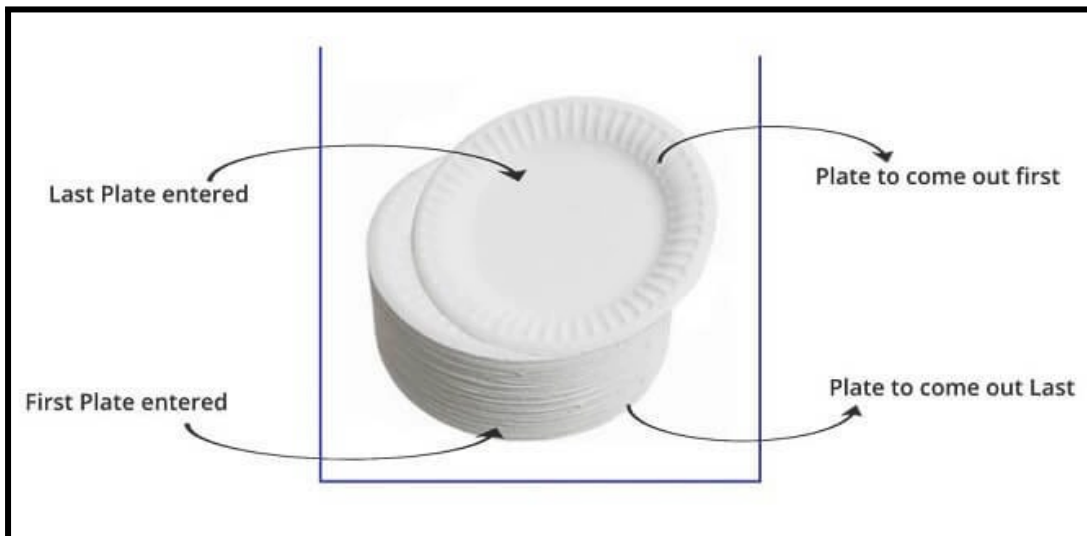
LEARNING CONTENT

WHAT IS STACK?

- Stack is a linear data structure which follows a particular order in which the operations are performed.
- The order may be LIFO or FILO.
- Last In First Out (LIFO) : This means that the last element added to the stack will be the first one to be removed.
- First In Last Out (FILO) : This means that the first element added to the stack will be the last one to be removed.

Example : Think of a stack of plates in a canteen. The plate on top is the first one you take, and the plate at the bottom stays the longest.

This is like a stack because it follows Last In, First Out (LIFO) or First In, Last Out (FILO) order.



ADVANTAGES OF STACK :

- Stacks are easy to implement and use. They can be implemented using arrays or linked lists, making them flexible for various applications.
- Stacks manage memory efficiently by only using as much memory as needed for the number of elements they contain. So, no memory is wasted on unused capacity.
- Stacks are simple data structures with a well-defined set of operations, which makes them easy to understand and use.
- Stacks are efficient for adding and removing elements, as these operations have a time complexity of $O(1)$.
- Stacks are ideal for reversing data or processing data in reverse order.
- Stacks are useful in scenarios that require backtracking, such as navigating through a maze or implementing undo features in applications.

DISADVANTAGES OF STACK :

- Stacks are not suitable for situations where random access or multiple points of access to elements are required.
- Stacks do not provide fast access to elements other than the top element.

- Stacks do not support efficient searching, as you have to pop elements one by one until you find the element you are looking for.
- If too many elements are added to a stack without sufficient capacity, it can lead to stack overflow.

STANDARD STACK OPERATIONS :

- push() : The operation where we insert an element in a stack is known as a push. If the stack is full then the overflow condition occurs.
- pop() : When we delete an element from the stack, the operation is known as a pop. If the stack is empty means that no element exists in the stack, this state is known as an underflow state.
- peek() : It returns the element at the given position.
- count() : It returns the total number of elements available in a stack.
- change() : It changes the element at the given position.
- display() : It prints all the elements available in the stack.
- isEmpty(): It determines whether the stack is empty or not.
- isFull(): It determines whether the stack is full or not.

STACK - PUSH & POP OPERATIONS :

Push :

- Adds an element to the top of the stack.

Example : Push into a stack

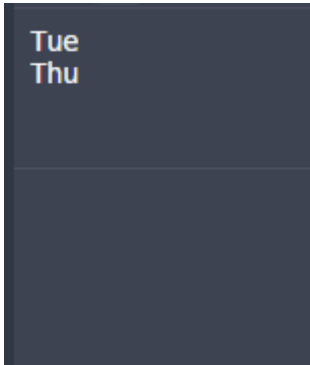
```

class Stack:
    def __init__(self):
        self.stack = []

    def add(self, dataval):
# Use list append method to add element
        if dataval not in self.stack:
            self.stack.append(dataval)
            return True
        else:
            return False
# Use peek to look at the top of the stack
    def peek(self):
        return self.stack[-1]

AStack = Stack()
AStack.add("Mon")
AStack.add("Tue")
AStack.peek()
print(AStack.peek())
AStack.add("Wed")
AStack.add("Thu")
print(AStack.peek())

```



Tue
Thu

Pop :

As we know we can remove only the top most data element from the stack, we implement a python program which does that.

The remove function in the following program returns the top most element.

We check the top element by calculating the size of the stack first and then use the in-built pop() method to find out the top element.

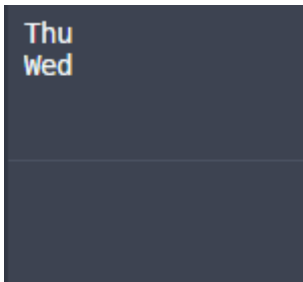
Example :

```
class Stack:
    def __init__(self):
        self.stack = []

    def add(self, dataval):
        # Use list append method to add element
        if dataval not in self.stack:
            self.stack.append(dataval)
            return True
        else:
            return False

    # Use list pop method to remove element
    def remove(self):
        if len(self.stack) <= 0:
            return ("No element in the Stack")
        else:
            return self.stack.pop()

AStack = Stack()
AStack.add("Mon")
AStack.add("Tue")
AStack.add("Wed")
AStack.add("Thu")
print(AStack.remove())
print(AStack.remove())
```



```
Thu
Wed
```

IN CLASS CHALLENGES

CHALLENGE1 - STACK CLASS :

Toshini is preparing for her examination, and she has so many topics to cover. She decided to study each topic properly, and then revise each topic in reverse order to finalise it. To make her work easy, write a python program and create a Stack class (stack data structure) where she can add the topic that she is learning for the first time. After completing all the topics she can check the peak value of the stack to check which topic she has to revise. After revising the topic she can pop the element from the stack. Once the stack becomes empty, it indicates she is ready for her examination.

CHALLENGE 2 - ANALYSING STOCK :

Niwas is working on his data visualisation project. He is analysing a stock, and one of the columns in his data is stock span. He is having all the historical prices of stock in a list and somehow he wants to extract the stock span for each day. Write a python program to create a list of stock span using the given list of historical prices.

HOME TASK

TASK1 - TRACK OF THE INDICES OF STOCK PRICES :

Aman completed his project successfully and now he started analysing a new stock. He has stock prices for the past 45 days. He wants to check how many times the stock made an all-time high in the form of a list. He has a list of historical prices. Write a python program to convert the list into such a form that can easily represent days when the stock made an all-time high.

List of historical price:

[498 , 536 , 520 , 512 , 543 , 555 , 560 , 551]

List which shows the days when the stock made an all-time high:

[-1 , -1 , 536 , 536 , -1 , -1 , -1 , 560]

Where -1 represents breakout in stock price

Note: As an example, you can create a list of 33 random numbers in a particular range or take the input. (optional)

(Question - greatest element to the left)

TASK 2 - REVERSE STACK :

Sailesh is a classmate of Neeta, and he is also preparing for the examination. He also wants to revise the topics after learning everything but in the same order instead of reverse order. Create a function `reverseStack()` that can reverse the entire stack from top to bottom. In this way, Shubham will be able to use the stack for his preparation.

Hint: reverse the stack using two extra temporary stacks.

WHAT'S NEXT?

Data Structures - Queue

LESSON 12

DATA STRUCTURES - QUEUE

LEARNING CONTENT

WHAT IS QUEUE?

- A queue is a linear data structure that follows the First In First Out (FIFO) principle, where The first element added to the queue will be the first one to be removed.
- A queue is a collection of items where the addition of new items happens at the rear end (tail), and the removal of existing items occurs at the front end (head).

Example :

- In a bank Customers stand in line and are served in the order they arrive.
- Booking systems for flights, hotels, or movie tickets often use queues to handle requests in the order they are received

- When multiple documents are sent to a printer, they are placed in a print queue. Documents are printed in the order they are received.
- Passengers go through security checks in the order they arrive in the queue.
- Customers queue up to purchase food and are served in the order they arrive.

All these scenarios resonate with the mechanics of a queue.

BASIC OPERATIONS OF QUEUE :

- Enqueue: The Enqueue operation is used to insert the element at the rear end of the queue. It returns void.
- Dequeue: It performs the deletion from the front-end of the queue. It also returns the element which has been removed from the front-end. It returns an integer value.
- Peek: This is the third operation that returns the element, which is pointed by the front pointer in the queue but does not delete it.
- Queue overflow (isfull): It shows the overflow condition when the queue is completely full.
- Queue underflow (isempty): It shows the underflow condition when the Queue is empty, i.e., no elements are in the Queue.

ENQUEUE & DEQUEUE OPERATIONS :

Enqueue and dequeue operations are fundamental to the functioning of queues in computer science. A queue is a linear data structure that follows the First In First Out (FIFO) principle, where the first element

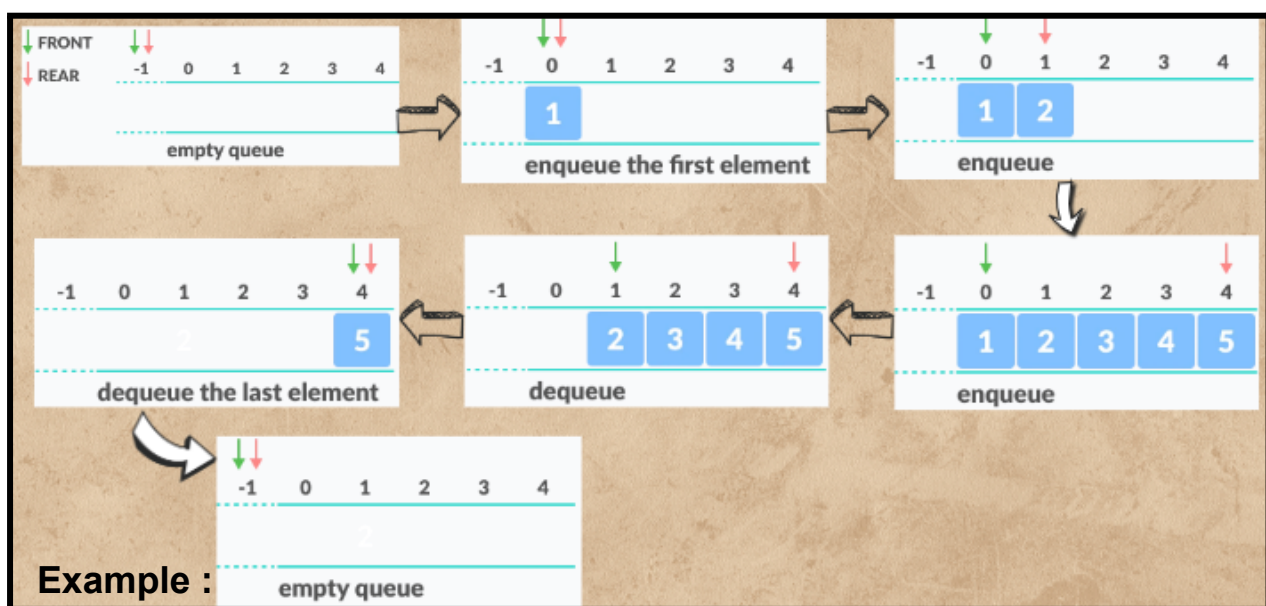
added is the first one to be removed. Here's a breakdown of these operations:

Enqueue Operation

- **Purpose** : Add an element to the end (rear) of the queue.
- **Steps**:
 1. Check if the queue is full (optional, mainly for fixed-size implementations).
 2. If not full, increment the rear pointer.
 3. Add the new element at the position pointed to by the rear pointer.

Dequeue Operation

- **Purpose**: Remove an element from the front of the queue.
- **Steps**:
 1. Check if the queue is empty.
 2. If not empty, retrieve the element at the position pointed to by the front pointer.
 3. Increment the front pointer.



```
# Queue implementation in Python
class Queue:
    def __init__(self):
        self.queue = []
    # Add an element
    def enqueue(self, item):
        self.queue.append(item)
    # Remove an element
    def dequeue(self):
        if len(self.queue) < 1:
            return None
        return self.queue.pop(0)
    # Display the queue
    def display(self):
        print(self.queue)
    def size(self):
        return len(self.queue)

q = Queue()
q.enqueue(1)
q.enqueue(2)
q.enqueue(3)
q.enqueue(4)
q.enqueue(5)
print("After adding elements to the queue")
q.display()
q.dequeue()
print("After removing an element from the queue")
q.display()
```

```
After adding elements to the queue
[1, 2, 3, 4, 5]
After removing an element from the queue
[2, 3, 4, 5]
```

HOW TO IMPLEMENT QUEUES IN PYTHON :

In Python, you can implement queues in several ways. Here are three common methods:

1. **Using a List :** A simple way to implement a queue is to use a list. Elements can be added to the end of the list using the `append()` method and removed from the beginning using the `pop()` method with an index of 0.

Example : *# implement a queue using a list*

```
queue = []
queue.append(1) # enqueue
queue.append(2)
queue.append(3)
print(queue)   # [1, 2, 3]
x = queue.pop(0) # dequeue
print(x)       # 1
print(queue)   # [2, 3]
```

```
[1, 2, 3]
1
[2, 3]
```

2. **Using `collections.deque` :** Python's `collections` module provides a double-ended queue (`deque`) data structure, which can be used as a queue by appending to the right and popping from the left.

Example :

```
from collections import deque

queue = deque()
queue.append(1) # enqueue
queue.append(2)
queue.append(3)
print(queue)   # deque([1, 2, 3])
x = queue.popleft() # dequeue
print(x)       # 1
print(queue)   # deque([2, 3])
```

```
deque([1, 2, 3])
1
deque([2, 3])
```

3. Using queue.Queue :

Python's built-in queue module provides the Queue class, which is a thread-safe queue that can be used in a multi-threaded environment. Elements can be added to the queue using the put() method and removed using the get() method.

Example :

```
import queue

queue = queue.Queue()
queue.put(1)    # enqueue
queue.put(2)
queue.put(3)
print(queue.queue) # [1, 2, 3]
x = queue.get()  # dequeue
print(x)         # 1
print(queue.queue) # [2, 3]
```

```
[1, 2, 3]
1
[2, 3]
```

SOME COMMON APPLICATIONS OF QUEUE :

- In operating systems, queues are used for job scheduling, where jobs are executed in the order they arrive (First-Come, First-Served or FCFS scheduling).
- Print jobs are managed using a queue, where documents are printed in the order they are received.
- Many task scheduling algorithms use queues to manage tasks. For example, round-robin scheduling in operating systems uses a circular queue.
- Queues are used in buffering for handling data streams, such as IO buffers, network packet buffers, and multimedia data streams.
- CPU tasks are managed using queues to ensure fair processing of tasks and efficient CPU usage.

- Queues are used in networking for managing packets in routers and switches to ensure smooth data transmission.
- In simulations, queues are used to model real-world systems like checkout lines, call centres, and traffic lights.
- Queues manage resources like memory allocation, disk scheduling, and device management.
- Event-driven systems, such as GUI applications, use queues to manage events like mouse clicks and key presses.
- Calls in a call centre are managed using a queue to ensure they are answered in the order they are received.
- Queues manage customers in service areas, such as banks or ticket counters, to ensure fair and organised service.
- In web servers and other distributed systems, queues help in load balancing by distributing incoming requests evenly across multiple servers.

IN CLASS CHALLENGES

CHALLENGE1 - ASSIGNMENT QUEUE :

Sindhu attends online classes. She is unable to complete after class assignments Sometimes she misses the due date and then that assignment gets marked as turned in late. She is unable to think how to manage assignments. To help her, write a python program and create a queue data structure so that she can enqueue (add) new assignments inside the queue. Also, create a peek() method which can help her in getting the oldest assignment she added in the queue. Create a dequeue method that can help her in removing the peek() assessment after she completes it. Also, create a method isEmpty() so that she can check whether she left with any assignment or not.

CHALLENGE 2 - CLASS MONITOR :

In a school, every student wants to leave the class first after the last session. A teacher gave responsibility to the class monitor to make a list of all the students in order they visit the class in the morning. And after the last session, they should leave the class in the same order (First in first out). Create a python program that can help the class monitor in managing the list more smoothly.

HOME TASK

TASK1 - CAROUSEL SIMULATION :

Mary is at Adventureland, the most exciting amusement park in town! She is standing in line for the magical carousel, surrounded by laughter and the sweet scent of cotton candy. Excitement bubbles within her as she eagerly awaits her turn. However, there's a line of kids ahead of her. Now, here's the challenge: Can you create a thrilling Python program to simulate this enchanting experience?

TASK 2 - TOYS ORDER :

Do you know how to play with a queue?
Here's a fun coding challenge! Imagine you have a magical box where you can put toys in one by one and take them out in the same order. Now, can you write a magical Python code that takes the toys out of the box, turns them upside down, and puts them back in? It's like reversing the order of the toys! Give it a try!
Hint : Keep in mind, there's a special trick called a stack that can assist you with this magic!

WHAT'S NEXT?

Data Structures - Linked List

LESSON 13

DATA STRUCTURES - LINKED LIST

LEARNING CONTENT

WHAT IS A LINKED LIST?

- Linked List is a list of elements (containing data) in which the elements are linked together.
- Nodes : the building blocks of a linked list. Each node contains data and a reference (or pointer) to the next node in the sequence.
- Head : The first node in a linked list. It is used as a starting point for any operations on the list.
- Tail : The last node in a linked list, which points to null (indicating the end of the list).
- Linked lists can grow and shrink in size dynamically, as nodes can be added or removed without reallocating or reorganising the entire data structure.



- As you can see in the above picture, each data is connected (or linked) to a different data and thus, forming a linear list of data, and this is a linked list.
- Since data in a linked list are stored in a linear fashion, a linked list is a linear data structure.
- There are also non-linear data structures like trees, graphs, etc.

NODE :

- A node is a fundamental component of a linked list and other data structures like trees and graphs.
- Nodes are the building blocks of a linked list. Each node contains data and a reference (or pointer) to the next node in the sequence.
- To connect data from a linked list we put our data in Nodes and these nodes are connected in the desired fashion.

Example : A Node can be viewed as a container or a box which contains data and other information in it. Nodes are connected (or organised) in a specific way to make data structures like linked lists, trees, etc.



- In the above picture you can see that each node has 2 parts, one stores the data & another is connected to a different node. The last node is not connected to any other node & thus its connection to the next node is null.
- In a linked list, a node is connected to a different node forming a chain of nodes.
- Thus to make a linked list, we first need to make a node which should store some data into it and also a link to another node.

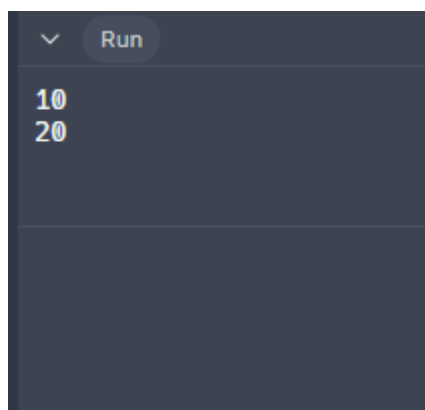
Example :

```
class Node():
    def __init__(self, data):
        self.data = data

a = Node(10)
b = Node(20)

a.next = b
b.next = None

print(a.data)
print(a.next.data)
```



- In the above example code we are able to access the node b with a.next.

OPERATIONS IN A LINKED LIST :

various operations to perform on a linked list:

1. Traversal

2. Insertion
3. Deletion

Traversal :

- Definition : Visiting nodes in a particular order to access their data OR Accessing each node of the linked list sequentially from the head to the tail.
- Generally, we call the first node of a linked list the "head" of the linked list, and we always keep the access of the head of the linked list so that we have access to all the nodes of a linked list.

Insertion :

Definition : Adding a new node to the linked list (at the beginning, end, or any position). It's more than a one-step activity. First, we create a node with the same structure and specify the location where we'll insert it.

Insertion in the Middle :

- **Step 1:** Create a new node (Node B) and allocate memory.
- **Step 2:** Assign the data to the new node.
- **Step 3:** Set the new Node B's next pointer to Node C (the node currently following Node A).
- **Step 4:** Update Node A's next pointer to point to the new Node B.
- **Result:** Node B is inserted between Node A and Node C.

Insertion at the Beginning :

- **Step 1:** Create a new node and allocate memory.
- **Step 2:** Assign the data to the new node.
- **Step 3:** Set the new node's next pointer to the current head (the first node in the list).
- **Step 4:** Update the head of the linked list to be the new node.
- **Result:** The new node is now the first node in the list.

Insertion at the End :

- **Step 1:** Create a new node and allocate memory.
- **Step 2:** Assign the data to the new node.
- **Step 3:** Set the new node's next pointer to null (indicating the end

of the list).

- **Step 4:** If the list is empty, set the head to the new node.
- **Step 5:** If the list is not empty, traverse to the last node.
- **Step 6:** Update the last node's next pointer to the new node.
- **Result:** The new node is now the last node in the list.

Deletion :

Definition : Removing a node from the linked list. Just like insertion, deletion is also a multi-step process. First, we track the node to be removed by using searching algorithms. Once it's located, we change the reference link of the previous node to the node that is next to our target node. Once the link pointing to the target node is removed, we'll also remove the link from our target node to the node that is next to it. Now we can either keep it in memory or deallocate memory and eliminate the target node.

Deletion in the Middle :

- **Step 1:** Traverse the list to find the node just before the node to be deleted.
- **Step 2:** Store the node to be deleted in a temporary variable.
- **Step 3:** Update the next pointer of the previous node to the next pointer of the node to be deleted.
- **Step 4:** Free the memory of the temporary variable (the node to be deleted).
- **Result:** The node is removed, and the previous node now points to the next node.

Deletion at the Beginning:

- **Step 1:** Check if the list is empty. If it is, there is nothing to delete.
- **Step 2:** Store the current head node in a temporary variable.
- **Step 3:** Update the head to the next node.
- **Step 4:** Free the memory of the temporary variable (the old head node).
- **Result:** The first node is removed, and the second node becomes the new head.

Deletion at the End:

- **Step 1:** Check if the list is empty. If it is, there is nothing to delete.
- **Step 2:** Traverse the list to find the second-to-last node.
- **Step 3:** Store the last node in a temporary variable.
- **Step 4:** Update the next pointer of the second-to-last node to null.
- **Step 5:** Free the memory of the temporary variable (the last node).
- **Result:** The last node is removed, and the second-to-last node is now the last node pointing to null

IN CLASS CHALLENGES

CHALLENGE1 - TRAIN STATION :

Hey Champion! Today, we're going to create an interactive program that simulates a train station. So, imagine a train station. There's a platform where passengers can wait for the train. We're going to represent this platform using a special kind of list called a "linked list". In this list, each passenger is like a train car, connected to the next one.

CHALLENGE 2 - TREASURE HUNT :

Hey there, Captain of coders! Imagine you're on a grand adventure, seeking hidden treasure on distant islands. Each island is like a clue on your map. But while sailing the seven seas in search of buried treasure, your trusty map has been torn to shreds by a fierce storm, and you need to rebuild your map using code!

Here's what you need to do:

1. Write a piece of Python code that creates a map (linked list) of the islands you've discovered so far.
2. Fill your map with the initial islands you've already found:[1, 2, 3, 4, 5].
3. Add a function to display your map, showing all the islands you've charted.
4. Create a function to remove an island from your map if it turns out to be empty of treasure.

HOME TASK

TASK1 - TOYS TRACKER :

Chris has a box where he puts different kinds of toys. He can also take out toys from the box whenever he wants. Now he wants to keep track of the toys. So write a Python program to create a special list where each toy you put in the box is like adding a new toy to the list, and whenever you take a toy out, you remove that toy from the list. So, how would you make this special list? Remember, you'll need to think about how to add new toys to the end of the list, remove specific toys out of the list, and also show all the toys in your list

TASK 2 - FRUIT BASKET :

Imagine your mom went to a supermarket and bought a basket filled with all sorts of fruits. Now your task is to manage this basket. Can you write a magical Python code that helps you manage this basket? write a code where each fruit is connected to the next one, forming a linked list. Also mention a spell to add fruits to the linked list and another spell to check if a specific fruit is present in the basket. Try to use your creativity to make the program fun and easy to understand.

WHAT'S NEXT?

More on Linked List

LESSON 14

MORE ON LINKED LIST

LEARNING CONTENT

TYPES OF LINKED LIST :

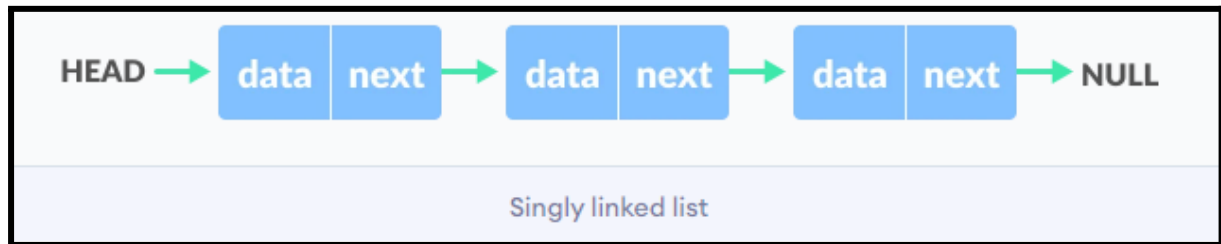
Linked lists are fundamental data structures in computer science, and they come in various types, each with unique characteristics and use cases. Here are the types of linked lists:

1. Singly Linked List
2. Doubly Linked List
3. Circular Linked List

Singly Linked List :

- A singly linked list is the most common type of linked list.
- Each node contains data and a reference (or pointer) to the next node in the sequence.
- Singly Linked List is simple and efficient for traversal and insertion.

However, it can only be traversed in one direction (forward).



Example :

```
class Node:
    def __init__(self, data=None):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def append(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node

    def display(self):
        current = self.head
        while current:
            print(current.data, end=" -> ")
            current = current.next
        print("None")
```



```

def search(self, key):
    current = self.head
    while current:
        if current.data == key:
            return True
        current = current.next
    return False

def delete(self, key):
    current = self.head
    if current and current.data == key:
        self.head = current.next
        current = None
        return
    prev = None
    while current and current.data != key:
        prev = current
        current = current.next
    if current is None:
        return
    prev.next = current.next
    current = None

# Example usage
l1list = LinkedList()
l1list.append(1)
l1list.append(2)
l1list.append(3)
l1list.append(4)

print("Initial linked list:")
l1list.display()

# Search for a node
key = 3
if l1list.search(key):
    print(f"{key} is found in the linked list.")
else:
    print(f"{key} is not found in the linked list.")

# Delete a node
key = 2
l1list.delete(key)
print(f"After deleting {key} from the linked list:")
l1list.display()

```

```
Initial linked list:  
1 -> 2 -> 3 -> 4 -> None  
3 is found in the linked list.  
After deleting 2 from the linked list:  
1 -> 3 -> 4 -> None
```

Doubly Linked List :

- In the Doubly Linked List each node contains data, a reference to the next node, and a reference to the previous node.
- Allows traversal in both directions (forward and backward).
- It is more complex than a singly linked list but provides more flexibility.



Example :

```
class Node:  
    def __init__(self, data):  
        self.data = data  
        self.prev = None  
        self.next = None  
  
class DoublyLinkedList:  
    def __init__(self):  
        self.head = None  
  
    def append(self, data):  
        new_node = Node(data)  
        if not self.head:  
            self.head = new_node  
        else:  
            current = self.head  
            while current.next:  
                current = current.next  
            current.next = new_node  
            new_node.prev = current
```

```

def prepend(self, data):
    new_node = Node(data)
    if not self.head:
        self.head = new_node
    else:
        new_node.next = self.head
        self.head.prev = new_node
        self.head = new_node

def display_forward(self):
    current = self.head
    while current:
        print(current.data, end=" ")
        current = current.next
    print()

def display_backward(self):
    current = self.head
    while current.next:
        current = current.next
    while current:
        print(current.data, end=" ")
        current = current.prev
    print()

# Example usage
dll = DoublyLinkedList()
dll.append(1)
dll.append(2)
dll.append(3)
dll.prepend(0)

print("Forward:")
dll.display_forward()

print("Backward:")
dll.display_backward()

```

Forward:

0 1 2 3

Backward:

3 2 1 0

Circular Linked List :

- A circular linked list is a variation of a linked list in which the last element is linked to the first element. This forms a circular loop.

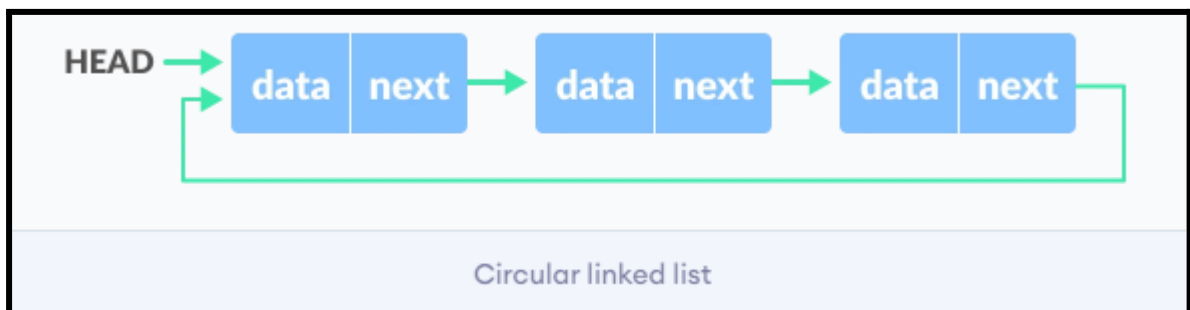
- The last node points back to the first node, forming a circular structure in the Circular Linked List.
- Useful for applications where the list needs to be navigated in a circular fashion, such as in round-robin scheduling.
- Circular Linked Lists can be either singly linked or doubly linked, depending on whether each node has one or two pointers.

Circular Singly Linked List

- Each node contains data and a reference (pointer) to the next node.
- The last node's next pointer points back to the first node to form a Circular Singly Linked List.

Circular Doubly Linked List

- Each node contains data, a reference to the next node, and a reference to the previous node.
- The last node's next pointer points to the first node, and the first node's previous pointer points to the last node to form a Circular Doubly Linked List.



Example :

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class CircularLinkedList:
    def __init__(self):
        self.head = None

    def append(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            new_node.next = self.head
        else:
            current = self.head
            while current.next != self.head:
                current = current.next
            current.next = new_node
            new_node.next = self.head

    def display(self):
        if not self.head:
            print("Circular Linked List is empty")
            return
        current = self.head
        while True:
            print(current.data, end=" -> ")
            current = current.next
            if current == self.head:
                break
        print()
```

```
# Create a circular linked list
clist = CircularLinkedList()

# Append some elements
clist.append(1)
clist.append(2)
clist.append(3)
clist.append(4)

# Display the circular linked list
clist.display()
```

```
1 -> 2 -> 3 -> 4 ->
```

IN CLASS CHALLENGES

CHALLENGE1 - ROUTE :

Let's plan a route that travels through a whimsical world filled with enchanted forests, mystical mountains, and dazzling cities. The train has a unique route with stops at various magical locations. Each stop is represented by a node in a linked list.

Your task is to design a program in Python that manages this magical train route using a linked list. The program should allow you to:

Add a new magical location to the train's route.

Remove a magical location from the route.

Display the entire route of the train, showcasing the magical locations it visits.

Each magical location should have a name and a description that adds to the enchantment of the journey. Your program should provide a delightful experience, capturing the imagination of all those who embark on this magical train ride.

INCLASS CHALLENGE 2- LINKED LIST:

Sonal learned about linked lists, and she found out that the linked list is better than the inbuilt python list in some operations. She decided to create a linked list class (data structure) like most of the methods an inbuilt python list contains.

She wants to add following methods:

- insert_at_beginning() -> To insert a new element at the beginning
- show() -> To print the entire linked list
- insert_at_end() -> To insert a new element at the end of the linked list.
- getLength() -> To get the number of elements present in a linked list.
- insert_at() -> To insert a new element at a particular index specified by user.
- remove_at() -> To remove a element from a particular index specified by user.
- remove_from_beginning() -> To remove the first element from a linked list.
- remove_from_end() -> To remove the last element from a linked list.

HOME TASK

HOME TASK1-More methods with Aakash:

Aakash is happy using linked lists, but he thinks there should be more methods in the linked list class. He is not able to apply operations like sort() , min() , max() , reverse().

So he wants to add these methods. To help Aakash create the following methods for the linked list class: min() -> To get the minimum value from a linked list max() -> To get the maximum value from a linked list reverse() -> To reverse the entire linked list. sort() -> To sort the linked list.

HOMETASK 2 - TO-DO LIST APPLICATION :

Henry wants to design a program to manage the listed activities. As part of his application, you need to create a data structure to store the tasks. Could you develop a Python implementation for a singly linked list to represent this to-do list? Include methods to add tasks, remove tasks, and display the current tasks in the list. Finally, could you demonstrate the usage of these methods by adding some initial tasks, adding a new task, and then removing an existing one?

WHAT'S NEXT?

Problem Solving - 2

LESSON 15

PROBLEM SOLVING

SESSION - 2

IN CLASS CHALLENGES

INCLASS CHALLENGE 1- ICE CREAM TRUCK:

Alice, Bob and Charlie all these friends are waiting in line for an ice cream truck. Now here's the task: create a program that simulates this scenario using Python!

First, let's create a class called 'IceCreamQueue' with features to add kids to the queue, serve them, check who's next, and see if the queue is full or empty.

Then, simulate the ice cream queue:

- Set a maximum queue size of 3.
- Add kids to the queue.

- Try adding more kids than the maximum queue size to see what happens.
- Check who's next in line.
- Serve the kids one by one to give them their ice cream.
- Make sure to handle cases where the queue is empty or full. Can you write Python code to bring this ice cream queue scenario to life? Let's do it!

```
✓ Run
Alice joined the ice cream queue.
Bob joined the ice cream queue.
Charlie joined the ice cream queue.
Queue is full. Please wait for the next batch of ice cream.
The next person in line is Alice.
Alice received their ice cream.
Bob received their ice cream.
Charlie received their ice cream.
Queue is empty. No one to serve.
```

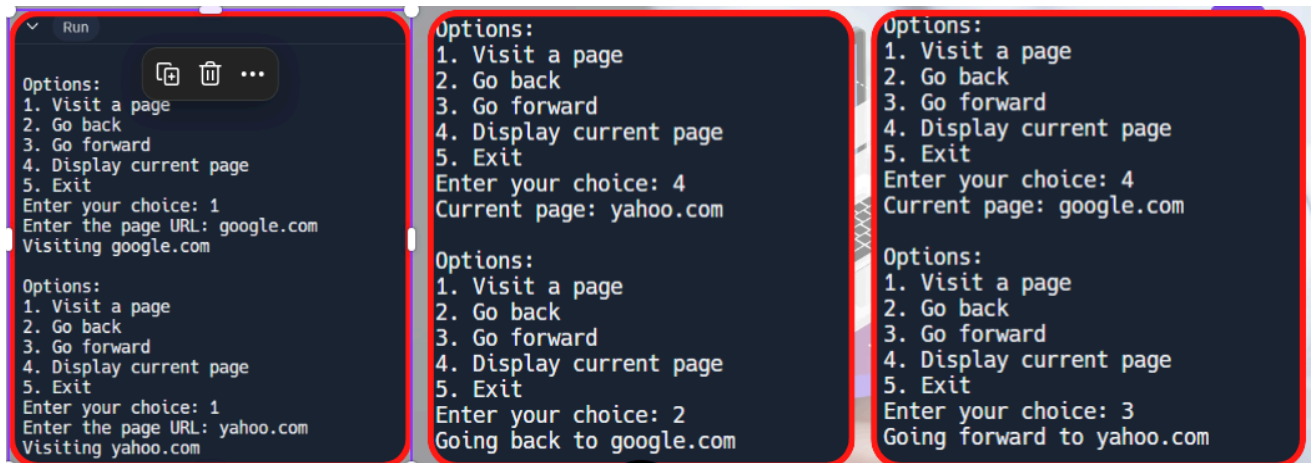
INCLASS CHALLENGE 2- WEB BROWSER:

Hey there! Do you know what a web browser is? It's the program you use to explore the internet! Imagine you're designing your very own browser, just like the ones you use every day. Your task is to create a simple version of a browser in Python.

Your browser should be able to:

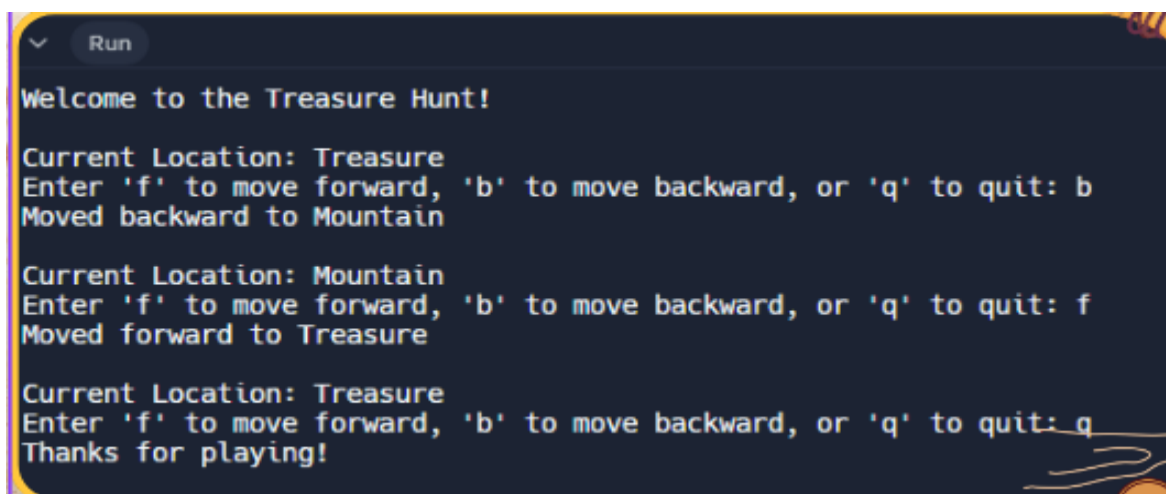
1. Visit different web pages.
2. Go back to the previous page.
3. Go forward to the next page.
4. Display the current page you're on.

Can you write a Python program to bring your browser to life? Don't worry, I'm here to help you along the way!



INCLASS CHALLENGE 3- TREASURE HUNT:

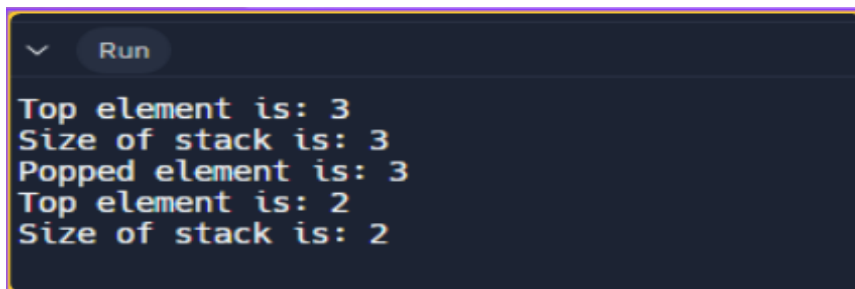
Captain Jack, a young adventurer! is on a thrilling treasure hunt adventure through mystical lands. He starts his treasure hunt in the 'Forest' where ancient trees whisper secrets, then crosses the 'River' where silver waters flow. His journey takes him through a dark 'Cave' echoing with mystery, and up a towering 'Mountain' where the air thins with excitement. Finally, he reaches the glittering 'Treasure'! Can you create a Python script using a doubly linked list to guide Captain Jack through these adventurous locations? Prompt him to use 'f' to move forward, 'b' to move backward, and 'q' to quit. Let the adventure begin!



HOME TASK

HOME TASK 1- STACK CREATION:

Eva is happy after creating a queue using a stack. Now she wants to know whether we can create a stack using a queue or not. She tried to do this by herself but got stuck in between. To help Eva in understanding the concept, write a python program to create a stack using queues.

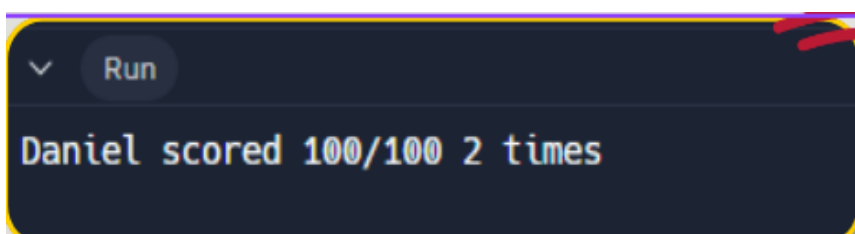


```
Run
Top element is: 3
Size of stack is: 3
Popped element is: 3
Top element is: 2
Size of stack is: 2
```

HOME TASK 2- HIGHEST SCORE CARD:

Daniel is in 8th grade. After every examination when he gets the result, he used to store the marks in a linked list. He passed out 8th grade and now he wants to know how many times he scored 100/100. He didn't have any method like countMarks() to count a specific number in a given linked list. Write a python program to help Daniel in counting the number of times he scored 100 in his examinations.

Hint: Create countMarks() method in linked list class.



```
Run
Daniel scored 100/100 2 times
```

WHAT'S NEXT?

Mini Project

LESSON 16

MINI PROJET - 2

IN CLASS CHALLENGES

Mini Project- Playground:

Hey Champion! You're tasked with creating a playground simulation program. You have to model a playground scenario where kids line up to slide down a slide.

How would you design such a system? Consider the various elements involved, such as representing the playground, managing the queue of kids waiting in line, handling the slide itself, and keeping track of which kid completes the slide first. Try to outline the classes and methods you would use to build this simulation.

```
Alice is sliding down the slide...
Alice completed the slide!

Line: Ella, Daisy, Charlie, Bob
Completed Slides: Alice

Bob is sliding down the slide...
Bob completed the slide!

Line: Ella, Daisy, Charlie
Completed Slides: Alice, Bob

Charlie is sliding down the slide...
Charlie completed the slide!

Line: Ella, Daisy
Completed Slides: Alice, Bob, Charlie

Daisy is sliding down the slide...
Daisy completed the slide!

Line: Ella
Completed Slides: Alice, Bob, Charlie, Daisy

Ella is sliding down the slide...
Ella completed the slide!

Line:
Completed Slides: Alice, Bob, Charlie, Daisy, Ella
All kids have completed their slides! Playground is now empty.
```

HOME TASK

HOME TASK 1-MAGICAL EVENT:

Tom has learned about stacks, queues, and linked lists, and now he wants to implement all these concepts in a single Python code. Help Tom by creating a magical zoo filled with fantastical creatures like mermaids, unicorns, and dragons!

Tom has learned about stacks, queues, and linked lists, and now he wants to implement all these concepts in a single Python code. Help Tom by creating a magical zoo filled with fantastical creatures like mermaids, unicorns, and dragons!

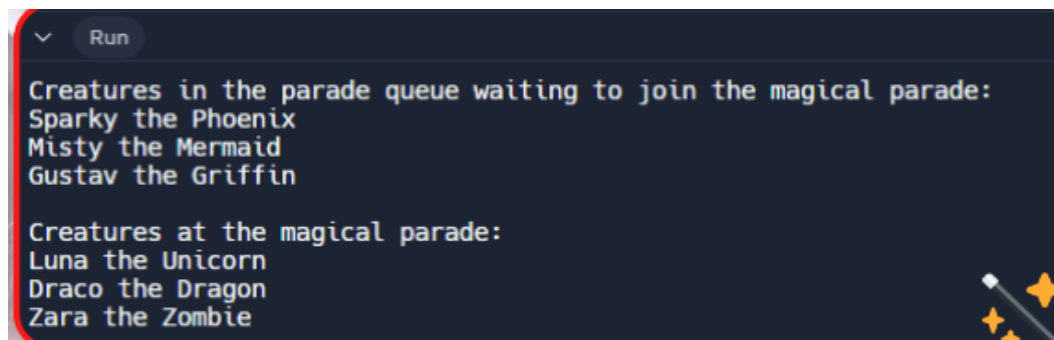
How would you craft a fun adventure where these creatures line up for a magical parade, and a wise wizard guides them through the enchanting event? Consider how you could utilize queues and stacks to manage the creatures waiting in line and those joining the parade.

Write a program to bring your magical zoo adventure to life and assist Tom in accomplishing his task.

Tom has learned about stacks, queues, and linked lists, and now he wants to implement all these concepts in a single Python code. Help Tom by creating a magical zoo filled with fantastical creatures like mermaids, unicorns, and dragons!

How would you craft a fun adventure where these creatures line up for a magical parade, and a wise wizard guides them through the enchanting event? Consider how you could utilize queues and stacks to manage the creatures waiting in line and those joining the parade.

Write a program to bring your magical zoo adventure to life and assist Tom in accomplishing his task.



```
▼ Run
Creatures in the parade queue waiting to join the magical parade:
Sparky the Phoenix
Misty the Mermaid
Gustav the Griffin

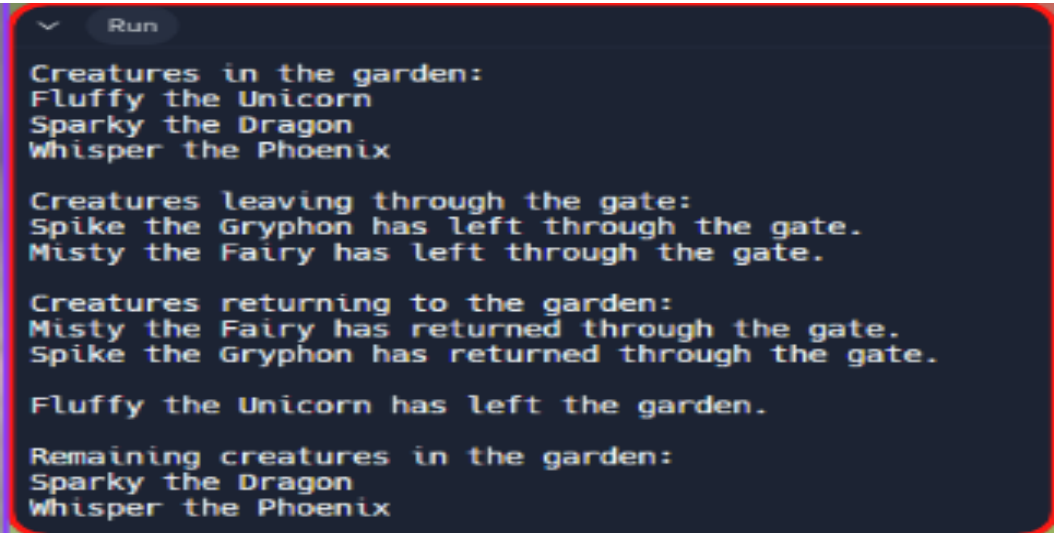
Creatures at the magical parade:
Luna the Unicorn
Draco the Dragon
Zara the Zombie
```

HOME TASK 2 - WHIMSICAL GARDEN:

Hey there, Captain of Young Coders! Can you create a whimsical adventure where creatures like unicorns, dragons, and phoenixes interact with each other through a magical gate?

Picture using essential programming concepts like : classes and objects, doubly linked lists for managing creatures within the garden, queues for controlling creature arrivals and departures through the gate, and stacks for temporarily storing creatures leaving and returning.

Write a fun Python program to bring your magical garden to life, where creatures join, leave, and encounter enchanting experiences at the touch of a wand!



```
Run
Creatures in the garden:
Fluffy the Unicorn
Sparky the Dragon
Whisper the Phoenix

Creatures leaving through the gate:
Spike the Gryphon has left through the gate.
Misty the Fairy has left through the gate.

Creatures returning to the garden:
Misty the Fairy has returned through the gate.
Spike the Gryphon has returned through the gate.

Fluffy the Unicorn has left the garden.

Remaining creatures in the garden:
Sparky the Dragon
Whisper the Phoenix
```

WHAT'S NEXT?

Divide And Conquer

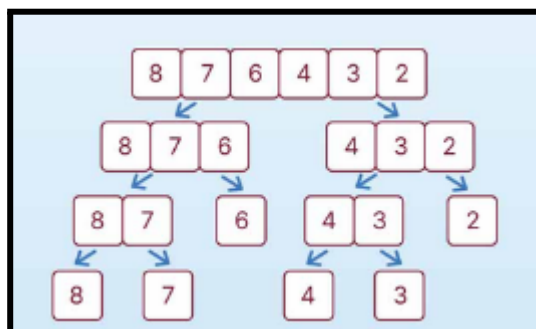
LESSON 17

DIVIDE AND CONQUER

LEARNING CONTENT

DIVIDE AND CONQUER ALGORITHM

In the Divide and Conquer approach, the problem in hand is divided/break into smaller sub- problems and then each problem is solved independently. When we keep on dividing the subproblems into even smaller sub-problems, we may eventually reach a stage where no more division is possible. Those "atomic" smallest possible sub-problem (fractions) are solved/conquered. The solution of all sub-problems is finally merged/combined in order to obtain the solution of an original problem.

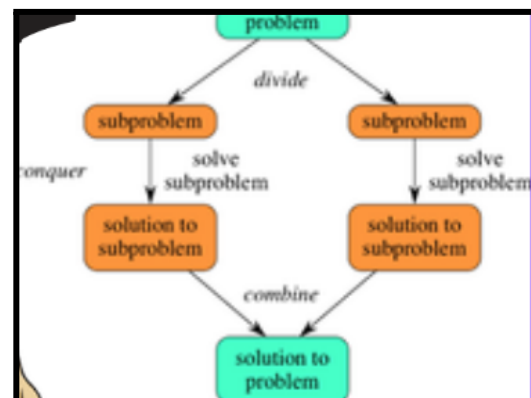


WORKING OF DIVIDE AND CONQUER ALGORITHM :

Divide & Conquer strategy usually consists of 3 steps:

1. Divide/Break - The problem is divided into smaller sub -problems that can be solved independently. This step usually involves breaking the problem into two or more sub-problems of roughly equal size.
2. Conquer/Solve - Each sub-problem is solved independently using recursion or other techniques. This step is where the actual work is done to solve the problem.
3. Combine/Merge - The solutions to the sub-problems are combined to form the solution to the original problem. This step usually involves merging the solutions to the sub-problems together.

Here's how to view one step,
assuming that each divide step
creates two subproblems
(though some divide-and-conquer
algorithms create more than two)



Advantages of Divide and Conquer

- Difficult problems can be solved with an easier approach.
- Algorithms that are made using this strategy are mostly found to be really efficient.
- Supports parallelism which makes it faster.
- Makes an efficient use of memory cache. As the sub-problems become simpler, they can be solved within the cache memory easily and there's no need to access the main memory which is comparatively slower.

Common Applications Of Divide and Conquer:

1. Merge sort and quick sort

Quick Sort sorts the components by comparing each component with the pivot while Merge Sort splits the array into two segments or subarrays on repetition until one component is left. Also, Merge Sort is more efficient as compared to Quick Sort

2. Binary Search

Binary Search is a search algorithm that is used to find the position of a target value within a sorted array. The algorithm works by repeatedly dividing the search interval in half until the target value is found.

Example :

```
main.py > ...  
1  def binary_search(arr, x):  
2      low = 0  
3      high = len(arr) - 1  
4  
5      while low <= high:  
6          mid = (high + low) // 2  
7  
8          if arr[mid] < x:  
9              low = mid + 1  
10         elif arr[mid] > x:  
11             high = mid - 1  
12         else:  
13             return mid  
14  
15         return -1  
16  
17 arr = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]  
18 x = 16  
19  
20 result = binary_search(arr, x)  
21  
22 if result != -1:  
23     print("Element is present at index", str(result))  
24 else:  
25     print("Element", x, "is not present in array")
```

IN CLASS CHALLENGES

IN CLASS CHALLENGE1 - Creatures Library :

Luna, a curious explorer, stumbles upon a library filled with the names of creatures. Eager to learn more, Luna decides to create a special spell (code) that allows her to quickly find a specific creature from the library. Can you help Luna create this spell? Your spell should organize the creatures alphabetically and use a special technique called binary search to find them faster.

Also, the output should appear with each step of the search process, showing which animal is being checked at each step and whether it's in the left or right half of the current range being searched.

IN CLASS CHALLENGE 2- BINARY SEARCH GAME:

Hey young coder! Have you ever played a game where you had to find a hidden treasure in a series of boxes?

Imagine if we could write a program that plays a similar game, but instead of boxes, we have a sorted list of random numbers. We'll use a special technique called binary search to quickly find a specific number in the list.

Let's create a program together that can find any number you choose from a sorted list of random numbers. Also the output should show each step to illustrate how the search narrows down the possibilities. Ready to embark on this coding adventure?

HOME TASK

HOME TASK1-MAGICAL BOOK:

Elsa has stumbled upon a magical book filled with words and their meanings. She's thrilled but wants to create a spell (code) that enables her to quickly find the meaning of any word in the book. Can you help her out? Think about how she should organise the words in the book and how she would efficiently search for a specific word.

Hints:

Organising the Book: Consider sorting the words alphabetically or in some ordered fashion to make searching easier. Choosing a Search Method: Think about efficient search methods like binary search, which works well with sorted data. Steps of the Spell (Code): Break down the process into clear steps, such as initialising variables, setting up the search loop, and handling cases where the word is found or not found. Testing the Spell: It's essential to test the spell with different words from the book to ensure it works correctly.

HOMETASK 2 - PLANT ORGANIZER :

Emma, a passionate gardener, is fascinated by the diverse range of plants in her backyard. To keep track of them efficiently, she plans to create a special tool (code) that enables her to quickly search for a specific plant name. Can you assist Emma in developing this tool? Your code should organise the plant names alphabetically and utilize binary search for faster retrieval.

WHAT'S NEXT?

More on Divide And Conquer

LESSON 18

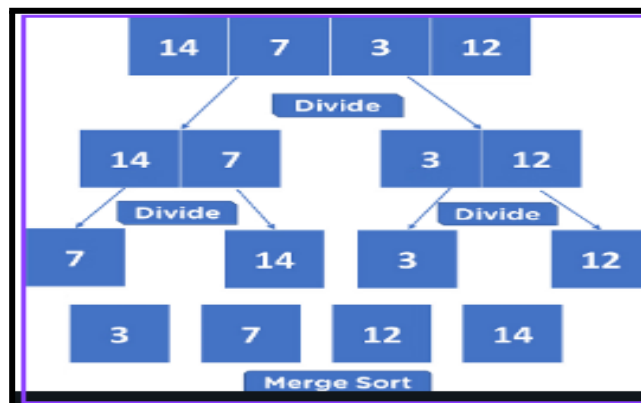
MORE ON DIVIDE AND CONQUER

LEARNING CONTENT

MERGE SORT

Merge Sort is a widely-used sorting algorithm known for its efficiency and stability. It operates on the principle of dividing an array into two halves, sorting each half separately, and then merging the sorted halves to produce a fully sorted array.

In Python, implementing Merge Sort involves breaking down the sorting process into recursive steps, making use of the "divide and conquer" strategy. Merge sort continuously cuts down a list into multiple sublists until each has only one item, then merges those sublists into a sorted list.



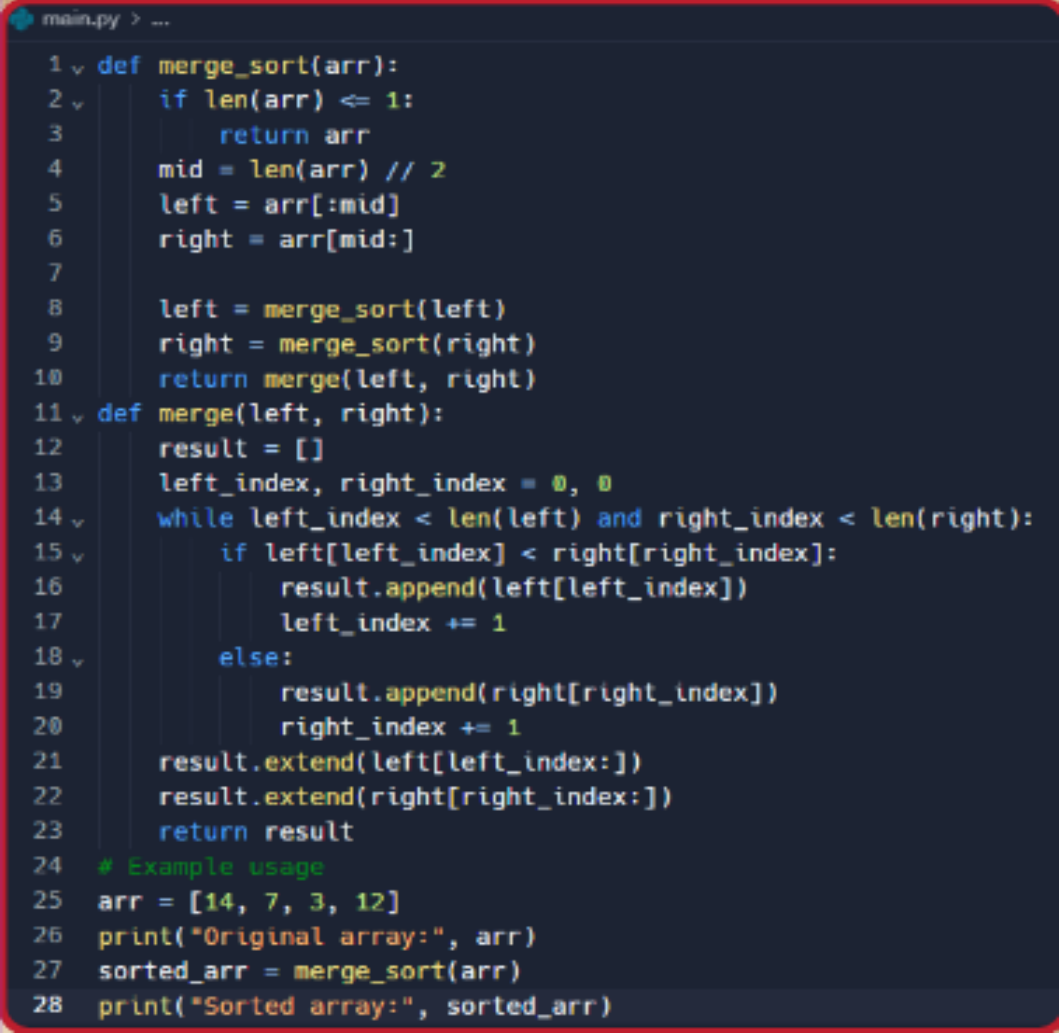
Here's how Merge Sort works:

Divide: The input array is divided in half.

Conquer: Recursively sort each half of the input array using Merge Sort.

Combine: Merge the sorted halves back together to obtain the final sorted array.

Example :

A screenshot of a code editor window titled 'main.py > ...'. The code implements the Merge Sort algorithm. It defines a 'merge_sort' function that recursively divides an array into halves until it reaches a base case of length 1 or less, then merges the sorted halves back together. A 'merge' function handles the merging of two sorted arrays. An example usage is provided at the bottom, showing an array [14, 7, 3, 12] being sorted to [3, 7, 12, 14].

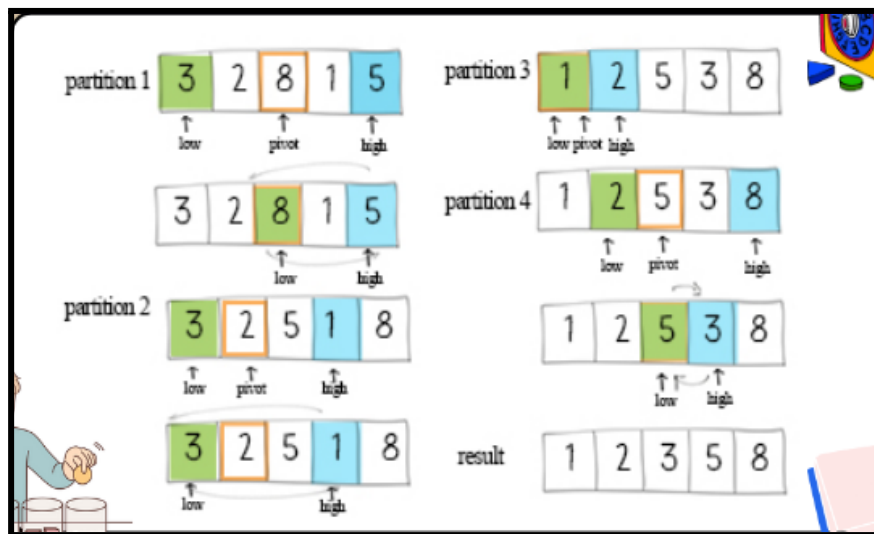
```
1 def merge_sort(arr):
2     if len(arr) <= 1:
3         return arr
4     mid = len(arr) // 2
5     left = arr[:mid]
6     right = arr[mid:]
7
8     left = merge_sort(left)
9     right = merge_sort(right)
10    return merge(left, right)
11 def merge(left, right):
12     result = []
13     left_index, right_index = 0, 0
14     while left_index < len(left) and right_index < len(right):
15         if left[left_index] < right[right_index]:
16             result.append(left[left_index])
17             left_index += 1
18         else:
19             result.append(right[right_index])
20             right_index += 1
21     result.extend(left[left_index:])
22     result.extend(right[right_index:])
23     return result
24 # Example usage
25 arr = [14, 7, 3, 12]
26 print("Original array:", arr)
27 sorted_arr = merge_sort(arr)
28 print("Sorted array:", sorted_arr)
```

QUICK SORT:

Quick Sort is based on the divide and conquer approach that works by dividing the input array into two groups, one with elements in the left half are smaller than pivot element, and another with elements in the right half are larger than the pivot element.

A pivot element is an element selected from the array that serves as a reference point for partitioning.

There are various strategies for choosing the pivot, such as selecting the first or last or middle or a randomly chosen element.



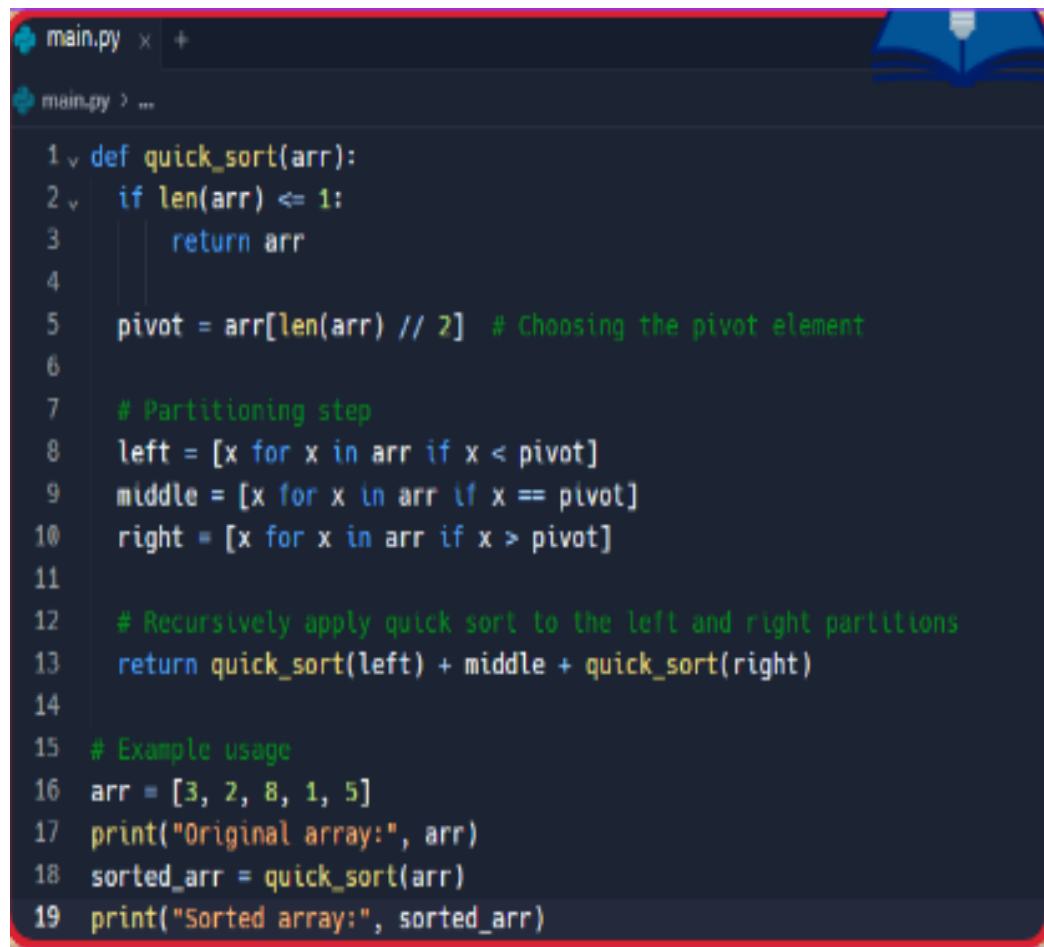
Here's how Quick Sort works:

Divide: Choose a pivot element and divide the input array into two partitions based on the pivot element.

Conquer: Recursively sort each partition using Quick Sort.

Combine: Combine the sorted partitions to obtain the final sorted array.

Example :

A screenshot of a Python IDE window titled 'main.py'. The code implements a quick sort algorithm. It starts with a function definition 'def quick_sort(arr):'. Inside, there's a base case 'if len(arr) <= 1: return arr'. Then, it chooses a pivot: 'pivot = arr[len(arr) // 2] # Choosing the pivot element'. A comment '# Partitioning step' precedes three list comprehensions: 'left = [x for x in arr if x < pivot]', 'middle = [x for x in arr if x == pivot]', and 'right = [x for x in arr if x > pivot]'. Another comment '# Recursively apply quick sort to the left and right partitions' is followed by the return statement 'return quick_sort(left) + middle + quick_sort(right)'. Finally, an example usage is shown: 'arr = [3, 2, 8, 1, 5]', 'print("Original array:", arr)', 'sorted_arr = quick_sort(arr)', and 'print("Sorted array:", sorted_arr)'.

```
1 def quick_sort(arr):
2     if len(arr) <= 1:
3         return arr
4
5     pivot = arr[len(arr) // 2] # Choosing the pivot element
6
7     # Partitioning step
8     left = [x for x in arr if x < pivot]
9     middle = [x for x in arr if x == pivot]
10    right = [x for x in arr if x > pivot]
11
12    # Recursively apply quick sort to the left and right partitions
13    return quick_sort(left) + middle + quick_sort(right)
14
15    # Example usage
16    arr = [3, 2, 8, 1, 5]
17    print("Original array:", arr)
18    sorted_arr = quick_sort(arr)
19    print("Sorted array:", sorted_arr)
```

IN CLASS CHALLENGES

IN CLASS CHALLENGE1 - ANIMAL SORTING GAME :

Do you love animals? Let's imagine you have a bunch of animal friends who want to line up for a fun game. Each animal has a size, like how big or small they are. How would you help them line up from smallest to largest? Think about how you can use your coding skills to create a game where you can input the names of different animals and their sizes, and then see them line up in the correct order!

Remember, you can use sorting algorithms like merge sort to help you out. Get creative and let's make a fun animal sorting game together!

INCLASS CHALLENGE 2- TEST SCORES:

In a school a teacher is having stack of test scores and she wants to organise them in ascending order, from the lowest score to the highest. But here's the catch: she needs your coding expertise to do it efficiently!

Here's your challenge: Can you write a program to help the teacher sort the students test scores using the Quick Sort algorithm? Think about which student's score would make a good pivot. Should we go with the highest score, the lowest score, or maybe something else?

Once you've written the program, make sure it displays each step of the sorting process. This will help us understand how quicksort works and how the choice of pivot impacts the sorting outcome.

HOME TASK

HOME TASK1-BOOKSHELF ORGANISER:

Peter, the book lover, has a magical bookshelf overflowing with all his cherished tales. But oh no, the books are all jumbled up! Can you create a special Python spell to help Peter in sorting the books alphabetically?

Your mission is to use the power of Merge Sort, a magical sorting algorithm, to tidy up the bookshelf and arrange the books in perfect order. Are you prepared to embark on this epic sorting quest?

HOMETASK 2 - MEDICAL CHECKUP :

Hey there, Code Champ! A medical team has just wrapped up a comprehensive health check in a school for the students of grade 10. They've measured all the students' heights and discovered that the ideal height for a 15-year-old boy is 163cm. But wait, there's a twist! Some students are taller, some shorter. So, the medical team needs your help in cracking this height mystery!

Can you build a magical Python code to sort out who's above and who's below this magical height? Use the Quick Sort spell to swiftly partition the list of heights and reveal the secret numbers.

WHAT'S NEXT?

Binary Trees

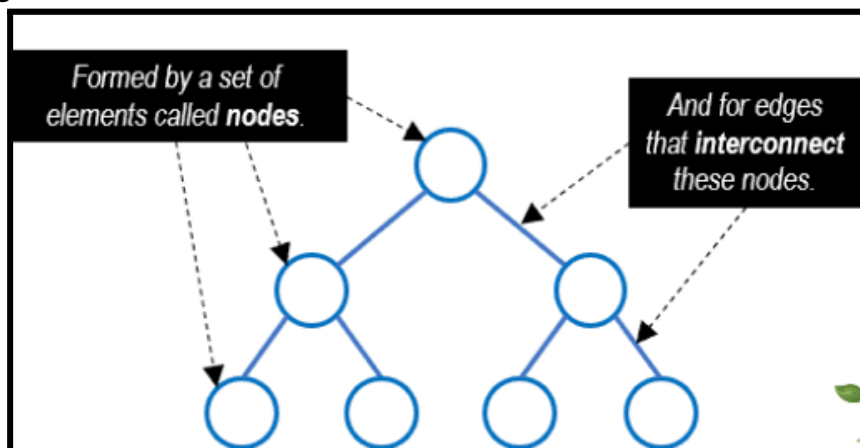
LESSON 19

BINARY TREES

LEARNING CONTENT

TREES

A Tree Data Structure is a hierarchical structure that is used to represent and organise data in a way that is easy to navigate and search. It is a collection of nodes that are connected by edges and has a hierarchical relationship between the nodes. It looks like an upside-down tree consisting of nodes.

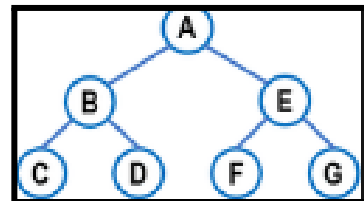


The big difference from a tree to other common data structures (such as arrays, lists, queues, and stacks) is that trees are not organized in a sequence. In a tree we cannot determine the next element and the previous element because its information is not organized in a queued manner.

1) In a list, each information is related to two other elements: the previous and the next.

A	B	C	D	E	F	G
0	1	2	3	4	5	6

2) In a tree, each information (node) can be related to more than two elements.

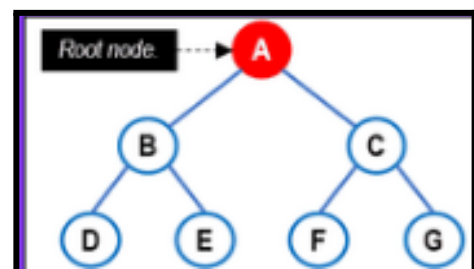


For this reason, Trees are called nonlinear structures. And this makes it a very special structure with many different properties and characteristics.

Basic Terminologies In Tree Data Structure

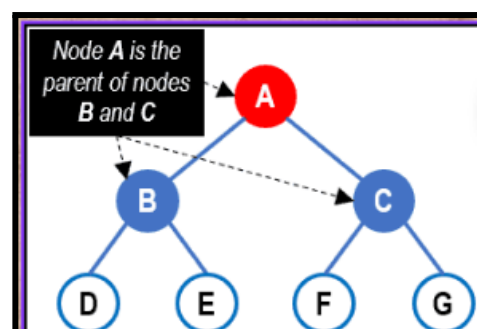
Root node:

Every tree has one main node called root. The root represents the starting node of all tree types.



Child nodes and parent node:

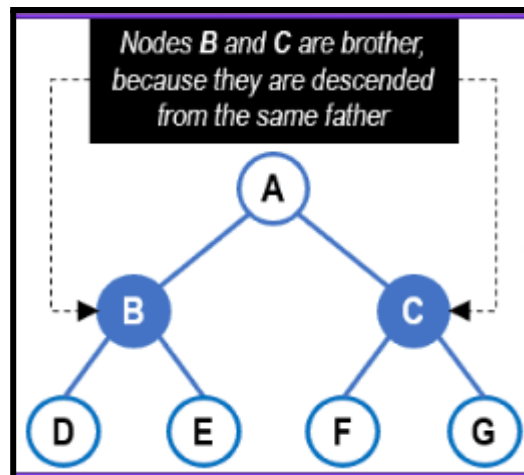
From the root, all lower nodes attached to a higher node are called child nodes. All higher nodes, which generate lower nodes, are



called parent nodes

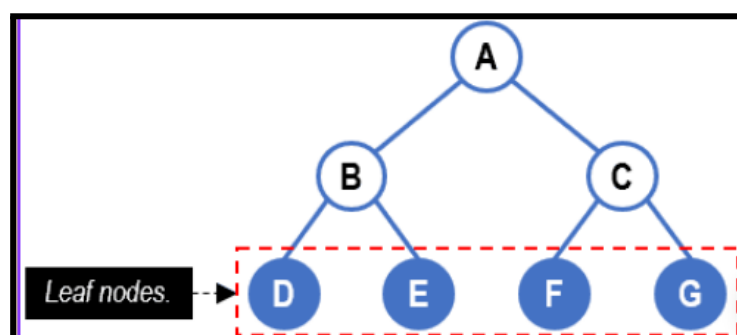
Sibling nodes:

Another important concept is the concept of brotherhood between nodes. Two nodes are siblings if they descend directly from the same parent node.



Leaf node:

Nodes that have no children are called leaf nodes. The term “leaf node” alludes to the leaves of a tree, as they represent the end of the branch.



Subtree

From the root, each new child represents a subtree, a new tree

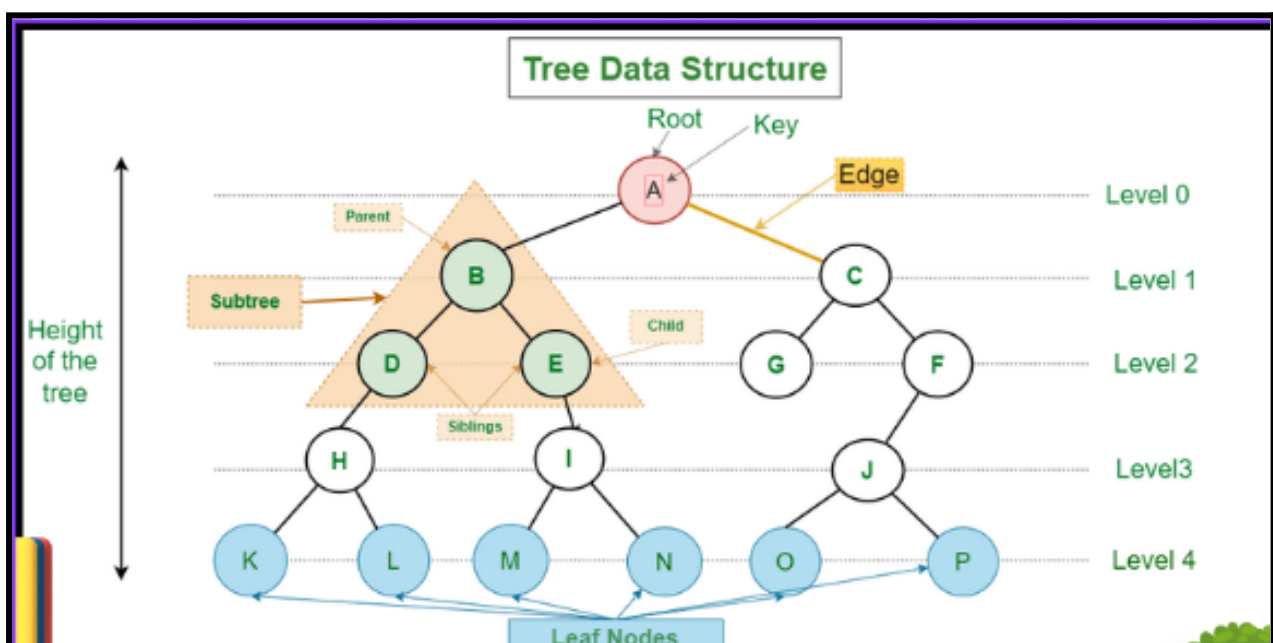
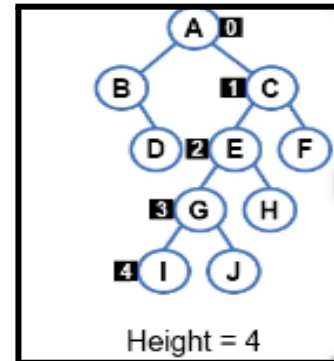
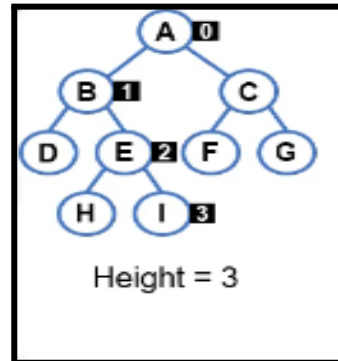
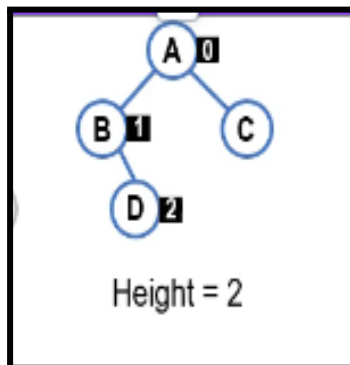
that recursively resets itself.

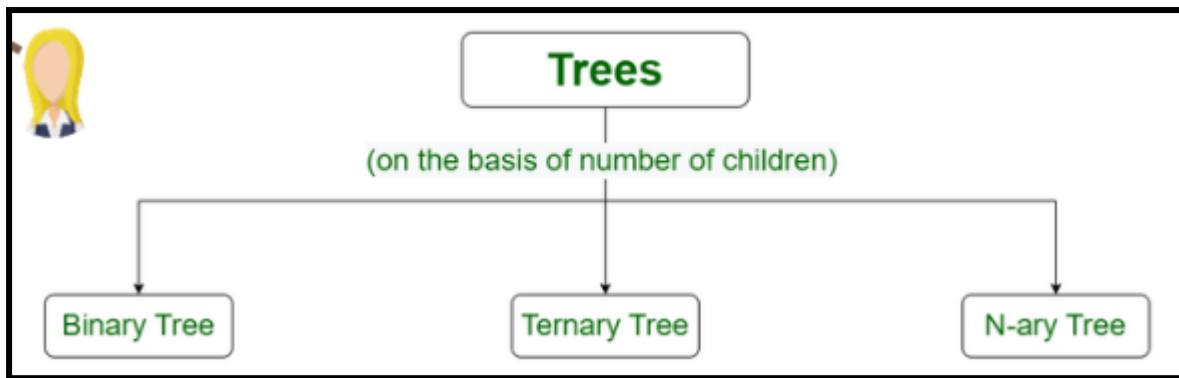
Level of a tree:

The level defines the depth slices of a tree, so that the children of a node are always placed at later levels.

Height of a tree:

Represents the largest depth of the tree. Represents the distance between the root and a leaf node of the highest level of the tree.

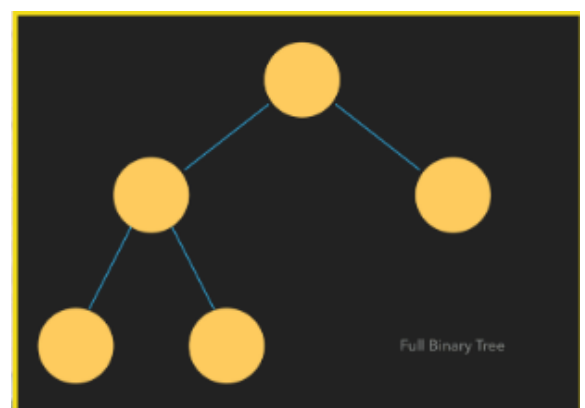




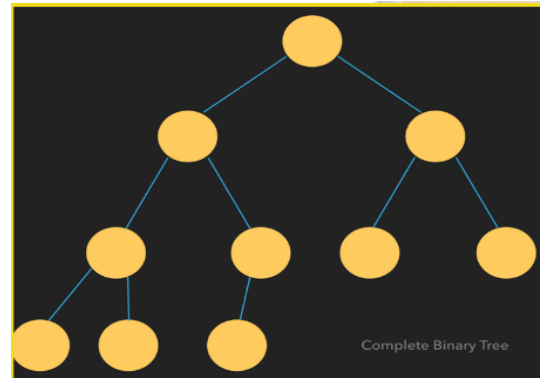
- **Binary tree:** In a binary tree, each node can have a maximum of two children linked to it. Some common types of binary trees include full binary trees, complete binary trees, balanced binary trees, and degenerate or pathological binary trees.
- **Ternary Tree:** A Ternary Tree is a tree data structure in which each node has at most three child nodes, usually distinguished as “left”, “mid” and “right”.
- **N-ary Tree or Generic Tree:** Generic trees are a collection of nodes where each node is a data structure that consists of records and a list of references to its children(duplicate references are not allowed). Unlike the linked list, each node stores the address of multiple nodes.

Binary Tree Identification:

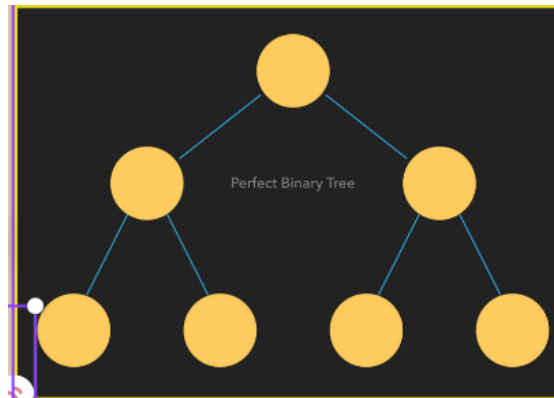
- Binary trees can be structured differently and possess different kinds of characteristics or qualities.
- A full or strictly binary tree is structured so that every node possesses either two or no children.



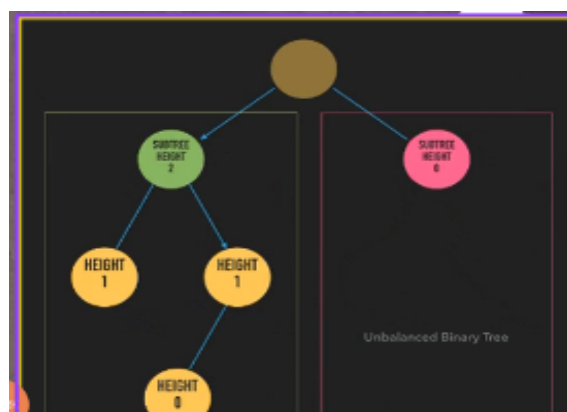
- A complete binary tree is a tree where nodes at every level but the last is required to have two children. On the last level, the children are positioned as far to the left as possible. So, complete binary tree nodes are connected from top to bottom, left to right.



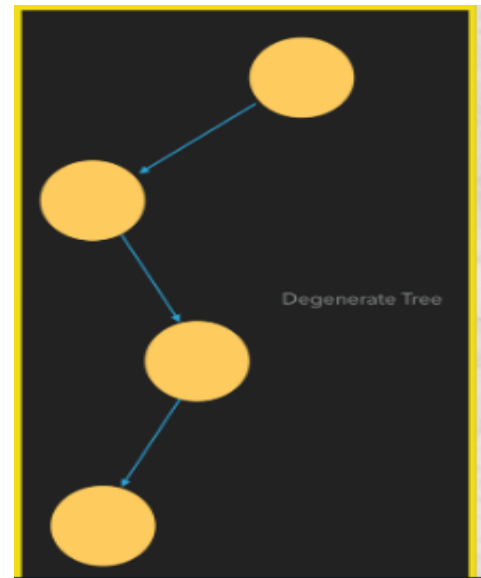
- A perfect binary tree is both full and complete. All the leaf nodes are located at the same depth, and all internal nodes have two children.



- A balanced binary tree describes a tree where two sibling subtrees don't differ in height by more than one level. If the difference in height is greater than that, then it is unbalanced.



- If every node in a tree has only one child, the structure is a degenerate or pathological tree, and is essentially a linked list.



IN CLASS CHALLENGES

IN CLASS CHALLENGE 1 - RECIPE COLLECTION :

Sara has a collection of recipes stored on her computer. She organizes them in a hierarchical manner, similar to a tree structure. Each recipe is either a folder (containing sub-recipes) or a file (containing the actual recipe). Sara wants to create a program to manage her recipe collection. Write a program that mimics Sara's recipe organisation. The program should have methods to: Add a new recipe (file). Add a new recipe folder. Delete a recipe (file). Delete a recipe folder. Display the entire recipe collection in a tree-like structure.

INCLASS CHALLENGE 2- GENERIC TREE:

Tarun and Vishal were working on a project. They were creating an online electronic store. They wanted to keep all types of products in different categories in the form of a tree. To help them, create a generic tree and add all the electronic items accordingly.

HOME TASK

HOME TASK1-HIKING TRIP:

A group of friends are planning to go on a hiking trip to explore a mysterious forest. To navigate through the forest, they need to understand the height of the trees they may encounter. Each tree in the forest is represented as a binary tree, where the height of a tree is defined as the length of the longest path from the root node to any leaf node. Your task is to write a Python program that takes the input of a binary tree and calculates the height of the tree.

HOMETASK 2 - HOBBY WISHLIST:

Robert needs help to create a Program tree structure for hobbies with categories such as Outdoor and Indoor. Each category can have subcategories , forming a hierarchical structure.

WHAT'S NEXT?

MORE ON BINARY TREES

LESSON 20

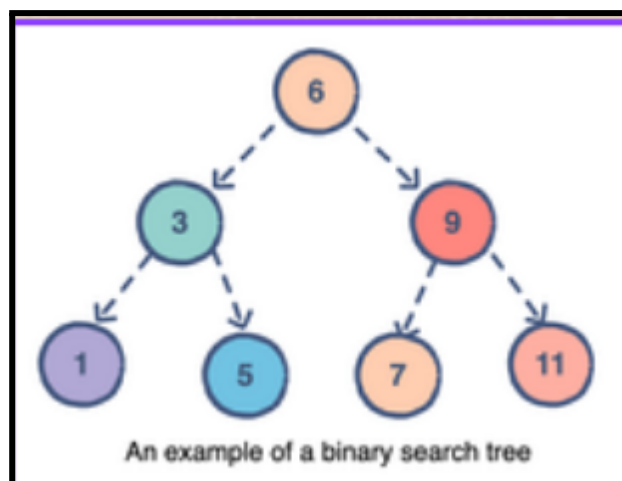
MORE ON BINARY TREES

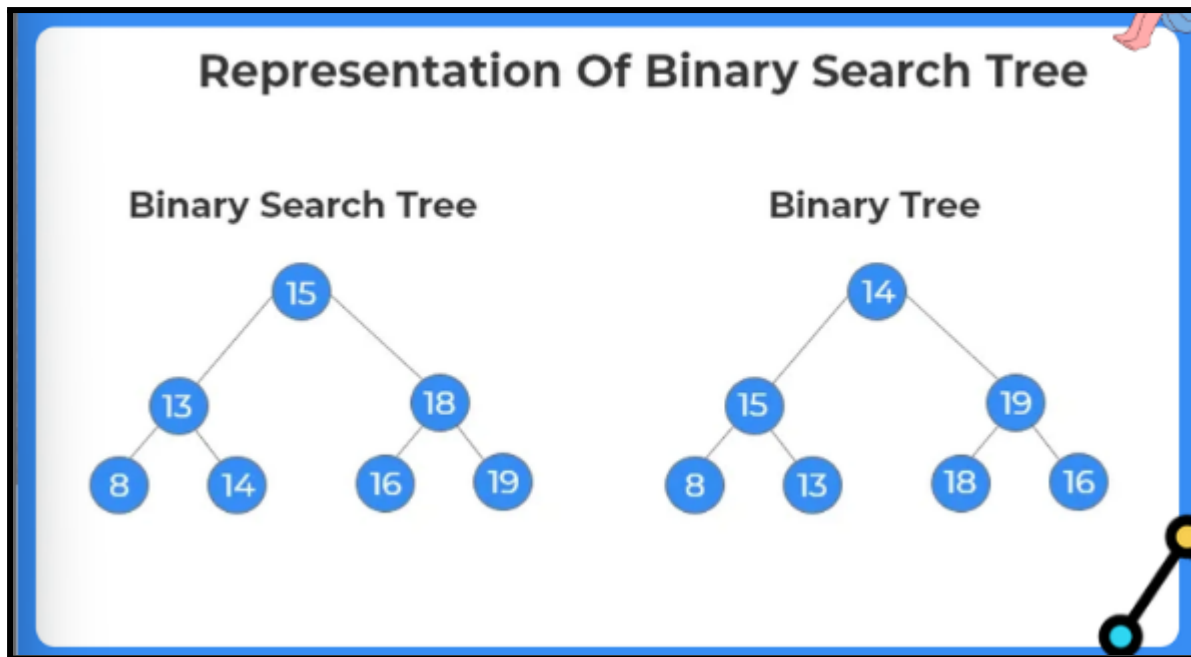
LEARNING CONTENT

Binary-Search Trees (BSTs)

Binary-Search Tree (BST) is a binary tree data structure that come with a special quality that is sortedness. It is naturally sorted, which makes searching for a value extremely efficient and quick. And the BST class possesses methods to insert and delete nodes in ways that always preserve and maintain that sorted state.

Each parent node points to a left child and/or a right child. If a child's value is less than the parent's, the child must be the left child node. On the other hand, if the child's value is greater, then that child must be the right child node.





Insert value in Binary Search Tree (BST)

The insert() method recursively traverses the tree until it finds the correct position for the new node. If the value is less than the current node's value, it goes to the left subtree. Otherwise, it goes to the right subtree. If the appropriate child node is None, it creates a new node with the given value and sets it as the child node. Otherwise, it continues recursively traversing the subtree.

Search for a given value in Binary Search Tree

- Create a search() function to initiate the search process.
- Inside the search() function, call the _search() method to recursively search for the node starting from the root node.
- Define the _search() method to traverse the tree recursively.

In the `_search()` method:

- Compare the given value with the current node's value.
- If the current node is None, return False.
- If the current node's value is equal to the given value, return True.
- If the given value is less than the current node's value, continue searching in the left subtree recursively.
- If the given value is greater than the current node's value, continue searching in the right subtree recursively.

Deleting the value in Binary Search Tree

The Delete function is used to remove the given node from a binary search tree. This function is also recursive, but it has an additional feature where it returns the updated state of the given node after performing the delete operation.

This ensures that a parent node whose child has been deleted can correctly update its left or right data member by setting it to None. But we have to delete a node from a binary search tree in a way that doesn't break any of its properties.

Removing a node from a binary search tree can happen in three different scenarios

- Node to be deleted is a leaf node
- Node to be deleted has only one child
- Node to be deleted has both child

Binary Search Tree Application

- ☐ In multilevel indexing in the database
- ☐ For dynamic sorting
- ☐ For managing virtual memory areas in Unix kernel

IN CLASS CHALLENGES

Inclass Challenge 1-pre-order traversal:

Vihaan just learned about binary search trees, and now he is writing a program on a binary search tree. Soon he realised that he can create a set using binary search (There will be no duplicate data on traversal). Also, he can get a sorted list using pre-order traversal. So write a python program to create a binary search tree. Add the following methods.
add_data() -> to add data in the binary tree pre_order_traversal -> to return a list in pre-order. (ascending order)

INCLASS CHALLENGE 2- bookstore categories:

Shreya and Aarav are developing a new online bookstore. They want to organize books into categories and subcategories to make it easier for users to browse. Help them by creating a tree structure to represent the categories and subcategories of books. Write a Python program that creates a tree for the bookstore categories and prints it. Include methods to add new categories and subcategories, as well as to delete categories and subcategories if needed.

HOME TASK

HOME TASK1- Jack's data organisation tree:

As Jack embarked on his coding journey, he discovered the power of binary search trees— unravelling a world where data organization met efficiency. Now, with your guidance, he sets out to craft a Python program that brings order to chaos, building a binary search tree to unlock the wonders of sorted lists and unique datasets. Add the following methods. add_data() -> to add data in the binary tree
pre_order_traversal -> to return a list in pre-order. (descending order)

HOME TASK 2 - Dictionary BST :

Soham is helping rohan in making an interesting Dictionary or Glossary: BSTs can be used to create a dictionary or glossary where words are stored alphabetically for easy lookup. This can be useful for students, writers, or anyone who frequently references definitions or meanings of words.

WHAT'S NEXT?

Problem solving session - 3

LESSON 21

PROBLEM SOLVING

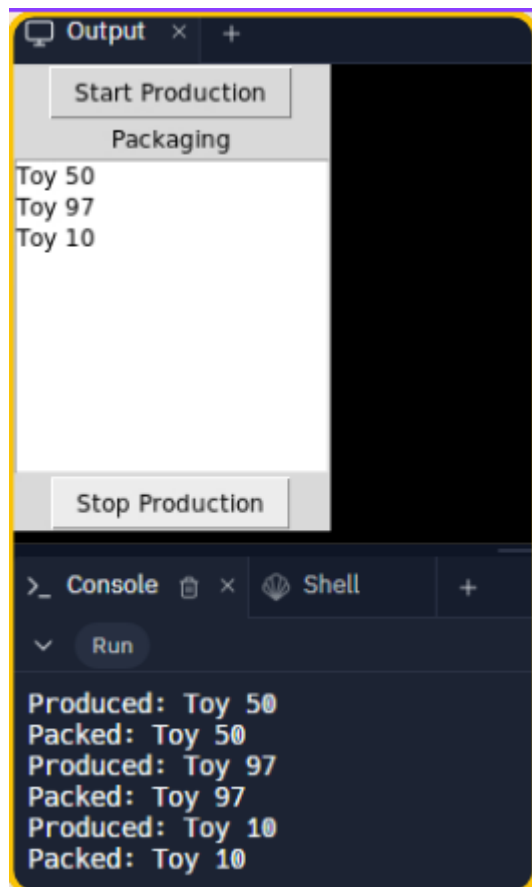
SESSION - 3

IN CLASS CHALLENGES

CHALLENGE1 - Toy Factory :

Bob is running a toy factory where he produces and packages toys. Can you create a program using Python that simulates this toy factory?

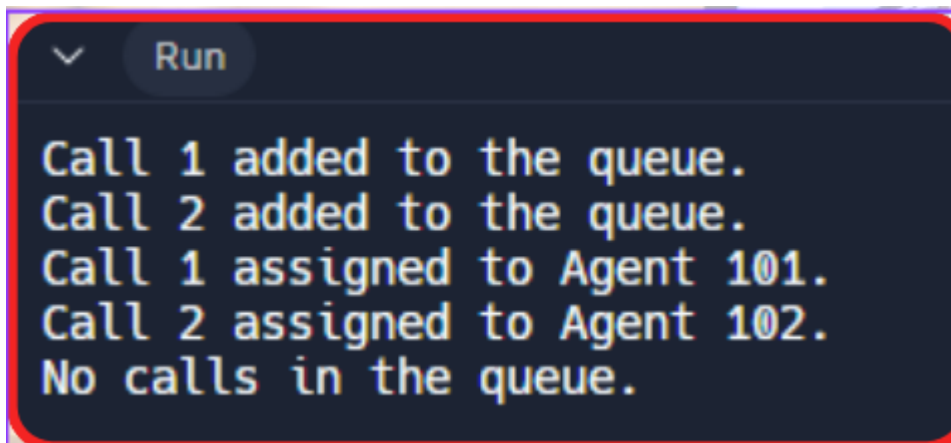
we'll need to use queues to manage the production and packaging processes, and include a graphical interface to show the production and packaging progress. How would you design the program that allow Bob to start and stop the production process while displaying the toys being packaged in real-time on the screen?



INCLASS CHALLENGE 2- Virtual Call Center:

Sarah wants to build a virtual call center but she's a little bit confused. So, let's design a virtual call center where you're the master coder, and Sarah is the eager assistant. Your task is to create a Python program that simulates a call center.

You'll have a list of agents who can handle incoming calls and a queue to hold those calls. Think about how you'd ensure fairness in assigning calls to agents. Discuss how you might use sets, queues, and conditionals to manage the agents and incoming calls efficiently. Sarah's excited to learn how you'll make it work!

A terminal window with a dark background and a red border. At the top left is a downward arrow icon, and at the top center is a button labeled "Run". The terminal displays five lines of text in a monospaced font: "Call 1 added to the queue.", "Call 2 added to the queue.", "Call 1 assigned to Agent 101.", "Call 2 assigned to Agent 102.", and "No calls in the queue."

```

Call 1 added to the queue.
Call 2 added to the queue.
Call 1 assigned to Agent 101.
Call 2 assigned to Agent 102.
No calls in the queue.

```

In class challenge 3 - Musical Playlist

Hey there! Let's imagine you and your friend, Emily, are working together on a fun project. You're both music enthusiasts, and you want to create a music playlist program in Python. You'll have a class called Song to represent individual songs, and another class called Playlist to manage a collection of songs. How would you design these classes?

Consider including methods to add songs, remove songs, and display the playlist. Think about how you'd use linked lists to organize the songs in the playlist. Imagine discussing this with Emily and brainstorming ideas to make the program cool and user-friendly. Can you write code to create and manipulate a playlist of your favorite songs?

```
Run
Current Playlist:
Playlist:
Shape of You - Ed Sheeran
Dance Monkey - Tones and I
Blinding Lights - The Weeknd
Enter the title of the song to add: Crazy Frog
Enter the artist of the song: Axel F

Updated Playlist:
Playlist:
Shape of You - Ed Sheeran
Dance Monkey - Tones and I
Blinding Lights - The Weeknd
Crazy Frog - Axel F

Enter the title of the song to delete: Shape of You

Final Playlist:
Playlist:
Dance Monkey - Tones and I
Blinding Lights - The Weeknd
Crazy Frog - Axel F
```

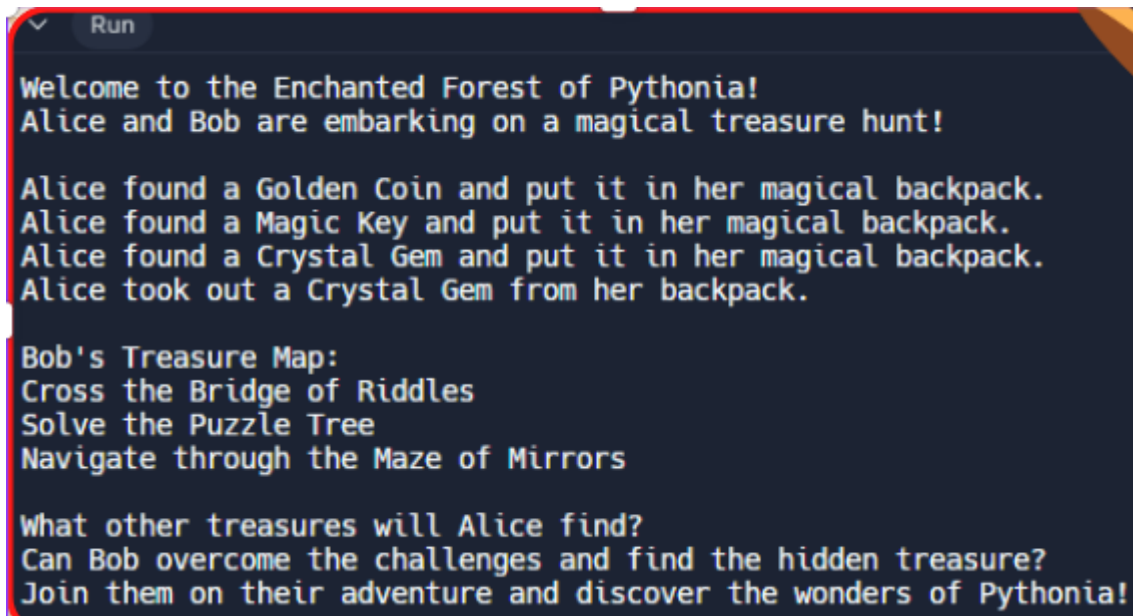
HOME TASK

Home task 1 - Magical Treasure Hunt

Once upon a time, in the enchanted forest of Pythonia, there lived two brave adventurers named Alice and Bob. They embarked on a magical treasure hunt, equipped with a backpack and a map.

Your task: How would you create this magical treasure hunt in Python? Imagine Alice finds magical treasures and puts them in her backpack, while Bob faces challenges guided by a treasure map. How could you represent Alice's backpack using a magical stack and Bob's treasure map using a linked list?

Can you write a Python code to bring their adventure to life?



```
Run
Welcome to the Enchanted Forest of Pythonia!
Alice and Bob are embarking on a magical treasure hunt!

Alice found a Golden Coin and put it in her magical backpack.
Alice found a Magic Key and put it in her magical backpack.
Alice found a Crystal Gem and put it in her magical backpack.
Alice took out a Crystal Gem from her backpack.

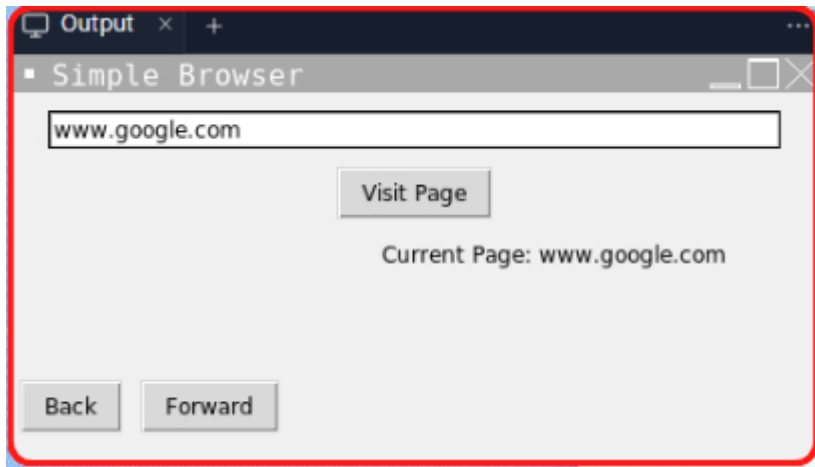
Bob's Treasure Map:
Cross the Bridge of Riddles
Solve the Puzzle Tree
Navigate through the Maze of Mirrors

What other treasures will Alice find?
Can Bob overcome the challenges and find the hidden treasure?
Join them on their adventure and discover the wonders of Pythonia!
```

HOMETASK 2 - Simple Web Browser :

Hey young coder, Let's create a basic web browser using Python. How would you integrate the concept of a stack to manage the history of visited web pages?

Think about employing a stack-like structure to store the URLs of visited pages, enabling users to navigate backward and forward effortlessly. Consider how you'd develop methods to add new URLs onto the stack when a user visits a page and remove URLs from the stack when they navigate back or forward. Additionally, how could you utilize tkinter to build the graphical interface for this browser?



WHAT'S NEXT?

Mini Project - 3

LESSON 22

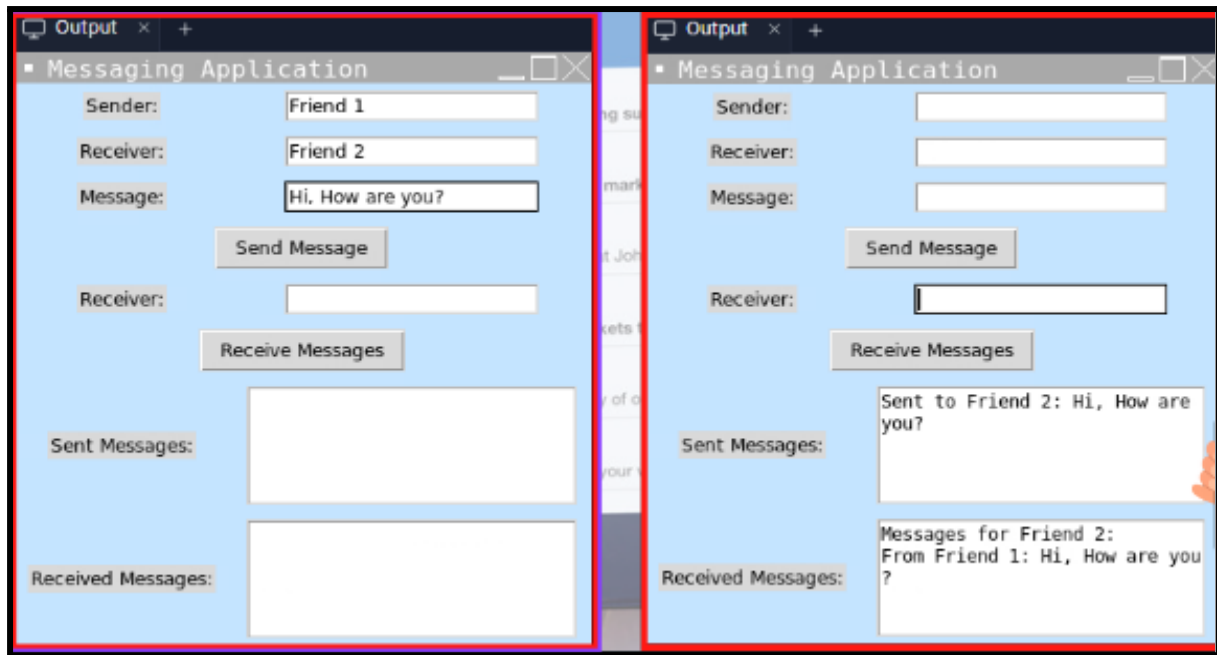
Mini Project-3

Mini project - Messaging App

Hey there! Have you ever wanted to create your own messaging app using Python? Imagine being able to send and receive messages just like you do on your phone, but with your very own program!

I've got a fun challenge for you: How about designing a messaging app where you can send messages to your friends and see what messages they've sent you back? You can use Python's tkinter module to create the user interface and organize the messages.

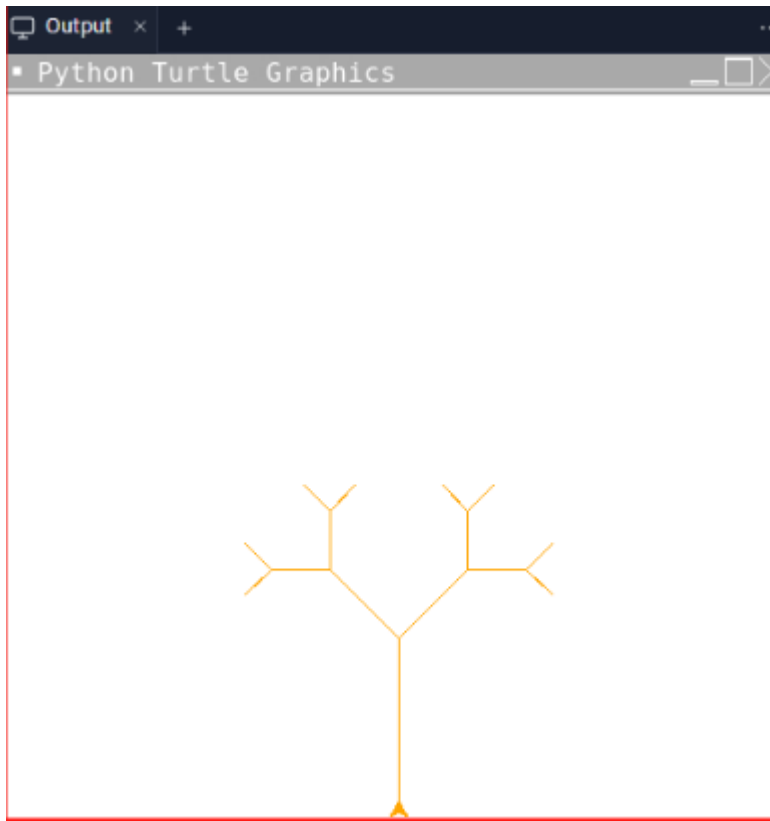
Think about how you'd design the layout with labels, entry fields, and buttons. And don't forget about storing messages for each friend! Ready to take on the challenge and build your own messaging app?



HOME TASK

HOME TASK1-TREE ART:

James wants to draw a beautiful, colorful tree. He has seen some amazing examples online. But, he is not exactly sure where to start. Do you think you could help him figure it out? How would you design a program that draws a colorful tree using a turtle? Think about how we can use the turtle's movements to represent the branches of the tree. Maybe we could use a binary tree structure and recursion to create the branches? We could have a main trunk, and then it splits into branches, which further split into smaller branches, and so on. Remember, you can make the tree as colorful and vibrant as you like! Let's give it a try and see what kind of colorful trees we can create!



HOMETASK 2 - ASCII Values :

After learning about algorithm sara now wants to create a queue-like behavior where characters are sorted based on their ASCII values and then dequeued to form a sorted string.

```
queue = ['z', 'a', 'c', 'b', 'e', 'd']
```

```
Sorted string from queue: abcdez
```

WHAT'S NEXT?

Assessment-2

LESSON 23

ASSESSMENT-2

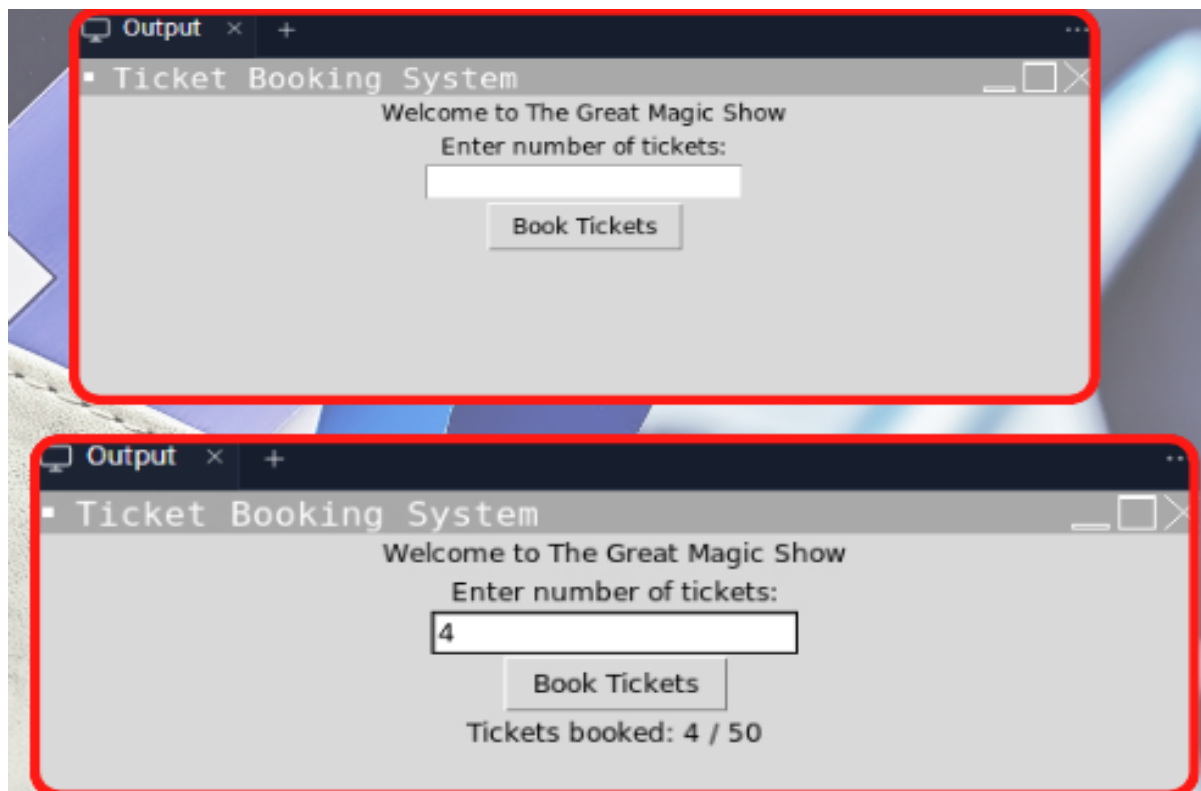
Assessment- Ticket Booking System

Picture yourself as a young programmer bubbling with excitement over a brilliant idea for a ticket booking system! You're envisioning a program where folks can snag tickets to a great magic show. How awesome does that sound? Your task: craft a Python program using tkinter to turn this idea into reality.

Here's what you need to do:

1. Display a welcoming message for the magic show.
2. Create an input field where users can enter the number of tickets they want to book.
3. Add a button to book the tickets.
4. Implement logic to handle booking tickets:
 - Check if the entered number of tickets is valid (not zero or negative).
 - Ensure that there are enough available tickets (maximum 50 tickets).
 - Display appropriate messages for different scenarios, such as successful booking, sold-out tickets, or insufficient available tickets.

5. Display the current status of booked tickets.



HOME TASK

HOME TASK1-Guess the Target Number:

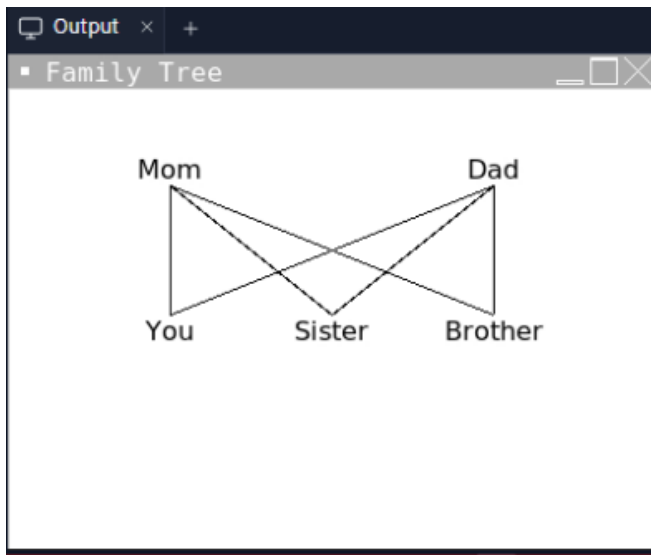
After learning binary search Dave now wants to create a program where the target number is hidden within a range of numbers, and the user has to guess the target number. The program will provide hints like "Too high" or "Too low" to guide the user to the correct answer.

```
Guess the target number hidden between 10 and 100!
Enter your guess: 34
Too high! Try again.
Enter your guess: 56
Too high! Try again.
Enter your guess: 78
Too high! Try again.
Enter your guess: 23
Too low! Try again.
Enter your guess: 44
Too high! Try again.
Enter your guess: 67
Too high! Try again.
Enter your guess: 
```

HOMETASK 2 - Family Tree :

Your friend, Samuel is feeling homesick after moving to a new city. So, help him to feel better by creating a digital family tree using Python. His family includes his mom, dad, himself, his sister, and his brother.

Start by defining each family member's position on the canvas and then use tkinter's Canvas widget to display their names. By connecting each family member with lines, you'll represent their relationships. Utilize object-oriented programming to organize the code, employ data structures like dictionaries to store member information, and iterate through the members to draw and connect them on the canvas.



WHAT'S NEXT?

Major PROJECT -3

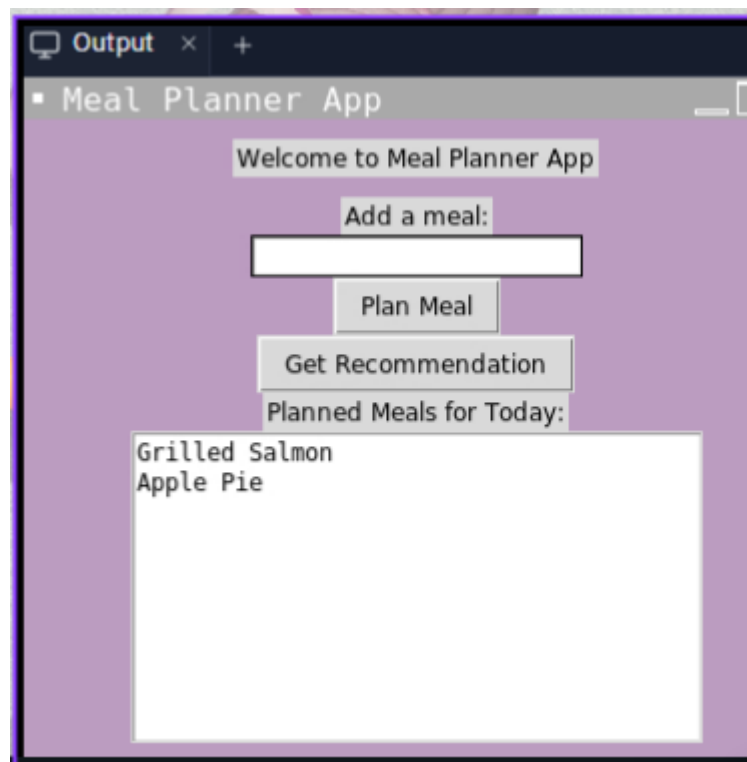
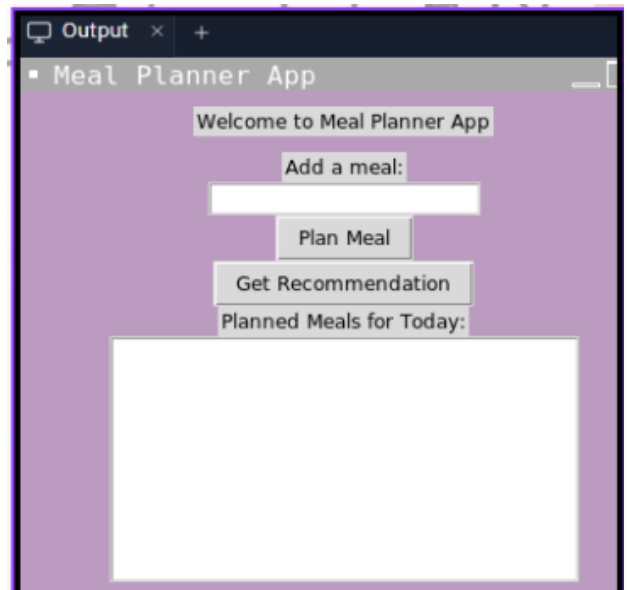
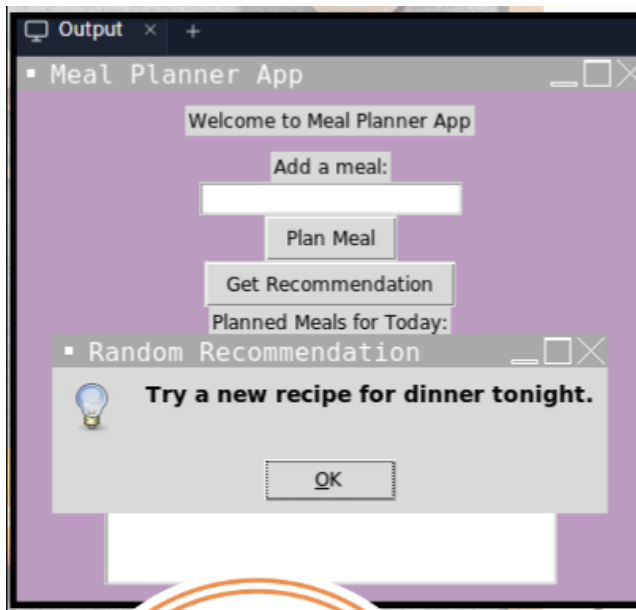
LESSON 24

MAJOR PROJECT - 3

Major Project - Meal Planner App

Can you create a Meal Planner app where you can plan your meals for the day and get random recommendations? You can use Python and Tkinter for the graphical interface. Start by designing the layout: a label to welcome the user, an entry field to add meals, buttons to plan a meal and get a recommendation, and a text area to display planned meals. Then, you'll need to implement a binary tree to store and organize the meals.

Think about how to insert meals into the tree and how to retrieve them for display. Lastly, add functionality to generate random recommendations when the user clicks the recommendation button. Remember, you can use the random module for this! Have fun creating your Meal Planner app!



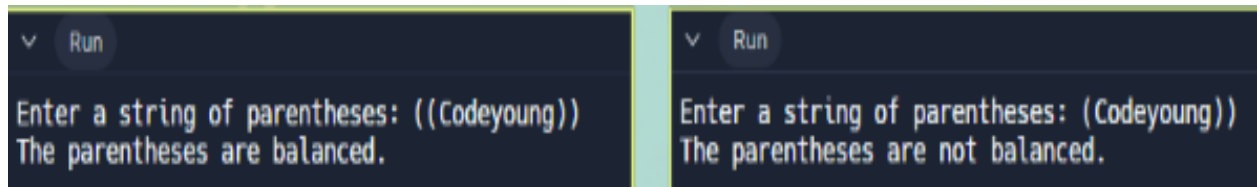
HOME TASK

HOME TASK1-Parantheses Checker:

Hey! Ready for a fun challenge? Create a program that checks if parantheses are balanced. Picture a string filled with parantheses of different types, like round brackets (). For example, ((())) is balanced, but))(is not. Your job is to write a program that ensures every opening paranthesis has a matching closing paranthesis. Think about how you'd solve this puzzle!

Hint: You can use a stack to keep track of opening parantheses and check if each closing paranthesis matches the most recent opening paranthesis.

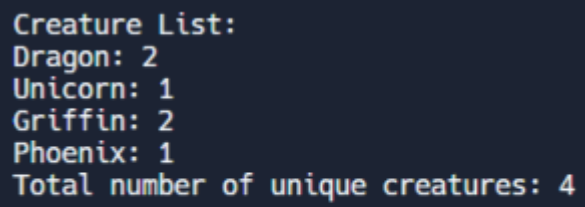
SOLUTION :



HOMETASK 2 - Unique Creatures:

A mad scientist has mixed up the body parts of several creatures and created a linked list of body parts. Each body part is labelled with the name of a creature it belongs to. Your task is to write a Python function to count the total number of creatures represented in the linked list. For example, given the linked list: Dragon -> Unicorn -> Griffin -> Dragon -> Phoenix -> Griffin -> None The function should return 4, as there are four unique creatures (Dragon, Unicorn, Griffin, Phoenix) represented in the linked list.

SOLUTION :

A screenshot of a terminal window with a dark blue background and yellow text. It displays a list of creatures and their counts, followed by a summary line.

```
Creature List:  
Dragon: 2  
Unicorn: 1  
Griffin: 2  
Phoenix: 1  
Total number of unique creatures: 4
```

WHAT'S NEXT?

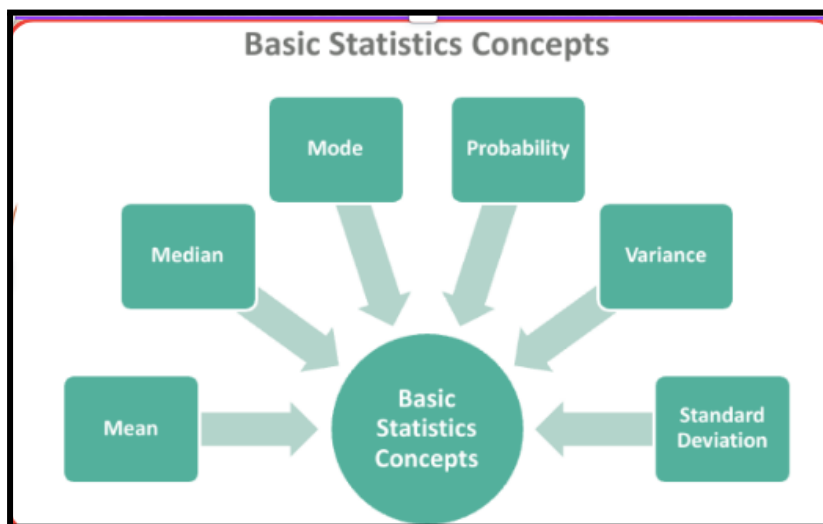
Basic Statistics

LESSON 25

BASIC STATISTICS

LEARNING CONTENT

BASIC STATISTICS:



Mean:

Mean is the average of the given numbers and is calculated by dividing the sum of given numbers by the total number of numbers.

Mean = (Sum of all the observations/Total number of observations)

The symbol of mean is usually given by the symbol ' $\bar{x}$ '. The bar above the letter x, represents the mean of x number of values.

Example : Find the mean of 2, 3, 4, 5, 6.

Answer: sum of observations = 2 + 3 + 4 + 5 + 6 = 20

number of observations = 5 Hence,

Mean = 20 / 5 = 4.

if we have n observations $x_1, x_2, x_3, \dots, x_n$, then:

$$\text{Mean } (\bar{x}) = \frac{x_1 + x_2 + x_3 + \dots + x_n}{n} = \frac{\sum_{i=1}^n x_i}{n}$$

Use of statistics.mean() function?

Syntax	
<pre>import statistics statistics.mean([data-set])</pre>	OR <pre>from statistics import mean mean([data-set])</pre>

where, [dataset]: list or tuple containing the set of number
Return Value mean() function returns the arithmetic mean of the dataset It will return TypeError if the list or the tuple doesn't have a numeric value. It will return TypeError if no argument is passed.

statistics.mean() function.

How to Calculate the Mean of a Dictionary in Python?

In the dictionary, the statistics.mean() only counts the key as a number and returns the mean.

```
#import statistics module
from statistics import mean

#define dictionary

data = {2:4, 3:9, 4:16, 5:25, 6:36}

#print

print("The Mean of the data is:", mean(data))
```

Output: The Mean of the data is: 4

Median:

The median of a set of data is the middlemost number or center value in the set. The median is also the number that is halfway into the set. To find the median, the data should be arranged, first, in order of least to greatest or greatest to the least value. A median is a number that is separated by the higher half of a data sample, a population or a probability distribution, from the lower half. The median is different for different types of distribution.

Median Formula

When number of observations are odd:

$$\text{Median} = \{(n+1)/2\}\text{th term}$$

When number of observations are even :

$$\text{Median} = [(n/2)\text{th term} + \{(n/2)+1\}\text{th}]/2$$

Example :

```
def median(data):
    sorted_data = sorted(data)
    data_len = len(sorted_data)

    middle = (data_len - 1) // 2

    if middle % 2:
        return sorted_data[middle]
    else:
        return (sorted_data[middle] + sorted_data[middle + 1]) / 2.0
```

Example usage:

```
numbers = [1, 2, 3, 4, 5, 6, 7]
med = median(numbers)

print(med)
```

To calculate the median value using built-in Median Function in Python

Import the statistics module. Call the statistics.median() function on a list of numbers.

For example, let's calculate the median of a list of numbers:

```
import statistics

numbers = [1, 2, 3, 4, 5, 6, 7]
med = statistics.median(numbers)

print(med)                                Output: 4
```

Mode:

In statistics, the mode is the value that is repeatedly occurring in a given set. We can also say that the value or number in a data set, which has a high frequency or appears more frequently, is called mode or modal value. It is one of the three

measures of central tendency, apart from mean and median.
 Mode formula for grouped data : Where, l = lower limit of the modal class h = size of the class interval f_1 = frequency of the modal class f_0 = frequency of the class preceding the modal class f_2 = frequency of the class succeeding the modal class
 Let us take an example to understand this clearly.

Mode

The term "mode" describes the value that appears to exist the most or the most frequently in a data series.

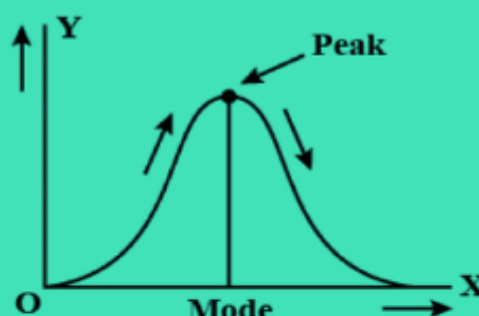
Formula

Individual Series Formula

By just looking at a series and noting the element that appears the most frequently, we can determine its mode.

Discrete Series Formula

A discrete series' mode is determined by the value of the element with the highest frequency.



Frequency Distribution or Continuous Series

First, we must ascertain the Modal class. The class with the highest frequency is the modal class. The mode is then determined by using the following formula:

$$\text{Mode} = l + h \frac{f_1 - f_0}{(2f_1 - f_0 - f_2)}$$

Example :

Finding the mode
 of the dataset given below

```
# importing the statistics library
import statistics

# creating the data set
my_set = [10, 20, 30, 30, 40, 40, 40, 50, 50, 60]

# estimating the mode of the given set
my_mode = statistics.mode( my_set)

# printing the estimated mode to the users
print("Mode of given set of data values is", my_mode)
```

Mode of given set of data values is 40

Standard Deviation

When we're presented with numerical data, we often find descriptive statistics to better understand it. One of these statistics is called the Standard Deviation, which measures the spread of our data around the mean (average).

It is a popular measure of variability because it returns to the original units of measure of the data set. Like the variance, if the data points are close to the mean, there is a small variation whereas the data points are highly spread out from the mean, then it has a high variance. Standard deviation calculates the extent to which the values differ from the average.

Standard Deviation is based on all values. Therefore a change in even one value affects the value of standard deviation. It is independent of origin but not of scale. It is also useful in certain advanced statistical problems.

Types of Standard Deviation

- **Standard Deviation Formula**

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (X_i - \mu)^2}$$

More specifically, this formula is the population standard deviation, one of the two types of standard deviation. We use this formula when we include all values in the entire set in our calculation – in other words, the whole population.

Here,
 σ = Population standard deviation
 N = Number of observations in population
 X_i = i th observation in the population
 μ = Population mean

- **Sample Standard Deviation Formula**

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}$$

This formula is used when we include only a portion of the entire population in our calculation – in other words, a representative sample.

Here,
 s = Sample standard deviation
 n = Number of observations in sample
 x_i = i th observation in the sample
 $\bar{x}$ = Sample mean

- Calculating standard deviation by hand can be tedious,

so people often choose to simplify the process with Python. The `statistics.stdev()` method calculates the standard deviation from a sample of data.

Syntax → `statistics.stdev(data, xbar)`

Parameter Values →

- `data` : Required. The data values to be used (can be any sequence, list or iterator)
- `xbar` : Optional. The mean of the given data. If omitted (or set to None), the mean is automatically calculated

Example : Calculate the standard deviation of the given data

```
# Import statistics Library
import statistics

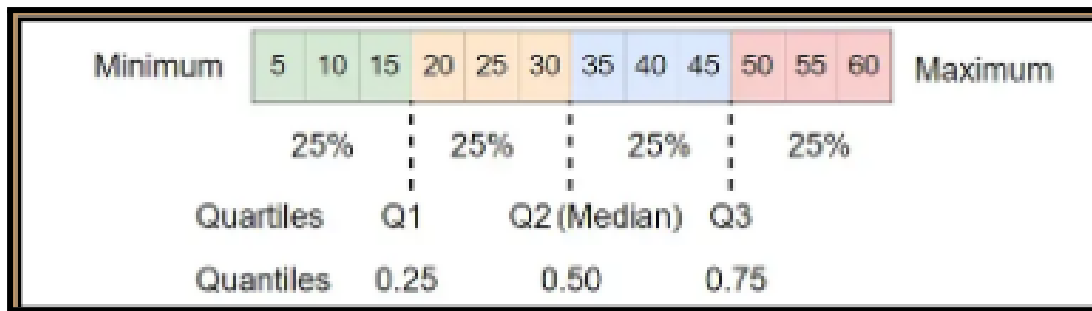
# Calculate the standard deviation from a sample of data
print(statistics.stdev([1, 3, 5, 7, 9, 11]))
print(statistics.stdev([2, 2.5, 1.25, 3.1, 1.75, 2.8]))
print(statistics.stdev([-11, 5.5, -3.4, 7.1]))
print(statistics.stdev([1, 30, 50, 100]))
```

```
3.7416573867739413
0.6925797186365384
8.414471660973929
41.67633221226008
```

Quartiles:

Quartiles are a kind of Quantile that divides a dataset into four equal parts, each containing 25% of the data. Quartiles are useful for understanding the spread and distribution of a dataset.

In general, there are three quartiles used. Q1 (first quartile), Q2 (second quartile), and Q3 (third quartile) are the values below which 25%, 50%, and 75% of the data fall. To find quartiles of a group of data, we have to arrange the data in ascending order.



Quartiles Formula:

Suppose, Q3 is the upper quartile is the median of the upper half of the data set. Whereas, Q1 is the lower quartile and median of the lower half of the data set. Q2 is the median. Consider, we have n number of items in a data set.

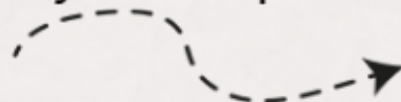
Then the quartiles are given by;

$Q1 = [(n+1)/4]$ th item $Q2 = [(n+1)/2]$ th item $Q3 = [3(n+1)/4]$ th item Hence, the formula for quartile can be given by; Where, Q_r is the rth quartile l_1 is the lower limit l_2 is the upper limit f is the frequency c is the cumulative frequency of the class preceding the quartile class.

Calculating Quartiles with Programming

In Python, the Quartiles can be calculated using the `quantile()` function from the NumPy and pandas package.

The general syntax of `quantile()` looks like this:



Where, x is the dataset in array format and the second array is the probability for the quartiles to compute.


```
# calculate quartiles using using NumPy
import numpy as np
np.quantile(x, [0.25, 0.5, 0.75])

# calculate quartiles using pandas
import pandas as pd
df['col_name'].quantile([0.25, 0.5, 0.75])
```

Quartiles can easily be found with many programming languages. Using software and programming to calculate statistics is more common for bigger sets of data, as finding it manually becomes difficult.

Example With Python use the NumPy library quantile() method to find the quartiles of the values 5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60

```
import numpy

values = [5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60]

x = numpy.quantile(values, [0,0.25,0.5,0.75,1])

print(x)
```

```
[ 5.   18.75 32.5  46.25 60. ]
```

Interquartile Range

1. The interquartile range, often denoted “IQR”, is a way to measure the spread of the middle 50% of a dataset.
2. It is calculated as the difference between the first and third quartiles (Q1 and Q3) of a data set.
3. The first quartile is the value in the data that separates the bottom 25% of values from the top 75%. The third quartile is the value in the data that separates the bottom 75% of

the values from the top 25%

Therefore, the interquartile range is equal to the upper quartile minus lower quartile.

Interquartile Range Formula

The difference between the upper and lower quartile is known as the interquartile range.

The formula for the interquartile range is given below

$$\text{Interquartile range} = \text{Upper Quartile} - \text{Lower Quartile} = Q3 - Q1$$

where Q1 is the first quartile and Q3 is the third quartile of the series.

$$\begin{aligned} \text{Interquartile Range (IQR)} &= Q3 - Q1 \\ &= 88.5 - 71 \\ &= 17.5 \end{aligned}$$

Solution: 17.5

Calculating the Interquartile Range with Programming

It's easy to calculate the interquartile range of a dataset in Python using the `numpy.percentile()` function.

The following code shows how to calculate the interquartile range of values in a single array:

```
import numpy

#define array of data
data = numpy.array([14, 19, 20, 22, 24, 26, 27, 30, 30, 31, 36, 38, 44, 47])

#calculate interquartile range
q3, q1 = numpy.percentile(data, [75, 25])
iqr = q3 - q1

print(iqr)
```

Output : 12.25

IN CLASS CHALLENGES

INCLASS CHALLENGE 1 - Statistical Calculator :

Hey there! Imagine you're on a journey to explore the world of statistics! Let's embark on an exciting coding adventure together! Here's a challenge for you: Can you write a Python program that helps us analyze some fascinating data? We have two sets of data to explore:

The weights of animals at a magical zoo 🦁🐘🦒 A collection of numbers that represent some interesting facts about a mysterious dataset

Your mission is to create a program that calculates various statistical measures for both datasets, such as the standard deviation, mean, median, mode, and quartiles.

Remember to use Python libraries like pandas, numpy, and statistics to make your job easier!

Are you up for the challenge? Let's dive into the world of statistics and uncover some amazing insights!

SOLUTION :

```

Run
Welcome to the Statistical Calculator!

1. Zoo Animal Weight Calculator:
Here are the predefined weights of the animals:
  Animal  Weight (kg)
0 Elephant    5000
1 Giraffe     1500
2 Lion        200
3 Tiger       300
4 Zebra       400
Standard deviation of weights: 2036.4184245876386 kg

2. Prime Number Mean Calculator:
Here are the first 10 prime numbers: [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
Mean of the first 10 prime numbers: 12.9

3. Dataset Analyzer:
Here is the predefined dataset: [5, 8, 6, 5, 7, 9, 8, 6, 5, 7, 8, 9, 6, 7, 8]
Median of the dataset: 7
Mode of the dataset: 8

4. Dataset Quartile Calculator:
Here are the quartiles of the dataset:
First quartile (Q1): 6.0
Second quartile (median, Q2): 7.0
Third quartile (Q3): 8.0

```

INCLASS CHALLENGE 2- Magical World Of Statistics

SoHey there, curious explorer! Let's dive into the magical world of statistics and discover some fascinating secrets together! Imagine you have three mysterious quests awaiting you in this mystical land:

- 🧙 The Mystery of Mean: Did you know that the mean of the first 10 natural numbers holds a secret? Can you write a spell in Python to uncover this magical secret?
- 🌈 The Marvellous Median: There is a magical dataset filled with numbers [4, 17, 77, 25, 22, 23, 92, 82, 40, 24, 14, 12, 67, 23, 29]. Can you create a potion to find the magical midpoint of this dataset?

- 🦄 The Fantastic Mode: In a land far, far away, there was a kingdom ruled by numbers ([3, 3, 6, 9, 15, 15, 15, 27, 27, 37, 48]). Can you embark on a quest to find out which number was the king of this enchanted kingdom?

SOLUTION:

```
Run
★ Welcome to the Magical Statistics Adventure! ★
Are you ready to embark on an exciting journey into the world of numbers and data? Let's go!

1. The Mystery of Mean 🧙:
Did you know that the mean of the first 10 natural numbers holds a secret?
Let's uncover it together!
The mean of the first 10 natural numbers is: 5.5

2. The Marvelous Median 🌈:
Imagine a magical dataset filled with numbers from a mysterious realm.
Let's discover the magical midpoint of this dataset!
The median of the magical dataset is: 24

3. The Fantastic Mode 🦄:
In a land far, far away, there was a kingdom ruled by numbers.
Let's find out which number was the king of this kingdom!
The mode of the enchanted kingdom is: 15

★ What an incredible adventure! ★
I hope you enjoyed exploring the magical world of statistics! Until next time, keep exploring and learning!
```

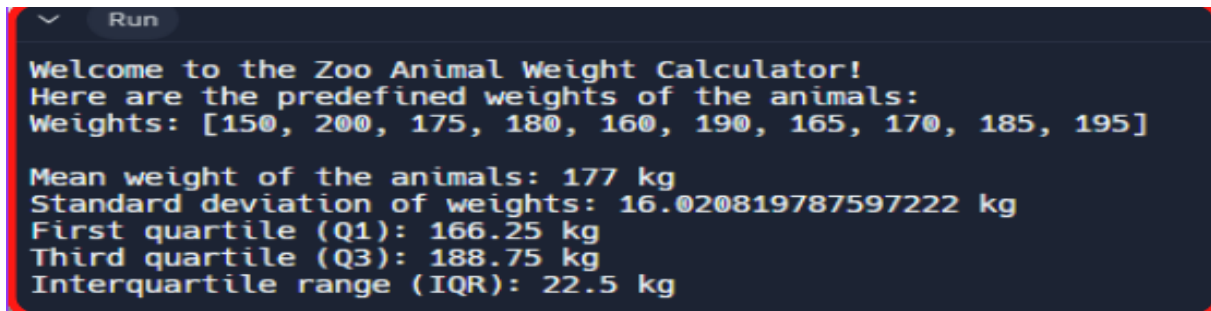
HOME TASK

HOME TASK1-Zoo Animals Weight:

Steve is exploring a magical zoo where animals come in all shapes and sizes. He wants to learn more about the weights of these fascinating creatures! Can you write a Python program that calculates the mean weight, standard deviation, quartiles (Q1 and Q3), and interquartile range of the weights of the animals in the zoo?

You can start by defining a list of predefined weights for the animals and then use numpy module for quartile and statistics module to perform the calculations.

SOLUTION :

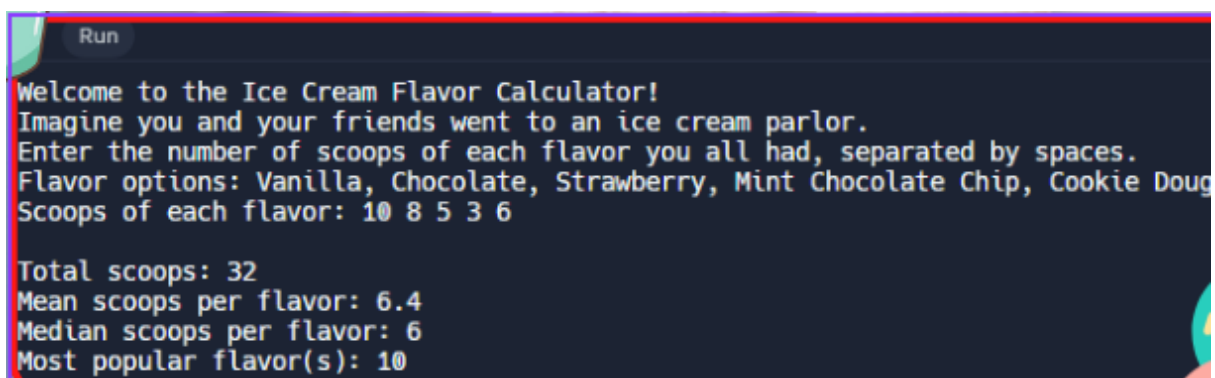
A screenshot of a Python program's output in a terminal window. The window has a dark background and a 'Run' button in the top left corner. The text is as follows:

```
Welcome to the Zoo Animal Weight Calculator!  
Here are the predefined weights of the animals:  
Weights: [150, 200, 175, 180, 160, 190, 165, 170, 185, 195]  
  
Mean weight of the animals: 177 kg  
Standard deviation of weights: 16.020819787597222 kg  
First quartile (Q1): 166.25 kg  
Third quartile (Q3): 188.75 kg  
Interquartile range (IQR): 22.5 kg
```

HOMETASK 2 - Ice Cream Flavor Calculator :

Imagine you and your friends had a fantastic day at an ice cream parlor, trying out different flavors. You want to create a program that helps you analyze your ice cream adventure! Can you write a Python program that takes input on how many scoops of each flavor you all had, and then calculates the total number of scoops, the average number of scoops per flavor, and identifies the most popular flavor(s)? You can use the statistics module to help you calculate the mean, median, and mode!

SOLUTION :

A screenshot of a Python program's output in a terminal window. The window has a dark background and a 'Run' button in the top left corner. The text is as follows:

```
Welcome to the Ice Cream Flavor Calculator!  
Imagine you and your friends went to an ice cream parlor.  
Enter the number of scoops of each flavor you all had, separated by spaces.  
Flavor options: Vanilla, Chocolate, Strawberry, Mint Chocolate Chip, Cookie Dough  
Scoops of each flavor: 10 8 5 3 6  
  
Total scoops: 32  
Mean scoops per flavor: 6.4  
Median scoops per flavor: 6  
Most popular flavor(s): 10
```

WHAT'S NEXT?

Probabilities and dealing with data

LESSON 26

PROBABILITY AND DEALING WITH DATA

LEARNING CONTENT

PROBABILITY

Probability is about how Likely something is to occur, or how likely something is true.

- The mathematical probability is a Number between 0 and 1.
- 0 indicates Impossibility and 1 indicates Certainty.
- The Probability of an Event

The probability of an event is: $\frac{\text{The number of ways the event can happen}}{\text{The number of possible outcomes}}$.

Tossing Coins



When tossing a coin, there are two possible outcomes:

Way	Probability
Heads	$1/2 = 0.5$
Tails	$1/2 = 0.5$

P(A) - The Probability

The probability of an event **A** is often written as **P(A)**.

When tossing two coins, there are 4 possible outcomes:

Event	P(A)
Heads + Heads	$1/4 = 0.25$
Tails + Tails	$1/4 = 0.25$
Heads + Tails	$1/4 = 0.25$
Tails + Heads	$1/4 = 0.25$

Example :

I have 6 balls in a bag: 3 reds, 2 are green, and 1 is blue. Blindfolded. What is the probability that I pick a green one? Number of Ways it can happen are 2 (there are 2 greens). Number of Outcomes are 6 (there are 6 balls). Probability = Ways / Outcomes The probability that I pick a green one is 2 out of 6: $2/6 = 0.333333$. The probability is written $P(\text{green}) = 0.333333$.

P(A)	W/O	Probability
P(red)	3/6	0.5000000
P(green)	2/6	0.3333333
P(blue)	1/6	0.1666666

6 balls Example using Python code

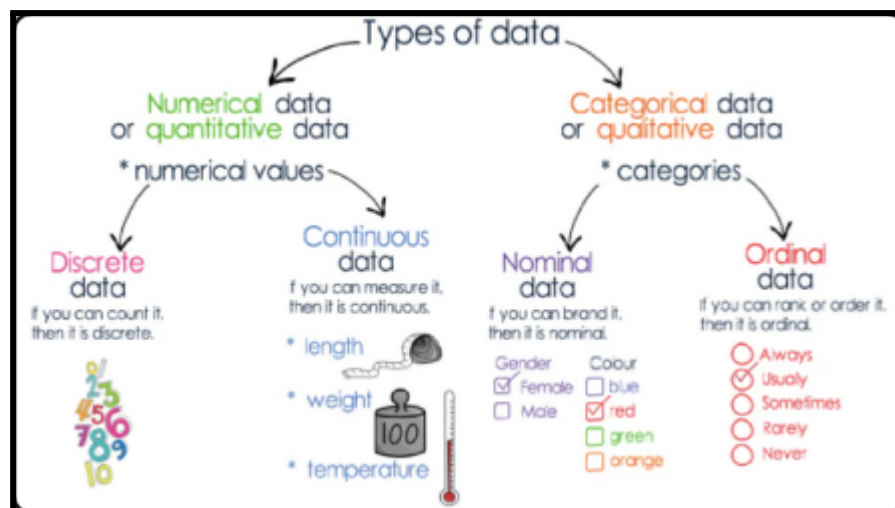

```
def calculate_probability(num_color, total_balls):
    probability = num_color / total_balls
    return probability

num_red = 3
num_green = 2
num_blue = 1
total_balls = 6

probability_red = calculate_probability(num_red, total_balls)
probability_green = calculate_probability(num_green, total_balls)
probability_blue = calculate_probability(num_blue, total_balls)

print("Probability of picking a red ball:", probability_red)
print("Probability of picking a green ball:", probability_green)
print("Probability of picking a blue ball:", probability_blue)
```

Probability of picking a red ball: 0.5
 Probability of picking a green ball: 0.3333333333333333
 Probability of picking a blue ball: 0.16666666666666666



Numerical Data

Numerical data, as the name suggests, consists of numbers. It represents quantitative information and can be measured and counted. This data type is often used to perform mathematical operations and statistical analyses.

Types of Numerical Data

Based on the requirement, or how the data needs to be analysed, numerical data is sorted in the best possible way. These types are Discrete and Continuous data.

1. Discrete Data

Discrete data consists of distinct and separate values. These values are typically integers and do not have fractional or decimal components.

Some common examples of discrete data include:

- Number of Students in a Class: You can't have a fraction of a student; it's a whole number.
- Number of Cars in a Parking Lot: You can't have half a car.
- Number of Customer Complaints: Complaints are usually counted as whole numbers.

Discrete data is often represented as a count or a whole number, and it's suitable for tasks that involve counting and enumeration.

2. Continuous Data

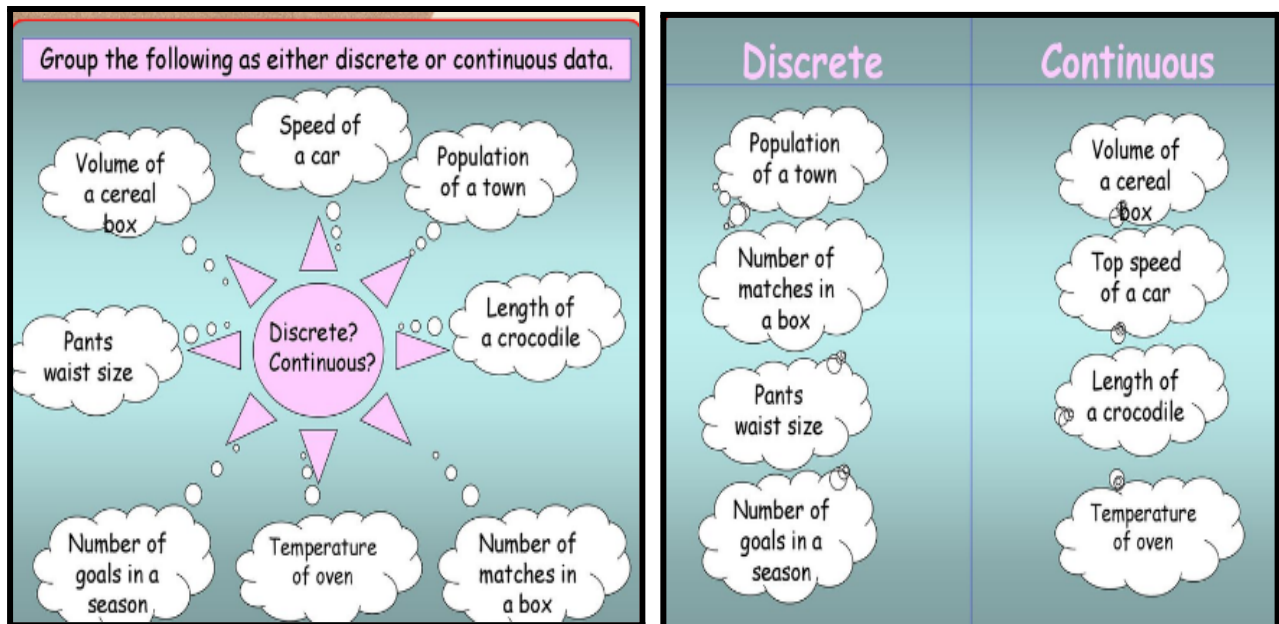
Continuous data, on the other hand, can take any value within a specific range. These values can be integers or decimals.

Some examples of continuous data include:

- Height of Individuals: Heights can be any value within a specific range and have fractional components (e.g., 5.7 feet).
- Temperature: Temperature can be measured with decimal values, taking any value within a range.

- **Weight of Products:** The weight of products can vary continuously and include decimal values.

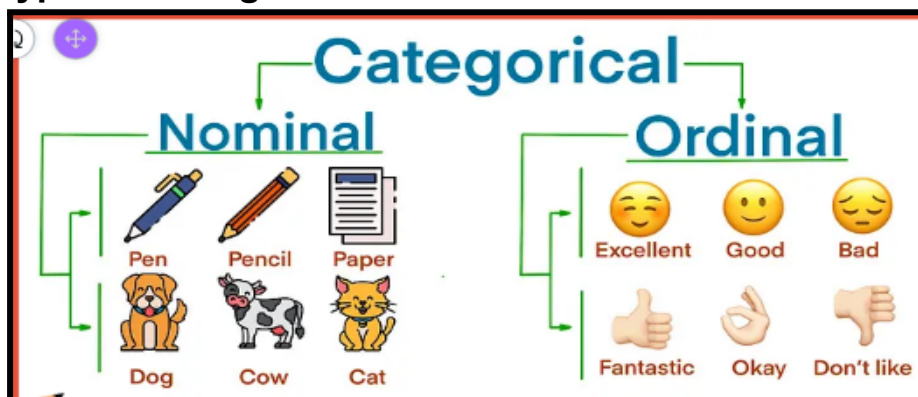
Continuous data is suitable for measurements with infinite possibilities within a given range. It is often used in scientific and engineering applications.



• Categorical Data

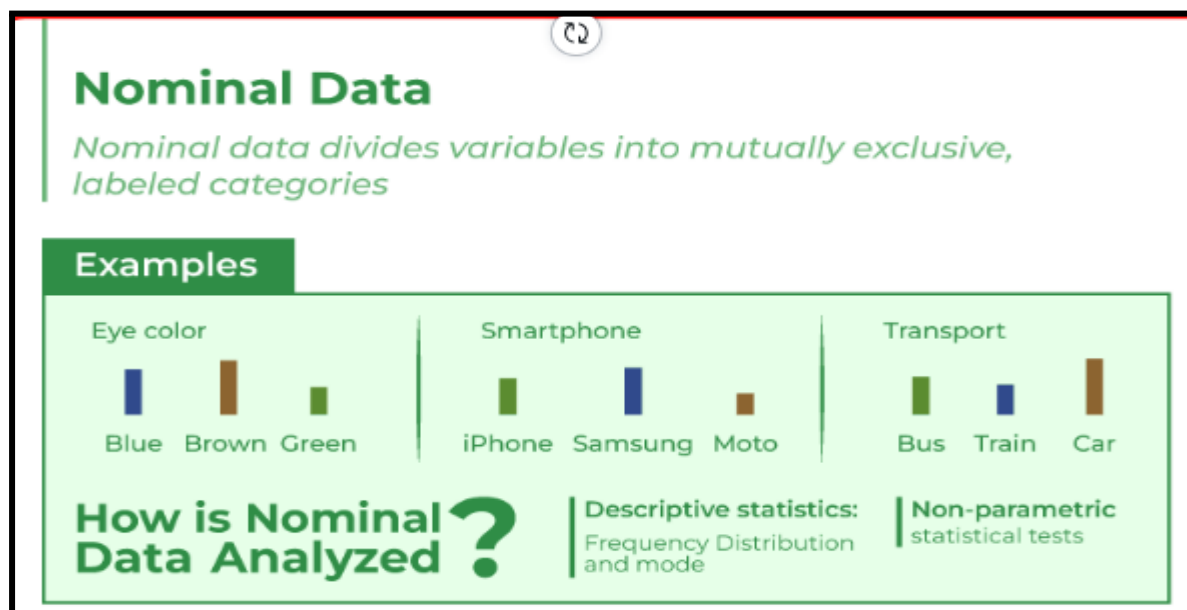
Data that can be categorized or grouped is called categorical data. It is a type of data in statistics that stores data into groups or categories using names or labels, and it can be derived from observations made of qualitative data that are summarized as counts or from observations of quantitative data grouped within given intervals. Categorical data is also well-known as qualitative data.

Types of Categorical Data



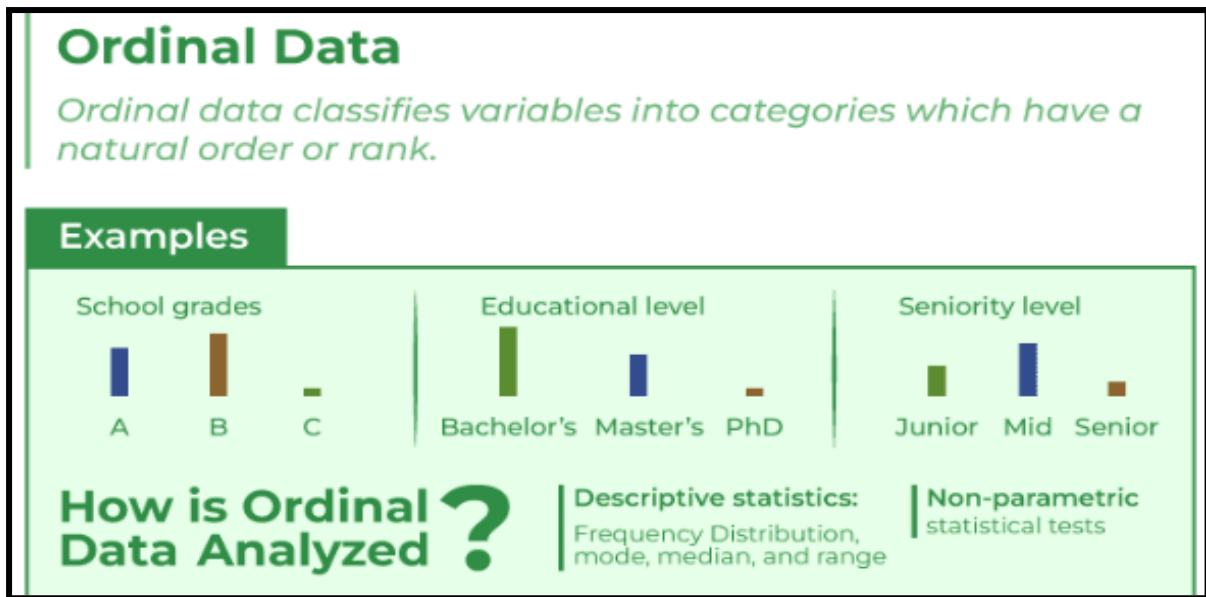
Nominal data

Nominal data is a crucial type of information utilised in diverse fields such as research, statistics, and data analysis. It comprises categories or labels that help in classifying and arranging data. The essential feature of nominal data is that it does not possess any inherent order or ranking among its categories. Instead, these categories are separate, distinct, and mutually exclusive.



Ordinal data

Ordinal data is a form of qualitative data that classifies variables into descriptive categories. It is characterized by the fact that the categories it employs are ranked on some sort of hierarchical scale, such as from high to low. Ordinal data is the second most complicated type of measurement, following nominal data. Although it is more intricate than nominal data, which lacks any inherent order, it is still relatively simplistic.



IN CLASS CHALLENGES

CHALLENGE1 - Ticket Game :

Would you like to play a fun game with the Great Magician? Imagine you're at a magical show where the magician has a special hat filled with tickets numbered from 1 to 20. Your task is to help the magician find out the chances of pulling out a special ticket! Here's the challenge: Can you write a magical spell (code) that lets the Great Magician draw tickets from the hat and reveals the probability of getting a ticket with a number that is a multiple of 3 or 5? Remember, the more tickets the magician draws, the better we understand the magic! Let's dive into the world of code and uncover the secrets of probability together!

INCLASS CHALLENGE 2- Coin Flip Simulator:

Imagine you're embarking on a digital coin-flipping experiment. You're tasked with creating a Python program that accurately simulates the process of flipping a coin, records the outcomes, and calculates the probabilities of getting heads or tails. To begin, utilize Python's random module to simulate the coin flips, ensuring a fair 50/50 chance for each outcome. Employ a loop structure to conduct multiple flips, keeping count of the number of heads and tails observed. Once the flips are completed, calculate the probability of heads and tails by dividing the respective counts by the total number of flips.

HOME TASK

HOME TASK1-Exam Scores:

Imagine you're in a classroom with 25 students, all gearing up for an upcoming test. Your teacher wants to understand what scores might emerge from this exam. Each student can score between 0 and 10. How would you write a program to help calculate these probabilities? You'll need to simulate the test-taking process multiple times to get accurate results. Can you come up with a plan to write such a program?

HOMETASK 2 - Dice Probability Analyzer:

Ever wondered how likely it is to roll a specific number on a dice? What if you could build a program that tells you the probability of rolling that number when you throw multiple dice at once? How would you go about designing such a program? Here are some hints to get you started on creating a program that can calculate this: Begin your journey by harnessing the power of Python's random module to emulate the unpredictability of dice rolls. Utilize loops to execute multiple rolls, meticulously tracking the outcomes of each toss. Once the rolls are complete, venture into the realm of probability by analyzing the data to determine the likelihood of rolling a particular number.

WHAT'S NEXT?

Accessing and modifying data in Pandas series

LESSON 27

Accessing and modifying data in series Using PANDAS

LEARNING CONTENT

What Is Pandas ?

Pandas is a powerful and open-source Python library. It is used for working with data sets. It has functions for analyzing, cleaning, exploring, and manipulating data. Pandas consist of data structures and functions to perform efficient operations on data. list of things that we can do using Pandas:

Data set cleaning, merging, and joining.

- Easy handling of missing data (represented as NaN) in floating point as well as non-floating point data.
- Columns can be inserted and deleted from DataFrame and higher-dimensional objects.
- Powerful group by functionality for performing split-apply-combine operations on data sets.

Pandas generally provide two data structures for manipulating data. They are: 1) Series 2) DataFrame

Pandas Series:

Pandas Series is a one-dimensional labeled array capable of holding data of any type (integer, string, float, python objects, etc.). Using the Series() function. We can simply turn a list, tuple, or dictionary into a Series.

Sample code to create a empty series :

```
# importing pandas library
import pandas as pd
# Creating empty series
series = pd.Series() print(series)
Output : Series([], dtype: object)
```

Creating Pandas Series

NumPy is an open-source Python library that facilitates efficient numerical operations on large quantities of data. There are a few functions that exist in NumPy that we use on pandas. the most important part about NumPy is that pandas is built on top of it. Creating a series from array: In order to create a series from array, we have to import a numpy module and have to use array() function.

```
import numpy as np
import pandas as pd

# simple array
data = np.array(['c','o','d','e','y','o','u','n','g'])

ser = pd.Series(data)
print(ser)
```

```
0    c
1    o
2    d
3    e
4    y
5    o
6    u
7    n
8    g
dtype: object
```


Example : Creating a series from Lists: In order to create a series from list, we have to first create a list after that we can create a series from list.

Creating a series from Dictionary: In order to create a series from the dictionary, we have to first create a dictionary after that we can make a series using dictionary. Dictionary keys are used to construct indexes of Series.

```
import pandas as pd

# a simple dictionary
dict = {'Code': 10,
        'young': 20,}

ser = pd.Series(dict) # create series from dictionary
print(ser)
```

```
Code      10
young     20
dtype: int64
```

ELEMENTS OF A SERIES CAN BE ACCESSED IN TWO WAYS

Accessing Element from Series with Position Accessing Element Using Label

In order to access an element in a series, we use index operator []
In order to access multiple elements from a series, we use Slice operation. Slice operation is performed on Series with the use of the colon(:)

- To print elements from beginning to a range use [:Index]
- To print elements from end use [:-Index]
- To print elements from specific Index till the end use [Index:]
- To print elements within a range, use [Start Index:End Index]
- To print whole Series with the use of slicing operation, use [:]
- To print the whole Series in reverse order, use [::-1]

Accessing Element from Series with Position

Accessing the First Element of Series: In this example, a Pandas Series named 'ser' is created from a NumPy array 'data' containing the elements 'g', 'e', 'e', 'k', 's', 'f', 'o', 'r', 'g', 'e', 'e', 'k', 's'. The first element of the series is accessed and printed using `print(ser[0])`

```
# import pandas and numpy
import pandas as pd
import numpy as np

# creating simple array
data = np.array(['g', 'e', 'e', 'k', 's', 'f',
                 'o', 'r', 'g', 'e', 'e', 'k', 's'])
ser = pd.Series(data)
# retrieve the first element
print(ser[0])
```

Output : g

Accessing First 5 Elements of Series:

In this example, a Pandas Series named 'ser' is created from a NumPy array 'data' containing the elements 'g', 'e', 'e', 'k', 's', 'f', 'o', 'r', 'g', 'e', 'e', 'k', 's'. The first five elements of the series are accessed and printed using `print(ser[:5])`

```
# import pandas and numpy
import pandas as pd
import numpy as np

# creating simple array
data = np.array(['g', 'e', 'e', 'k', 's', 'f',
                 'o', 'r', 'g', 'e', 'e', 'k', 's'])
ser = pd.Series(data)
# retrieve the first element
print(ser[:5])
```

Output:

0	g
1	e
2	e
3	k
4	s
dtype: object	

Accessing Last 10 Elements of Series:

In this example, a Pandas Series named 'ser' is created from a NumPy array 'data' containing the elements 'g', 'e', 'e', 'k', 's', 'f', 'o', 'r', 'g', 'e', 'e', 'k', 's'. The last 10 elements of the series are accessed and printed using `print(ser[-10:])`

```
# import pandas and numpy
import pandas as pd
import numpy as np

# creating simple array
data = np.array(['g', 'e', 'e', 'k', 's', 'f',
                 'o', 'r', 'g', 'e', 'e', 'k', 's'])
ser = pd.Series(data)

# retrieve the first element
print(ser[-10:])
```

Output:

3	k
4	s
5	f
6	o
7	r
8	g
9	e
10	e
11	k
12	s

dtype: object

Access an Element in Series Using Label:

In order to access an element from series, we have to set values by index label. A Series is like a fixed-size dictionary in that you can get and set values by index label. Here, we will access an element in Pandas using label.

Accessing a Single Element Using index Label

In this example, a Pandas Series 'ser' is created from a NumPy array 'data' with custom indices provided. The element at index 16 is accessed and printed using `print(ser[16])`

```
# import pandas and numpy
import pandas as pd
import numpy as np

# creating simple array
data = np.array(['g', 'e', 'e', 'k', 's', 'f',
                'o', 'r', 'g', 'e', 'e', 'k', 's'])
ser = pd.Series(data, index=[10, 11, 12, 13, 14,
                             15, 16, 17, 18, 19, 20, 21, 22])

# accessing a element using index element
print(ser[16])
```

Output : o

Accessing a Multiple Element Using index Label:

In this example, a Pandas Series 'ser' is created from a NumPy array 'data' with custom indices provided. Multiple elements at indices 10, 11, 12, 13, and 14 are accessed and printed using `print(ser[[10, 11, 12, 13, 14]])`

```
# import pandas and numpy
import pandas as pd
import numpy as np

# creating simple array
data = np.array(['g', 'e', 'e', 'k', 's', 'f',
                'o', 'r', 'g', 'e', 'e', 'k', 's'])
ser = pd.Series(data, index=[10, 11, 12, 13, 14,
                             15, 16, 17, 18, 19, 20, 21, 22])

# accessing a multiple element using
# index element
print(ser[[10, 11, 12, 13, 14]])
```

Output:

10	g
11	e
12	e
13	k
14	s
dtype: object	

Access Multiple Elements by Providing Label of Index:

In this example, a Pandas Series 'ser' is created using NumPy's `arange()` function with values from 3 to 8 and custom indices. Elements at indices 'a', 'd', 'g', and 'l' are accessed and printed using `print(ser[['a', 'd', 'g', 'l']])`. Note that for indices 'g' and 'l' that do not exist in the original series, NaN (Not a Number) will be displayed.

In this example, a Pandas Series 'ser' is created using NumPy's `arange()` function with values from 3 to 8 and custom indices. Elements at indices 'a', 'd', 'g', and 'l' are accessed and printed using `print(ser[['a', 'd', 'g', 'l']])`. Note that for indices 'g' and 'l' that do not exist in the original series, NaN (Not a Number) will be displayed.

```
# importing pandas and numpy
import pandas as pd
import numpy as np

ser = pd.Series(np.arange(3, 9), index=['a', 'b', 'c', 'd', 'e', 'f'])
print(ser[['a', 'd', 'g', 'l']])
```

Output:

a	3.0
d	6.0
g	NaN
l	NaN

dtype: float64

MODIFYING DATA IN PANDAS SERIES:

Pandas series is a One-dimensional ndarray with axis labels. The labels need not be unique but must be a hashable type. The object supports both integer- and label-based indexing and provides a host of methods for performing operations involving the index.

Pandas `Series.update()` function modify Series in place using non-NA values from passed Series object. The function aligns on index.

Example 1: Use Series.update() function to update the values of some cities in the given Series object

```
# importing pandas as pd
import pandas as pd

# Creating the Series
sr = pd.Series(['New York', 'Chicago', None, 'Toronto', 'Lisbon', 'Rio', 'Chicago', 'Lisbon'])

# Print the series
print(sr)
```

```
0    New York
1    Chicago
2         None
3    Toronto
4     Lisbon
5         Rio
6    Chicago
7     Lisbon
dtype: object
```

Now we will use Series.update() function to update the values identified the passed indexed in the given Series object.

```
# update the values at the passed index
# from the values in the passed series object
sr.update(pd.Series(['Melbourne', 'Moscow'], index = [2, 7]))
```

```
0    New York
1    Chicago
2  Melbourne
3    Toronto
4     Lisbon
5         Rio
6    Chicago
7     Moscow
dtype: object
```

Example 2: Use Series.update() function to update the values of some elements in the given Series object

```
# importing pandas as pd
import pandas as pd

# Creating the Series
sr = pd.Series([100, 214, 325, 88, None, 325, None, 325, 100])

# Print the series
print(sr)
```

```
0    100.0
1    214.0
2    325.0
3     88.0
4     NaN
5    325.0
6     NaN
7    325.0
8    100.0
dtype: float64
```

Now we will use Series.update() function to update the values identified the passed indexed in the given Series object.

```
# update the values at the passed index
# from the values in the passed series object
sr.update(pd.Series([5000, 6000], index = [4, 6]))
```

```
0    100.0
1    214.0
2    325.0
3     88.0
4    5000.0
5    325.0
6    6000.0
7    325.0
8    100.0
dtype: float64
```

As we can see in the output, the Series.update() function has successfully updated the values in the original series object from the passed series object.

IN CLASS CHALLENGES

CHALLENGE1 - Favorite Toys :

Imagine you have a group of friends, each with their favorite toys. You want to create a program that helps you quickly find out which toy each friend likes the most and vice versa. To accomplish this, start by organizing the list of favorite toys along with the corresponding friends' names into a Pandas Series. Next, develop two functions: one to find the friend who prefers a specific toy the most by inputting the toy's name, and another to discover the toy at a given position in the list by providing its position number. Ensure to handle scenarios where the toy name is not found or the provided position is out of range. By following these steps and testing your functions with different inputs, you'll be able to create a handy tool for exploring your friends' toy preferences.

INCLASS CHALLENGE 2- Board Games:

Hey there! Are you ready for a cool coding challenge? Let's imagine you and your friends are crazy about board games, and you want to create a program to keep track of everyone's favorite board games. Here's what you need to do:

Create a Pandas Series: Start by creating a Pandas Series representing your friends' favorite board games. Each friend's name will be the index, and their favorite game will be the value.

Update the Dataset: Next, suppose some of your friends have changed their favorite games. You need to update the dataset with the latest information.

Access First and Last Element: Lastly, write a function to access the first and last elements of the series using slicing notation (`s[:]`). This function should return the first and last games in the list.

HOME TASK

HOME TASK1-Task Update:

You and your friends are working on an exciting group project, and you need to assign tasks to each friend based on their skills. Your task is to create a program that helps you manage these task assignments using Python. Below are the steps you need to follow: Create a Pandas Series: First, create a Pandas Series representing the assigned tasks for each friend. Think about what tasks each friend would be best suited for and assign them accordingly. Write a Function to Update Tasks: Write a function called `update_task()` that takes two inputs: a friend's name and their new task. This function should check if the friend exists in the Series and update their task if found. If the friend is not in the list, it should print a message saying so. Test Your Function: Test your `update_task()` function with different inputs to ensure it correctly updates the task assignments.

HOMETASK 2 - Candy Tracker :

Picture this: it's Halloween night, and you're out trick-or-treating with your best buddies - Alice, Bob, and Charlie. You've all collected a bunch of candies, but now comes the fun part: sharing and trading! Can you create an exciting program that keeps track of how many candies each of you has after swapping some treats with each other? And let's not forget the second round of trick-or-treating - how will your candy counts change after that adventure? Get ready for a spooktacular coding challenge!

WHAT'S NEXT?

ACCESSING AND MODIFYING DATA

LESSON 28

ACCESSING AND MODIFYING DATA

LEARNING CONTENT

PANDAS DATAFRAME

Pandas DataFrame is two-dimensional size-mutable, potentially heterogeneous tabular data structure with labeled axes (rows and columns). A Data frame is a two-dimensional data structure, i.e., data is aligned in a tabular fashion in rows and columns. Pandas DataFrame consists of three principal components, the data, rows, and columns.

The diagram illustrates a Pandas DataFrame structure. It features a table with 7 rows and 5 columns. The columns are labeled 'Name', 'Team', 'Number', 'Position', and 'Age'. The rows are indexed from 0 to 6. The data is as follows:

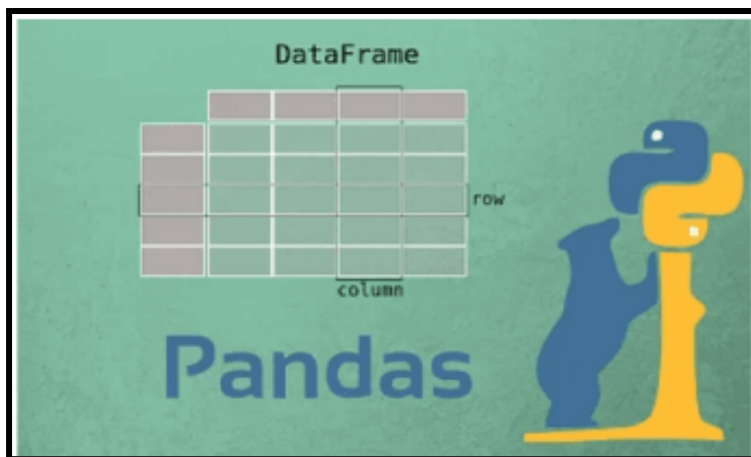
	<i>Name</i>	<i>Team</i>	<i>Number</i>	<i>Position</i>	<i>Age</i>
0	Avery Bradley	Boston Celtics	0.0	PG	25.0
1	John Holland	Boston Celtics	30.0	SG	27.0
2	Jonas Jerebko	Boston Celtics	8.0	PF	29.0
3	Jordan Mickey	Boston Celtics	NaN	PF	21.0
4	Terry Rozier	Boston Celtics	12.0	PG	22.0
5	Jared Sullinger	Boston Celtics	7.0	C	NaN
6	Evan Turner	Boston Celtics	11.0	SG	27.0

Annotations in the diagram:

- Columns:** A blue label with arrows pointing to the column headers.
- Rows:** An orange label with arrows pointing to the row indices (0-6).
- Data:** A pink label with a bracket pointing to the data cells, specifically highlighting the 'Team' column for rows 2-6 and the 'Position' and 'Age' columns for row 5.

Creating a Pandas DataFrame

In the real world, a Pandas DataFrame will be created by loading the datasets from existing storage, storage can be SQL Database, CSV file, and Excel file. Pandas DataFrame can be created from the lists, dictionary, and from a list of dictionary etc. To create a DataFrame, you can use the `pd.DataFrame()` constructor. This function takes a list as input and creates a DataFrame with the same number of rows and columns as the input list.



Creating a dataframe using List: DataFrame can be created using a single list or a list of lists.

```
# import pandas as pd
import pandas as pd

# list of strings
lst = ['Codeyoung', 'is', 'an', 'online', 'learning',
       'platform', 'for', 'young', 'kids']

# Calling DataFrame constructor on list
df = pd.DataFrame(lst)
print(df)
```

```
0    Codeyoung
1         is
2         an
3    online
4  learning
5  platform
6         for
7    young
8     kids
```

Example : Creating DataFrame from dict of ndarray/lists: To create DataFrame from dict of ndarray/list, all the ndarray must be of same length. If index is passed then the length index should be equal to the length of arrays. If no index is passed, then by default, index will be range(n) where n is the array length.

```
# Python code demonstrate creating
# DataFrame from dict ndarray / lists
# By default addresses.

import pandas as pd

# initialise data of lists.
data = {'Name':['Sam', 'Emma', 'krish', 'jack'],
        'Age':[20, 21, 19, 18]}

# Create DataFrame
df = pd.DataFrame(data)

# Print the output.
print(df)
```

	Name	Age
0	Sam	20
1	Emma	21
2	krish	19
3	jack	18

Creating DataFrame using csv file

CSV files are the “comma-separated values”, these values are separated by commas, this file can be viewed like an excel file. In Python, Pandas is the most important library coming to data science. We need to deal with huge datasets while analyzing the data, which usually can get in CSV file format. Creating a pandas data frame using CSV files can be achieved in multiple ways. Using read_csv() method: read_csv() is an important pandas function to read csv files and do operations on it.

Example :

```
# import pandas module
import pandas as pd

# creating a data frame
df = pd.read_csv("Codeyoung.csv")
print(df.head())
```

Exploring the DataFrame

Now that we have our DataFrame loaded, let's explore its content. Understanding the structure and characteristics of our data is crucial for meaningful analysis. Here are some fundamental operations to perform on a DataFrame: Displaying the first few rows: By calling `df.head()`, we can view the first few rows of our DataFrame, giving us a glimpse of the data's structure and content. Examining the last few rows: Similar to `df.head()`, the `df.tail()` method allows us to inspect the last few rows of our DataFrame. Checking the shape of the DataFrame: By using `df.shape`, we can determine the number of rows and columns in our DataFrame, providing an overview of the dataset's size. Inspecting the column names and data types: Employ `df.columns` to retrieve the column names and `df.dtypes` to obtain the data types of each column

```
# Python code demonstrate creating
# DataFrame from dict narray / lists
# By default addresses.

import pandas as pd

# initialise data of lists.
data = {'Name':['Sam', 'Emma', 'krish', 'jack'],
        'Age':[20, 21, 19, 18]}

# Create DataFrame
df = pd.DataFrame(data)

# Print the output.
print(df)
print('\n',df.head(2))
print('\n', df.tail(2))
print('\n', df.shape)
print('\n',df.columns)
print('\n',df.dtypes)
```

```
Run

  Name  Age
0   Sam   20
1  Emma   21
2 krish   19
3  jack   18

  Name  Age
0   Sam   20
1  Emma   21
2 krish   19
3  jack   18

(4, 2)

Index(['Name', 'Age'], dtype='object')

Name    object
Age     int64
dtype: object
```

ACCESSING DATA FROM DATAFRAME

We can access data of DataFrames in multiple ways

- Value
- Columns
- Rows

```
##Creating a DataFrame

import pandas as pd
dictObj = {'EmpId' : ['E01','E02','E03','E04'],
           'EmpName' : ['Sam','Emma','Krish','Jack'],
           'Department' : ['IT','IT','HR','Accounts']}
df=pd.DataFrame(dictObj, index=['First','Second','Third','Fourth'])
print(df)
```

Run

	EmpId	EmpName	Department
First	E01	Sam	IT
Second	E02	Emma	IT
Third	E03	Krish	HR
Fourth	E04	Jack	Accounts

ACCESSING DATAFRAME VALUE

We can access a individual value of a DataFrame by using the row name along with the column name. And also with the help of “at” and “iat” attributes.

Syntax: DataFrame Object . column name [row name]

DataFrame Object . at [row name, column name]

DataFrame Object . iat [row index no, column index no]

```
import pandas as pd
dictObj = {'EmpId' : ['E01','E02','E03','E04'],
           'EmpName' : ['Sam','Emma','Krish','Jack'],
           'Department' : ['IT','IT','HR','Accounts']}
df=pd.DataFrame(dictObj, index=['First','Second','Third','Fourth'])
print(df)
print('\n',df.EmpName['Third']) #access using row name
print('\n',df.at['Second','EmpName']) #access using at method
print('\n',df.iat[3,1]) #access using iat method
```

	EmpId	EmpName	Department
First	E01	Sam	IT
Second	E02	Emma	IT
Third	E03	Krish	HR
Fourth	E04	Jack	Accounts

Krish
Emma
Jack

ACCESSING SINGLE COLUMN & MULTIPLE COLUMNS OF A DATAFRAME

We can access single column data by passing a column name and to access multiple columns of a DataFrame we need to pass a list of columns names inside the square bracket.

Syntax: DataFrame Object [Column Name] OR DataFrame Object . Column Name ##for single column data.

DataFrame Object [[Column Name1, Column Name2, Column Name3]] ##for multiple column data.

```
import pandas as pd
dictObj = {'EmpId' : ['E01','E02','E03','E04'],
           'EmpName' : ['Sam','Emma','Krish','Jack'],
           'Department' : ['IT','IT','HR','Accounts']}
df=pd.DataFrame(dictObj, index=['First','Second','Third','Fourth'])
print(df)
print('\n',df['EmpId']) #accessing a column
print('\n',df.EmpName) #accessing a column
print('\n',df[['EmpName','Department']]) #accessing multiple columns
```

	EmpId	EmpName	Department
First	E01	Sam	IT
Second	E02	Emma	IT
Third	E03	Krish	HR
Fourth	E04	Jack	Accounts

First	E01
Second	E02
Third	E03
Fourth	E04

Name: EmpId, dtype: object

First	Sam
Second	Emma
Third	Krish
Fourth	Jack

Name: EmpName, dtype: object

	EmpName	Department
First	Sam	IT
Second	Emma	IT
Third	Krish	HR
Fourth	Jack	Accounts

ACCESSING DATAFRAME ROWS

We can access single row and multiple rows of a DataFrame in two ways. By using loc and iloc

Syntax
DataFrame Object . loc [[row name]]
DataFrame Object . iloc [[row index no]]

Accessing row using loc and iloc

Example:

```
import pandas as pd
dictObj = {'EmpId' : ['E01','E02','E03','E04'],
           'EmpName' : ['Sam','Emma','Krish','Jack'],
           'Department' : ['IT','IT','HR','Accounts']}
df=pd.DataFrame(dictObj, index=['First','Second','Third','Fourth'])
print(df)
print('\n',df.loc['First'])    #accessing a row using label
print('\n',df.iloc[3])        #accessing a row using index
```

```
      EmpId EmpName Department
First   E01     Sam         IT
Second E02     Emma         IT
Third  E03    Krish         HR
Fourth E04     Jack    Accounts

      EmpId      E01
EmpName     Sam
Department   IT
Name: First, dtype: object

      EmpId      E04
EmpName     Jack
Department  Accounts
Name: Fourth, dtype: object
```

By using loc and iloc we can also access subset of a dataframe or we can say slicing the dataframe.

Syntax : DataFrame Object . loc [start row name : end row name , start column name : end column name]

DataFrame Object . iloc [start row number : end row number , start column number : end column number]

EXAMPLES OF SLICING	
<code>df.loc [:]</code>	Fetching all rows and columns from dataframe
<code>df.loc ['First' : , :] OR df.iloc [0 : , :]</code>	Fetching all row starting from 'First' and fetching all columns.

<code>df.loc [:]</code>	Fetching all rows and columns from dataframe
<code>df.loc ['First' : , :] OR df.iloc [0 : , :]</code>	Fetching all row starting from 'First' and fetching all columns.
<code>df.loc [: 'Third' , :] OR df.iloc [: 3 , :]</code>	Fetching rows starting from 'First' till the 'Third' with all columns.
<code>df.loc ['Second' : 'Third' , :] OR df.iloc [1:3 , :]</code>	Fetching row starting from 'Second' and end to 'Third' with all columns.
<code>df.loc [: , 'EmpName' :] OR df.iloc [: , 1 :]</code>	Fetching all rows with 'EmpName' column to end column.
<code>df.loc [: , : 'EmpName'] OR df.iloc [: , : 2]</code>	Fetching all rows till column EmpName.
<code>df.loc ['Second' : 'Third' , 'EmpName' : 'Department'] OR df.iloc [1:3 , 1:3]</code>	Fetching rows starting with 'Second' till 'Third' and columns from 'EmpName' till 'Department'.

Implementing The Examples Of Slicing In A Code.

Example:

```
import pandas as pd
dictObj = {'EmpId' : ['E01','E02','E03','E04'],
           'EmpName' : ['Sam','Emma','Krish','Jack'],
           'Department' : ['IT','IT','HR','Accounts']}
df=pd.DataFrame(dictObj, index=['First','Second','Third','Fourth'])

print(df.loc [ : ]) #Printing the entire DataFrame using loc
print('\n',df.loc [ : 'Third' , : ]) #Printing the first three rows using loc
print('\n',df.iloc [ 1:3 , : ]) #Printing the second and third rows using iloc
print('\n',df.loc [ : , 'EmpName' : ]) #Printing the EmpName column till the end
print('\n',df.iloc [ : , : 2 ]) #Printing all rows till the second column using iloc
print('\n',df.iloc [ 1:3 , 1:3 ]) #Printing the second & third rows till the second column
```

	EmpId	EmpName	Department
First	E01	Sam	IT
Second	E02	Emma	IT
Third	E03	Krish	HR
Fourth	E04	Jack	Accounts

	EmpId	EmpName	Department
First	E01	Sam	IT
Second	E02	Emma	IT
Third	E03	Krish	HR

	EmpId	EmpName	Department
Second	E02	Emma	IT
Third	E03	Krish	HR

	EmpName	Department
First	Sam	IT
Second	Emma	IT
Third	Krish	HR
Fourth	Jack	Accounts

	EmpId	EmpName
First	E01	Sam
Second	E02	Emma
Third	E03	Krish
Fourth	E04	Jack

	EmpName	Department
Second	Emma	IT
Third	Krish	HR

MODIFYING DATA IN DATAFRAME

Updating a Pandas DataFrame Row There are several ways to update a row in a Pandas DataFrame. Here are some common methods: Using `.loc[]` or `.iloc[]` To update a Pandas DataFrame row with new values, we first need to locate the row we want to update. This can be done using the `loc` or `iloc` methods, depending on whether we want to locate the row by label or integer index. Once we have located the row, we can update the values of the row using the assignment operator `=`. We simply need to assign the new values to the row using the column names as keys.

Example:

Let's say we have a Pandas DataFrame with three columns: Name, Age, and Designation. We want to update the row with Name = "John" and set the Age to 32 and Designation to "Full Stack Developer".

```

import pandas as pd

# create a sample dataframe
df = pd.DataFrame({
    'Name': ['John', 'Mary', 'Peter'],
    'Age': [30, 25, 35],
    'Designation': ['Back-end Developer', 'Software Engineer', 'HR Manager']})

# print the dataframe
print(df)

# locate the row to update
row_index = df.loc[df['Name'] == 'John'].index[0]

# update the row with new values
df.loc[row_index, 'Age'] = 32
df.loc[row_index, 'Designation'] = 'Full Stack Developer'

# print the updated dataframe
print('\nUpdated DataFrame:')
print(df)

```

```

   Name  Age  Designation
0  John   30  Back-end Developer
1  Mary   25  Software Engineer
2  Peter   35    HR Manager

Updated DataFrame:
   Name  Age  Designation
0  John   32  Full Stack Developer
1  Mary   25  Software Engineer
2  Peter   35    HR Manager

```

IN CLASS CHALLENGES

CHALLENGE1 - Student Performance Analysis :

Picture this: You have this amazing list, kind of like a magical scroll, a DataFrame, listing each student's name alongside their math and science scores. Now, imagine one day Alice comes to you and says, 'Hey, I think my math score should be higher!' How would you use your wizardry to update her score from 85 to 92? And then, let's say Bob feels like he deserves a higher science score. How would you make his score from 85 to 88? Also let's find out who is the top performer. How would you use your coding skills to find out who scored higher than 90 in math and higher than 85 in science? With the power of coding & with pandas DataFrame we can make all of these magical things happen!

INCLASS CHALLENGE 2- Educational Toys:

SoHey kiddo! So, you know how toys can be super fun, right? Well, did you know that some toys can also help us learn cool stuff? Like Puzzles and Building Blocks - they're not just for play, they're educational too! Now, imagine we have a big list of toys, with all their Names, Age Group, Price and Educational Value. In this list the Educational value of Puzzles and Building Blocks is marked as No. But, we need to mark them Yes under Educational column. That's where the fun coding part comes in! We're going to use a special coding trick called `.loc` to find those special toys - the puzzles and building blocks - and then we'll tell the computer to label them as Educational. So, are you excited to jump into this adventure of coding and toys with me?

HOME TASK

HOME TASK1-Classroom RosterMagical Book:

Imagine you're organizing a classroom roster for your school. You want to create a list of students with their names, ages, and favorite subjects. How would you write code to accomplish this task using Python? Consider using a library like Pandas to help manage the data. Think about how you would display the roster, access specific information about students, and make changes to the roster, like updating ages or changing favorite subjects. Try writing code that creates the roster and then demonstrates how you can access and modify the information within it.

HOMETASK 2 - Animals Data :

Hey there! Are you ready for a magical adventure? Imagine you're in charge of a magical creature zoo, filled with incredible creatures like dragons, unicorns, and mermaids! But wait, some of their habitats and diets need updating to make sure they're happy and healthy. How would you use your coding magic to update the habitats of dragons to 'Volcanic mountains', give unicorns a special diet of 'Herbivore, Magic', and ensure that mermaids feast on 'Seafood'? Dive into the world of pandas DataFrame and let's make our magical zoo even more enchanting!

WHAT'S NEXT?

Adding/removing rows and columns

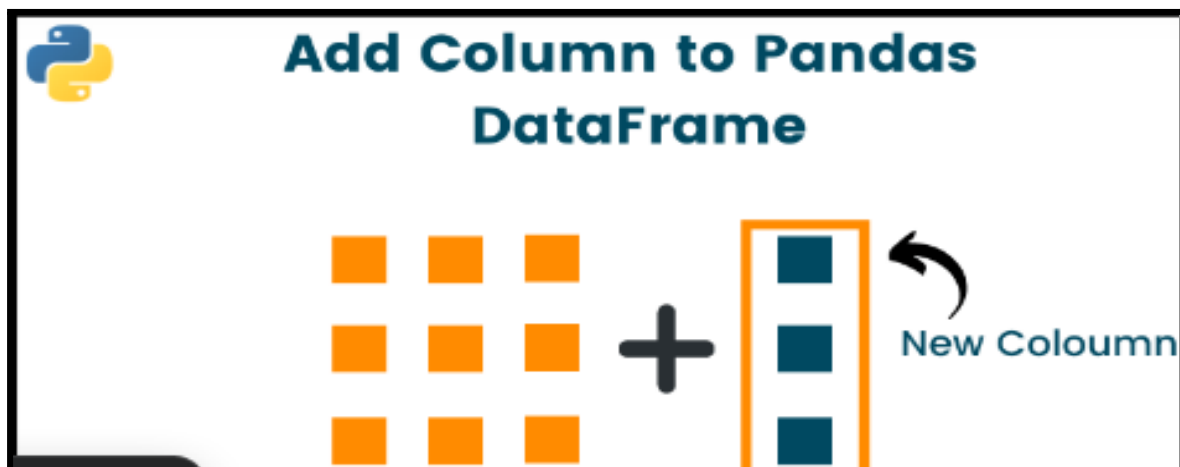
LESSON 29

Adding/removing rows and columns

LEARNING CONTENT

COLUMN TO PANDAS DATAFRAME

First, let's create an example DataFrame that we'll use to explain a few ideas related to adding columns to Pandas DataFrames.



Suppose we need to add a new column named 'colC' containing the values 'a', 'b', and 'c' for the indices 0, 1, and 2, respectively. How will we do it? Let's see!

```
# importing pandas library
import pandas as pd

# creating the DataFrame
df = pd.DataFrame({'colA':[True, False, False], 'colB': [1, 2, 3],})

print(df)
```

	colA	colB
0	True	1
1	False	2
2	False	3

1) Using the simple assignment

You can add a new column to Dataframe by simply giving your Series data to the existing frame. It is one of the easiest and efficient methods widely used by python programmers. Note that the name of the new column should be enclosed with single quotes inside the square brackets.

SYNTAX: `df['New_Column'] = [value1, value2, value3, A...]`

Note that in most circumstances, the above will work if the new column's indices match those of the DataFrame; or else, NaN values will be given to missing indices

Example :

```
import pandas as pd

df = pd.DataFrame({'colA':[True, False, False], 'colB': [1, 2, 3],})
print('Original DataFrame:\n', df)

# adding a new column
df['colC'] = ['a','b','c']
print('\nDataFrame after adding a new column:\n', df)
```

	colA	colB
0	True	1
1	False	2
2	False	3

	colA	colB	colC
0	True	1	a
1	False	2	b
2	False	3	c

2) Using loc[] method

The method `loc[]` is a commonly used approach to add a new column to

a pandas DataFrame using the .loc[] accessor. SYNTAX: df.loc[:, 'New_Column'] = [value1, value2, value3, ...]

```
import pandas as pd

df = pd.DataFrame({'colA':[True, False, False], 'colB': [1, 2, 3],})
print('Original DataFrame:\n', df)

# adding a new column using .loc method
df.loc[:, 'colC'] = ['x', 'y', 'z']
print('\nDataFrame after adding a new column:\n', df)
```

```
Original DataFrame:
   colA  colB
0   True     1
1  False     2
2  False     3

DataFrame after adding a new column:
   colA  colB colC
0   True     1    x
1  False     2    y
2  False     3    z
```

3) Using assign() method

SYNTAX: df = df.assign(New_Column=[value1, value2, value3, ...])

Using the pandas.DataFrame.assign() method, you can insert multiple columns in a DataFrame, ignoring the index of a column to be added, or modify the values of existing columns. The method returns a new DataFrame object with all of the original columns as well as the additional(newly added) ones. Note that the index of the new columns will be ignored as well as, all the current columns will be overwritten if they are re-assigned.

Example :


```
import pandas as pd

df = pd.DataFrame({'colA':[True, False, False], 'colB': [1, 2, 3],})
print('Original DataFrame:\n', df)

# adding a new column
e = pd.Series([1.0, 3.0, 2.0], index=[0, 2, 1])
s = pd.Series(['a', 'b', 'c'], index=[0, 1, 2])
df=df.assign(colC=s.values, colB=e.values) # Reassign the result back to df
print('\nDataFrame after adding a new column:\n', df)
```

```
Original DataFrame:
   colA  colB
0  True     1
1 False     2
2 False     3

DataFrame after adding a new column:
   colA  colB colC
0  True   1.0   a
1 False   3.0   b
2 False   2.0   c
```

Above lines of code create two Series (e and s) and add them as new columns (colC and colB, respectively) to the DataFrame df. The values from the Series are aligned with DataFrame indices during the assignment.

The assign() method returns a new DataFrame with the assigned columns, rather than modifying the original DataFrame in place. Therefore, you need to capture the returned DataFrame after using assign() or reassign it to df if you want to see the changes.

4) Using insert() method

You can also use the method pandas.DataFrame.insert() for adding columns to DataFrame. This method comes in handy when you need to add a column at a specific position or index.

SYNTAX: df.insert(loc, 'New_Column', [value1, value2, value3, ...])

loc: The index position where the new column should be inserted. This is an integer value.

Example :

```
import pandas as pd

df = pd.DataFrame({'colA':[True, False, False], 'colB': [1, 2, 3],})
print('Original DataFrame:\n', df)

# adding a new column using insert()
df.insert(len(df.columns), 'colC', ['a', 'b', 'c'])
print('\nDataFrame after adding a new column:\n', df)
```

```
Original DataFrame:
   colA  colB
0  True     1
1 False     2
2 False     3

DataFrame after adding a new column:
   colA  colB colC
0  True     1    a
1 False     2    b
2 False     3    c
```

- In the above code insert() method is used to insert a new column ('colC') into the DataFrame df.
- The position parameter specifies the index position at which the new column should be inserted. In this case, len(df.columns) is used to insert the new column at the end.

Now, if you want to add a column 'colC' in between two columns - 'colA' and 'colB'.

```
import pandas as pd

df = pd.DataFrame({'colA':[True, False, False], 'colB': [1, 2, 3],})
print('Original DataFrame:\n', df)

# adding a new column using insert()
df.insert(1, 'colC', ['a', 'b', 'c'])
print('\nDataFrame after adding a new column:\n', df)
```

```
Original DataFrame:
   colA  colB
0  True     1
1 False     2
2 False     3

DataFrame after adding a new column:
   colA colC  colB
0  True    a     1
1 False    b     2
2 False    c     3
```

Note that the `insert()` method cannot be used to add the column with a similar name. By default, a `ValueError` will be thrown when a column already exists in the `DataFrame`. Nevertheless, the `DataFrame` will allow having two columns with the same name if you pass the command `allow_duplicates=True` to the `insert()` method.

```
import pandas as pd

df = pd.DataFrame({'colA':[True, False, False], 'colB': [1, 2, 3],})
print('Original DataFrame:\n', df)

# adding two columns with same name using insert
df.insert(1, 'colC',['a','b','c'])
df.insert(1, 'colC',['a','b','c'], allow_duplicates=True)
print('\nDataFrame after adding a new column:\n', df)
```

```
Original DataFrame:
   colA  colB
0  True     1
1 False     2
2 False     3

DataFrame after adding two columns with same name:
   colA  colC  colC  colB
0  True    a     a     1
1 False    b     b     2
2 False    c     c     3
```

There are various scenarios where adding rows becomes necessary, such as appending new records to an existing dataset or incorporating new observations into an analysis. Therefore Pandas provides a lot of techniques for adding or inserting a new row to an existing `DataFrame`.

Adding Single Row in Pandas DataFrame Using DataFrame.loc

- 1) One way to add a single row to a DataFrame is by using the DataFrame.loc method. This method allows us to specify the position where the new row should be inserted.
- 2) To add a row at the end of the DataFrame, we can simply utilize the length of the index of the DataFrame to determine the position.

```
import pandas as pd

# Create a DataFrame
df = pd.DataFrame({'Name':['Martha', 'Tim', 'Rob', 'Georgia'],
                  'Maths':[87, 91, 97, 95], 'Science':[83, 99, 84, 76] })

# Display the DataFrame
print('Original DataFrame:\n', df)

# Add a new row
df.loc[len(df.index)] = ['Amy', 89, 93]

# Display the New DataFrame
print('\nNew DataFrame:\n', df)
```

```
Original DataFrame:
   Name  Maths  Science
0  Martha    87      83
1    Tim    91      99
2    Rob    97      84
3 Georgia    95      76

New DataFrame:
   Name  Maths  Science
0  Martha    87      83
1    Tim    91      99
2    Rob    97      84
3 Georgia    95      76
4    Amy    89      93
```

Inserting a Row at a Specific Index

Adding a row at a specific index is a bit different. As shown in the previous example, we need to use the loc accessor. However, inserting a row at a given index will only overwrite this. What we can do instead is pass in a value close to where we want to insert the new row.

For example, if we have current indices from 0-3 and we want to insert a new row at index 2, we can simply assign it using index 1.5. Let's see how this works:

```
import pandas as pd

# Create a DataFrame
df = pd.DataFrame({'Name': ['Martha', 'Tim', 'Rob', 'Georgia'],
                  'Maths': [87, 91, 97, 95], 'Science': [83, 99, 84, 76]})
# Display the DataFrame
print('Original DataFrame:\n', df)

# Inserting a Row at a Specific Index
df.loc[1.5] = ['Jane', 92, 93]
df = df.sort_index().reset_index(drop=True)

# Display the New DataFrame
print('\nNew DataFrame:\n', df)
```

```
Original DataFrame:
   Name  Maths  Science
0  Martha    87      83
1    Tim    91      99
2    Rob    97      84
3  Georgia    95      76

New DataFrame:
   Name  Maths  Science
0  Martha    87      83
1    Tim    91      99
2   Jane    92      93
3    Rob    97      84
4  Georgia    95      76
```

Insert a Row at the Top using pd.concat() function

Adding a row to the top of a Pandas DataFrame is quite simple. Here we append the larger DataFrame to the new row. However, we must first create a DataFrame. We can do this using the `pd.DataFrame()` class. Let's take a look:

```
import pandas as pd

# Create a DataFrame
df = pd.DataFrame({'Name': ['Martha', 'Tim', 'Rob', 'Georgia'],
                  'Maths': [87, 91, 97, 95], 'Science': [83, 99, 84, 76]})
print('Original DataFrame:\n', df)

# Add a new record at the top of a DataFrame
new_record = pd.DataFrame([['Jane', 92, 93]], columns=df.columns)
df = pd.concat([new_record, df], ignore_index=True)
print('\nNew DataFrame:\n', df)
```

```
Original DataFrame:
   Name  Maths  Science
0  Martha    87      83
1    Tim    91      99
2    Rob    97      84
3  Georgia    95      76

New DataFrame:
   Name  Maths  Science
0   Jane    92      93
1  Martha    87      83
2    Tim    91      99
3    Rob    97      84
4  Georgia    95      76
```

The `pd.concat()` function is used to concatenate the new record DataFrame (`new_record`) with the original DataFrame (`df`). By setting `ignore_index=True`, Pandas will reset the index of the resulting DataFrame, ensuring that the new DataFrame has a continuous index without any gaps. By concatenating `new_record` with `df`, the new record is added to the top of the DataFrame, effectively becoming the first row.

Insert Multiple Rows in a Pandas DataFrame

Adding multiple rows to a Pandas DataFrame is the same process as adding a single row using the `pandas.concat` function. This `concat` method involves creating a new DataFrame containing all the rows that need to be added, and then concatenating it with the original DataFrame.

For example, if we add items using a dictionary, then we can simply add them as a list of dictionaries.

```
import pandas as pd

# Create a DataFrame
df = pd.DataFrame({'Name': ['Martha', 'Tim', 'Rob', 'Georgia'],
                   'Maths': [87, 91, 97, 95], 'Science': [83, 99, 84, 76]})
print('Original DataFrame:\n', df)

# Adding multiple rows to a Pandas DataFrame
new_rows = pd.DataFrame([{'Name': 'Jane', 'Maths': 92, 'Science': 93},
                          {'Name': 'Sam', 'Maths': 88, 'Science': 89}])
df = pd.concat([df, new_rows], ignore_index=True)
print('\nNew DataFrame:\n', df)
```

Removing Column/Row using drop() method

The drop() method in pandas is used to remove rows or columns from a DataFrame. It allows you to specify the index labels or column labels that you want to remove.

By specifying the column axis (axis='columns'), the drop() method removes the specified column. By specifying the row axis (axis='index'), the drop() method removes the specified row.

Syntax: dataframe.drop(labels, axis, index, columns, level, inplace., errors)

Parameter	Value	Description
labels		Optional, The labels or indexes to drop. If more than one, specify them in a list.
axis	01'index"columns'	Optional, Which axis to check, default 0.
index	StringList	Optional, Specifies the name of the rows to drop. Can be used instead of the labels parameter.
columns	StringList	Optional, Specifies the name of the columns to drop. Can be used instead of the labels parameter.
level	Numberlevel name	Optional, default None. Specifies which level (in a hierarchical multi index) to check along
inplace	TrueFalse	Optional, default False. If True: the removing is done on the current DataFrame. If False: returns a copy where the removing is done.
errors	'ignore"raise'	Optional, default 'ignore'. Specifies whether to ignore errors or not

EXAMPLE CODE:


```
import pandas as pd

data = {"name": ["Sally", "Mary", "John"],
        "age": [50, 40, 30],
        "qualified": [True, False, False]}

df = pd.DataFrame(data)
print('Original DataFrame\n', df)

df.drop("age", axis='columns', inplace=True) #removing column
df.drop(1, axis='index', inplace=True) #removing row
print('\nDataFrame after removing a column and a row\n', df)
```

```
Original DataFrame
   name  age  qualified
0  Sally   50        True
1  Mary   40        False
2  John   30        False

DataFrame after removing a column and a row
   name  qualified
0  Sally        True
2  John        False
```

IN CLASS CHALLENGES

INCLASS CHALLENGE1 - Employee Update:

Imagine you're managing a super cool team of employees, and you keep track of all your team members in a special roster. You've got this dataset called 'employees.csv' where you record the details of each team member. Link for the Data set: [Now, picture this:](#) You recently recruited two new members to your team, Ronald and Alice. Ronald, a valiant warrior, joined your team on January 5, 2020, with a salary of 35,000. He's specialized in handling all the magical gadgets and gizmos, so he's part of the 'IT' squad. Alice, on the other hand, is a brilliant sorceress who joined your team on February 10, 2019, with a salary of 42,000, and she's joining the 'HR' team.

To keep 'The employees' dataset up-to-date, you need to add these two new members to the top of the DataFrame. However, after adding them, you realize that the 'Date Joined' column isn't really necessary. How would you update 'The employees DataFrame' to reflect these changes? Write down the code to update 'The employees dataset accordingly. Ready for the adventure?

INCLASS CHALLENGE 2- Fruits List:

Imagine you want to create a special list just for your favorite fruits! You'll use a computer program to make this list. First, you'll need to use a special tool called Python. In Python, there's a magical tool called 'Pandas' that helps you create lists. Your task is to write a program that lets you add your favorite fruits to the list and see what's on the list whenever you want.

Here are some steps to guide you:

- Start by creating an empty list for your fruits. We'll call it the 'Fruits list'.
- Add your favorite fruit to the list along with how many you have. For example, if you have 2 apples, you'll write 'Apples' and '2'.
- Then, create a way to add more fruits to the list whenever you want.
- Finally, make sure you can see what fruits are on your list anytime you ask.

Ready to give it a try? Think about how you would ask someone to add a fruit to the list, and how you would show them the list when they want to see it. Let's get coding!

HOME TASK

HOME TASK1-Months In A Year:

Hey buddy, your friend has sent you a special file named 'month.csv'. Inside this file, there is data that shows information about each month of the year! But, your friend accidentally included some extra information that we don't need. DataSet Link: Your mission is to help clean up unnecessary data by writing a special code using Python!

Here's what you need to do:

- Start by loading the 'month.csv' file into a magical tool called a DataFrame. Think of it as a table where each row is a month and each column is a piece of information about that month.
- Next, you'll look at the table and find the columns named 'Numeric' and 'Numeric-2'. These are the extra bits we don't need.
- Then, you'll write a special command to tell Python to remove these two columns from the table.
- Finally, you'll check the table again to make sure those extra bits are gone!

Imagine being the hero who cleans up the list and makes it perfect for your friend to use again! Are you ready to dive into Python and make the magic happen?

HOMETASK 2 - ShopList :

Hey there! Imagine you and your friend are planning a super fun cooking adventure together. You're going to make the most delicious dishes ever, but first, you need to make sure you have all the ingredients. How about turning this into a coding challenge? You: Hey, I have this cool idea! Let's create a magical grocery list using Python to make sure we don't forget anything when we go shopping! Friend: Wow, that sounds awesome! But how do we do that? You: It's easy! We can use Python's pandas library to create a DataFrame, which is like a fancy table. We'll start with an empty grocery list and then add items as we discuss them, their quantities, and even include whether they're fruits or vegetables. Friend: Cool! What about prices? You: We can add prices too, but let's also include how to delete the price column when we're done shopping. Friend: I am excited! Let's dive into coding and create our magical grocery list! You: Absolutely! Let's get started!

WHAT'S NEXT?

DEALING WITH MISSING DATA

LESSON 30

DEALING WITH MISSING DATA

LEARNING CONTENT

Missing Data In Pandas

Missing Data can occur when no information is provided for one or more items or for a whole unit. Missing Data is a very big problem in a real-life scenarios. Missing Data can also refer to as NA(Not Available) values in pandas. In DataFrame sometimes many datasets simply arrive with missing data, either because it exists and was not collected or it never existed.

For Example, Suppose different users being surveyed may choose not to share their income, some users may choose not to share the address in this way many datasets went missing.

In Pandas missing data is represented by two value:

- **None** : None is a Python singleton object that is often used for missing data in Python code.

- NaN : NaN (an acronym for Not a Number), is a special floating-point value recognized by all systems that use the standard IEEE floating-point representation

Pandas treat None and NaN as essentially interchangeable for indicating missing or null values. To facilitate this convention, there are several useful functions for detecting, removing, and replacing null values in Pandas DataFrame :

- isnull()
- notnull()
- dropna()
- fillna()
- replace()
- interpolate()

In order to check missing values in Pandas DataFrame, we use a function isnull() and notnull(). Both function help in checking whether a value is NaN or not. These function can also be used in Pandas Series in order to find null values in a series.

Checking for missing values using isnull()

In order to check null values in Pandas DataFrame, we use isnull() function this function return data frame of Boolean values which are True for NaN values.

Checking for missing values using notnull()

In order to check null values in Pandas Dataframe, we use notnull() function this function return data frame of Boolean values which are False for NaN values.

Using isnull() and notnull()

```
import pandas as pd

# dictionary of lists
dict = {'First Score':[100, 90, None, 95],
        'Second Score': [30, 45, 56, None],
        'Third Score':[None, 40, 80, 98]}

# creating a dataframe from list
df = pd.DataFrame(dict)

df.isnull() # using isnull() function
print(df) # prints the dataframe
print('DataFrame using isnull() function')
print(df.isnull()) # prints the dataframe boolean values

df.notnull() # using notnull() function
print('\n',df) # prints the dataframe
print('DataFrame using notnull() function')
print(df.notnull()) # print the dataframe boolean values
```

	First Score	Second Score	Third Score
0	100.0	30.0	NaN
1	90.0	45.0	40.0
2	NaN	56.0	80.0
3	95.0	NaN	98.0

DataFrame using isnull() function

	First Score	Second Score	Third Score
0	False	False	True
1	False	False	False
2	True	False	False
3	False	True	False

	First Score	Second Score	Third Score
0	100.0	30.0	NaN
1	90.0	45.0	40.0
2	NaN	56.0	80.0
3	95.0	NaN	98.0

DataFrame using notnull() function

	First Score	Second Score	Third Score
0	True	True	False
1	True	True	True
2	False	True	True
3	True	False	True

Checking for missing values in a “csv file” using isnull() and notnull()

```
import pandas as pd

# making data frame from csv file
data = pd.read_csv("employeedata.csv")

# creating bool series True for NaN values in Gender column
bool_series = pd.isnull(data["Gender"])
print(pd.isnull(data["Gender"].head(5)))
data[bool_series] # filtering dataframe only with Gender = NaN
# printing first 5 rows where Gender = NaN
print(data[bool_series].head(5), '\n')

# creating bool series False for NaN values in Gender column
bool_series = pd.notnull(data["Gender"])
print(pd.notnull(data["Gender"].head(5)))
data[bool_series] # filtering dataframe only with Gender != NaN
# printing first 5 rows where Gender != NaN
print(data[bool_series].head(5))
```

```
0    False
1    False
2    False
3     True
4     True
Name: Gender, dtype: bool
   Name Gender  Salary      Team
3  Jerry   NaN  138705    Finance
4  Larry   NaN  101004  Client Services
20  Lois   NaN   64714      Legal
22 Joshua   NaN   90816  Client Services
27  Scott   NaN  122367      Legal

0     True
1     True
2     True
3    False
4    False
Name: Gender, dtype: bool
   Name Gender  Salary      Team
0  Douglas  Male   97308  Marketing
1   Thomas  Male   61933        NaN
2   Maria  Female  130590    Finance
5  Dennis  Male   115163      Legal
6   Ruby   Female   65476    Product
```

Filling missing values using fillna(), replace() and interpolate()

fillna() is used to fill NaN (missing) values with a specified value or using a predefined method like forward fill or backward fill. replace() is used to replace specific values in a DataFrame or Series with other values.

fillna() is primarily used for handling missing data, while replace() is more versatile and can be used for various types of value replacement. The interpolate() function in Pandas is used to fill NaN (missing) values in a DataFrame or Series by interpolating between existing values. This function provides several methods for interpolation such as linear, polynomial, spline, etc., which you can specify using the method parameter.

EXAMPLE: Filling null values with a single value using fillna()

```
import pandas as pd

# dictionary of lists
dict = {'First Score':[100, 90, None, 95],
        'Second Score': [30, 45, 56, None],
        'Third Score':[None, 40, 80, 98]}

# creating a dataframe from dictionary
df = pd.DataFrame(dict)
print(df)

# filling missing value using fillna()
df.fillna(5, inplace = True)
print('\nDataFrame after filling missing value with 5\n', df)
```

	First Score	Second Score	Third Score
0	100.0	30.0	NaN
1	90.0	45.0	40.0
2	NaN	56.0	80.0
3	95.0	NaN	98.0

DataFrame after filling missing value with 5			
	First Score	Second Score	Third Score
0	100.0	30.0	5.0
1	90.0	45.0	40.0
2	5.0	56.0	80.0
3	95.0	5.0	98.0

Filling NaN values using predefined methods like Forward Fill[ffill] and Backward Fill[bfill]

- Forward Fill (ffill): Propagates the last valid observation forward to fill missing values.
- Backward Fill (bfill): Propagates the next valid observation backward to fill missing values.


```
import pandas as pd

# Create a sample DataFrame with missing values
data = {'First Score': [100, 90, None, 95],
        'Second Score': [30, 45, 56, None],
        'Third Score': [None, 40, 80, 98]}
df = pd.DataFrame(data)
print('Original DataFrame:')
print(df)

# Forward fill missing values
df_filled_ffill = df.fillna(method='ffill')

print("\nDataFrame with forward fill:")
print(df_filled_ffill)

# Backward fill missing values
df_filled_bfill = df.fillna(method='bfill')

print("\nDataFrame with backward fill:")
print(df_filled_bfill)
```

Original DataFrame:

	First Score	Second Score	Third Score
0	100.0	30.0	NaN
1	90.0	45.0	40.0
2	NaN	56.0	80.0
3	95.0	NaN	98.0

DataFrame with forward fill:

	First Score	Second Score	Third Score
0	100.0	30.0	NaN
1	90.0	45.0	40.0
2	90.0	56.0	80.0
3	95.0	56.0	98.0

DataFrame with backward fill:

	First Score	Second Score	Third Score
0	100.0	30.0	40.0
1	90.0	45.0	40.0
2	95.0	56.0	80.0
3	95.0	NaN	98.0

Filling null values in a CSV File using fillna()

filling all the null values in Gender column with “No Gender” using fillna()

```
import pandas as pd

# making data frame from csv file
df = pd.read_csv("employeedata.csv")
print(df) # printing the data frame

# filling a null values using fillna()
df.fillna({'Gender': 'No Gender'}, inplace = True)
# printing the data frame after filling null values
print('\nDataFrame after filling null values with No Gender:\n', df)
```

```

0      Name  Gender  salary      Team
1  Douglas   Male   97308   Marketing
2  Thomas   Male   61933         NaN
3  Maria    Female  130590   Finance
4  Jerry     NaN   138705   Finance
5  Larry     NaN   101004  Client Services
..      ...     ...     ...     ...
194  Irene   Female  131038  Distribution
195  Ronald   Male  121068   Product
196  Steven   Male   62719  Client Services
197  Carolyn  Female   69268  Client Services
198  Maria    Female   36067   Product

[199 rows x 4 columns]

DataFrame after filling null values with No Gender:
   Name  Gender  Salary      Team
0  Douglas   Male   97308   Marketing
1  Thomas   Male   61933         NaN
2  Maria    Female  130590   Finance
3  Jerry   No Gender  138705   Finance
4  Larry   No Gender  101004  Client Services
..      ...     ...     ...     ...
194  Irene   Female  131038  Distribution
195  Ronald   Male  121068   Product
196  Steven   Male   62719  Client Services
197  Carolyn  Female   69268  Client Services
198  Maria    Female   36067   Product

[199 rows x 4 columns]

```

Filling null values in a CSV File using replace() method

Here we are replacing all NaN values in the data frame with value as 'xyz' using replace() method

```

import pandas as pd

# making data frame from csv file
df = pd.read_csv("employeedata.csv")
print('Initial DataFrame:\n', df)

# replace NaN values in DataFrame with 'xyz'
df = df.replace(to_replace=pd.NA, value='xyz')
print('\nDataFrame after replacing NaN values:\n', df)

```

```
Initial DataFrame:
   Name  Gender  Salary  Team
0  Douglas  Male  97308  Marketing
1  Thomas  Male  61933  NaN
2  Maria  Female  130590  Finance
3  Jerry  NaN  138705  Finance
4  Larry  NaN  101004  Client Services
..  ...  ...  ...  ...
194  Irene  Female  131038  Distribution
195  Ronald  Male  121068  Product
196  Steven  Male  62719  Client Services
197  Carolyn  Female  69268  Client Services
198  Maria  Female  36067  Product
[199 rows x 4 columns]

DataFrame after replacing NaN values:
   Name  Gender  Salary  Team
0  Douglas  Male  97308  Marketing
1  Thomas  Male  61933  xyz
2  Maria  Female  130590  Finance
3  Jerry  xyz  138705  Finance
4  Larry  xyz  101004  Client Services
..  ...  ...  ...  ...
194  Irene  Female  131038  Distribution
195  Ronald  Male  121068  Product
196  Steven  Male  62719  Client Services
197  Carolyn  Female  69268  Client Services
198  Maria  Female  36067  Product
[199 rows x 4 columns]
```

Using interpolate() function to fill the missing values using linear method.

NOTE: Here, the missing values in columns 'A' and 'B' are filled by linear interpolation. You can explore other methods such as 'polynomial', 'spline' etc. by changing the method parameter accordingly.

Linear interpolation is a method of estimating values that lie between two known values.

For the missing value at index 2: Since it lies between the known values 2 and 4, linear interpolation estimates the missing value to be the midpoint between these two values, which is 3.

Example :

```
import pandas as pd

# Create a sample DataFrame with missing values
data = {'A': [1, 2, None, 4, 5],
        'B': [10, 20, 30, None, 50]}
df = pd.DataFrame(data)
print(df)

# Interpolate missing values using linear method
df_interpolated = df.interpolate(method='linear')
print('\nAfter interpolating missing values:\n',df_interpolated)
```

	A	B
0	1.0	10.0
1	2.0	20.0
2	NaN	30.0
3	4.0	NaN
4	5.0	50.0

After interpolating missing values:

	A	B
0	1.0	10.0
1	2.0	20.0
2	3.0	30.0
3	4.0	40.0
4	5.0	50.0

Dropping missing values using dropna()

The `dropna()` function in Pandas is used to remove missing (NaN) values from a DataFrame or Series. It provides flexibility in how you want to handle missing data by allowing you to specify different parameters.

Basic Example of how to use `dropna()`:

```
import pandas as pd

# Create a sample DataFrame with missing values
data = {'A': [1, 2, None, 4, 5],
        'B': [10, None, 30, None, 50]}
df = pd.DataFrame(data)
print(df)

# Drop rows with any NaN value
df_dropped = df.dropna()
print('\nDropped rows with any NaN value:')
print(df_dropped)
```

```
   A      B
0  1.0  10.0
1  2.0   NaN
2  NaN  30.0
3  4.0   NaN
4  5.0  50.0

Dropped rows with any NaN value:
   A      B
0  1.0  10.0
4  5.0  50.0
```

By default, `dropna()` removes any row containing at least one NaN value. However, you can customize its behavior using parameters such as `axis`, `how`, `thresh`, and `subset`.

1. `axis`: Specifies whether to drop rows (`axis=0`) or columns (`axis=1`) containing missing values. By default, it's set to 0.
2. `how`: Specifies how to determine which rows or columns to drop. Options include 'any' (drops if any NaN values are present) and 'all' (drops if all values are NaN). The default is 'any'.
3. `thresh`: Specifies the minimum number of non-NaN values required for a row or column to be kept. Rows or columns with fewer non-NaN values than `thresh` will be dropped.
4. `subset`: Specifies the subset of columns or rows to consider when dropping NaN values.

Example: demonstrating the use of `dropna()` parameters:

These options provide flexibility in managing missing data based on your specific requirements and the structure of your DataFrame.

```

import pandas as pd

# Create a sample DataFrame with missing values
data = {'A': [1, 2, None, 4, 5],
        'B': [10, None, None, None, 50]}
df = pd.DataFrame(data)
print('Original DataFrame\n',df)

# Drop rows with all NaN values
df_dropped_all = df.dropna(how='all')

# Drop rows with at least 2 non-NaN values
df_dropped_thresh = df.dropna(thresh=2)

# Drop rows where NaN appears in column 'B'
df_dropped_subset = df.dropna(subset=['B'])

print('\nDropped rows with all NaN values\n',df_dropped_all)
print('\nDropped rows with atleast 2 non-NaN values\n',df_dropped_thresh)
print('\nDropped rows where NaN appears in column B\n',df_dropped_subset)

```

```

Original DataFrame
   A  B
0  1  10
1  2  NaN
2  NaN NaN
3  4  NaN
4  5  50

Dropped rows with all NaN values
   A  B
0  1  10
1  2  NaN
3  4  NaN
4  5  50

Dropped rows with atleast 2 non-NaN values
   A  B
0  1  10
4  5  50

Dropped rows where NaN appears in column B
   A  B
0  1  10
4  5  50

```

IN CLASS CHALLENGES

IN CLASS CHALLENGE1 - Weather Report:

Link for the DataSet:

on this scenario with columns: Name, Favorite Food, Activity, Bringing Games? Now, Charlie has confirmed that he'll bring games to the picnic. So, in the Bringing Games column, fill the missing values with Yes. [HINT: use fillna() to fill missing values] But uh-oh, there's a chance Charlie might not make it to the picnic trip, so now you'll need to remove his plan from the list. [HINT: use drop() method to remove the row 'Charlie'] Your task is to create a Python DataFrame reflecting this scenario and then update Charlie's confirmation and potential absence from the picnic.

HOME TASK

HOME TASK1-World Famous Locations:

Hey there, You have a list of world-famous locations stored in a file called "locations.csv" and this file consists of two columns: Location and Latitude/Longitude. Some of the data might be missing due to old or damaged records.

Link to the
file:

HOMETASK 2 - Coins Collection :

Imagine you're tracking the number of coins collected by a group of kids over several days. Now, create a DataFrame where each row represents a different day, and each column represents a different kid's collection. However, some kids might have missed collecting coins on certain days, leading to missing data in the DataFrame. How would you handle this scenario using Python? Think about using Pandas DataFrame methods to check for missing data and then fill in those missing values. Specifically, you might want to use `notnull()` to check for missing data and `interpolate()` with the `method='linear'` parameter to fill in the missing values using linear interpolation. Can you write Python code to implement this solution?

WHAT'S NEXT?

Combining data frames

LESSON 31

COMBINING DATA FRAMES

LEARNING CONTENT

Combining DataFrames in Python, particularly with Pandas, is a powerful technique for data manipulation and analysis.

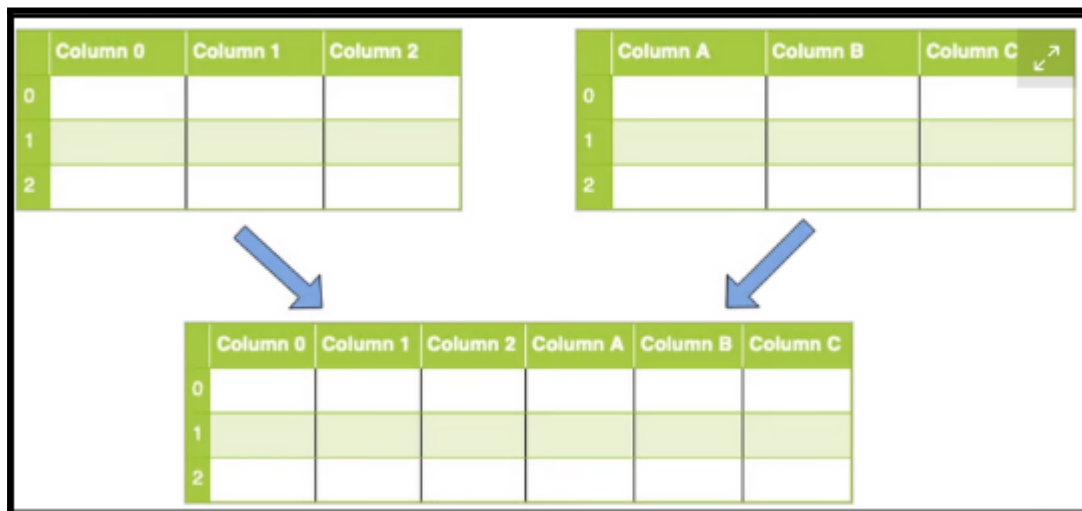
Data processing becomes critical when training a model. We occasionally need to restructure and add new data to the datasets to increase the efficiency of the data. We'll look at how to combine multiple datasets and merge multiple datasets with the same and different column names . We'll use the pandas library's following functions to carry out these operations

- ☐ `pandas.concat()`
- ☐ `pandas.merge()`
- ☐ `pandas.DataFrame.join()`

pandas concat(): Combining Data Across Rows or Columns

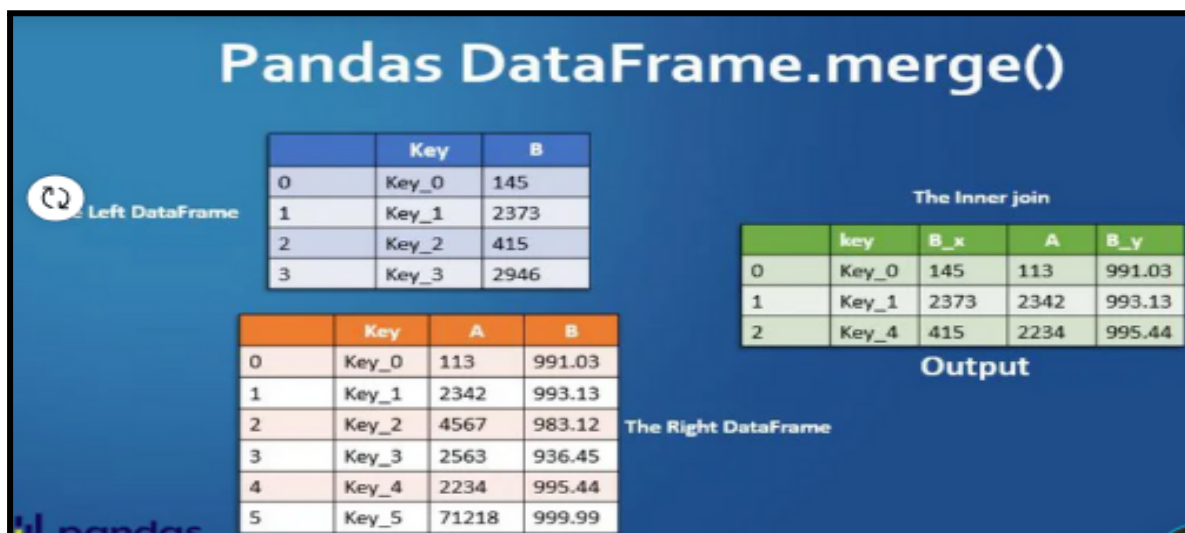
With concatenation, your datasets are just stitched together along an axis — either the row axis or column axis. Visually, a concatenation with no parameters along rows would look like this:

What if you wanted to perform a concatenation along columns instead?

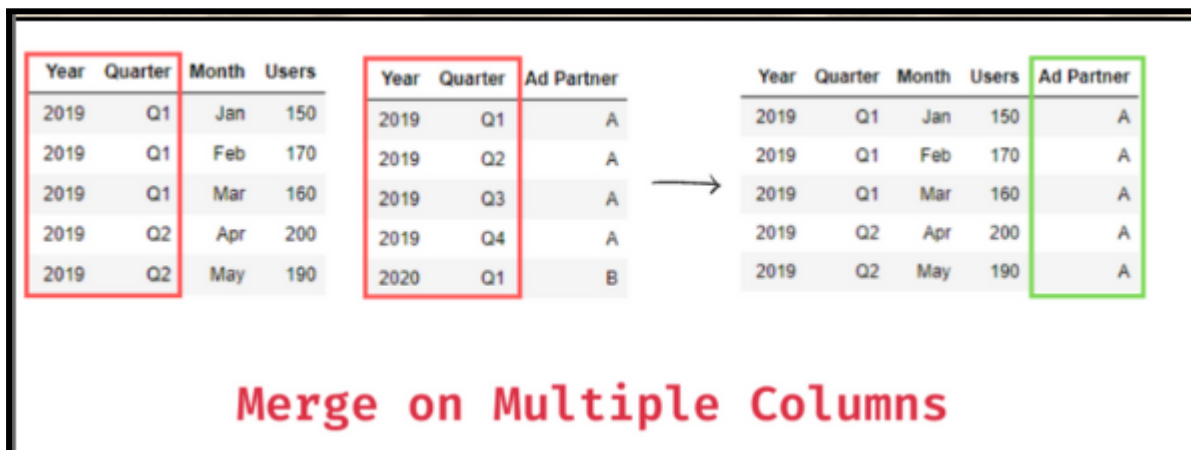


pandas merge(): Combining Data on Common Columns or Indices

When you want to combine data objects based on one or more keys, similar to what you'd do in a relational database, merge() is the tool you need. More specifically, merge() is most useful when you want to combine the share data.



- How the merge should happen: It takes values 'left', 'right', 'inner', 'outer'. default value is 'inner'. 'left' :Collects all data from left dataframe and common data of left and right. 'right' : Collects all data from right dataframe and common data of left and right. 'inner' : Collects only the common data from both the left and right dataframes. This more like an intersection. 'outer' :Collects all from left and right including common.
- The pandas merge() function is used to do database-style joins on dataframes. To merge dataframes on multiple columns, pass the columns to merge on as a list to the on parameter of the merge() function. The following is the syntax: df_merged = pd.merge(df_left, df_right, on=['Col1', 'Col2', ...], how='inner')



Here df1 and df2 are the dataframes you want to merge, and on='index' specifies that you want to merge the data frames based on the index.

Example :

```
import pandas as pd

# create two dataframes with the same index
df1 = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]}, index=[0, 1, 2])
df2 = pd.DataFrame({'C': [7, 8, 9], 'D': [10, 11, 12]}, index=[0, 1, 2])

# merge the dataframes based on the index
result = pd.merge(df1, df2, on='index')

print(result)
```

Output:

	A	B	C	D
0	1	4	7	10
1	2	5	8	11
2	3	6	9	12

pandas .join(): Combining Data on a Column or Index

The `join()` method in Pandas is used to combine two DataFrames based on a key column or index. It's similar to merging, but it combines DataFrames on their indexes by default, rather than on columns. This method is particularly useful when you want to join two DataFrames on their indexes or when the key column is the index.

By default, `.join()` will attempt to do a left join on indices. If you want to join on columns like you would with `merge()`, then you'll need to set the columns as indices.

- Inner join: Keep only IDs that are contained in both data sets.
- Outer join: Keep all IDs.
- Left join: Keep only IDs that are contained in the first data set.
- Right join: Keep only IDs that are contained in the second data set.

However, using the `how` parameter, you can specify other types of joins, such as right, inner or outer. Let's understand this by an example, as shown below.

Example :

			ID	X1	ID	X2
			1	a1	2	b1
			2	a2	3	b2

Inner Join			Outer Join			Left Join			Right Join		
ID	X1	X2	ID	X1	X2	ID	X1	X2	ID	X1	X2
2	a2	b1	1	a1	NA	1	a1	NA	2	a2	b1
			2	a2	b1	2	a2	b1	3	NA	b2
			3	NA	b2						

IN CLASS CHALLENGES

INCLASS CHALLENGE1 - Fictional sports league:

You are given two datasets related to a fictional sports league: teams data and players data. teams data contains the following columns: 'team_id', 'team_name', 'city'.

Link for teams data:

in the show. So, I've collected some data about the pets from two different neighborhoods.

So, first let's create DataFrames to represent this data. In one neighborhood, we have pets like dogs named Buddy, cats named Whiskers, and birds named Tweetie. In the other neighborhood, we have pets like rabbits named Floppy, dogs named Max, and fish named Bubbles.

Before we merge these datasets together, let's take a look at each neighborhood's data separately. Once we've seen all the pets from each neighborhood, I'll need your help to combine all this data together using a method called “concatenation”. Concatenation is like stacking blocks on top of each other, combining them into one big tower. It's a great method for combining data because it keeps everything organized in one place, just like stacking up all the pets in our show!

HOME TASK

HOME TASK1-Merge Magic:

Hey there! Imagine you're running a cool online store that sells all sorts of gadgets like laptops, smartphones, and headphones. You have a dataset of customers who bought these gadgets and another dataset that describes each gadget in detail.

Customer Orders dataset Link:

Your Task: How would you use your coding skills to combine these two datasets together so you can see who bought what? Think about how you would match up the gadgets each customer bought with the details about those gadgets.

1. Display the original DataSets.
2. Look for something common between the two lists that you can use to match them up. Is there a unique identifier for each gadget that appears in both lists?
3. Once you've found that common factor, consider using a special function in Python called 'merge' from a library called 'Pandas' to combine the lists together based on that factor.
4. Finally, check your merged data to see if it looks right! You should be able to see which customers bought which gadgets, along with all the details about those gadgets.

Give it your best shot! You're on your way to becoming a coding wizard!

HOMETASK 2 -Friends Grades:

Imagine you have a list of your friends names, their ages, and their grades in different subjects like Math, Science, English. Your task is to create a special report that combines all this information neatly into one table. How would you write a program to do this? Think about how you can use Python to merge your friends' data with their grades effortlessly. Give it a try!

Steps to follow:

First create two sample DataFrames: `student_data` with columns as 'ID', 'Name', 'Age' and `grades_data` with columns as 'ID', 'Math', 'Science', 'English'. Then, we set the index of both DataFrames to be the 'ID' column using the `.set_index()` method.

Finally, we use the `.join()` method to join the DataFrames based on their indices, which are now the 'ID' column.

The result is the combined data, where each student's information from `student_data` is combined with their grades from `grades_data`.

WHAT'S NEXT?

Special inbuilt functions

LESSON 32

SPECIAL INBUILT FUNCTIONS

LEARNING CONTENT

Special Pandas inbuilt functions

Pandas functions refer to the collection of methods and operations provided by the Pandas library in Python, designed for data manipulation and analysis. The library introduces two primary data structures, Series and DataFrame, and offers many functions to perform tasks like data cleaning, exploration, transformation, and statistical analysis.

unique() function

The pandas library in Python is used to work with data frames that structure data in rows and columns. It is widely used in data analysis and machine learning. While analyzing the data, many times the user wants to see the unique values in a particular column, which can be done using Pandas unique() function. Therefore the unique function in pandas is

used to find the unique values from a series [A series is a single column of a data frame].

We can use the unique function on any possible set of elements in Python. It can be used on a series of strings, integers, tuples, or mixed elements.

Syntax
The syntax of the unique function is as follows:
`pd.unique(pd.Series([2, 1, 3, 3]))`

We can use it for a data frame as follows:
`dataframe["column"].unique()`

Return value
The unique function returns a list containing all unique elements only.
The resultant list is not sorted.

Example :

```
import pandas as pd

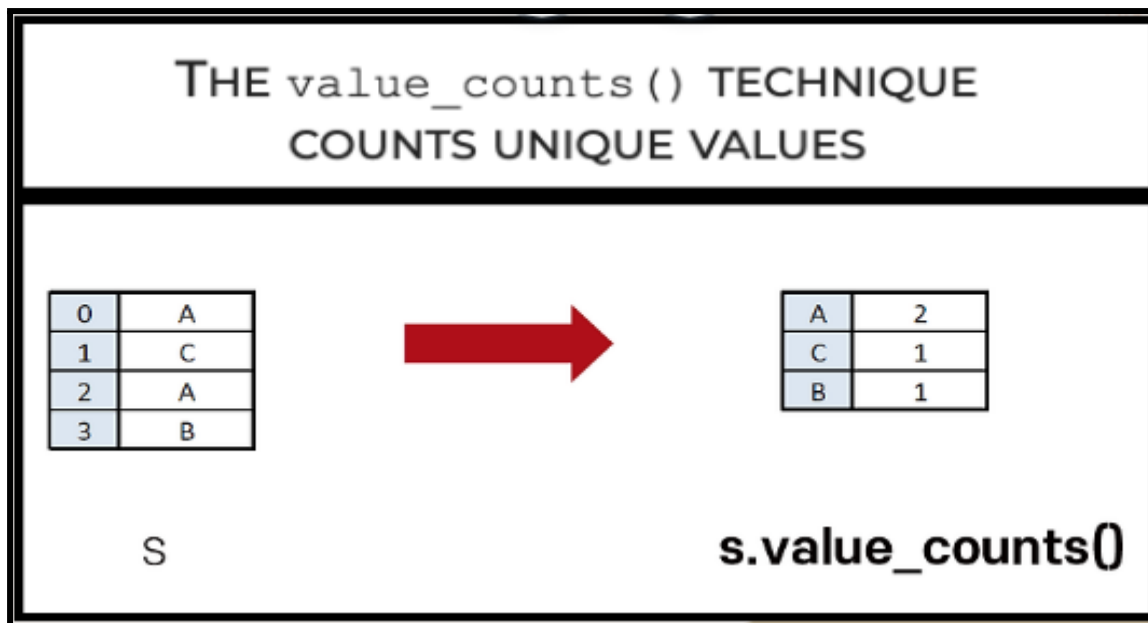
# Creating a dataframe
df = pd.DataFrame({'Sports': ['Football', 'Cricket', 'Baseball', 'Basketball',
                              'Tennis', 'Table-tennis', 'Archery', 'Swimming', 'Boxing'],
                  'Player': ["Messi", "Afridi", "Chad", "Johnny", "Federer",
                              "Yong", "Mark", "Phelps", "Khan"],
                  'Country': ["Argentina", "Pakistan", "England", "England",
                              "Switzerland", "China", "China", "USA", "Pakistan" ],
                  'Rank': [1, 9, 7, 12, 1, 2, 11, 1, 1] })

# Finding unique countries
print(df["Country"].unique())
# Finding unique rankings
print(df["Rank"].unique())
```

```
['Argentina' 'Pakistan' 'England' 'Switzerland' 'China' 'USA']
[ 1  9  7 12  2 11]
```

value_counts() function

The `value_counts()` function in pandas return a Series that contains the number of unique values. The resulting object will be in descending order so that the first element is the most frequently-occurring element. Excludes NA values by default.



Syntax

`Series.value_counts(normalize=False, sort=True, ascending=False, bins=None, dropna=True)`

The `value_counts` function takes the following parameters:

- `normalise` (optional): If set to `True`, the function returns the relative frequencies of the values. It is set to `False` by default.
- `sort` (optional): If set to `True`, the function returns the values in a sorted manner. It is set to `True` by default.
- `ascending` (optional): If set to `True`, values are sorted in an ascending manner. It is set to `False` by default.
- `bins` (optional): Groups values into bins instead of counting them. It is set to `None` by default.
- `dropna` (optional): If set to `True`, counts of `NaN` are not included. It is set to `True` by default.

Return Value

The function returns a series of counts of unique values.

Example :

```
# importing pandas as pd
import pandas as pd

# Creating the Series
sr = pd.Series([100, 214, 325, 88, None, 325, None, 325, 100])

# Print the series
print(sr)

#print value counts
print("\nCounts of Unique Values:")
print(sr.value_counts())
```

```
0    100.0
1    214.0
2    325.0
3     88.0
4     NaN
5    325.0
6     NaN
7    325.0
8    100.0
dtype: float64

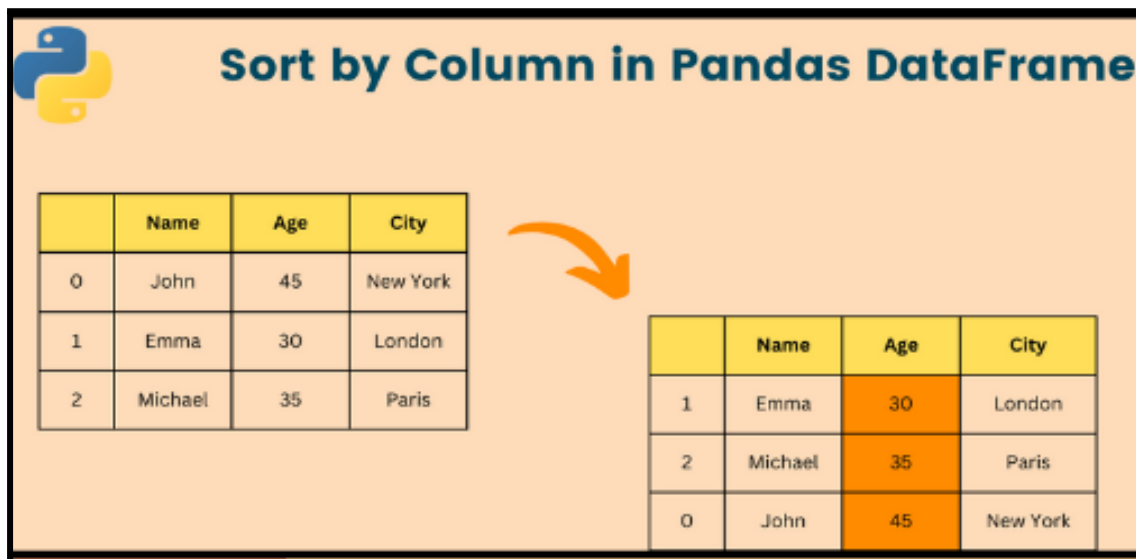
Counts of Unique Values:
325.0    3
100.0    2
88.0     1
214.0    1
dtype: int64
```

sort_values() function

Pandas is a Python library that is used in manipulating and analyzing data. This data exists in the form of two-dimensional structures called data frames. These data frames make the organization of data straightforward and allows easy understanding of the tabular format. To further increase our comprehension of the data, the `sort_values()` function sorts the data frame in ascending or descending order of the column passed as an argument.

For Example,

suppose you want data on your students based on increasing age. You could do this easily by using the `sort_values()` function and passing the column with the students' ages as an argument.



Sort by Column in Pandas DataFrame

	Name	Age	City
0	John	45	New York
1	Emma	30	London
2	Michael	35	Paris

	Name	Age	City
1	Emma	30	London
2	Michael	35	Paris
0	John	45	New York

Syntax

`dataframe.sort_values(by, axis, ascending, inplace, kind, na_position, ignore_index, key)`

1. `by`: Specifies the column or index label to sort by. Datatype must be a string.
2. `axis`: Specifies the axis by which to sort. 1 for column or 0 for index. (Optional)
3. `ascending`: If set to True, data frame will be sorted in ascending manner. True is the default. (Optional)
4. `inplace`: If set to True, operation will be performed on the original data frame. False is the default. (Optional)
5. `kind`: Specifies the sorting algorithm to use. The options are mergesort, quicksort, or heapsort. quicksort is the default. (Optional)
6. `na_position`: Specifies where to put NULL values. Options are last or first. last is the default. (Optional)
7. `ignore_index`: If set to True, index is ignored. False is the default. (Optional)

8. key: Specifies a function to be executed before the data is sorted. (Optional)

Example :

```
import pandas as pd

#initialize the data
data = {"Name": ['John', 'Kelly', 'Kris', 'Betty', 'Bob'],
        "Age": [16, 19, 15, 21, 17],
        "Marks": [89, 76, 85, 67, 53]}

#create a dataframe object
df = pd.DataFrame(data)

#print the data frame
print("Original DataFrame")
print(df)

#sort the data frame
sorted_df = df.sort_values(by='Marks', ascending=False)

#print the sorted data frame
print("\nSorted Data According to the Marks(High to Low)")
print(sorted_df)
```

```
Original DataFrame
   Name  Age  Marks
0  John   16    89
1  Kelly   19    76
2   Kris   15    85
3  Betty   21    67
4   Bob   17    53

Sorted Data According to the Marks(High to Low)
   Name  Age  Marks
0  John   16    89
2   Kris   15    85
1  Kelly   19    76
3  Betty   21    67
4   Bob   17    53
```

EXAMPLE:

```

import pandas as pd

#initialize the data
data = {"Name": ['John', 'Kelly', 'Kris', 'Betty', 'Bob'],
        "Age": [16, 19, 15, 21, 17],
        "Marks": [89, 76, 85, 67, 53]}

#create a dataframe object
df = pd.DataFrame(data)

#print the data frame
print("Original DataFrame")
print(df)

#sort the data frame
sorted_df = df.sort_values(by='Marks', ascending=False)

#print the sorted data frame
print("\nSorted Data According to the Marks(High to Low)")
print(sorted_df)

```

```

Original DataFrame
   Name  Age  Marks
0  John   16    89
1  Kelly   19    76
2  Kris   15    85
3  Betty   21    67
4   Bob   17    53

Sorted Data According to the Marks(High to Low)
   Name  Age  Marks
0  John   16    89
2  Kris   15    85
1  Kelly   19    76
3  Betty   21    67
4   Bob   17    53

```

info() function

The info function in Pandas displays a summary of the dataframe. It displays column names, data types, the number of non-null values, and memory usage.

Name	Age
Jim	26
Dwight	28
Angela	27
Tobi	32



```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 2 columns):
#   Column  Non-Null Count  Dtype
---  -
0    Name     4 non-null     object
1    Age      4 non-null     int64
dtypes: int64(1), object(1)
memory usage: 192.0+ bytes
```

info()

Syntax DataFrame.info(verbose, buf, max_cols, memory_usage, show_counts, null_counts)

All parameters are optional for the info function. The info function takes the following parameters:

- **verbose:** Determines whether the full summary is to be printed. Takes a bool value. By default, the setting in `pandas.options.display.max_info_columns` is followed.
- **buf:** Determines where to send the output. By default, the output is printed to `sys.stdout`.
- **max_cols:** Determines when to switch from the verbose to the truncated output. Takes an int variable. If the dataframe has more than `max_cols` columns, the truncated output is used. By default, the setting in `pandas.options.display.max_info_columns` is used.
- **memory_usage:** Determines whether total memory usage of the dataframe elements (including the index) should be displayed. By default, this follows the `pandas.options.display.memory_usage` setting.
- **show_counts:** Determines whether to show non-null value counts or not. A value of `True` will always show the counts, while `False` never shows the counts.

Return value The info function does not return anything. Instead, it outputs a concise summary of the dataframe.

Example:

```
import pandas as pd

# Creating a dataframe
int_values = [1, 2, 3, None, 5]
text_values = ['alpha', 'beta', 'gamma', 'delta', 'epsilon']
float_values = [0.0, 0.25, 0.5, 0.75, 1.0]
df = pd.DataFrame({"int_col": int_values, "text_col": text_values,
                  "float_col": float_values})

print("Original Dataframe")
print(df)
print('\n')

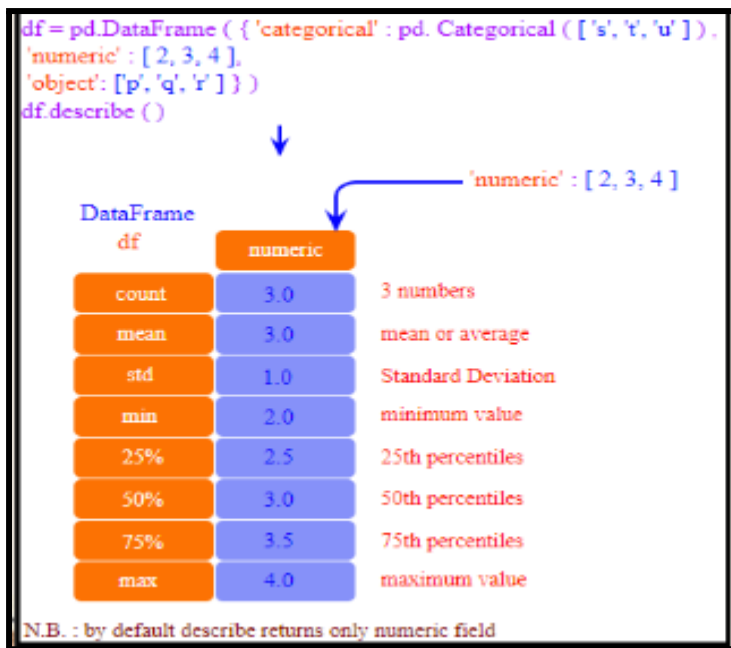
# Use info() to get information about the DataFrame
print("information about the DataFrame using info function:")
print(df.info())
```

```
Original Dataframe
  int_col text_col float_col
0      1.0   alpha      0.00
1      2.0    beta      0.25
2      3.0   gamma      0.50
3      NaN    delta      0.75
4      5.0  epsilon      1.00

information about the DataFrame using info function:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   int_col      4 non-null        float64
1   text_col     5 non-null        object
2   float_col    5 non-null        float64
dtypes: float64(2), object(1)
memory usage: 248.0+ bytes
None
```

describe() function

The describe() method displays a statistical summary of the data that consists of the mean, the standard deviation, the minimum and maximum value, and so on.



Syntax `dataFrame.describe(percentiles, include, exclude, datetime_is_numeric)`

Arguments `percentiles`: Values between 0 and 1. Specifies the percentile to be returned in the result. (Optional) `include`: List of data types to include in the result. Options are `None` | `'all'` | `datatypes`. (Optional) `exclude`: List of data types to exclude in the result. Options are `None` | `'all'` | `datatypes`. (Optional) `datetime_is_numeric`: To treat datetime data as numeric. Set to `True` or `False`, with default as `False`. (Optional) `Return Value`
The functions return a `DataFrame` object, where each row has a type of statistic that provides a summary of the columns.

Example:

```

#import library
import pandas as pd

#define data
data = {'Name': ['Kris', 'Kelly', 'Josh', 'Bob', 'Lisa'],
        'Age': [16, 21, 17, 19, 20],
        'Marks': [78, 56, 87, 89, 79]}

#create a DataFrame object
df = pd.DataFrame(data)

#describe the data
print(df.describe())

```

	Age	Marks
count	5.000000	5.000000
mean	18.600000	77.800000
std	2.073644	13.103435
min	16.000000	56.000000
25%	17.000000	78.000000
50%	19.000000	79.000000
75%	20.000000	87.000000
max	21.000000	89.000000

IN CLASS CHALLENGES

INCLASS CHALLENGE1 - Covid-19 Data :

One day your friend Sophie tells you that she has access to a dataset called 'Covind-19 Info' , which contains vital information about the spread of Covid-19 across different states and union territories of India. The dataset includes columns such as 'State/UTs', 'Total Cases', 'Active Cases', 'Discharged Cases', 'Deaths', 'Active Ratio', 'Discharge Ratio', 'Death Ratio', and 'Population'.

Dataset Link:

1. How many people in total have been affected by Covid-19 in India?
2. Can you Find all the states where the death ratio is greater than 1 and the population is greater than 4000000?
3. Sophie also wants to know some basic facts like count, mean, standard deviation, minimum values, and maximum values of all the columns [Hint: Use the describe() method].

Lastly, can you help her to find the data type of each column? [Hint: Use info() method].

INCLASS CHALLENGE 2- Color Analysis:

Hey, young coder! Imagine your teacher has collected data on the colors each student likes and now she wants to analyze this data.

```
student_data = {'Alice': 'Blue', 'Bob': 'Green', 'Charlie': 'Yellow',  
'David': 'Red', 'Emma': 'Pink', 'Frank': 'Red', 'Grace': 'Green',  
'Harry': 'Blue', 'Ivy': 'Blue', 'Jack': 'Black'}
```

Here is your task. Help your teacher in finding the following things:

- Display what colors students like the most.
- Collect the colors each student likes and count them up. [Hint: use value_counts() function]
- figure out which color is the least favorite among the students. Sort them from least to most popular, in Ascending Order. [Hint: use sort_values() funtion]

How would you use your coding skills to solve this colorful mystery?

HOME TASK

HOME TASK1-Data Summary:

Imagine you are a weather scientist and you've collected data about the weather of the year 2022. You have saved this data in a file called WeatherReport.csv. This file includes information about the temperature, month, year, and rainfall count for each month of the year. Now, you want to analyze this data to understand it better.

DataSet

Link:

☐☐☐☐

Pokémon is the fastest? Use the pandas library to help you organise and sort the data.

To help you get started, remember these steps:

- ☐ First create a dictionary containing Pokémon names, their types, and their speed stats. Then convert this dictionary into a pandas DataFrame.
- ☐ Now print the original DataFrame to see the initial order of the Pokémon.
- ☐ Use a Pandas function that is use to sort DataFrame columns in Ascending or Descending order. In this case you need to sort the DataFrame by the 'Speed' column in Descending order.

Finally, print the sorted DataFrame to see the Pokémon ordered by their speed from fastest to slowest.

Have fun coding and finding out which Pokémon is the speediest!

WHAT'S NEXT?

Special inbuilt functions

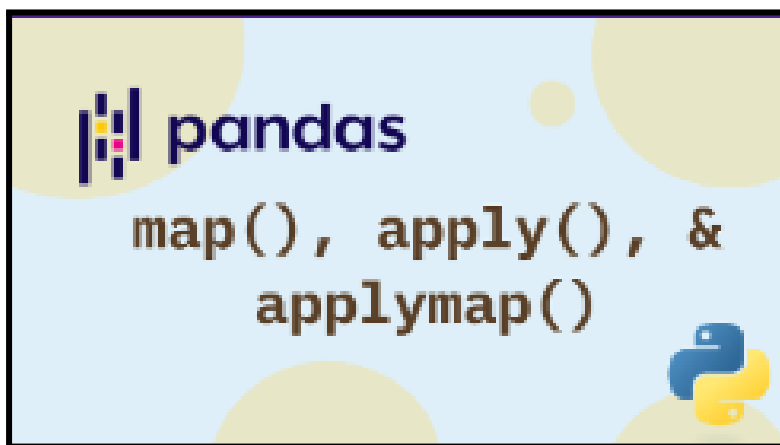
LESSON 33

CUSTOM FUNCTIONS

LEARNING CONTENT

Custom functions

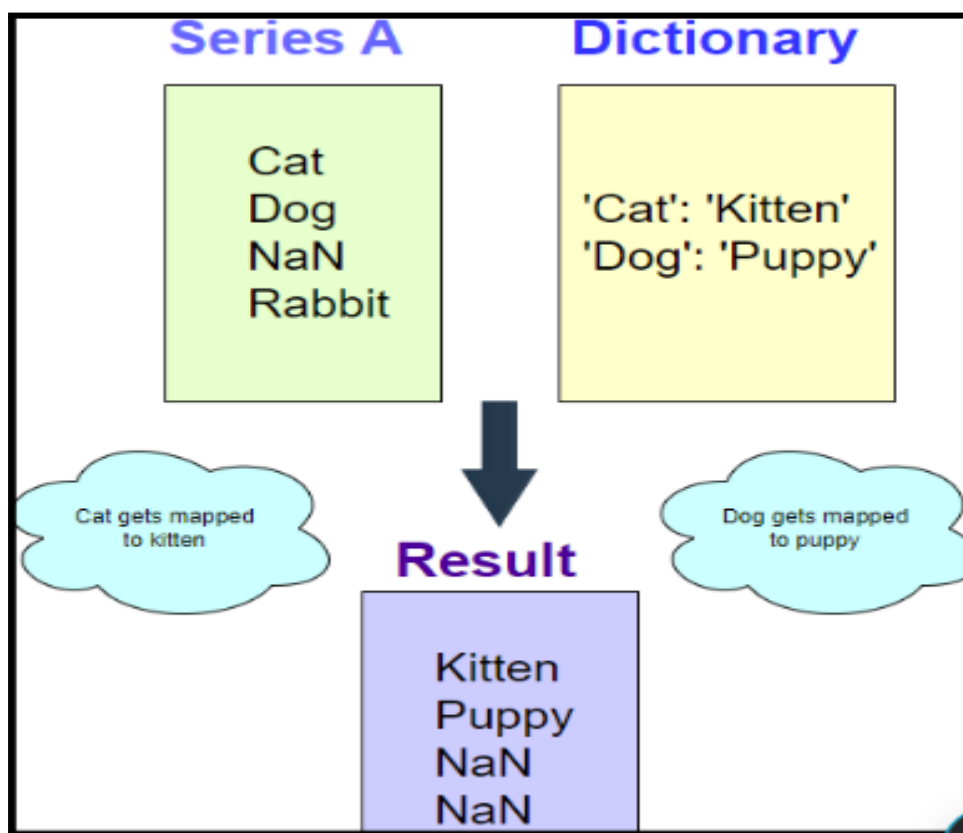
Pandas, a powerful data manipulation library in Python, allows users to create custom functions to apply to their DataFrame operations. Custom functions in Pandas can be used to perform specific operations that are not covered by built-in methods, providing flexibility in data processing. There are generally 3 ways to apply custom functions in Pandas. That is by using `map()`, `apply()` and `applymap()`.



map():

map works element-wise on a series, and is optimized for mapping values to a series (e.g. one column of a DataFrame). The map method in pandas maps values from one series to another based on a common column. Mapping is based on values within a series. Since mapping is one-to-one, the series must have only unique values. Otherwise an error will occur.

illustration showing how the map function works:



Syntax

```
Series.map(arg, na_action=None)
```

Parameters

The map method takes two parameters:

- **arg** : An object such as a dictionary, series, or function that determines the mapping.
- **na_action** : Determines how to deal with NaN values. It can take two forms; by default, it is None. If it is ignore, mapping is not applied to NaN values.
- **map** accepts values as dictionaries in the first parameter. If a value is not found in the dictionary, it is mapped to NaN. Only the first parameter is compulsory.

Return value The map function returns a series with the mappings incorporated.

Example :

```
import pandas as pd
import numpy as np

s = pd.Series(['cat', 'dog', np.nan, 'rabbit'])
print("Original Series")
print(s)
print('\n')

print("Mapping onto a dictionary")
print(s.map({'cat': 'kitten', 'dog': 'puppy'}))
print('\n')

print("We can also map to a function")
print(s.map('I am a {}'.format))
print('\n')

print("Use the na_action parameter to avoid mapping NaN")
print(s.map('I am a {}'.format, na_action='ignore'))
```

Original Series

```
0    cat
1    dog
2    NaN
3  rabbit
dtype: object
```

Mapping onto a dictionary

```
0    kitten
1    puppy
2    NaN
3    NaN
dtype: object
```

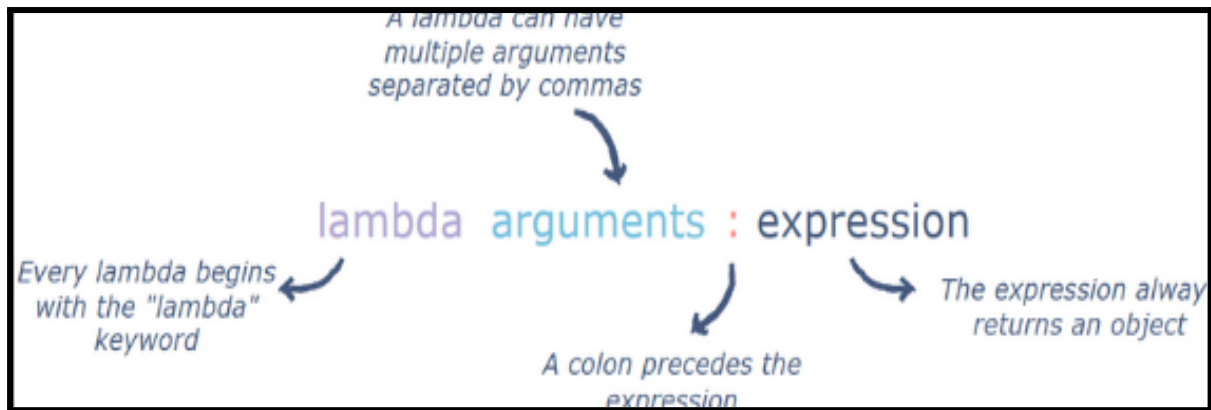
We can also map to a function

```
0    I am a cat
1    I am a dog
2    I am a nan
3    I am a rabbit
dtype: object
```

Use the na_action parameter to avoid mapping NaN

```
0    I am a cat
1    I am a dog
2    NaN
3    I am a rabbit
dtype: object
```

Lambda function :A lambda function is a small anonymous function which returns an object. The object returned by lambda is usually assigned to a variable or used as a part of other bigger functions. Instead of the conventional def keyword used for creating functions, a lambda function is defined by using the lambda keyword. The structure of lambda can be seen below:



A lambda is much more readable than a full function since it can be written in-line. Hence, it is a good practice to use lambdas when the function expression is small. The beauty of lambda functions lies in the fact that they return function objects. This makes them helpful when used with functions like map or filter which require function objects as arguments.

Example: Using Lambda function for mapping

```
import pandas as pd
import numpy as np

# List of animal names
s = pd.Series(['cat', 'dog', np.nan, 'rabbit'])
print("Original List:")
print(s)

# Use a quick recipe (lambda function) to change the list
print("\nMapping Using a lambda function:")
print(s.map(lambda x: f"I am a {x}" if pd.notna(x) else x))
```

Original List:

0	cat
1	dog
2	NaN
3	rabbit

dtype: object

Mapping Using a lambda function:

0	I am a cat
1	I am a dog
2	NaN
3	I am a rabbit

dtype: object

apply():

The apply function in pandas is used to apply a function on an axis of the dataframe. NOTE: axis refers to rows or columns. A series object must be passed to the apply function, which would have either the dataframe index (axis = 0) or a dataframe column (axis = 1).

apply works on both Series and DataFrames, but it's relatively slow and should only be used if you're running out of other options.

Syntax

```
DataFrame.apply(func, axis=0, raw=False, result_type=None, args=(),  
**kwargs)
```

The apply functions take the following parameters:

- **func** : The function to apply to a row or column.
- **axis** : The axis on which the function is applied. 0 refers to applying the function column-wise. 1 refers to applying the function row-wise.
- **raw** : Determines if the row or column passed is a series or ndarray. False refers to a series. True refers to a ndarray. By default, it is False.
- **result_type** : Only used when axis = 1. Can take four values: expand, reduce, broadcast, None. By default, it is None.
- **args** : Positional arguments to pass to the function in addition to the array or series.
- ****kwargs** : Any additional arguments.

Only the func parameter is required. The rest have default values.

Return value The apply function returns a series or dataframe after applying the specified function.

values of result_type parameter:

- **expand**: list-like results convert into columns.
- **reduce**: returns a series if possible, instead of expanding list-like results.
- **broadcast**: results are broadcasted to the original shape of the dataframe, and the original index and columns will be retained.
- **None**: depends on the return value of the applied function; list-like results will be returned as a series. However, if the apply function returns a series, then these are expanded to columns.

Example :

```
import pandas as pd
import numpy as np

df = pd.DataFrame([[4, 9]] * 3, columns=['A', 'B'])
print("Original Dataframe")
print(df)
print('\n')

print("Applying square root")
print(df.apply(np.sqrt))
print('\n')

print("Column-wise Sum")
print(df.apply(np.sum, axis=0))
print('\n')

print("Row-wise Sum")
print(df.apply(np.sum, axis=1))
print('\n')

print("Returning a list-like column to each index")
print(df.apply(lambda x: [1, 2], axis=1))
print('\n')
```

```
Original Dataframe
   A  B
0  4  9
1  4  9
2  4  9

Applying square root
   A    B
0  2.0  3.0
1  2.0  3.0
2  2.0  3.0

Column-wise Sum
A    12
B    27
dtype: int64

Row-wise Sum
0     13
1     13
2     13
dtype: int64

Returning a list-like column to each index
0    [1, 2]
1    [1, 2]
2    [1, 2]
dtype: object
```

applymap()

The `applymap()` method in Pandas is used to apply a function to each element of a DataFrame. This method is particularly useful when you need to perform element-wise operations that are not covered by built-in functions or when you want to apply a custom function to every cell in the DataFrame.

NOTE:

`applymap()` is element-wise for DataFrames, and is equivalent to `apply()` for Series.

`applymap()` is not suitable for aggregations or reducing operations since it operates element-wise rather than on the entire DataFrame or row/column-wise.

By using `applymap()`, you gain the flexibility to manipulate each element of a DataFrame in a highly customized manner, making it a powerful tool for data cleaning and transformation.

Syntax

`DataFrame.applymap(func)`

`func`: The function to apply to each element of the DataFrame.

```
import pandas as pd

df = pd.DataFrame([[1, 2, 3], [4, 5, 6], [7, 8, 9]], columns=['A', 'B', 'C'])
print("Original DataFrame:")
print(df)

squared_df = df.applymap(lambda x: x**2)
print("Squared DataFrame using applymap():")
print(squared_df)
```

```
Original DataFrame:
   A  B  C
0  1  2  3
1  4  5  6
2  7  8  9

Squared DataFrame using applymap():
   A  B  C
0  1  4  9
1 16 25 36
2 49 64 81
```

Example: If you want to apply a more complex function, such as formatting strings in a DataFrame, you can do so with `applymap()`.

```
import pandas as pd

df = pd.DataFrame([['Alice', 'Bob'], ['Carol', 'Dave']], columns=['Name1', 'Name2'])
print("Original DataFrame:")
print(df)

formatted_df = df.applymap(lambda x: x.upper())
print("\nFormatted DataFrame:")
print(formatted_df)
```

Original DataFrame:

	Name1	Name2
0	Alice	Bob
1	Carol	Dave

Formatted DataFrame:

	Name1	Name2
0	ALICE	BOB
1	CAROL	DAVE

Example: Consider a DataFrame with mixed data types and you want to apply different transformations based on the data type.

```
import pandas as pd

df = pd.DataFrame({'A': [1, 2, 3],
                   'B': ['x', 'y', 'z'],
                   'C': [1.1, 2.2, 3.3]})
print("Original DataFrame:")
print(df)

def transform(x):
    if isinstance(x, int):
        return x + 1
    elif isinstance(x, float):
        return x * 2
    elif isinstance(x, str):
        return x.upper()
    else:
        return x

transformed_df = df.applymap(transform)
print("\nTransformed DataFrame:")
print(transformed_df)
```

Original DataFrame:

	A	B	C
0	1	x	1.1
1	2	y	2.2
2	3	z	3.3

Transformed DataFrame:

	A	B	C
0	2	X	2.2
1	3	Y	4.4
2	4	Z	6.6

Difference between map, applymap and apply in Pandas

map	applymap	apply
Defined only in Series	Defined only in Dataframe	Defined in both Series and DataFrame
Accepts dictionary, Series, or callable	Accept callables only	Accept callables only
Series.map() Operate on one element at time	DataFrame.apply map() Operate on one element at a time	operates on entire rows or columns at a time for Dataframe, and one at a time for Series.apply
Missing values will be recorded as NaN in the output.	Performs better operation than apply().	Suited to more complex operations and aggregation.

IN CLASS CHALLENGES

INCLASS CHALLENGE1 - Student Grades:

Imagine you are given a CSV file named school-grades.csv which contains the grades of students in Math, Science, and English.

DataSet

Link:

Assign Letter Grades: Based on the average grade, assign a letter grade (A, B, C, D, or F) according to the following scale: A= 90 and above, B= 80 to 89, C= 70 to 79, D= 60 to 69 and F= below 60. HINT: [Use the map() method on the Average column to convert numeric grades to letter grades based on a custom function].

Assign Pass/Fail: Determine if each student has passed or failed based on their average grade. A passing grade is 75 or higher. HINT: [Use the apply() method on the Average column to determine pass or fail based on a custom lambda function].

INCLASS CHALLENGE 2- Online Store:

Picture yourself managing an online store and you have a DataFrame representing items with their prices and categories. data = { 'Item': ['Laptop', 'Book', 'Phone', 'Headphones'], 'Price': [1200, 15, 800, 150], 'Category': ['Electronics', 'Books', 'Electronics', 'Electronics']}

You need to write a Python script using Pandas to perform the following tasks:

1. Create the initial DataFrame with the given data.
2. Print the initial DataFrame to see the data.
3. Apply a 10% discount: Create a new column to store the discounted price for electronics items using the apply() method.
4. Add Sales Tax: Create a new column to store the final price (including sales tax) for all items using the map() method.
5. Standardize the item names to be in uppercase using the apply() method.
6. Increase all prices by a fixed amount (eg: \$10) using the applymap() method.

Print the updated DataFrame after each step.

HOME TASK

HOME TASK1-Employee Salary:

Link to the CSV file:

HOMETASK 2 - Temperature Conversion :

Link to the CSV
file:

LESSON 34

GROUPING DATA

LEARNING CONTENT

GROUPING DATA

Python allows many different libraries that enable data manipulation. One such library, pandas, has a command used to group the dataset by the selected column. It can be used to group large datasets and apply operations on them.

Pandas group by is used for grouping the data according to the categories and applying a function to the categories. It also helps to aggregate data efficiently. The Pandas groupby() is a very powerful function with a lot of variations. It makes the task of splitting the Dataframe over some criteria really easy and efficient.

The GroupBy function in Pandas employs the split-apply-combine strategy meaning it performs a combination of — splitting an object, applying functions to the object and combining the results.

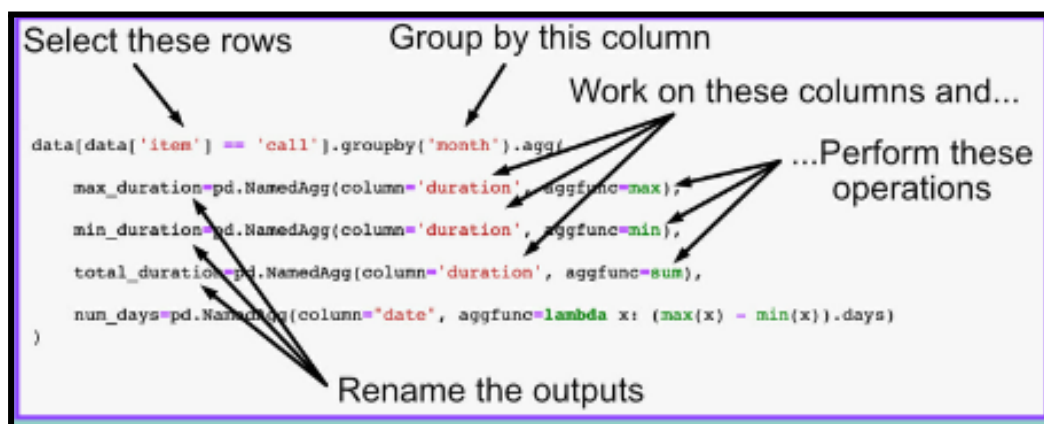
But before you start in GroupBy, you need to be familiar with aggregation.

Aggregation:

Aggregation involves creating statistical summaries of data with methods such as mean, median, mode, min (minimum), max (maximum), std (standard deviation), var (variance), sum, count, etc.

To perform the aggregation operation on groups:

- Import module
- Create or load data
- Create a GroupBy object which groups data along a key or multiple keys
- Apply a statistical operation.



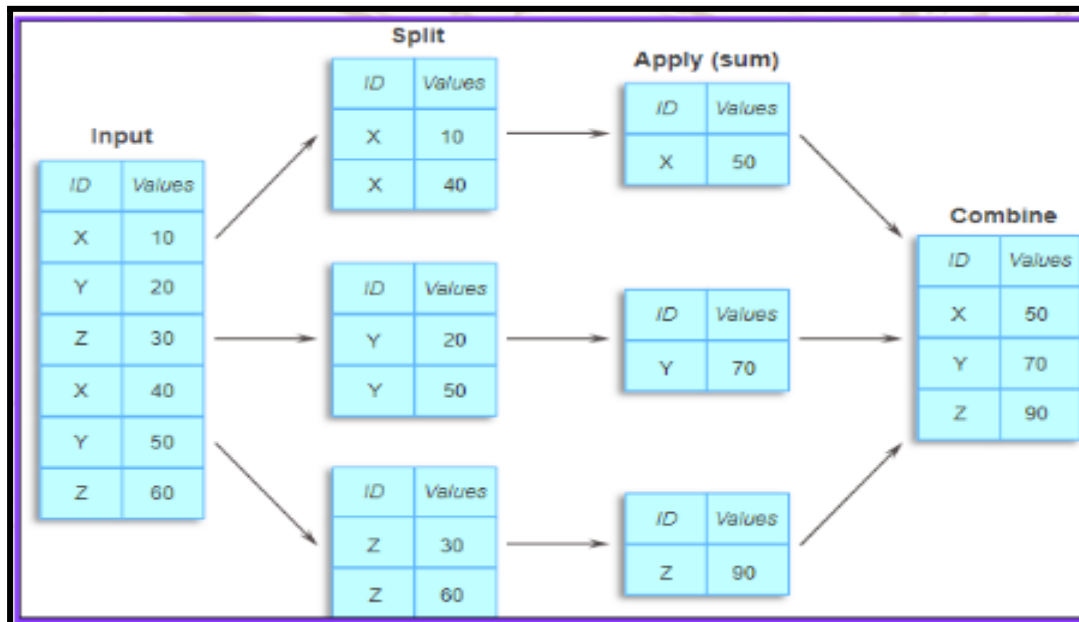
Pandas dataframe.groupby():

The groupby method in pandas is a powerful tool that allows for grouping data based on the values in one or more columns. The basic concept of groupby is to split the data into groups based on some criteria, apply a function to each group independently, and then combine the results.

1. **Splitting**: The data is split into groups based on the specified key(s). The keys can be column names, a list of column names, or even a custom function that returns a value for each row in the DataFrame.
2. **Applying**: A function is applied to each group independently. This can be an aggregation function (e.g.,

sum, mean), a transformation function, or any function that produces a scalar or an array of values.

3. **Combining**: The results of the applied functions are combined into a new DataFrame or Series.



Syntax:

DataFrame.groupby(by=None, axis=0, level=None, as_index=True, sort=True, group_keys=True, squeeze=False, observed=False)

Parameters :

- **by**: Specifies the column(s) to group the data by.
- **axis**: Determines whether the grouping is performed along rows (axis=0) or columns (axis=1).
- **level**: Enables grouping based on a specific level or multi-levels in the DataFrame's index.
- **as_index**: Determines whether the grouped column(s) become the index of the resulting DataFrame.
- **sort**: Defines whether to sort the resulting DataFrame by the columns used for grouping.

- `group_keys`: Indicates whether to add a group key to the index of the resulting DataFrame.
- `squeeze`: Returns a Series instead of a DataFrame if possible.
- `observed`: Controls whether to include all values from the original DataFrame's data when grouping on categorical variables.

Example : DataFrame containing information about students and their test scores:

```
import pandas as pd

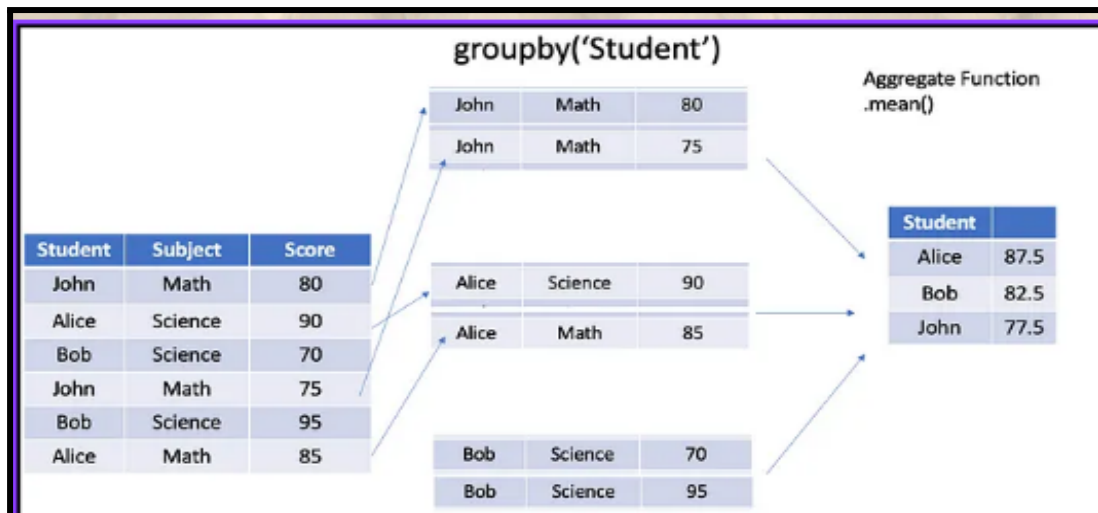
data = {'Student': ['John', 'Alice', 'Bob', 'John', 'Bob', 'Alice'],
        'Subject': ['Math', 'Science', 'Science', 'Math', 'Science', 'Math'],
        'Score': [80, 90, 70, 75, 95, 85]}
df = pd.DataFrame(data)
```

The DataFrame `df` consists of three columns: 'Student', 'Subject' and 'Score'.

	Student	Subject	Score
0	John	Math	80
1	Alice	Science	90
2	Bob	Science	70
3	John	Math	75
4	Bob	Science	95
5	Alice	Math	85

groupby() based on a Single Column:

We can group the data by a single column, such as 'Student.' To calculate the average score per student, we can use the following code:



Example : The output displays the average score for each student.

```
student_group = df.groupby('Student')
average_score_per_student = student_group['Score'].mean()
print(average_score_per_student)
```

Student

Alice 87.5

Bob 82.5

John 77.5

Name: Score, dtype: float64

groupby() based on Multiple Columns:

Let's group by both 'Student' and 'Subject' columns to calculate the average score per student, per subject

```
student_subject_group = df.groupby(['Student', 'Subject'])
average_score_per_student_subject = student_subject_group['Score'].mean()
print(average_score_per_student_subject)
```

Student Subject

Alice Math 85

 Science 90

Bob Science 70

John Math 77.5

Name: Score, dtype: float64

IN CLASS CHALLENGES

INCLASS CHALLENGE1 - Analyzing Sales Data :

File

Link:

school	class	name	date_Of_Birth	age	height	weight	address
S1	s001	Alberto Franco	15/05/2002	12	173	35	street1
S2	s002	Gino Mcneill	17/05/2002	12	192	32	street2
S3	s003	Ryan Parkes	16/02/1999	13	186	33	street3
S4	s001	Eesha Hinton	25/09/1998	13	167	30	street1
S5	s002	Gino Mcneill	11/05/2002	14	151	31	street2
S6	s004	David Parkes	15/09/1997	12	159	32	street4

Things to be done:

1. DataFrame: Create the DataFrame with the above given sample data.
2. Group the Data: Use the class column to group the students.
3. Check the Type: Find out what kind of object is created when you group the data.
4. Calculate Age Stats: For each class, find the average (mean), youngest (min), and oldest (max) ages of the students.

HOME TASK

HOME TASK1-Sales Summary:

Dataset Link:

- Finally, print the result to see the total purchase amount for each date.

Can you use these hints to write a code that organizes the sales data and calculates the total purchase amount for each date? So, dive in and get those sales numbers sorted!

HOMETASK 2 - Customer Expenditure:

Can you write a Python program using the Pandas library to analyze customer spending based on the given data? You are provided with a sample dataset.

```
data = {'Customer': ['Alice', 'Bob', 'Alice', 'Alice', 'Bob', 'Bob', 'Alice', 'Bob'], 'Item': ['Pen', 'Pencil', 'Notebook', 'Eraser', 'Pen', 'Pencil', 'Notebook', 'Eraser'], 'Price': [10, 5, 50, 20, 10, 5, 50, 20], 'Quantity': [3, 4, 2, 5, 10, 6, 1, 2]}
```

Your task is to create a DataFrame from this data and calculate the total amount spent by each customer.

Here are some hints to get you started:

- Import the Pandas library as pd.
- Create a DataFrame from the given data using pd.DataFrame(). Calculate the total amount spent for each purchase (Price * Quantity) & add it as a new column to the DataFrame.
- Group the DataFrame by 'Customer'.
- Sum the total amount spent by each customer.
- Finally, print the total amount spent by each customer.

Your output should display the original data and the total amount spent by each customer. Ensure that your program is well-structured and easy to understand.

WHAT'S NEXT?

Mini Project-4

LESSON 35

Mini Project - 4

Mini project - Garv's DataFrames Manipulation

Gaurav have been given 2 CSV files: one with the sales data of a ice cream store and one with the expenses data. His task is to merge these files based on the days of the week, then update a value in the merged dataframe, remove a column which is not needed any more, handle any missing values, and then get some statistics about the combined data. So how would he write a Python program using pandas to accomplish all these tasks? Can you help him complete this task.

Sales data link:

To get you started, here's a step-by-step plan:

1. Read the data from 'Data of sales.csv' and 'Data of expenses.csv' into two separate DataFrames.
2. Merge these DataFrames based on the 'Days' column.
3. Update the 'Cups Sold' value for Wednesday from 50 to 55.
4. Remove the 'Cups Sold' column from the merged DataFrame.
5. Check for missing values and fill all missing values with 0.
6. Get a statistical summary of the data and display information about the DataFrame.
7. Print the final DataFrame.

SOLUTION :



```
DataFrame after Merging:
  Days  Cups Sold  Income  Expenses
0  Monday    30.0   90.0    20.0
1  Tuesday   45.0  135.0    25.0
2  Wednesday  50.0  150.0    30.0
3  Thursday   60.0  180.0    15.0
4  Friday     NaN   NaN     NaN
5  Saturday   70.0  210.0    40.0
6  Sunday    75.0  225.0    45.0

Merged DataFrame after modification:
  Days  Cups Sold  Income  Expenses
0  Monday    30.0   90.0    20.0
1  Tuesday   45.0  135.0    25.0
2  Wednesday  55.0  150.0    30.0
3  Thursday   60.0  180.0    15.0
4  Friday     NaN   NaN     NaN
5  Saturday   70.0  210.0    40.0
6  Sunday    75.0  225.0    45.0

DataFrame after Dropping 'Cups Sold' Column:
  Days  Income  Expenses
0  Monday   90.0    20.0
1  Tuesday  135.0    25.0
2  Wednesday 150.0    30.0
3  Thursday  180.0    15.0
4  Friday     NaN     NaN
5  Saturday  210.0    40.0
6  Sunday   225.0    45.0

Missing Values in Each Column:
Days      0
Income    1
Expenses  1
dtype: int64

DataFrame after Filling NaN Values:
  Days  Income  Expenses
0  Monday   90.0    20.0
1  Tuesday  135.0    25.0
2  Wednesday 150.0    30.0
3  Thursday  180.0    15.0
4  Friday    0.0     0.0
5  Saturday  210.0    40.0
6  Sunday   225.0    45.0

Statistical Summary:
              Income  Expenses
count      7.000000   7.000000
mean     141.428571   25.000000
std       77.390476  15.275252
min        0.000000   0.000000
25%      112.500000  17.500000
50%      150.000000  25.000000
75%      195.000000  35.000000
max       225.000000  45.000000

Information about the DataFrame:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Days        7 non-null      object
1   Income       7 non-null      float64
2   Expenses     7 non-null      float64
dtypes: float64(2), object(1)
memory usage: 296.0+ bytes
None
```

```
Final DataFrame:
   Days  Income  Expenses
0  Monday    90.0    20.0
1  Tuesday   135.0    25.0
2  Wednesday  150.0    30.0
3  Thursday   180.0    15.0
4  Friday     0.0     0.0
5  Saturday   210.0    40.0
6  Sunday    225.0    45.0
```

HOME TASK

HOME TASK1-Sports Survey:

Your school wants to improvise its sports program and wants to know that in which sports the students are interested in participating, and you have been selected for this survey. Now you have collected the following data :

Student	Sports
Alex	Basketball, Soccer
Ben	Baseball
Emma	Volleyball
Chloe	Tennis, Swimming
David	Basketball, Track

Using the above data and create a DataFrame then generate a frequency table that shows how many students participate in each sport. Can you provide the Python code to achieve this?

SOLUTION :

Frequency Table:		
	Sport	Frequency
0	Basketball	2
1	Soccer	1
2	Baseball	1
3	Volleyball	1
4	Tennis	1
5	Swimming	1
6	Track	1

HOMETASK 2 - Lunch Preference :

You are the person in charge of a school cafeteria, and you need to understand the lunch preferences of students from different grade levels. You have collected data on whether students prefer Packed lunches, School Cafeteria meals, or Fast Food. Here is the data you've collected:

```
data = { "Grade": ["9th", "9th", "9th", "10th", "10th", "10th", "11th", "11th",
"12th", "12th", "12th", "12th"], "Lunch Preference": ["Packed", "School
Cafeteria", "Fast Food", "Packed", "School Cafeteria", "Packed", "Fast
Food", "School Cafeteria", "Packed", "Fast Food", "School Cafeteria",
"Packed"] }
```

Using Python and Pandas, can you write a program to:

- Create a DataFrame from the given dictionary.
- Count how many students in each grade prefer each type of lunch. Calculate the proportion of each lunch preference within each grade.
- Print the frequency table showing the counts of each lunch preference for each grade.
- Print the proportions table showing the proportions of each lunch preference for each grade.

Can you provide the code to accomplish these tasks?

SOLUTION :

```

Frequency Table :
Lunch Preference Grade Fast Food Packed School Cafeteria
0 10th 0 2 1
1 11th 1 0 1
2 12th 1 2 1
3 9th 1 1 1

Proportions Table :
Lunch Preference Grade Fast Food Packed School Cafeteria
0 10th 0.000000 0.666667 0.333333
1 11th 0.500000 0.000000 0.500000
2 12th 0.250000 0.500000 0.250000
3 9th 0.333333 0.333333 0.333333

```

WHAT'S NEXT?

Assessment-3

LESSON 36

ASSESSMENT-3

Assessment - Book Genre Analysis

Sam is a data analyst at a bookstore chain, and he has been tasked with understanding the composition of the store's inventory. The bookstore maintains a comprehensive dataset titled 'Books.csv', which includes details such as the title, author, genre, and other relevant information for each book in stock. His goal is to provide insights into the distribution of fiction and non-fiction books within the inventory.

CSV file link:

This analysis will help the management team understand the book genre distribution and make informed decisions about inventory management and marketing strategies.

SOLUTION :

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 550 entries, 0 to 549
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Name            550 non-null   object
1   Author          550 non-null   object
2   UserRating      550 non-null   float64
3   Reviews         550 non-null   int64
4   Price           550 non-null   int64
5   Year            550 non-null   int64
6   Genre           550 non-null   object
dtypes: float64(1), int64(3), object(3)
memory usage: 30.2+ KB
None

Probability of selecting a fiction book: 0.43636363636363634
Probability of selecting a non-fiction book: 0.5636363636363636
```

HOME TASK

Home Task 1 - Analyzing Books Data:

You're sitting in a cozy coffee shop when your friend Sam joins you, looking puzzled and asking for your help with a coding task. Sam wants to analyze a list of books to find out which type—Fiction or Non-fiction—is usually more expensive, which type gets more reviews, and who the best author is based on user ratings.

Link for the dataset:

1. Which type of books (Fiction or Non-fiction) are usually more expensive? Which type of books gets more reviews?
2. Who is the best author based on the highest average user rating?
3. Can you write a code to load the data, perform these calculations, and print out the results?

SOLUTION :

```
The type of book that is usually more expensive: Non-fiction
The type of book that gets more reviews: Fiction
The best author according to the analysis: Alice Schertle
```

Home Task 2 - Highest Books Rating:

HeNow your friend Sam is trying to sort his list of books, but it's getting a bit chaotic. Sam wants to organize them based on their ratings, meaning he wants to filter out the books with ratings lower than 4.7 to focus on the best ones. Could you help him write a program to do that?

Steps to be followed:

1. Import the pandas library.
2. Read the CSV file into a pandas DataFrame.
3. Filter the DataFrame to include only books with ratings greater than 4.7 : Use boolean indexing to filter out the rows where the 'UserRating' column has values greater than 4.7 and Assign the filtered DataFrame to a new variable.
4. Display the filtered DataFrame.

Link for the dataset:

```

Original Data:
      Name ... Genre
0      10-Day Green Smoothie Cleanse ... Non Fiction
1      11/22/63: A Novel ... Fiction
2      12 Rules for Life: An Antidote to Chaos ... Non Fiction
3      1984 (Signet Classics) ... Fiction
4      5,000 Awesome Facts (About Everything!) (Natio... ... Non Fiction
..      ... ...
545      Wrecking Ball (Diary of a Wimpy Kid Book 14) ... Fiction
546      You Are a Badass: How to Stop Doubting Your Gr... ... Non Fiction
547      You Are a Badass: How to Stop Doubting Your Gr... ... Non Fiction
548      You Are a Badass: How to Stop Doubting Your Gr... ... Non Fiction
549      You Are a Badass: How to Stop Doubting Your Gr... ... Non Fiction

[550 rows x 7 columns]

Filtered Data with Ratings > 4.7:
      Name ... Genre
4      5,000 Awesome Facts (About Everything!) (Natio... ... Non Fiction
19      Alexander Hamilton ... Non Fiction
30      Barefoot Contessa Foolproof: Recipes You Can T... ... Non Fiction
32      Becoming ... Non Fiction
33      Becoming ... Non Fiction
..      ... ...
541      Wonder ... Fiction
542      Wonder ... Fiction
543      Wonder ... Fiction
544      Wonder ... Fiction
545      Wrecking Ball (Diary of a Wimpy Kid Book 14) ... Fiction

[179 rows x 7 columns]

```

WHAT'S NEXT?

Introduction to Seaborn and Univariate plots

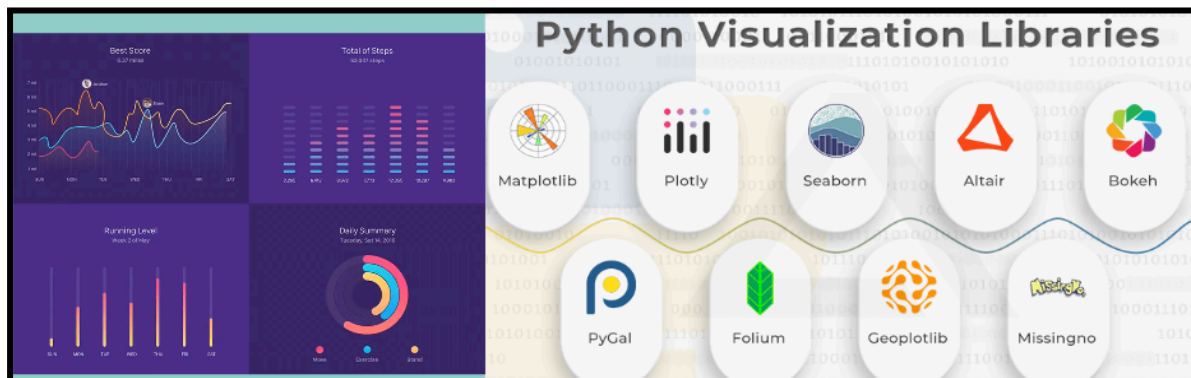
LESSON 37

Introduction to Seaborn and Univariate plots

LEARNING CONTENT

Data Visualization

In today's world, a lot of data is being generated on a daily basis. And sometimes to analyze this data for certain trends, patterns may become difficult if the data is in its raw format. To overcome this data visualization comes into play. Data visualization provides a good, organized pictorial representation of the data which makes it easier to understand, observe, analyze. Python provides various libraries that come with different features for visualizing data and can support various types of graphs.



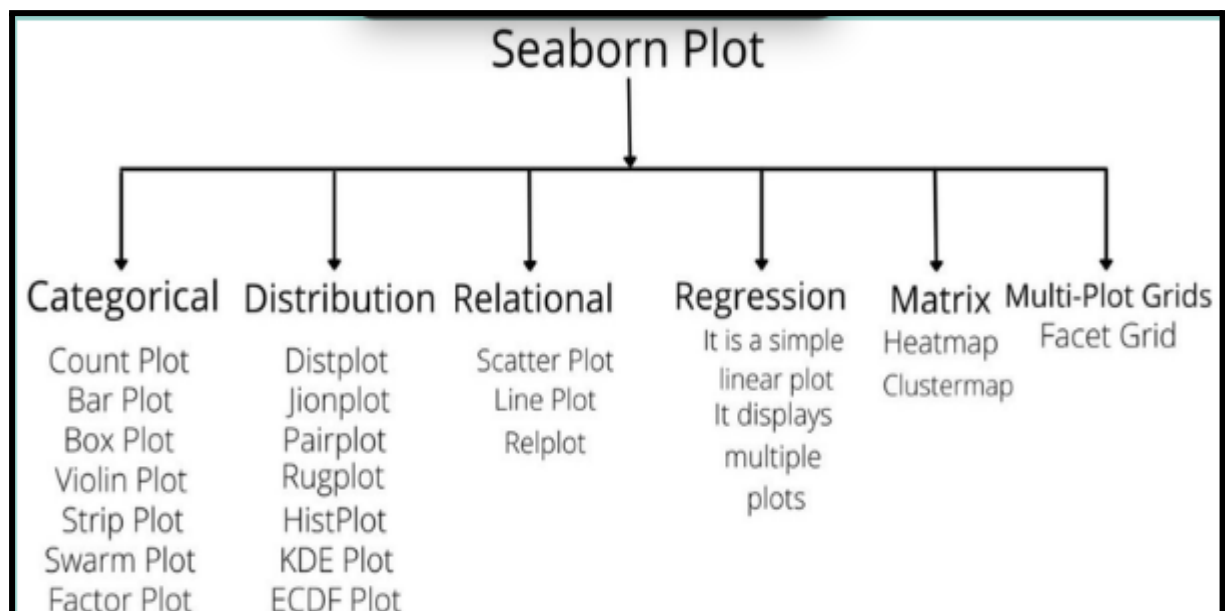
Introduction to Seaborn

What is Seaborn?

Seaborn is an amazing visualization library for statistical graphics plotting in Python. It provides beautiful default styles and color palettes to make statistical plots more attractive. It is built on top matplotlib library and is also closely integrated with the data structures from pandas.

Seaborn aims to make visualization the central part of exploring and understanding data. It provides dataset-oriented APIs so that we can switch between different visual representations for the same variables for a better understanding of the dataset.

Categories of Plots in Python's seaborn library



Distribution plots

1) Univariate plots are graphical representations of data that

involve only one variable. These plots help to understand the distribution, central tendency, spread, and outliers of the data.

- 2) Bivariate plots are graphical representations that involve two variables and are used to understand the relationship between them. These plots help to identify patterns, correlations, and potential causal relationships.

NOTE: When using Seaborn for data visualization, you typically need to import a few key modules to handle data manipulation and visualization.

Here are the essential imports:

1. Seaborn (sns): The main library for creating statistical
2. plots. Matplotlib (plt): Often used for showing and customizing plots, as Seaborn builds on top of Matplotlib.

```
import numpy as np          # For numerical operations and generating sample data
import pandas as pd         # For data manipulation and handling data frames
import seaborn as sns       # For data visualization
import matplotlib.pyplot as plt # For displaying and customizing plots
```

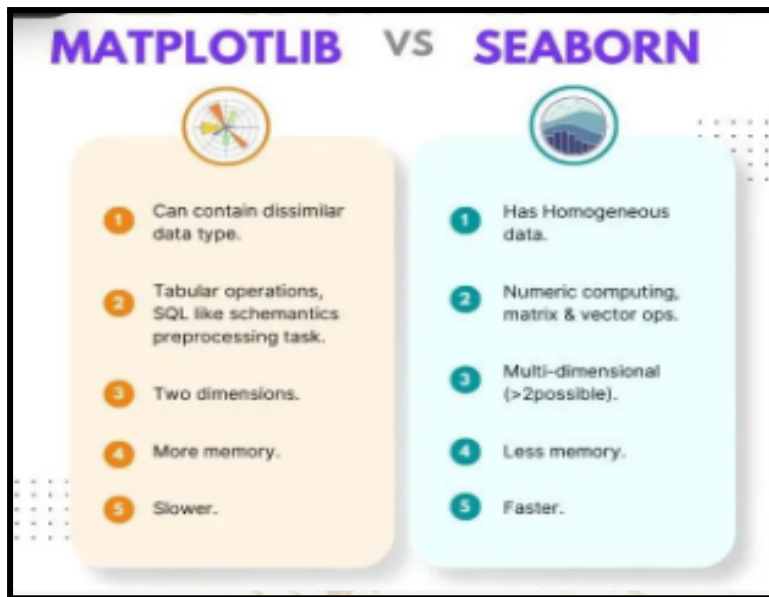
Introduction to Matplotlib

What is Matplotlib?

Matplotlib is a powerful plotting library in Python used for creating static, animated, and interactive visualizations.

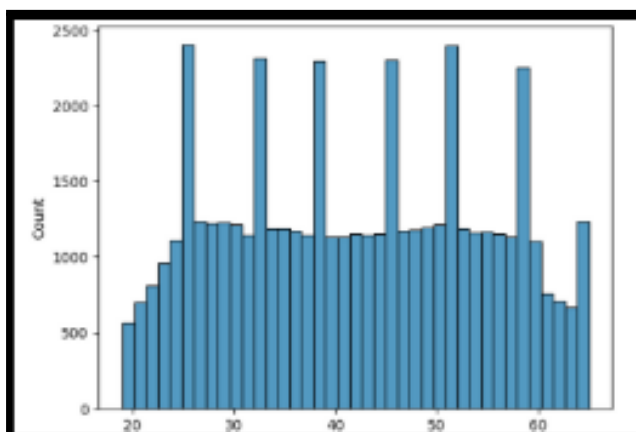
Matplotlib's primary purpose is to provide users with the tools and functionality to represent data graphically, making it easier to analyze and understand.

It is built on NumPy arrays and consists of several plots like line, bar, scatter, histogram, etc.



Univariate Analysis:

Univariate Analysis is a type of data visualization where we visualize only a single variable at a time. Univariate Analysis helps us to analyze the distribution of the variable present in the data so that we can perform further analysis.



Histplot:

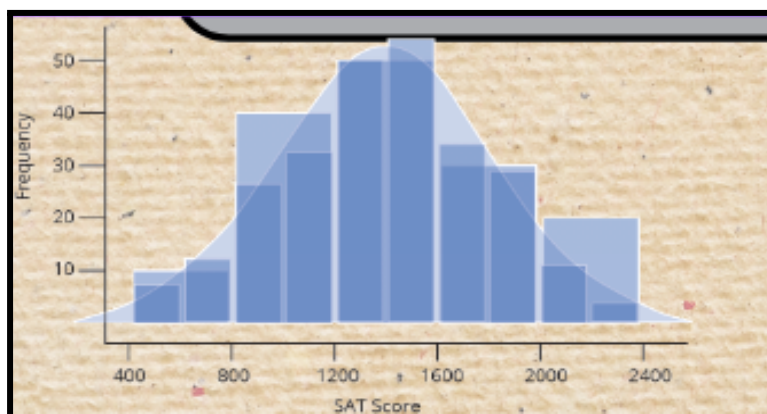
Histplot: Seaborn Histplot is used to visualize the univariate set of distributions(single variable). A Histogram represents data provided in the form of some groups. It is an accurate method

for the graphical representation of numerical data distribution. It is a type of bar plot where the X-axis represents the bin ranges while the Y-axis gives information about frequency.

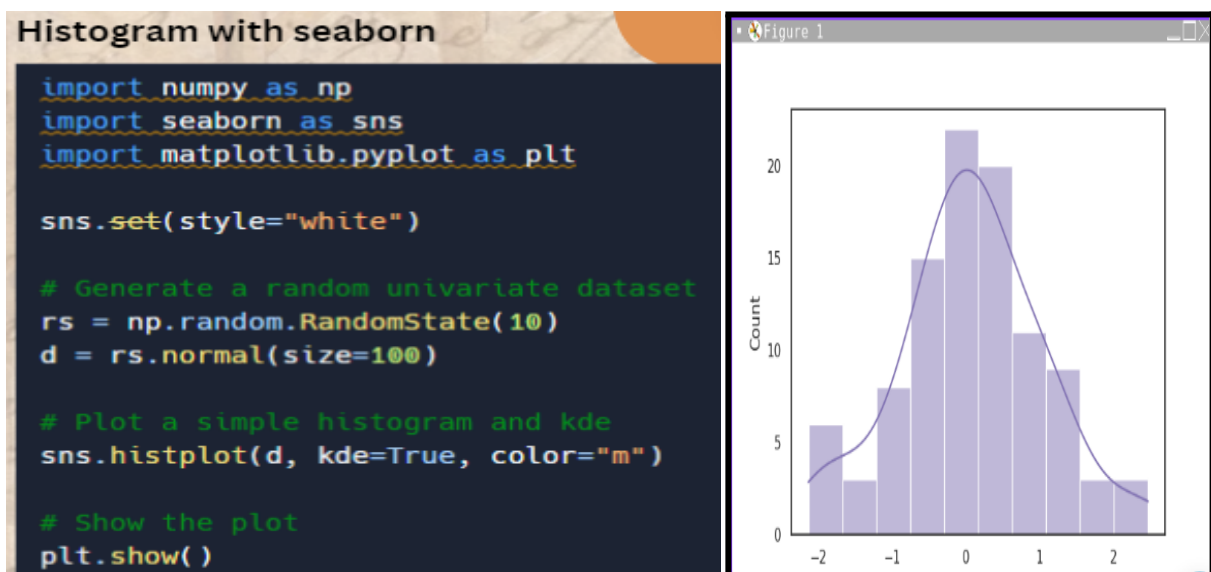
Creating a Matplotlib Histogram

To create a Matplotlib histogram the first step is to create a bin of the ranges, then distribute the whole range of the values into a series of intervals, and count the values that fall into each of the intervals. Bins are identified as consecutive, non-overlapping intervals of variables.

The `matplotlib.pyplot.hist()` function is used to compute and create a histogram of `x`.



Example:



Attribute	Parameter
x	array or sequence of array
bins	optional parameter contains integer or sequence or strings
density	Optional parameter contains boolean values
range	Optional parameter represents upper and lower range of bins
histtype	optional parameter used to create type of histogram [bar, barstacked, step, stepfilled], default is "bar"
align	optional parameter controls the plotting of histogram [left, right, mid]
weights	optional parameter contains array of weights having same dimensions as x

bottom	location of the baseline of each bin
rwidth	optional parameter which is relative width of the bars with respect to bin width
color	optional parameter used to set color or sequence of color specs
label	optional parameter string or sequence of string to match with multiple datasets
log	optional parameter used to set histogram axis on log scale

Displot:

The displot function in Seaborn is used for creating distribution plots, which are useful for visualizing the distribution of a single variable. This function combines aspects of distplot, kdeplot and histplot to provide a more flexible and powerful way to visualize distributions. The distribution and range of a collection of numeric values are represented against a dimension in a distribution plot.

Syntax:

```
seaborn.displot(data=None, *, x=None, y=None, hue=None,
row=None, col=None, col_wrap=None, row_order=None,
col_order=None, height=5, aspect=1, facet_kws=None,
kind="hist", rug=False, rug_kws=None, kde=False,
kde_kws=None, bins=None, binwidth=None, binrange=None,
discrete=False, stat="count", common_bins=True,
common_norm=True, multiple="layer", element="bars",
fill=True, shrink=1, color=None, palette=None,
hue_order=None, hue_norm=None, log_scale=None,
legend=True, ax=None)
```

Parameters and Description

Data Parameters

- data: Dataset for plotting.
- x, y: Variables for x and y axes.
- hue: Variable for color subsets.
- row, col: Variables for facet rows and columns.
- col_wrap: Number of columns before wrapping.
- row_order, col_order: Order of rows and columns.

Size Parameters

- height: Height of each facet.
- aspect: Aspect ratio of each facet.
- facet_kws: Additional FacetGrid parameters.

Plot Type and Style Parameters kind:

- Type of plot ('hist', 'kde', 'ecdf').
- rug: Add a rug plot (True/False).
- rug_kws: Rug plot parameters.
- kde: Add a KDE plot (True/False).
- kde_kws: KDE plot parameters.

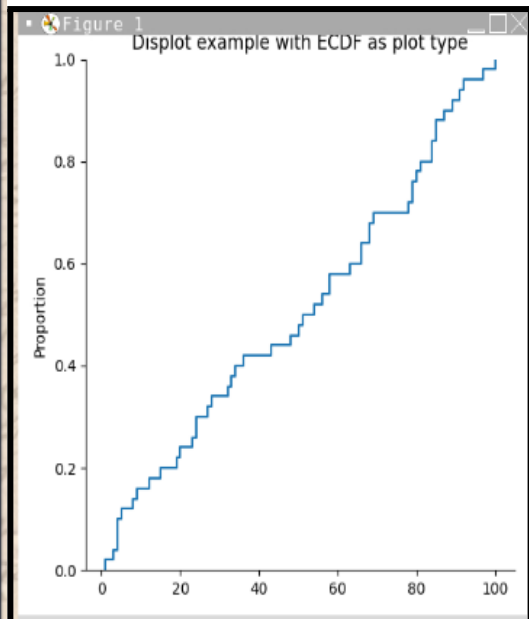
```
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

# Generate sample data
# Random numbers between 1 and 100
sample_data = np.random.randint(1, 101, 50)

# Create ECDF plot
sns.displot(sample_data, kind='ecdf')

# Add title
plt.title("Displot example with ECDF as plot type")

# Show the plot
plt.show()
```



IN CLASS CHALLENGES

Inclass Challenge 1 - Analyzing Scores:

Imagine you are a teacher and you have exam scores for three subjects: Math, Science and English. You have saved these scores in a CSV file called test scores.csv. The CSV file has two columns: Subject and Scores.

Your task is to create a histplot using histograms to visualize the distribution of scores for each subject. You will use Python with the Pandas, Seaborn and Matplotlib libraries to do this. Can you write a Python program that reads the exam scores from the CSV file and creates a separate histogram for each subject, showing the distribution of scores? Each histogram should also display the mean and median scores with vertical lines.

File

link:

Hint:

Start by importing the necessary libraries: pandas, seaborn, and matplotlib.pyplot.

1. Read the CSV file into a Pandas DataFrame. ‘
2. Set up the plot style using Seaborn.
3. Create a figure with multiple subplots.
4. Loop through each subject to create individual histograms.
5. Calculate the mean and median scores for each subject.
6. Add vertical lines for the mean and median scores on each histogram.
7. Display the plot with all the histograms.

Inclass Challenge 2 - Sales Visualization

Imagine you are a data analyst at an e-commerce company. You have a dataset containing sales data for different product categories like 'Electronics', 'Clothing', 'Home & Garden', 'Books', and 'Toys'. Your task is to visualize how sales amounts are distributed across these categories. You will use Python to simulate the data and create a Seaborn's displot that shows the distribution of sales amounts for each category using Seaborn.

Instructions:

First, you will simulate some sales data: Create a dataset with 1000 transactions. Each transaction should have a category randomly chosen from 'Electronics', 'Clothing', 'Home & Garden', 'Books', and 'Toys'. Generate random sales amounts for these transactions, making sure the minimum sales amount is 10.

Next, use Seaborn's displot to visualize the distribution of sales amounts for each category: Plot the data to show how the sales amounts are spread for each product category. Use a KDE

(Kernel Density Estimate) plot and fill the area under the curves. Label the axes and give the plot a title.

Can you write the Python code to accomplish this task?

HOME TASK

Home Task 1 - Sea Shell Collection

One day you and your friends went on to the beach, and each of you collected seashells. Now, you want to see how many seashells everyone collected in a cool and colorful graph.

Here's what you know about the number of seashells each friend collected: `seashells_collected = [10, 15, 10, 20, 25, 10, 5, 30, 10, 20, 15, 25]`

Can you write a Python program that:

- Creates a list of the number of seashells each friend collected.
- Uses Seaborn to create a histogram that shows the number of seashells collected.
- Adds a title and labels to the histogram to make it easy to understand.

Let's see if you can make an awesome graph to show everyone's seashell collection!

Home Task 2 - Plotting Ice Cream Flavors

Imagine we conducted a survey among your friends to find out their favorite ice cream flavors. We collected the following responses, repeated five times:

1. Chocolate
2. Vanilla
3. Vanilla
4. Strawberry
5. Mint
6. Cookie Dough
7. Cookie Dough

We want to visualize these responses using a histogram with a density curve to see which ice cream flavor is the most popular among your friends. Follow these steps to create the plot using Python and Seaborn:

- Import the necessary libraries
- Create data: Store the ice cream flavors in a list. The list should contain the given flavors repeated 5 times.
- Create a DataFrame: Use the pandas library to create a DataFrame from the list.
- Create the plot: Use `sns.displot` from the seaborn library to create a histogram of the favorite ice cream flavors. Set the `x` parameter to 'Flavor'. Ensure you add a kernel density estimate (KDE) line by setting the `kde` parameter to `True`. Specify that the plot type is a histogram by setting the `kind` parameter to 'hist'.
- Add labels: Use `plt.xlabel` and `plt.ylabel` to add the label to the x and y-axis.

Finally, use `plt.show` to display the plot.

WHAT'S NEXT?

More on Univariate plots for numerical data

LESSON 38

More on Univariate plots for numerical data

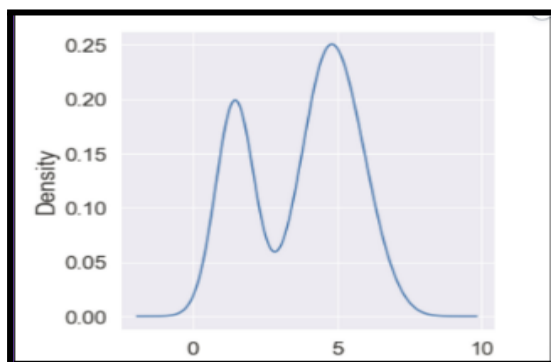
LEARNING CONTENT

More on Univariate plots for numerical data

A density plot is like a smoother version of a histogram. Generally, the kernel density estimate is used in density plots to show the probability density function of the variable. A continuous curve, which is the kernel is drawn to generate a smooth density estimation for the whole data.

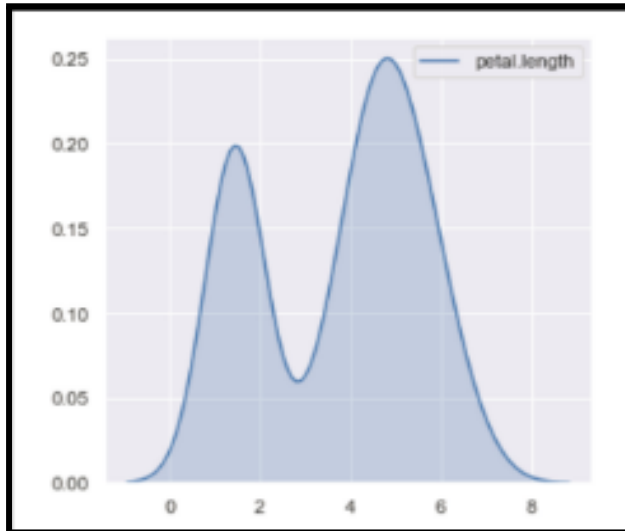
Plotting density plot of the variable 'petal.length' :

```
plt.figure(figsize=(5,5))  
df['petal.length'].plot(kind='density')
```



we use the pandas `df.plot()` function (built over matplotlib) or the seaborn library's `sns.kdeplot()` function to plot a density plot .

```
sns.set(rc={'figure.figsize': (5,5)})  
sns.kdeplot(df['petal.length'], shade=True)
```



Syntax

`sns.kdeplot( data=None, x=None, y=None, hue=None, shade=True, bw_adjust=None, cut=None, clip=None, color='blue', fill=True`

Parameters

- `data`: DataFrame or array-like, the dataset for plotting.
- `x, y`: Variables that specify positions on the x and y axes.
- `hue`: Semantic variable that determines the color of plot elements.
- `shade`: Whether to shade the area under the KDE curve.
- `bw_adjust`: Factor adjusting the bandwidth of the KDE curve.
- `cut`: Factor extending the density plot beyond the data extremes.
- `clip`: Tuple (min, max) for bounding the density plot.
- `color`: Color of the KDE plot. `fill`: If True, fills in the KDE plot.

These parameters allow you to control the appearance and behavior of your density plot in Seaborn, making it easy to customize and visualize your data distribution effectively.

Example : In the above example we are loading a dataset from Seaborn's built-in datasets and then Using the `sns.kdeplot` function to create a density plot

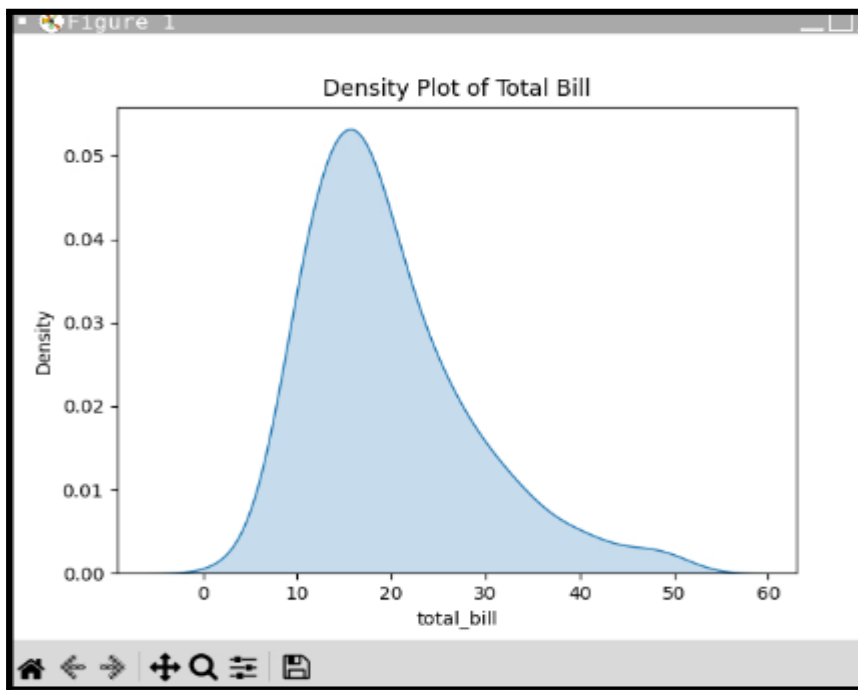
```
import seaborn as sns
import matplotlib.pyplot as plt

# Load the example tips dataset
tips = sns.load_dataset("tips")

# Create a density plot
sns.kdeplot(data=tips, x="total_bill", fill=True)

# Add a title
plt.title('Density Plot of Total Bill')

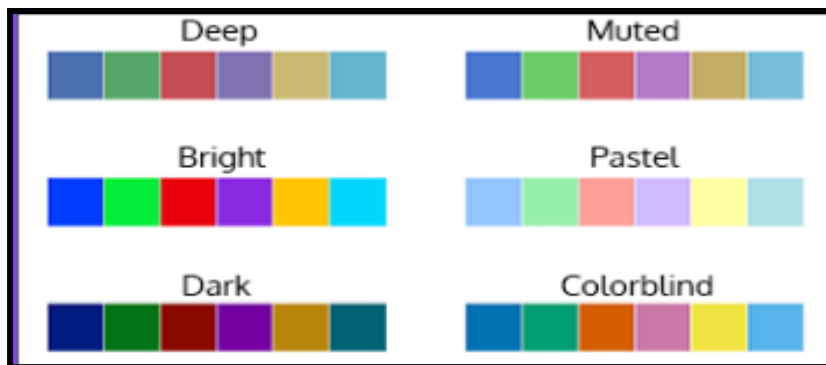
# Show the plot
plt.show()
```



Color Palette In Seaborn:

Seaborn is a powerful Python visualization library built on top of Matplotlib that provides a high-level interface for drawing attractive statistical graphics. One of its features is the ability to easily create and customize color palettes. Seaborn has several built-in color palettes

such as deep, muted, bright, pastel, dark, and colorblind.



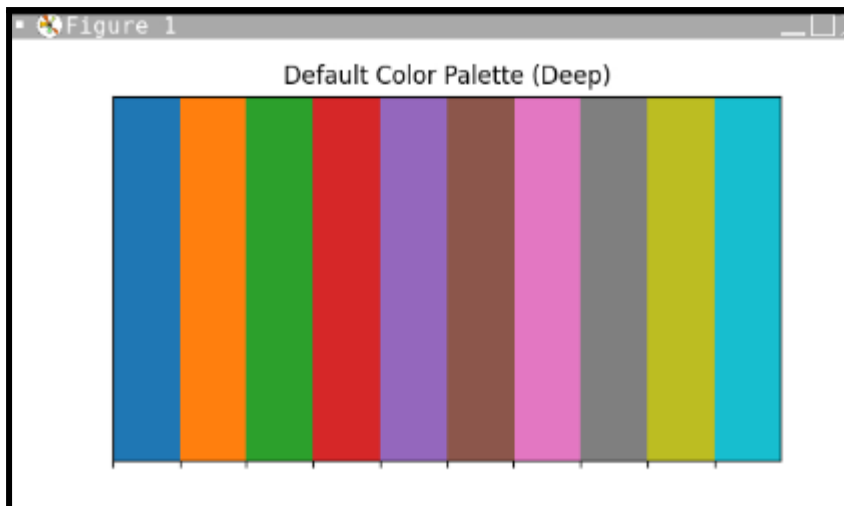
By default, Seaborn uses the deep palette, which consists of 10 colors. You can access this palette using the `color_palette` function without any arguments.

The `sns.palplot` function in Seaborn is used to visualize color palettes. It displays the colors in a given palette as a horizontal array of rectangles, allowing you to see the sequence of colors in the palette.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Access the default color palette
default_palette = sns.color_palette()

# Visualize the default color palette
sns.palplot(default_palette)
plt.title('Default Color Palette (Deep)')
plt.show()
```



Changing to Another Preset Palette:

If you want to change the default palette to another built-in or preset palette, you can use the `set_palette` function. For example, you can set the palette to pastel.

Example :

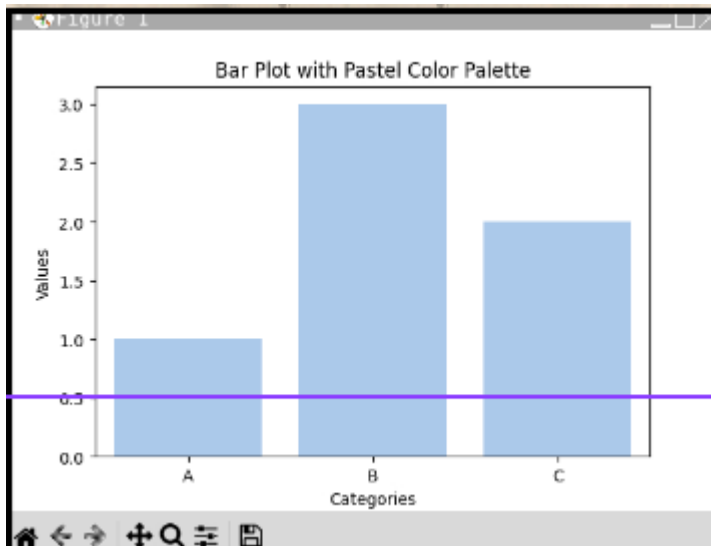
```
import seaborn as sns
import matplotlib.pyplot as plt

# Set a different color palette
sns.set_palette("pastel")

# Create a bar plot using the new color palette
sns.barplot(x=["A", "B", "C"], y=[1, 3, 2])

# Add labels and title
plt.xlabel('Categories')
plt.ylabel('Values')
plt.title('Bar Plot with Pastel Color Palette')

# Show the plot
plt.show()
```



IN CLASS CHALLENGES

INCLASS CHALLENGE1 - Math Test Scores :

Ms. Taylor, a math teacher, recently gave her students a math test. She wants to visualize how her students performed to understand the distribution of their scores. She decides to use a density plot to see if most students scored around the average or if there were many highs and lows. Let's help Ms. Taylor by plotting the density of the math test scores using Seaborn.

Hints:

- Import Libraries
- Create Sample Data: Use numpy to create random test scores for 30 students. The scores are between 50 and 100.
- Use `sns.kdeplot()` to create a density plot of the scores, use `shade=True` parameter which fills the area under the curve and `color="blue"` sets the color of the plot.
- Add a title to the plot and labels for the x-axis (Scores) and y-axis (Density) to make it understandable.
- Finally, use `plt.show()` to display the plot.

INCLASS CHALLENGE 2- Colorful Plotting:

Hey kiddo! Let's play with some data and create cool plots using Python. Imagine we have two groups of kids, Group A and Group B. We measured their heights and found that Group A has an average height of 50 inches, and Group B has an average height of 55 inches.

Can you write a Python code to generate random height data for both groups and then create two density plots to compare their heights? One plot should use the default color palette, and the other should use a 'dark' color palette.

Steps to be followed:

- ☐ Use the numpy library to create random height data for both groups. Create a DataFrame, Combine the heights of both groups & add a column to indicate the group.
- ☐ Use matplotlib.pyplot to create the figure and subplots.
- ☐ Use seaborn.kdeplot to create the density plots.
- ☐ For the first plot, use the default color palette.
- ☐ For the second plot, set the palette to 'dark' using `sns.set_palette('dark')`.

HOME TASK

HOME TASK1-Collection Of Items:

You're organizing a scavenger hunt in your neighborhood park, and a group of kids participated. Each kid sets out to find hidden treasures, and after the hunt, you record the number of items found by each of them:

Items found by each kid = { 'Alice': 8, 'Bob': 10, 'Charlie': 7, 'David': 12, 'Eva': 9, 'Frank': 11, 'Grace': 6, 'Henry': 8, 'Ivy': 10, 'Jack': 9 }

So how would you generate a Python code to create a Seaborn density plot showcasing the distribution of items found by each kid?

Hints to get started:

1. Data Preparation: You already have the data in a dictionary format. You'll need to extract the values (number of items found) from this dictionary.
2. Import Libraries: Import necessary libraries like Seaborn, Matplotlib, and NumPy.
3. Create Data Structure: Convert the extracted values into a list.
4. Plotting: Use Seaborn to create a density plot (`sns.kdeplot`) of the data. Customization: Add labels, title, and adjust plot aesthetics as needed. Display: Show the plot using `plt.show()`.

HOMETASK 2 - Time Spent On Activities :

Imagine you have collected data on how much time you and your friends spend on different activities each week. You have data for the following activities: Studying, Playing, Watching TV, and Reading. The data looks like this:

1. Studying: [2, 3, 3, 4, 5, 6, 7, 8, 3, 4]
2. Playing: [1, 2, 2, 3, 4, 5, 5, 6, 7, 8]
3. Watching TV: [0, 1, 1, 2, 2, 3, 3, 4, 4, 5]
4. Reading: [1, 1, 2, 2, 3, 3, 4, 4, 5, 6]

Can you write a Python program using seaborn and matplotlib to create a colorful density plot that shows how much time is spent on each of these activities? Use the 'bright' color palette to make it fun and engaging.

Here's a hint: You'll need to create a dictionary for the data, convert it into a DataFrame, Set the color palette using `sns.set_palette('bright')` and then use a loop to plot each activity's density separately with `sns.kdeplot`. Make sure to add a legend to distinguish between different activities.

WHAT'S NEXT?

Bivariate plots of Numerical Data

LESSON 39

Bivariate plots of Numerical Data

LEARNING CONTENT

Bivariate Analysis of Numerical data

Bivariate in Python refers to the analysis involving two variables. It uses statistical methods and visualizations to explore the relationship and interactions between these two variables in a dataset. In bivariate analysis, it might be observed that one variable (especially the Xs) is causing Y to change. As shown in the image, though it would be seen that both sunburn and ice cream sales are correlated, ice-creams do not cause sunburn

Scatter Plot

Scatter plots are used to display the relationship between two continuous numerical variables. A scatter plot (or scatter diagram) is a type of plot that uses Cartesian coordinates to display values for typically two variables for a set of data. The data is displayed as a collection of points, each having the value of one variable determining the position on the horizontal axis and the value of the other variable determining the position on the vertical axis.

Syntax

`sns.scatterplot(data=None, *, x=None, y=None, hue=None, style=None, size=None, palette=None, sizes=None, markers=True, edgecolor=None, linewidth=0, ax=None, **kwargs)`

Parameters

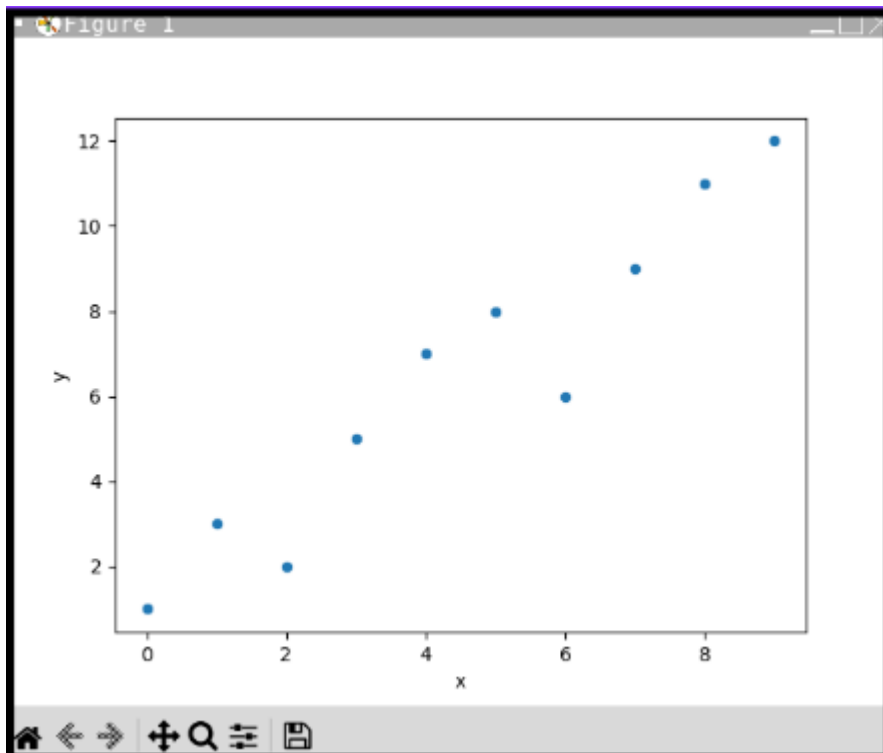
- `data`: Dataframe where each column is a variable and each row is an observation.
- `x, y`: Input data variables that should be numeric.
- `hue`: Name of variables in data, optional - Grouping variable to produce points with different colors.
- `style`: Name of variables in data, optional - Grouping variable to produce points with different markers.
- `size`: Name of variables in data, optional - Grouping variable to produce points with different sizes.
- `palette`: Palette name, list, or dict, optional - Colors to use for the different levels of the hue variable.
- `sizes`: List, dict, or tuple, optional - Sizes to use for the different levels of the size variable.
- `markers`: Boolean or dict, optional - Marker style.
- `edgecolor`: Color for the edges of the points.
- `linewidth`: Width of the lines around the points.
- `ax`: Matplotlib Axes, optional - Axes object to draw the plot onto.
- `kwargs`: Additional keyword arguments to pass to the underlying plotting function.

Example :

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Sample data
data = pd.DataFrame({'x': range(10),
                     'y': [1, 3, 2, 5, 7, 8, 6, 9, 11, 12]})

sns.scatterplot(x='x', y='y', data=data)
plt.show()
```



LinePlot:

A line plot (or line chart) is a type of plot that displays information as a series of data points called 'markers' connected by straight line segments. It is commonly used to visualize the trend of a variable over time or another continuous variable. Line plots are primarily used with numerical data. They are ideal for representing continuous data where the order of data points is significant.

Syntax

```
sns.lineplot(data=None, *, x=None, y=None, hue=None, size=None,
style=None, units=None, estimator='mean', ci=95, n_boot=1000,
sort=True, err_style='band', err_kws=None, legend='brief', ax=None,
**kwargs)
```

Parameters :

- x, y: Input data variables, must be numeric.
- data: Dataframe where each column is a variable and each row is an observation.

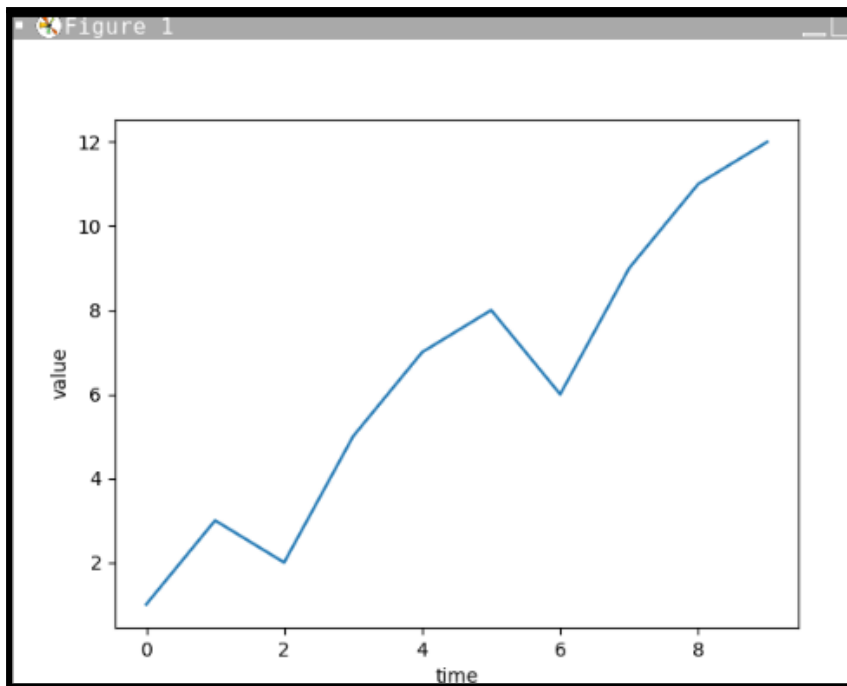
- size: Name of variables in data, optional - Grouping variable that will produce lines with different widths.
- style: Name of variables in data, optional - Grouping variable that will produce lines with different dashes and/or markers.
- hue: Name of variables in data, optional - Grouping variable to produce lines with different colors.
- units: Name of variables in data, optional - When plotting long-form data, this will split the data into multiple units.
- estimator: Callable that maps vector -> scalar, optional - Method for aggregating data points.
- ci: int or 'sd' or None, optional - Size of confidence intervals to draw. n_boot: int, optional - Number of bootstrap iterations to use when computing confidence intervals.
- sort: boolean, optional - If True, sort data by x before plotting.
- err_style: 'band' or 'bars', optional - Style of the error bars.
- err_kws: dict of keyword arguments, optional - Additional parameters to pass to error bar function.
- legend: 'brief', 'full', or False, optional - How to draw the legend.
- ax: Matplotlib Axes, optional - Axes object to draw the plot onto.
- kwargs: Additional keyword arguments to pass to the underlying plotting function.

Example :

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# sample data
data = pd.DataFrame({'time': range(10),
                    'value': [1, 3, 2, 5, 7, 8, 6, 9, 11, 12]})

sns.lineplot(x='time', y='value', data=data)
plt.show()
```



IN CLASS CHALLENGES

Inclass Challenge 1 - Tip & Bill Visualization

Nick, a data analyst is tasked to visualize the relationship between the total bill amount and the tip amount given by customers at a restaurant. So can you help Nick to determine if there is a correlation between these two variables and also see how this relationship varies across different days of the week.

Steps to be followed:

- Load the Data: Use Seaborn's built-in 'tips' dataset.
- Visualize with Scatter Plot: Create a scatter plot to visualize the relationship between the total bill and the tip amount.
- Enhance the Plot: Use different colors to distinguish data points from different days of the week.

NOTE: tips dataset is one of the example datasets built into the seaborn package. It can be easily loaded using the `seaborn load_dataset` command.

Inclass Challenge 2 - Monthly Sales Trends

Sachin is having a CSV file called monthly sales.csv. The CSV file has columns for Month, Product A, Product B, and Product C and this file contains monthly sales data for three different products (Product A, Product B, and Product C) over the past year. Can you write a Python script that uses the Seaborn library to create a line plot showing the sales trends for each product over the year? Your plot should have different colored lines for each product, include markers at each data point, and have appropriate titles and labels.

CSV file link:

☐☐

- Import the necessary libraries (Seaborn, Matplotlib, Pandas).
- Create a DataFrame with the given data.
- Use Seaborn to create a scatter plot of study hours versus exam scores. Add titles and labels to your plot.
- Display the plot.

Can you create a scatter plot using the given data?

Home Task 2 - Tracking Plant Growth

Imagine you are tracking the growth of your favorite plant over five weeks. Each week, you measure the height of the plant in centimeters and record the following data:

Week 1 = 3 cm

Week 2 = 6 cm

Week 3 = 9 cm

Week 4 = 12 cm

Week 5 = 15 cm

Now using this data, can you create a line plot to visualize the plant's growth over time? just make sure your plot should have markers at each data point, and also include appropriate titles and labels for the axes. Give it a try and see how your plant's growth looks on the graph!

WHAT'S NEXT?

More on Bivariate plots of Numerical Data

LESSON 40

More on Bivariate plots of Numerical Data

LEARNING CONTENT

Jointplot

A jointplot is used to display a bivariate relationship between two variables along with the univariate distribution of each variable. It combines a scatter plot (or another kind of bivariate plot) with marginal histograms or density plots.

This function provides a convenient interface to the 'JointGrid' class, with several canned plot kinds. This is intended to be a fairly lightweight wrapper; if you need more flexibility, you should use :class:'JointGrid' directly.

Syntax

```
sns.jointplot(x=None, y=None, data=None, kind='scatter', hue=None,
hue_order=None, palette=None, height=6, ratio=5, space=0.2,
dropna=True, xlim=None, ylim=None, marginal_kws=None,
joint_kws=None, annot_kws=None, **kwargs)
```


Parameters:

- x, y: Names of variables in data.
- data: DataFrame containing the data.
- kind: Type of plot to draw, options include "scatter", "kde", "hist", "hex", "reg".
- hue: Variable in data for color encoding.
- hue_order: Order for the levels of the hue variable.
- palette: Colors to use for different levels of the hue variable.
- height: Size of the figure.
- ratio: Ratio of joint axis size to marginal axes height.
- space: Space between the joint and marginal axes.
- dropna: If True, drop missing values from the data.
- xlim, ylim: Limits for the x and y axes.
- marginal_kws: Additional keyword arguments for the marginal plots.
- joint_kws: Additional keyword arguments for the joint plot.
- annot_kws: Additional keyword arguments for the annotations.

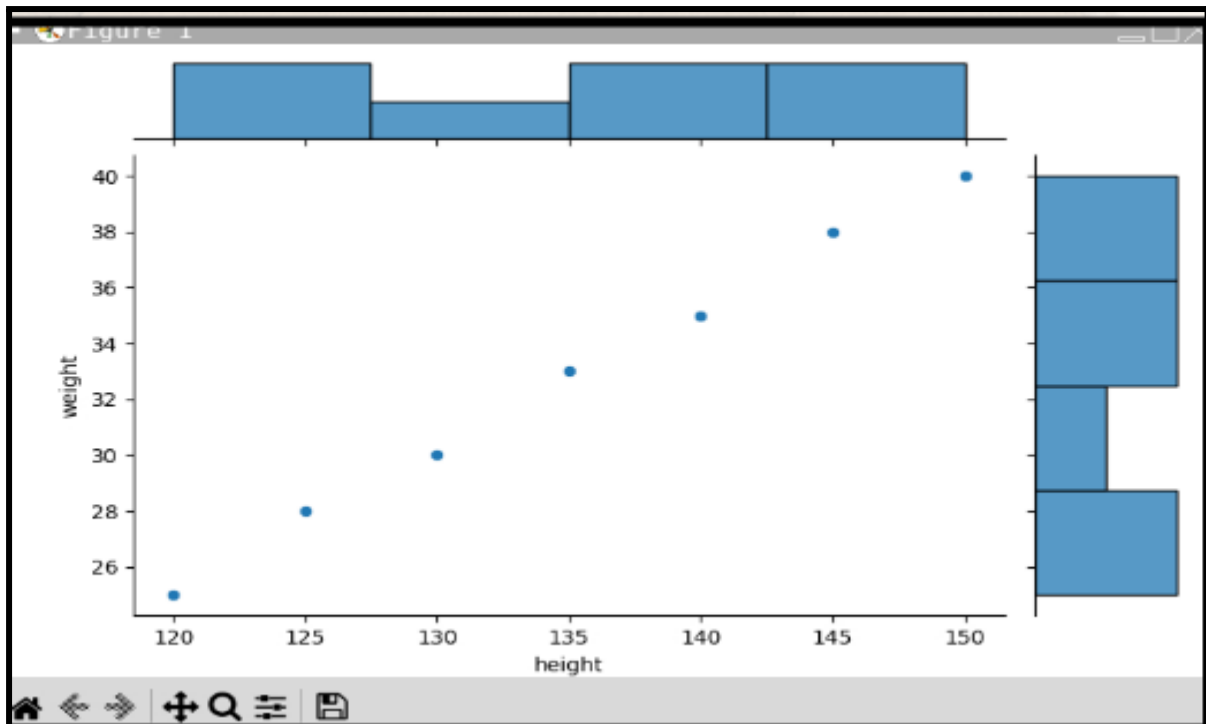
Example :

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Create data
data = pd.DataFrame({"height": [120, 130, 125, 140, 135, 150, 145],
                    "weight": [25, 30, 28, 35, 33, 40, 38]})

# Create a joint plot for height vs weight
sns.jointplot(x="height", y="weight", data=data, kind="scatter")

# Show the plot
plt.show()
```



Pairplot:

A pairplot is used to visualize pairwise relationships in a dataset. It creates a matrix of plots showing the relationship between each pair of variables in the dataset. To plot multiple pairwise bivariate distributions in a dataset, you can use the `pairplot()` function. This shows the relationship for (n, 2) combination of variable in a DataFrame as a matrix of plots and the diagonal plots are the univariate plots.

Syntax

```
sns.pairplot( data, hue=None, hue_order=None, palette=None,
vars=None, x_vars=None, y_vars=None, kind='scatter', diag_kind='auto',
markers=None, height=2.5, aspect=1, dropna=True, plot_kws=None,
diag_kws=None, grid_kws=None, corner=False )
```

Parameters:

- `data`: DataFrame containing the data.
- `hue`: Variable in data for color encoding.
- `hue_order`: Order for the levels of the hue variable.
- `palette`: Colors to use for different levels of the hue variable.

- vars: Variables to plot, or None to plot all numeric variables.
- x_vars, y_vars: Variables to plot on the x and y axes respectively.
- kind: Kind of plot for non-diagonal subplots ("scatter", "kde", "hist", "reg").
- diag_kind: Kind of plot for diagonal subplots ("auto", "hist", "kde").
- markers: Markers to use for different levels of the hue variable.
- height: Height of each facet, aspect of each facet (width/height).
- aspect: Aspect ratio of each facet.
- dropna: If True, drop missing values from the data.
- plot_kws: Additional keyword arguments for non-diagonal plots.
- diag_kws: Additional keyword arguments for diagonal plots.
- grid_kws: Additional keyword arguments for the entire grid.
- corner: If True, only plot the lower triangle of the pair grid.

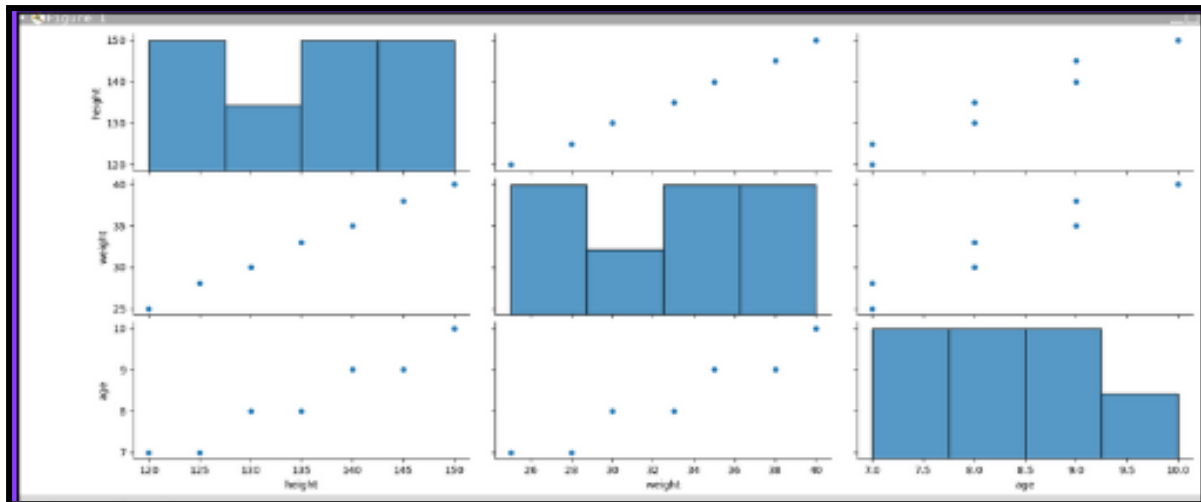
Example :

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Create data
data = pd.DataFrame({"height": [120, 130, 125, 140, 135, 150, 145],
                     "weight": [25, 30, 28, 35, 33, 40, 38],
                     "age": [7, 8, 7, 9, 8, 10, 9]})

# Create a pair plot for all columns
sns.pairplot(data)

# Show the plot
plt.show()
```



IN CLASS CHALLENGES

INCLASS CHALLENGE 1 - Exercise & Energy :

A scientist is studying the secret for having super high energy levels! He has gathered data from a group of superhero trainees about how many hours they spend exercising each week and how energetic they feel. Here is the data:

exercise hours = [2, 3, 2.5, 4, 3.5, 5, 4.5] and

energy levels = [70, 80, 75, 85, 78, 90, 88]

Now, can you write a Python code to create a cool graph to visualize this data and uncover the secret for having super high energy levels?

Hints:

- You'll need to use Python libraries like pandas, seaborn, and matplotlib.pyplot to create your graph.
- Look into the `pd.DataFrame()` function to create a DataFrame to store your data.
- Explore the `sns.jointplot()` function to create a graph that shows the relationship between exercise hours and energy levels.
- Consider setting the `kind` parameter of `sns.jointplot()` to "kde" to create a plot with smooth curves.
- Don't forget to add labels to your graph to make it easy to understand. You can use `plt.xlabel()` and `plt.ylabel()` for this.
- Finally, use `plt.show()` to display your awesome graph to the world!

INCLASS CHALLENGE 2- Magical Zoo Adventure:

Imagine you are a young scientist working at a magical zoo where animals have special powers. Each type of animal has unique characteristics, and you want to understand how these characteristics relate to each other. You have collected data on three magical animals: Griffins, Phoenixes and Unicorns.

For each animal, you measured three magical traits:

- Power Level
- Speed
- Friendliness

Now, can you visualize the relationships between these traits to see if there are any interesting patterns? HINT: Your mentor told you about a special plot called a "pair plot" that can help you do this. To make it even more magical, you can use something called "Kernel Density Estimation" (KDE) to make the plots smoother and more beautiful.

Write a Python program to create a pair plot with KDE for these magical traits. Use the following data for each trait:

- ☐ Power Level = `np.random.normal(size=100)`
- ☐ Speed = `np.random.normal(size=100)`
- ☐ Friendliness = `np.random.normal(size=100)`

HOME TASK

Home Task 1 - Home Value Exploration

Imagine you have some data that tells you how big houses are and how much they cost. You want to see if there's a connection between these two things: do bigger houses always cost more?

Data : Price = [300000, 400000, 500000, 600000, 700000],

Size = [1500, 1800, 2000, 2200, 2500],

Bedrooms = [3, 4, 3, 4, 4]

Your job is to write some simple instructions for Python to make a picture that shows how house size and price are related.

This picture will make it easy for you to see if there's a pattern or if it's just random. You'll need to use Seaborn's joint plot function, which will create a scatter plot. In this scatter plot, the size of the houses should be on the x-axis, the price should be on the y-axis, and each dot on the plot represents a house.

Once it's done, you'll be able to see if there's a secret rule about how big houses are priced. Ready to give it a try

HOMETASK 2 - Cars:

Sam is in charge of organizing a school science fair, and one of the projects involves analyzing data about different types of cars. He want to create a visually appealing presentation to showcase the relationships between various features of these cars. To do this, he decide to use Python and its data visualization libraries.

Your task is to help Sam to write a Python script that loads a dataset containing details about cars (like their horsepower, weight, and miles per gallon) and create a pair plot using Seaborn.

This pair plot will display scatterplots of these numerical features against each other, helping viewers understand any potential correlations or patterns in the data.

Can you write the Python code to accomplish this task?

Link of the
dataset:

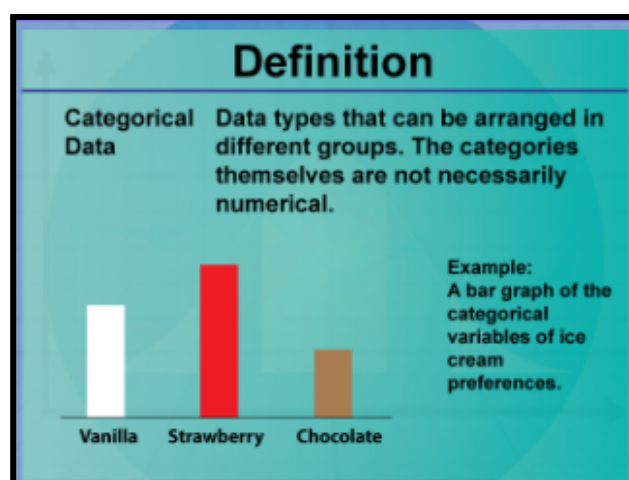
LESSON 41

Plots for Categorical Data

LEARNING CONTENT

Categorical Data

Data that can be categorized or grouped is called categorical data. It is a type of data in statistics that stores data into groups or categories using names or labels, and it can be derived from observations made of qualitative data that are summarized as counts or from observations of quantitative data grouped within given intervals. Categorical data is also well-known as qualitative data.



Barplot

Barplot is basically used to aggregate the categorical data according to some methods and by default it's the mean. It can also be understood as a visualization of the group by action. To use this plot we choose a categorical column for the x-axis and a numerical column for the y-axis, and we see that it creates a plot taking a mean per categorical column.

The bar plot can also be used for univariate data visualization plot on a two-dimensional axis. One axis is the category axis indicating the category, while the second axis is the value axis that shows the numeric value of that category, indicated by the length of the bar.

The Seaborn bar plots are versatile and can be easily customized to fit the specific requirements of your data visualization needs.

Syntax

```
seaborn.barplot(x=None, y=None, hue=None, data=None,
order=None, hue_order=None, estimator=<function mean>,
ci=95, n_boot=1000, units=None, orient=None, color=None,
palette=None, saturation=0.75, errcolor='.26', errwidth=None,
capsize=None, dodge=True, ax=None, **kwargs)
```

Parameter:

- x, y: Variables that specify positions on the x and y axes.
- hue: Variable to split data by color.
- data: DataFrame or array for plotting.
- order, hue_order: Order of categories for x axis and hue levels.
- estimator: Function to estimate within each category (default is mean).
- ci: Size of confidence intervals (default is 95, can be "sd" or None).
- n_boot: Number of bootstrap iterations (default is 1000).
- units: Identifier for sampling units for multilevel bootstrap.

- orient: Orientation of the plot, vertical or horizontal.
- color, palette: Color for bars or palette for different hues.
- saturation: Proportion of color saturation (default is 0.75).
- errcolor, errwidth, capsize: Error bars color, width, and cap size.
- dodge: Whether to separate hue levels along the categorical axis.
- ax: Matplotlib Axes to draw the plot on.
- kwargs: Other keyword arguments passed to ax.bar().

EXAMPLE: In this example, we'll visualize the average tips by day of the week using the built-in "tips" dataset from Seaborn to create a bar plot.

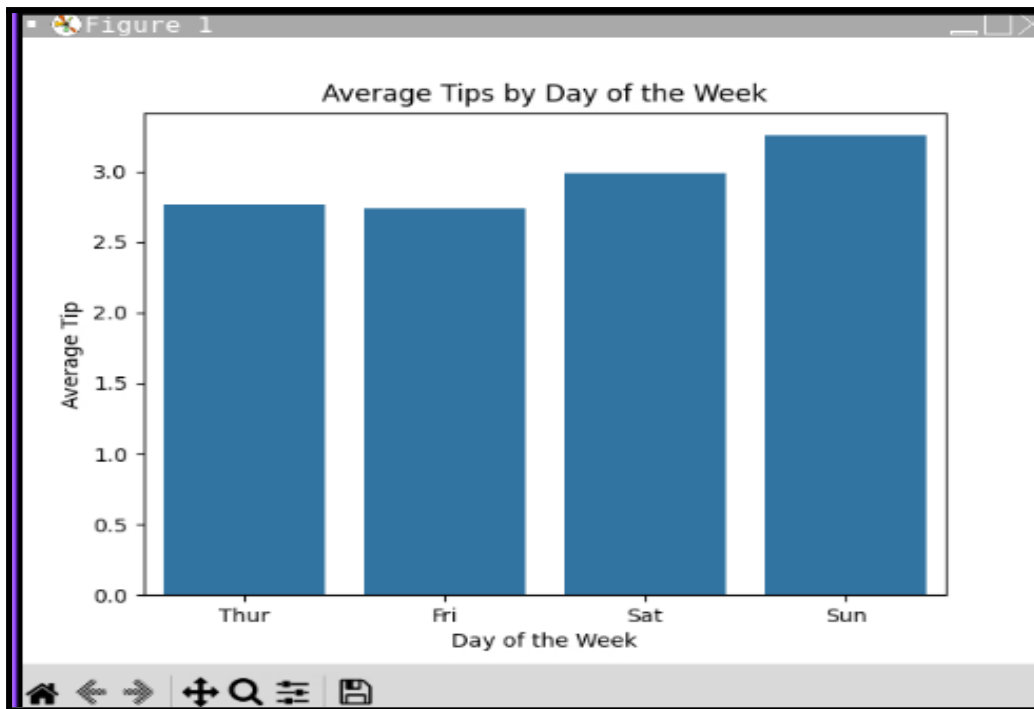
```
import seaborn as sns
import matplotlib.pyplot as plt

# Load the built-in 'tips' dataset
tips = sns.load_dataset("tips")

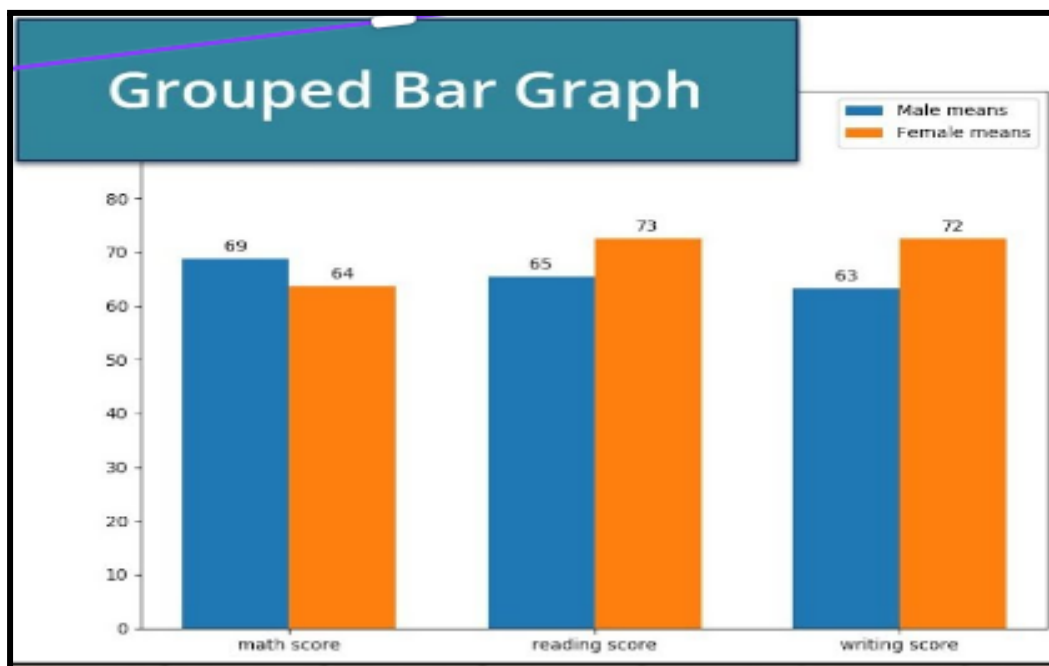
# Create a bar plot
sns.barplot(x="day", y="tip", data=tips, ci=None)

# Add title and labels
plt.title("Average Tips by Day of the Week")
plt.xlabel("Day of the Week")
plt.ylabel("Average Tip")

# Display the plot
plt.show()
```



A grouped bar plot in seaborn is a visualization that allows you to compare values across categories, where each category contains multiple sub-categories. This type of plot is useful for visualizing data where you want to compare the values of different groups within each category.



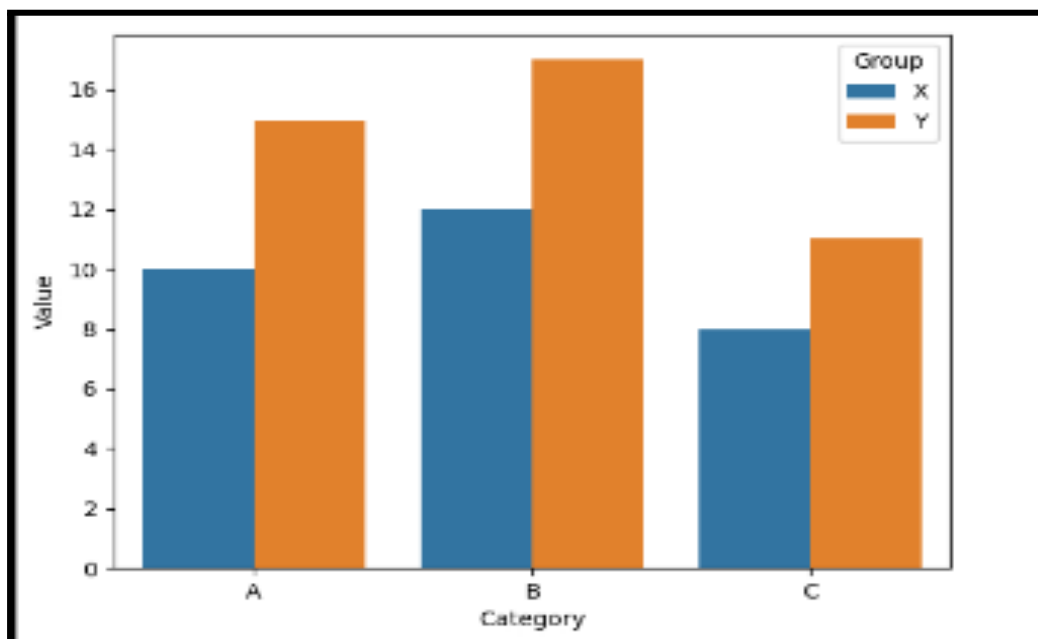
To create a grouped bar plot in seaborn, you typically start with a DataFrame where each row represents a data point, and each column represents a variable. You then use seaborn's `barplot()` function to specify the DataFrame, the variables for the x-axis, y-axis, and the hue (which represents the groups within each category).

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Sample data
data = {'Category': ['A', 'A', 'B', 'B', 'C', 'C'],
        'Group': ['X', 'Y', 'X', 'Y', 'X', 'Y'],
        'Value': [10, 15, 12, 17, 8, 11]}

# Create DataFrame
df = pd.DataFrame(data)

# Create grouped bar plot
sns.barplot(x='Category', y='Value', hue='Group', data=df)
plt.show()
```



`pd.melt()` is a method provided by the pandas library in Python, not seaborn directly, though it is often used in conjunction with seaborn for data visualization purposes. The `melt()` function is used to transform or reshape a DataFrame from a wide format to a long format.

Wide Format vs. Long Format

- 1) **Wide Format:** Each variable is in a separate column. This format is common when data is collected and stored.

Example:

css			
id	A	B	C
1	10	20	30
2	40	50	60

- 2) **Long Format:** Each row is an observation and a single column contains the values of different variables. This format is often preferred for data visualization and analysis.

Example:

css		
id	variable	value
1	A	10
1	B	20
1	C	30
2	A	40
2	B	50
2	C	60

The `melt()` function converts a DataFrame from wide format to long format. Here's the general

syntax:

```
pd.melt(frame, id_vars=None, value_vars=None,  
var_name=None, value_name="value", col_level=None,  
ignore_index=True)
```

- `frame`: The DataFrame to melt.
- `id_vars`: Column(s) to use as identifier variables (these

columns are not unpivoted).

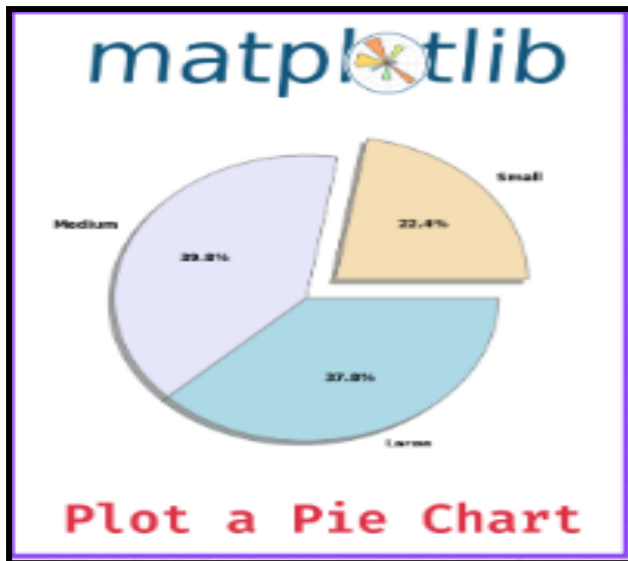
- `value_vars`: Column(s) to unpivot (convert from wide to long format).
- `var_name`: Name for the variable column (default is 'variable').
- `value_name`: Name for the value column (default is 'value').
- `col_level`: If columns are a MultiIndex, this determines which level to melt.
- `ignore_index`: If True, original index is ignored.

Q. Why Use `pd.melt()` with Seaborn?

Ans - Seaborn, a statistical data visualization library based on matplotlib, often requires data in long format to create various plots. For example: `boxplot()`, `barplot()`: These functions work well with long format data to create visualizations where each row represents an observation.

Pie Chart:

A Pie Chart is a circular statistical plot that can display only one series of data. The area of the chart is the total percentage of the given data. Pie charts are commonly used in business presentations like sales, operations, survey results, resources, etc. as they provide a quick summary. In order to make a pie chart in Seaborn, we need to utilize the Matplotlib's `pie()` function. A pie chart is one kind of chart that shows parts of a whole, like slices of a pie.



Syntax

```
plt.pie(sizes, explode=None, labels=None, colors=None, autopct=None, shadow=False, startangle=None)
```

Parameters:

- sizes: List of values representing the size of each slice.
- explode: (Optional) List of fractions to offset each slice. Default is None.
- labels: (Optional) List of labels for each slice. Default is None.
- colors: (Optional) List of colors for each slice. Default is None.
- autopct: (Optional) String format to display percentages on the slices. Default is None.
- shadow: (Optional) Boolean to draw a shadow beneath the pie. Default is False.
- startangle: (Optional) Starting angle for the slices (in degrees). Default is None.

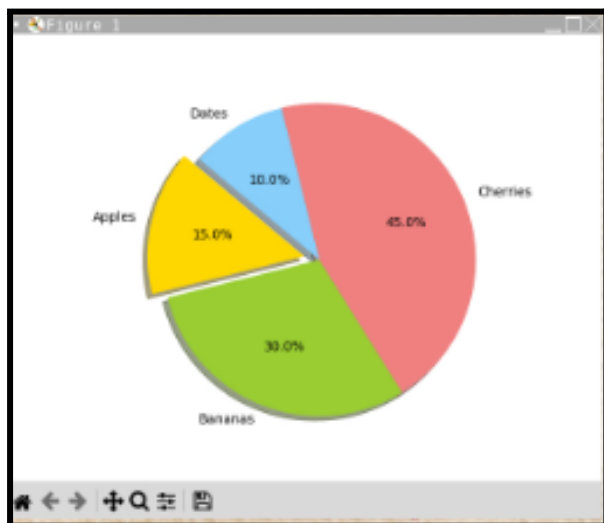
Example :

```
import matplotlib.pyplot as plt

# Data to plot
labels = 'Apples', 'Bananas', 'Cherries', 'Dates'
sizes = [15, 30, 45, 10] # Percentages
colors = ['gold', 'yellowgreen', 'lightcoral', 'lightskyblue']
explode = (0.1, 0, 0, 0) # "explode" the 1st slice (Apples)

# Plot
plt.pie(sizes, explode=explode, labels=labels, colors=colors,
        autopct='%1.1f%%', shadow=True, startangle=140)
plt.axis('equal') # Equal aspect ratio ensures that pie is drawn as a circle.

plt.show()
```



IN CLASS CHALLENGES

INCLASS CHALLENGE1 - Visualizing Sales Data :

You are a young data scientist helping your friend analyze the monthly sales of three products: A, B, and C. Your friend has provided you with a CSV file named “monthly sales” containing the sales data in a wide format.

Link to the csv file:

Your task is to create a grouped bar plot showing the sales of each product for each month using Seaborn in Python. Follow these steps to achieve this:

- Read the data from the CSV file.
- Set the plot style to "whitegrid" and use a "pastel" color palette for your plot.
- Convert the data from wide format to long format using `pd.melt`.
- Create a grouped bar plot using Seaborn.
- Customize the plot by adding a title, labels, and a legend.
- Rotate the x-axis labels for better readability.
- Finally, display

INCLASS CHALLENGE 2- Toy Sales:

Imagine you are a data analyst at a toy company called 'FunToys Inc.' The company sells three popular toys: Toy-A, Toy-B and Toy-C. The sales team has collected data on the number of each toy sold every month throughout the year and is stored in a csv file named "Toy sales". Your job is to create a visual representation of the total sales for each toy to help the company understand which toy is the most popular.

Link to the csv file:

Steps to follow:

- Read the data from the CSV file.
- Sum up the sales for each toy across all months in the dataframe.
- Use the `plt.pie()` function to create a pie chart with the calculated totals.
- Add labels to the pie chart to indicate which section corresponds to which toy.
- Display the percentage of total sales for each toy on the pie chart.

Can you write the Python code to create this pie chart?

HOME TASK

HOME TASK1-Favorite Animal:

In Ms. Johnson's class, the kids were asked about their favorite animal. The results are in, and we need to visualize them to see which animal is the most popular. The favorite animals are: Dogs, Cats, Rabbits, Lions, and Dolphins. The number of kids who prefer each animal is as follows:

Dogs: 18 kids

Cats: 12 kids

Rabbits: 9 kids

Lions: 5 kids

Dolphins: 6 kids

Your Task is to:

- Import necessary Libraries for handling the data.
- Create a dictionary with animals and the number of kids who like each animal.
- Now convert the dictionary into a Pandas DataFrame.
- Set the style of the plot to "whitegrid" and choose a "bright" color palette for a vibrant look.

- Create a bar plot with `sns.barplot()`, specifying the x and y values, and the color palette.
- Add a title and axis labels to make the plot informative.
- Finally, display the plot using `plt.show()`.

HOMETASK 2 - Favorite Flavors :

Suppose you conducted a survey among a group of kids to find out their favorite ice cream flavors. Here are the results:

- Chocolate: 35%
- Vanilla: 30%
- Strawberry: 20%
- Cookie Dough: 15%

Now, can you create a pie chart using matplotlib to represent the above results?

Steps to be followed:

- Import necessary library.
- Prepare your data: Make a list of the favorite ice cream flavors and the percentage of the kids who like each flavor. Don't forget to name them and assign percentages!
- Choose colors: Think about what colors represent each flavor. [HINT: use 'chocolate' for Chocolate flavor, 'lightyellow' for Vanilla, 'lightpink' for Strawberry and 'wheat' for Cookie Dough flavor].
- Create the pie chart, pass your data (flavor names, percentages and colors to it). [HINT: use `plt.pie()`].
- Add a title using `plt.title()` and make sure that pie chart is drawn as a circle. [HINT: use `plt.axis()`].
- Finally, display your colorful pie chart using `plt.show()`.

WHAT'S NEXT?

More on plots for Categorical Data

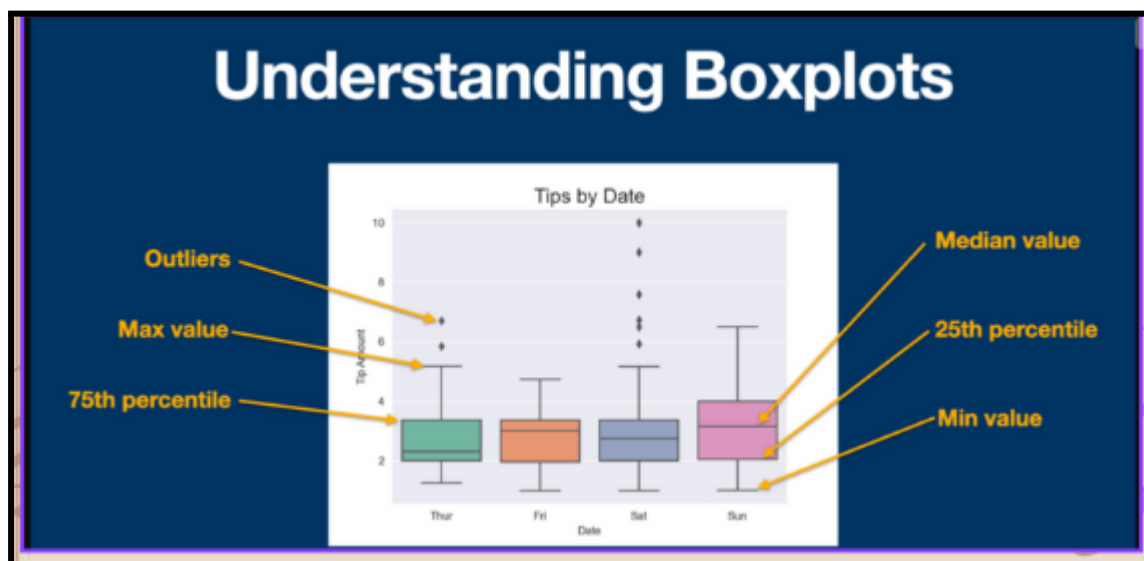
LESSON 42

More on Plots for Categorical Data

LEARNING CONTENT

Box plot

The box plot is widely used for summarizing the distribution of a dataset with a visual representation of the median, quartiles, and outliers. It's particularly useful for comparing distributions across multiple categories.



Syntax: `seaborn.boxplot(x=None, y=None, hue=None, data=None, order=None, hue_order=None, orient=None, color=None, palette=None, saturation=0.75, width=0.8, dodge=True, fliersize=5, linewidth=None, whis=1.5, ax=None, **kwargs)`

Parameters:

x, y, hue: Inputs for plotting long-form data.

data: Dataset for plotting. If x and y are absent, this is interpreted as wide-form.

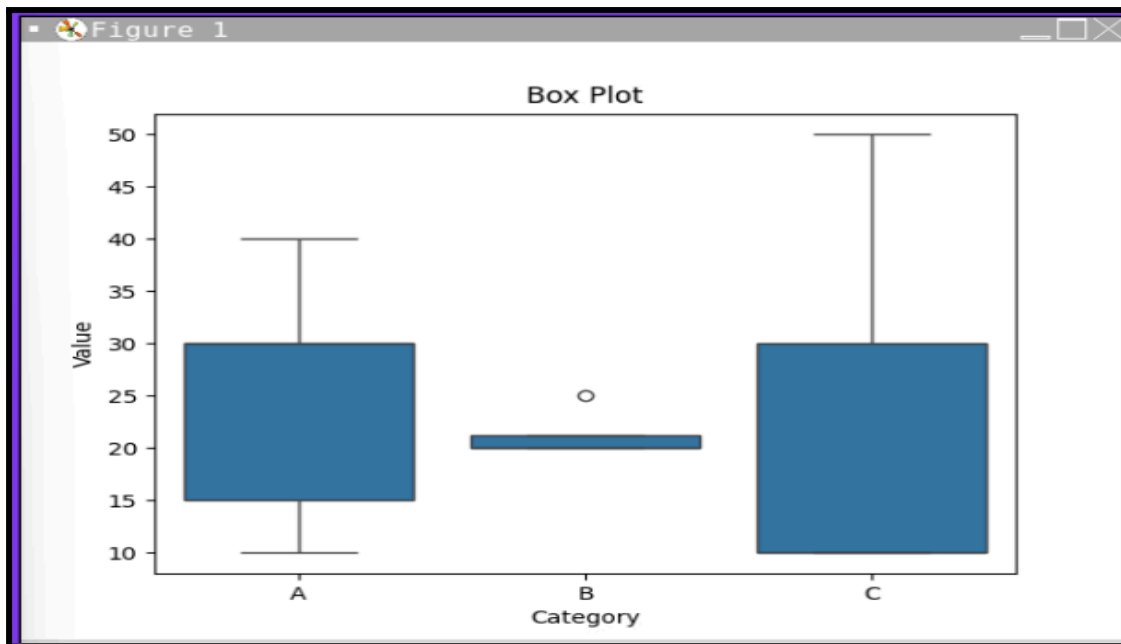
color: Color for all of the elements.

Example :

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

df = pd.DataFrame({
    'Category': ['A', 'B', 'A', 'C', 'B', 'B', 'A',
                'A', 'C', 'C', 'B', 'A'],
    'Value': [10, 20, 15, 10, 20, 25, 30, 40, 50,
              10, 20, 30]
})

sns.boxplot(x='Category', y='Value', data=df)
plt.xlabel('Category')
plt.ylabel('Value')
plt.title('Box Plot')
plt.show()
```



Count plot:

For categorical data, the count plot is one of the most straightforward and widely used plots. It shows the count of observations in each categorical bin using bars, making it easy to see the distribution of categorical data.

```
Syntax : seaborn.countplot(x=None, y=None, hue=None, data=None, order=None, hue_order=None, orient=None, color=None, palette=None, saturation=0.75, dodge=True, ax=None, **kwargs)
```

Parameters :

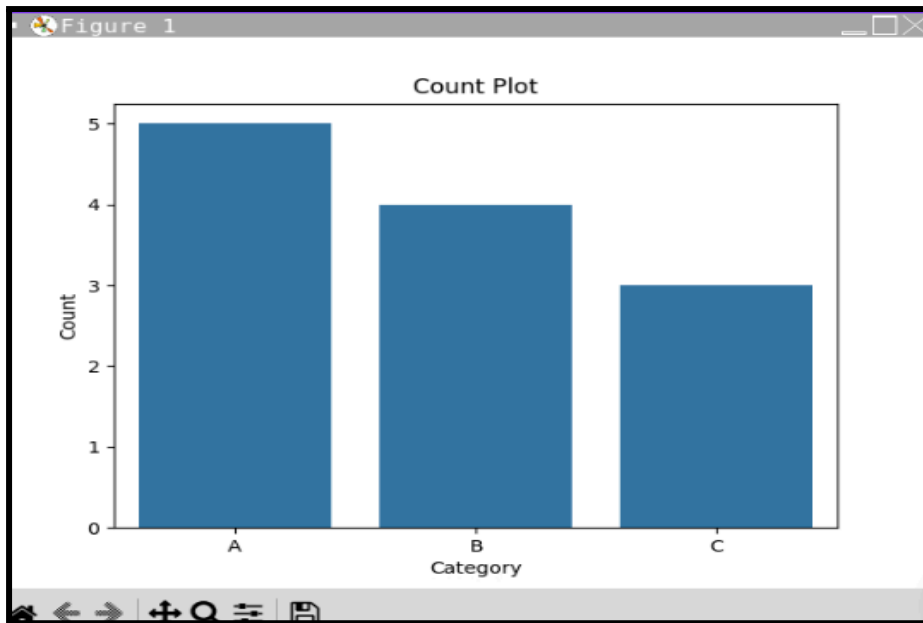
- x, y: Variable names or vectors for the data to be plotted on the x or y axis.
- hue (optional): Variable name for creating grouped plots based on this categorical variable.
- data (optional): DataFrame containing the data to be plotted.
- order (optional): List specifying the order to plot the categorical levels.
- hue_order (optional): List specifying the order for the hue variable levels.

- orient (optional): Orientation of the plot; 'v' for vertical (default) or 'h' for horizontal.
- color (optional): Single color for all elements.
- palette (optional): Colors to use for different levels of the hue variable.
- saturation (optional): Proportion of the original saturation for drawing colors (default=0.75).
- dodge (optional): Whether to shift elements when hue nesting is used (default=True).
- ax (optional): Matplotlib Axes object to draw the plot onto.
- kwargs: Additional keyword arguments passed to matplotlib.axes.Axes.bar.

Example :

```
import seaborn as sns
import matplotlib.pyplot as plt

data = ['A', 'B', 'A', 'C', 'B', 'B',
        'A', 'A', 'C', 'C', 'B', 'A']
sns.countplot(x=data)
plt.xlabel('Category')
plt.ylabel('Count')
plt.title('Count Plot')
plt.show()
```



IN CLASS CHALLENGES

INCLASS CHALLENGE1 - Visualizing Distribution of Scores :

Ms. Smith's classroom has recently completed a series of tests in three subjects: Math, Science and English. She has collected the test scores and stored them in a CSV file called test scores.csv. The file contains two columns: Subject and Scores.

File Link:



- ☐ Create the box plot: Use Seaborn to create a box plot to show the distribution of scores for each subject. Add a title and labels to the plot.

Can you complete this code and run it to see the box plot of the students test scores?

INCLASS CHALLENGE 2- Employees Distribution:

Imagine you are the manager of a big toy company called "FunToys Inc." You have employees working in different departments such as HR, IT, Sales, and Marketing. Each employee has a different employment status: some are Full-time, some are Part-time, and others are Contractors. And all this data is stored in a csv file.

Link of the data:

Create the plot and show it to the company's directors to help them understand the distribution of employees across departments and their employment statuses. Good luck!

HOME TASK

HOME TASK1-Childrens Height:

Jacob is having a list of heights for kids in Grade 3, Grade 4, and Grade 5. He want to make a special type of chart called a "box plot" to see how their heights compare. Can you help Jacob to create a box plot using Python?

Here's the sample data of the heights:

- Grade 3: [120, 122, 125, 121, 123, 124, 125, 126, 120, 122, 124, 121, 123, 125, 126, 127, 128, 123, 122, 124]
- Grade 4: [130, 132, 135, 131, 133, 134, 135, 136, 130, 132, 134, 131, 133, 135, 136, 137, 138, 133, 132, 134]
- Grade 5: [140, 142, 145, 141, 143, 144, 145, 146, 140, 142, 144, 141, 143, 145, 146, 147, 148, 143, 142, 144]

Here's what you need to do:

Create a list of heights for kids in each grade.

Combine these lists into a special table called a DataFrame.

Draw a box plot to see the height differences between Grade 3, Grade 4, and Grade 5.

Label your chart so everyone knows what it shows.

HOMETASK 2 - Favorite Fruit :

Imagine you're in a classroom where each student gets to vote for their favorite fruit. The choices are Apple, Banana, and Cherry. You have collected the votes from 20 students and the vote result is stored in a csv file.

Link of the file:

LESSON 43

Exploring more on Plots for Categorical Data

LEARNING CONTENT

Strip Plot

A strip plot is used to display the distribution of a dataset along a single axis with individual data points represented as dots. This type of plot is useful for visualizing the spread of data points and identifying potential outliers.

Syntax

```
sns.stripplot(  
    x=None, y=None, hue=None, data=None, order=None, hue_order=None,  
    jitter=True, dodge=False, orient=None, color=None, palette=None,  
    size=5, edgecolor='gray', linewidth=0, ax=None, **kwargs  
)
```

Key Parameters:

- x, y: Names of variables in data for the x and y axes.
- hue: Categorical variable that will determine the color of plot elements.
- data: DataFrame containing the data.
- jitter: Amount of jitter (spread) to apply along the categorical axis.
- dodge: When using hue, whether to separate the strips for different levels of the hue variable.
- orient: Orientation of the plot (vertical or horizontal).
- palette: Colors to use for the different levels of the hue variable.

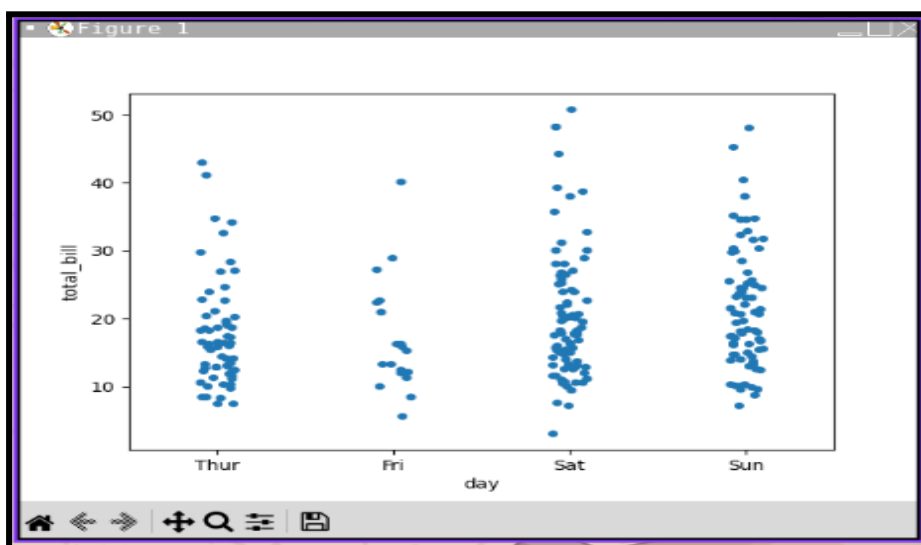
Example: In the below example we are using Seaborn's built in dataset ("tips").

```
import seaborn as sns
import matplotlib.pyplot as plt

# Example dataset
tips = sns.load_dataset("tips")

# Basic strip plot
sns.stripplot(x="day", y="total_bill", data=tips, jitter=True)

plt.show()
```



Swarm plot

A swarm plot is similar to a strip plot but with an added feature: it arranges the points in a way that avoids overlap and reveals the structure of the distribution more clearly. This is achieved through an algorithm that spreads the points out along the categorical axis.

A swarm plot can be drawn on its own, but it is also a good complement to a box, preferable because the associated names will be used to annotate the axes. This type of plot sometimes known as “beeswarm”.

Syntax

```
sns.swarmplot(  
    x=None, y=None, hue=None, data=None, order=None, hue_order=None,  
    dodge=False, orient=None, color=None, palette=None,  
    size=5, edgecolor='gray', linewidth=0, ax=None, **kwargs  
)
```

Key Parameters:

- x, y: Names of variables in data for the x and y axes.
- hue: Categorical variable that will determine the color of plot elements.
- data: DataFrame containing the data.
- dodge: When using hue, whether to separate the swarms for different levels of the hue variable.
- orient: Orientation of the plot (vertical or horizontal).
- palette: Colors to use for the different levels of the hue variable.

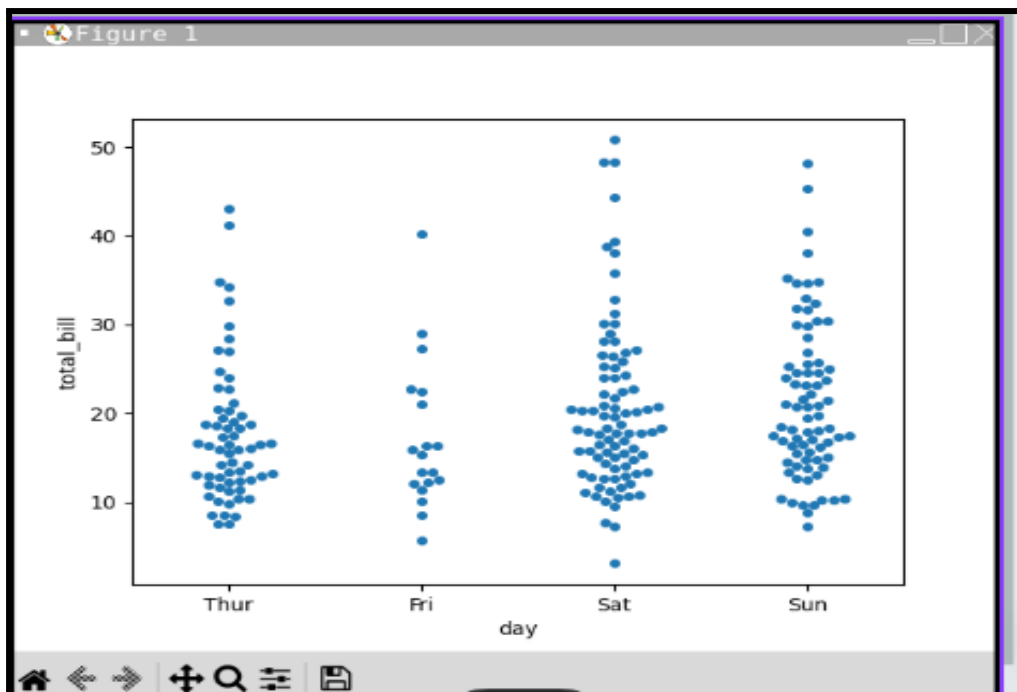
Example :

```
import seaborn as sns
import matplotlib.pyplot as plt

# Example dataset
tips = sns.load_dataset("tips")

# Basic swarm plot
sns.swarmplot(x="day", y="total_bill", data=tips)

plt.show()
```



IN CLASS CHALLENGES

INCLASS CHALLENGE1 - Plant Growth:

Imagine you're organizing a science fair at your school, and you're in charge of showcasing the results of an exciting plant growth experiment! You've collected data from different

experiments where students have tested how watering frequency affects the growth of various plants.

Suppose you have data from three types of plants: roses, tulips and sunflowers. Each plant was watered at different frequencies: daily, every alternate day and weekly. Here's the data:

```
data = {'Plant Type': ['Rose', 'Rose', 'Rose', 'Tulip', 'Tulip', 'Tulip',  
    'Sunflower', 'Sunflower', 'Sunflower'],  
    'Watering Frequency': ['Daily', 'Alternate Days', 'Weekly', 'Daily',  
    'Alternate Days', 'Weekly', 'Daily',  
    'Alternate Days', 'Weekly'],  
    'Plant Height in cm': [15, 18, 12, 10, 11, 9, 25, 27, 23]}
```

Your Task:

Using Python, create a visual representation of this data to show how the watering frequency affects the height of each type of plant. Use a strip plot to showcase this information. Be sure to label your axes and provide a legend for clarity.

INCLASS CHALLENGE 2- Plotting Employees:

You've just started your new role as a data analyst at a major retail company, and your first task is to analyze the workforce data provided by the HR department. Data is stored in a file named 'employees info'. This file contains crucial information about employees, including their department, employment status and gender.

Link of the dataset:

cylinders in the vehicle. But this time use Swarm Plot to show the graph. so that Rahul can analyze the data in a better way.

Link of the dataset:

LESSON 44

Problem solving session - 4

IN CLASS CHALLENGES

INCLASS CHALLENGE 1- Analyzing & Visualizing

Imagine you are a teacher and you want to analyze the test scores of your students in three different subjects: Math, Science, and English. Here is the table with the scores:

You need to create a program that will help you:

- Create the dataframe & display the data.
- Calculate and print the average scores(mean) for Math, Science and English.
- Create a grouped bar plot using seaborn showing each student's scores in different subjects.

NOTE:

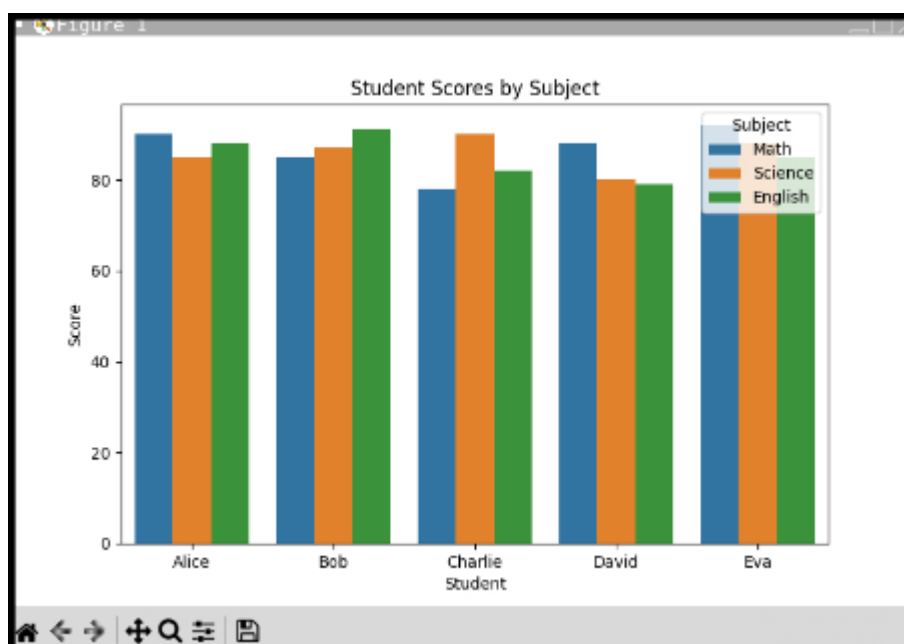
Since the data is in a wide format, you need to "melt" the DataFrame to convert it into a long format for better visualization.

Student	Math	Science	English
Alice	90	85	88
Bob	85	87	91
Charlie	78	90	82
David	88	80	79
Eva	92	88	85

SOLUTION :

```
Original Dataframe:
  Student  Math  Science  English
0   Alice    90     85     88
1    Bob    85     87     91
2  Charlie    78     90     82
3   David    88     80     79
4    Eva    92     88     85

Average Scores:
Math      86.6
Science   86.0
English   85.0
dtype: float64
```



INCLASS CHALLENGE 2- Data Summary & Ratings

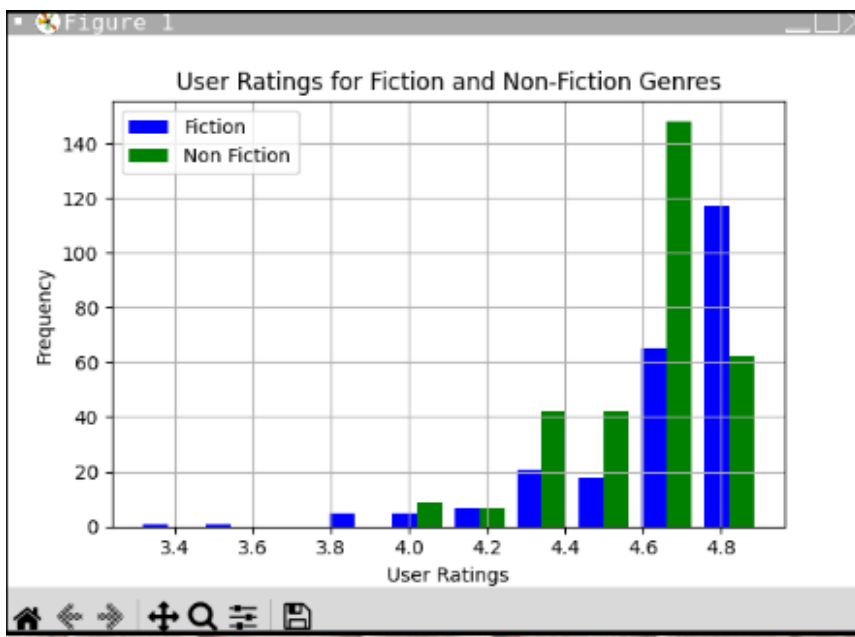
Your friend, Alex has this awesome collection of book data stored in a csv file called 'Books'. Now, Alex knows you're pretty good with coding and all, so he comes up to you and says, Hey, I've got this massive dataset of books, but I need to quickly understand what's going on. Can you help me out? He hands you the file and ask, Could you first give me a quick summary of this data frame? It's pretty big, so I need just a snapshot. HINT: use pandas info() function to get the summary of the data frame. Now, Alex wants to see which genre—fiction or non-fiction—has the highest user ratings? So, help Alex by visualizing which genre has the highest user rating. HINT: use seaborn hist plot to make a cool graph.

[Link for the Data Set:](#)

```

Summary of DataFrame:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 550 entries, 0 to 549
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Name        550 non-null    object
1   Author      550 non-null    object
2   UserRating  550 non-null    float64
3   Reviews     550 non-null    int64
4   Price       550 non-null    int64
5   Year        550 non-null    int64
6   Genre       550 non-null    object
dtypes: float64(1), int64(3), object(3)
memory usage: 30.2+ KB
None

```



INCLASS CHALLENGE 3- Merge And Visualize:

[Link for the Data Sets:](#)

customer Id, order Id, product Id and order date. And another with product details such as, product name and Id, category and prize. Your manager wants you to graphically analyze the pricing distribution of products to better understand the market. How would you approach this task using Python?

First you need to merge these datasets based on the 'Product ID' column to combine the relevant information. Once merged, your task is to plot the distribution of product prices. Think about how you would approach merging the datasets, visualize the product prices using Python libraries like matplotlib and seaborn. HINT: use pandas merge() function for merging the data sets then use seaborn density plot i.e, sns.kdeplot() for visually plotting the data.

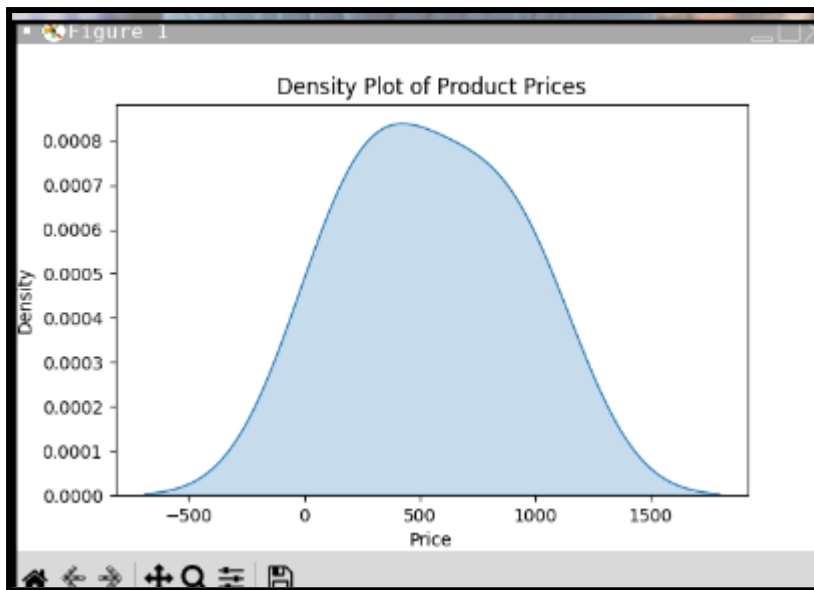
SOLUTION :

```
Customer Orders:
  Customer ID  Order ID  Product ID  Quantity  Order Date
0           1         101         101         2    1/1/2024
1           2         102         102         1    1/2/2024
2           3         103         103         3    1/3/2024
3           4         104         104         2    1/4/2024
4           5         105         105         1    1/5/2024

Product Details:
  Product ID  Product Name  Category  Price
0         101        Laptop  Electronics  1000
1         102   Smartphone  Electronics   800
2         103   Headphones  Electronics   100
3         104        Tablet  Electronics   500
4         105   Smartwatch  Electronics   300

Merged Dataframe:
  Customer ID  Order ID  ...  Category  Price
0           1         101  ...  Electronics  1000
1           2         102  ...  Electronics   800
2           3         103  ...  Electronics   100
3           4         104  ...  Electronics   500
4           5         105  ...  Electronics   300

[5 rows x 8 columns]
```



HOME TASK

HOME TASK 1 -Christmas toy adventure:

Hey there! Imagine you and your pals, like Alice, Bob, Charlie, Diana and Eva, had a blast collecting toys for kids at Christmas.

Sample Data:

kids = Alice, Bob, Charlie, Diana, Eva and toys collected = 10, 15, 8, 12, 20

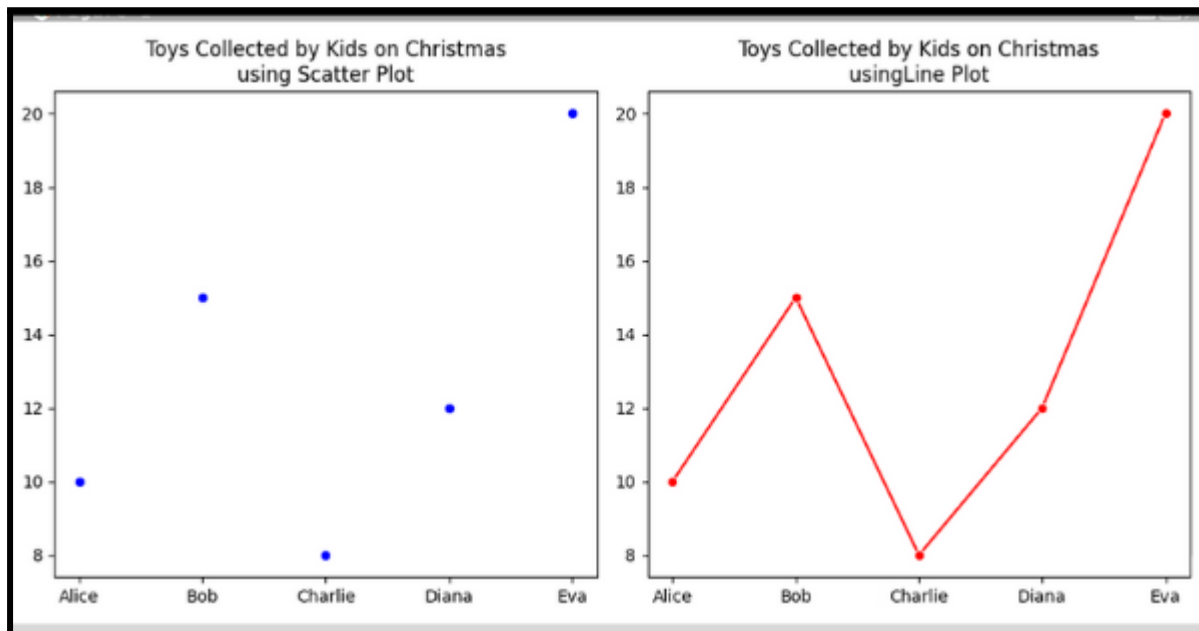
Now, can you show off how many toys each of you collected by making a cool graph using seaborn in Python?

First, you'll gather everyone's toy counts. Then, you need to make two types of plots:

- one with dots for each person's count. HINT: use Scatter plot
- another with lines connecting the dots. HINT: use Line plot

These plots will make it super easy to see who collected the most toys and how everyone did overall. So, are you ready to make your Christmas toy adventure shine with some awesome coding?

SOLUTION :



HOME TASK 2 -Zoo Animal Exploration:

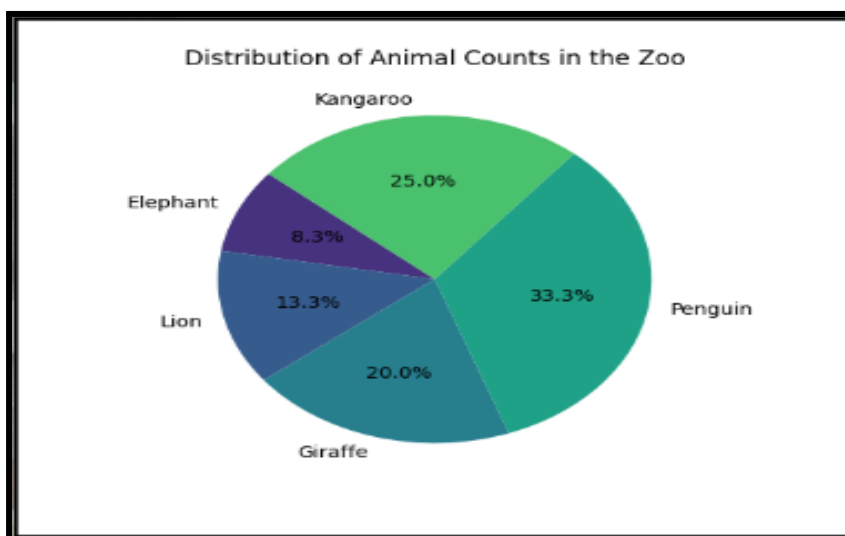
You and your friends visited the zoo and were amazed by all the different animals you saw! You decided to collect some information about the animals to make some cool visualizations. Here's the data you collected:

Write a python code to create a pie chart using the pandas, seaborn and matplotlib libraries to show the distribution of the animal counts in the zoo. First, create a Data Frame with the provided data, and then create and display the pie chart. The pie chart will help you and your friends see which animals are the most and least common.

Animal	Count	Average Weight (kg)	Average Lifespan (years)
Elephant	5	5400	70
Lion	8	190	14
Giraffe	12	800	25
Penguin	20	30	20
Kangaroo	15	85	23

SOLUTION :

Data about the Animals:				
	Animal	Count	Average Weight	Average Lifespan
0	Elephant	5	5400	70
1	Lion	8	190	14
2	Giraffe	12	800	25
3	Penguin	20	30	20
4	Kangaroo	15	85	23



WHAT'S NEXT?

Mini Project-5

LESSON 45

MINI PROJECT-5

Mini project - Visuals Of Penguin Data

Hey there, young explorer! Imagine you're a scientist studying penguins in Antarctica. You've collected lots of data about different penguin species, including their body mass, beak (culmen) measurements, and flipper lengths. Now, you need to analyze this data to learn more about these amazing creatures. Your mission is to use Python to look at this data and create some cool charts that show different aspects of the penguins.

Link of the Data:

Here's what you need to do :

Load the Data : Load the penguin data from a CSV file using Pandas and display the Data set and a Statistical summary of the dataset.

Data Preprocessing : Show the first few rows of the dataset, check for any missing values, and analyze the distribution of penguin species.

Data Visualization : Use Matplotlib and Seaborn to create three plots :

- A bar plot showing the body mass of different penguin species.
- A box plot showing measurements of the penguins' culmen and flippers.
- A scatter plot showing the relationship between culmen length and depth.

SOLUTION :

```
Null values in the dataset:
  species  island  ...  body_mass_g  sex
0   False   False  ...      False  False
1   False   False  ...      False  False
2   False   False  ...      False  False
3   False   False  ...       True   True
4   False   False  ...      False  False
..      ...      ...  ...      ...   ..
339  False   False  ...       True   True
340  False   False  ...      False  False
341  False   False  ...      False  False
342  False   False  ...      False  False
343  False   False  ...      False  False

[344 rows x 7 columns]
Distribution of species:
species
Adelie      152
Gentoo      124
Chinstrap    68
Name: count, dtype: int64
```

```

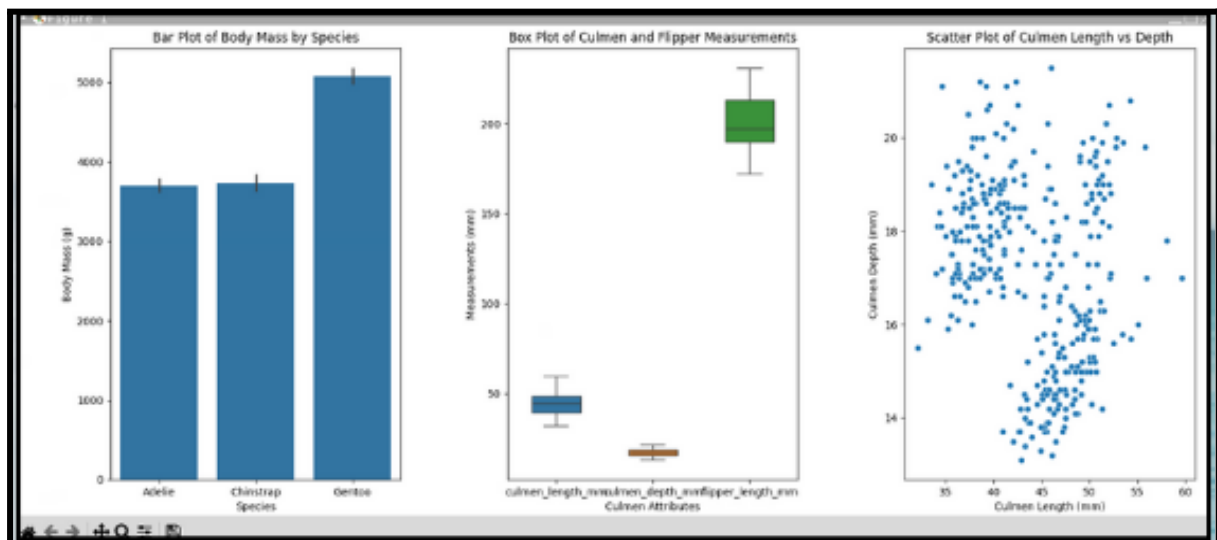
Statistical Summary of the dataset:
count    species  island  ...  body_mass_g  sex
unique      3      3      ...      NaN         3
top    Adelie  Biscoe  ...      NaN        MALE
freq     152    168  ...      NaN        168
mean      NaN    NaN  ...  4201.754386  NaN
std       NaN    NaN  ...  801.954536  NaN
min       NaN    NaN  ...  2700.000000  NaN
25%       NaN    NaN  ...  3550.000000  NaN
50%       NaN    NaN  ...  4050.000000  NaN
75%       NaN    NaN  ...  4750.000000  NaN
max       NaN    NaN  ...  6300.000000  NaN

[11 rows x 7 columns]

First few rows of the dataset:
species  island  ...  body_mass_g  sex
0  Adelie  Torgersen  ...  3750.0  MALE
1  Adelie  Torgersen  ...  3800.0  FEMALE
2  Adelie  Torgersen  ...  3250.0  FEMALE
3  Adelie  Torgersen  ...    NaN    NaN
4  Adelie  Torgersen  ...  3450.0  FEMALE

[5 rows x 7 columns]

```



HOME TASK

Home Task 1 - Visualizing Customer Demographics

Can you create a cool visualization of shopping habits across different age groups? Imagine you have information about people's spending scores, annual incomes, and membership statuses from a shopping club. Here's what you need to do :

- Create a dataset with the following details :

Age groups : '18-25', '26-35', '36-45', '46-55', '55+'

Spending scores : some scores like 45, 55, 65, etc.

Annual incomes : some values like 35000, 45000, 55000, etc.

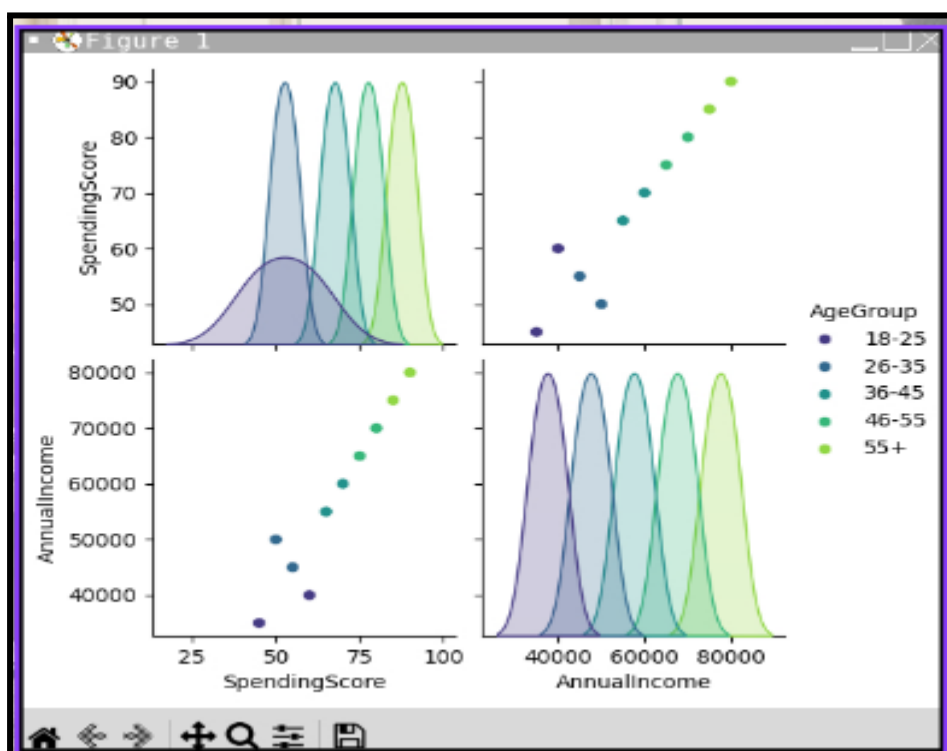
Membership statuses : 'Silver', 'Gold', 'Platinum'

- Make a DataFrame from this data.
- Visualize the relationships between spending scores and annual incomes for different age groups using seaborn pair plot.

Can you write the code to do this? Don't forget to color the plots by age group!

SOLUTION :

```
Generated DataFrame:
  AgeGroup  SpendingScore  AnnualIncome  MembershipStatus
0   18-25             45         35000             Silver
1   26-35             55         45000              Gold
2   36-45             65         55000             Platinum
3   46-55             75         65000             Silver
4    55+             85         75000              Gold
5   18-25             60         40000             Platinum
6   26-35             50         50000             Silver
7   36-45             70         60000              Gold
8   46-55             80         70000             Platinum
9    55+             90         80000             Silver
```



Home Task 2 - Dual Pie Charts

You've recently been appointed as the HR manager at a bustling tech startup, and you're eager to get a grasp of the employee demographics to ensure a balanced and inclusive workplace. As you settle into your new role, you discover that the company keeps a detailed record of all employees in a digital database (csv file).

Link of the Data:

```

Data Frame:
  Department Employment Status Gender
0         HR      Full-time    Male
1         IT      Part-time    Male
2        Sales      Contractor  Female
3    Marketing      Full-time  Female
4         HR      Full-time  Female
5         IT      Full-time    Male
6        Sales      Contractor    Male
7    Marketing      Part-time  Female
8         HR      Part-time    Male
9         IT      Full-time    Male
10        Sales      Full-time  Female
11   Marketing      Part-time  Female
12        HR      Contractor    Male
13        IT      Contractor    Male
14        Sales      Full-time  Female
15   Marketing      Full-time  Female
16        HR      Part-time    Male
17        IT      Contractor    Male
18        Sales      Part-time  Female
19   Marketing      Full-time  Female

```

```

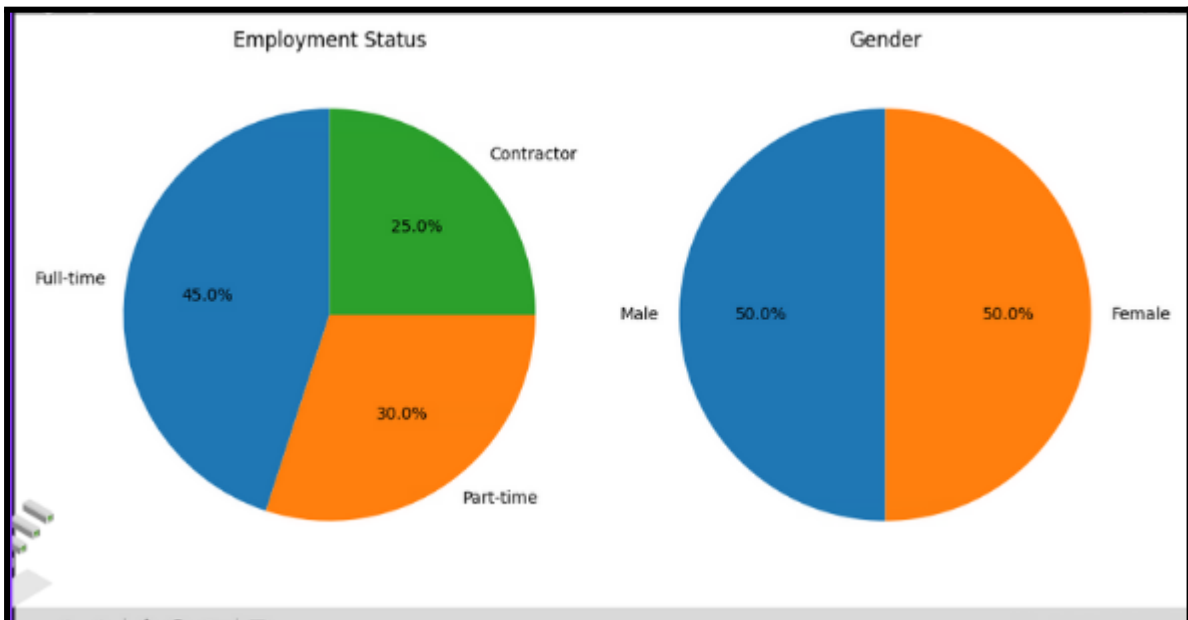
Employment Status Counts:
Employment Status
Full-time      9
Part-time      6
Contractor      5
Name: count, dtype: int64

```

```

Gender Counts:
Gender
Male      10
Female     10
Name: count, dtype: int64

```



WHAT'S NEXT?

Assessment - 4

LESSON 46

ASSESSMENT-4

Assessment - Product Price Distribution

Imagine you have a small online store where you sell five different products. You've been keeping track of the prices of these products in a csv file named "product details".

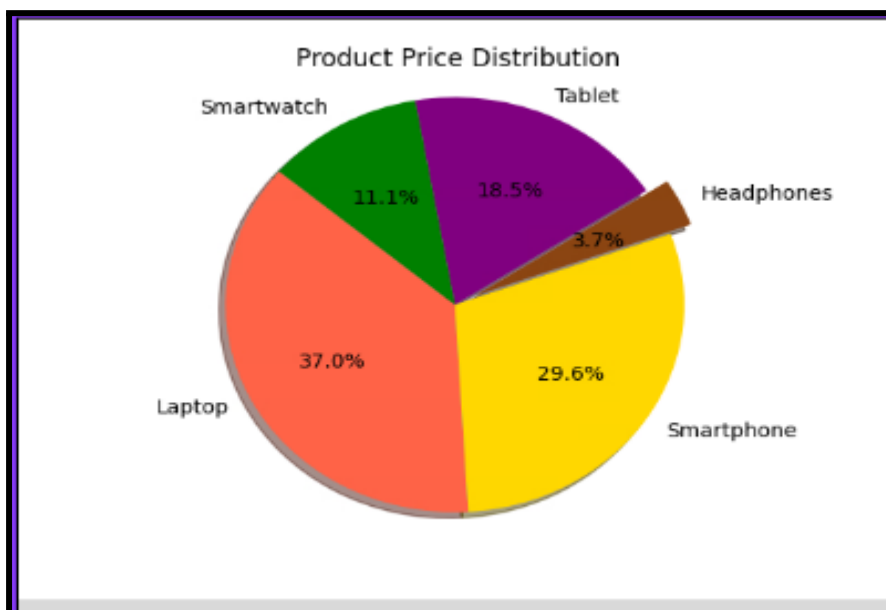
CSV file link :

- Display the percentage of the total price for each product on the pie chart.

Steps to Follow :

- Load the data from the CSV file using pandas.
- Extract the product names and prices into separate variables.
- Define a list of colors for the products.
- Create an "explode" parameter list to highlight the slice with the lowest price.
- Use matplotlib to plot the pie chart with the specified parameters

SOLUTION :-



HOME TASK

Home Task 1 - Fruit Data Visualization

Hey there! Imagine you're a scientist who loves studying different types of fruits. You have collected some cool data about ten different fruits, including their weights, colors, and sweetness levels. Now, can you create an interesting and awesome bar plot to show the weights of these fruits?

Here's the data you've collected :

- Fruit : Apple, Banana, Cherry, Date, Elderberry, Fig, Grape, Honeydew, Ita Palm, Jackfruit
- Weight (in grams) : 500, 120, 50, 250, 100, 50, 30, 1000, 20, 1500
- Color : Red, Yellow, Red, Brown, Purple, Purple, Green, Green, Brown, Yellow
- Sweetness (1 to 10) : 7, 8, 6, 9, 5, 7, 6, 8, 4, 7

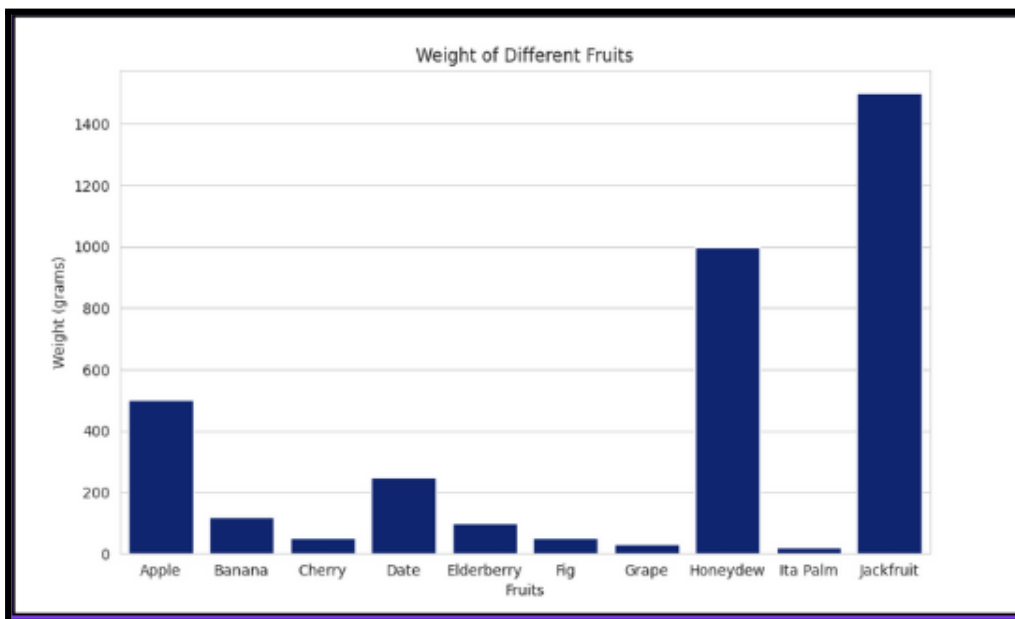
Your task is to first create a DataFrame using the above data and display it. Then, you need to create a seaborn bar plot that shows the weights of these different fruits using the data provided.

Note : You should use the 'dark' color palette and set the style to 'whitegrid' to make your plot look awesome.

SOLUTION :

Generated DataFrame:

	Fruit	Weight	Color	Sweetness
0	Apple	500	Red	7
1	Banana	120	Yellow	8
2	Cherry	50	Red	6
3	Date	250	Brown	9
4	Elderberry	100	Purple	5
5	Fig	50	Purple	7
6	Grape	30	Green	6
7	Honeydew	1000	Green	8
8	Ita Palm	20	Brown	4
9	Jackfruit	1500	Yellow	7



Home Task 2 - Salaries of Employees

Imagine you're taking part in a school project where you're tasked with analyzing data from a company. The data you're working with is all about employee salaries from three departments: HR, Engineering and Marketing. Here's what the data looks like :

- Department : HR, HR, HR, HR, HR, Engineering, Engineering, Engineering, Engineering, Engineering, Marketing, Marketing, Marketing, Marketing, Marketing

- Salary : 45000, 48000, 47000, 46000, 49000, 75000, 77000, 76000, 78000, 79000, 52000, 54000, 53000, 55000, 56000

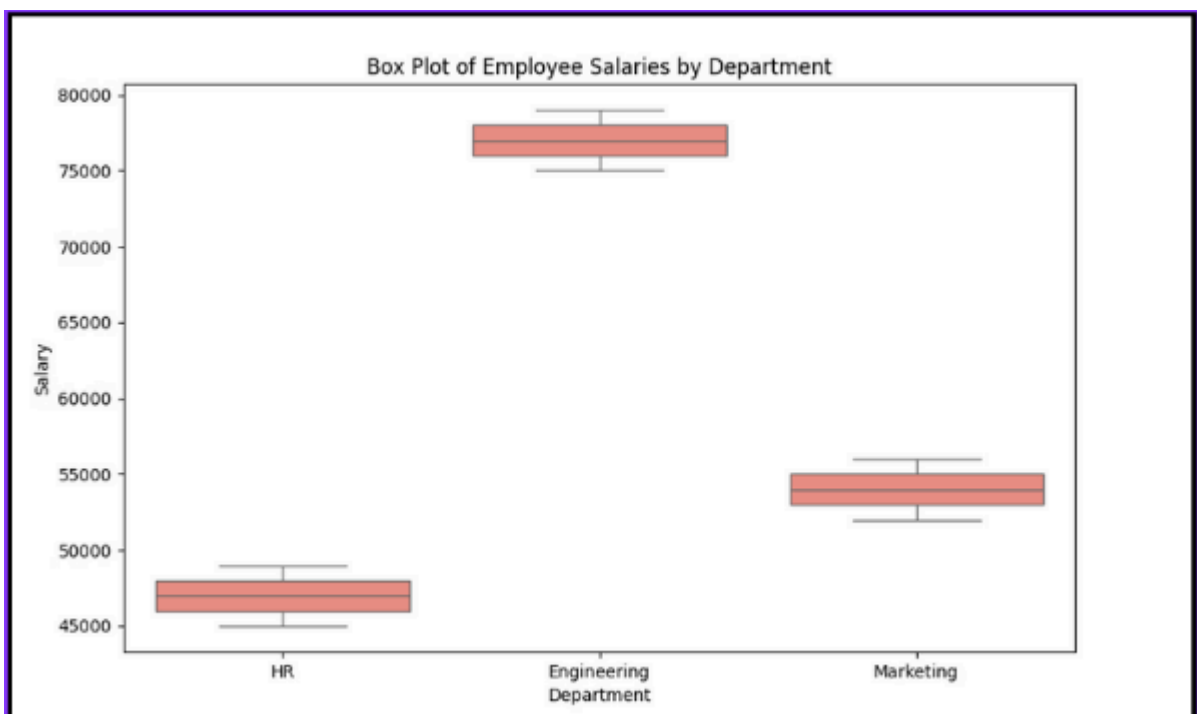
Here's a step-by-step breakdown of what you need to do:

- Import Libraries : You'll need to import some Python libraries to help you work with the data and create visualizations. Specifically, you'll want to import Pandas, Seaborn, and Matplotlib.
- Prepare the Data : Use Pandas to create a DataFrame using the given data and display the data.
- Create a Box Plot : You'll use Seaborn to create a box plot, where the x-axis represents the departments, and the y-axis represents the salaries.
- Customize the Plot : Customize the appearance of the plot using various parameters provided by Seaborn and Matplotlib, like setting the title, labels for the axes, and specifying color as 'salmon'.
- Display the Plot : Finally, you'll use Matplotlib to display the plot on the screen.

Your goal is to create a visually appealing box plot that clearly illustrates any differences in salaries between departments. How would you tackle each step of this process?

SOLUTION :

Data Frame :		
	Department	Salary
0	HR	45000
1	HR	48000
2	HR	47000
3	HR	46000
4	HR	49000
5	Engineering	75000
6	Engineering	77000
7	Engineering	76000
8	Engineering	78000
9	Engineering	79000
10	Marketing	52000
11	Marketing	54000
12	Marketing	53000
13	Marketing	55000
14	Marketing	56000



WHAT'S NEXT?

Final Project

LESSON 47

Final Project

Final project -Sam's Pokemon Stats:

Link of the dataset :

Before diving into any modifications, take a look at the first few rows of the data to get a feel for what it contains. Also, check the detailed information about the dataset, including its structure and memory usage.

- Clean and Organize the Data :

Convert all the column names to uppercase to maintain consistency.

Set the Pokémon names as the index of the dataset, so it's easier to look up information about any specific Pokémon.

Remove the column that contains just a serial number for the Pokémon (we don't need that for our analysis).

- Handle Missing Data :

Check if there are any missing values in the dataset, especially in the 'TYPE 2' column which indicates a secondary type for the Pokémon.

Fill any missing values in the 'TYPE 2' column with the values from the 'TYPE 1' column (which indicates the primary type). This way, every Pokémon will have at least one type.

- Analyze the Data :

Check if there are any remaining missing values after your cleanup.

Finally, generate some basic statistics about the Pokémon attributes to understand the overall trends in the dataset.

SOLUTION :

```
# Display the first few rows of the original dataframe
#
```

#	Name	Type 1	Type 2	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
0 1	Bulbasaur	Grass	Poison	318	45	49	49	65	65	45	1	False
1 2	Ivysaur	Grass	Poison	405	60	62	63	80	80	60	1	False
2 3	Venusaur	Grass	Poison	525	80	82	83	100	100	80	1	False
3 3	VenusaurMega	Venusaur	Grass	625	80	100	123	122	120	80	1	False
4 4	Charmander	Fire	NaN	309	39	52	43	60	50	65	1	False

```
# Display the dataframe info with verbose and memory usage
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 800 entries, 0 to 799
Data columns (total 13 columns):
# Column Non-Null Count Dtype
---
0 # 800 non-null int64
1 Name 800 non-null object
2 Type 1 800 non-null object
3 Type 2 414 non-null object
4 Total 800 non-null int64
5 HP 800 non-null int64
6 Attack 800 non-null int64
7 Defense 800 non-null int64
8 Sp. Atk 800 non-null int64
9 Sp. Def 800 non-null int64
10 Speed 800 non-null int64
11 Generation 800 non-null int64
12 Legendary 800 non-null bool
dtypes: bool(1), int64(9), object(3)
memory usage: 75.9+ KB
```

```
# Convert column names to uppercase
# Display the first few rows of the dataframe
#
```

#	NAME	TYPE 1	TYPE 2	...	SP. DEF	SPEED	GENERATION	LEGENDARY	
0 1	Bulbasaur	Grass	Poison	...	65	45	1	False	
1 2	Ivysaur	Grass	Poison	...	80	60	1	False	
2 3	Venusaur	Grass	Poison	...	100	80	1	False	
3 3	VenusaurMega	Venusaur	Grass	Poison	...	120	80	1	False
4 4	Charmander	Fire	NaN	...	50	65	1	False	

```
[5 rows x 13 columns]

# Set 'NAME' as the index
# Display the first few rows of the dataframe with 'NAME' as the index
#
```

NAME	#	TYPE 1	TYPE 2	...	SPEED	GENERATION	LEGENDARY
Bulbasaur	1	Grass	Poison	...	45	1	False
Ivysaur	2	Grass	Poison	...	60	1	False
Venusaur	3	Grass	Poison	...	80	1	False
VenusaurMega	3	Grass	Poison	...	80	1	False
Charmander	4	Fire	NaN	...	65	1	False

```
[5 rows x 12 columns]
```

```
# Show the dataframe columns (after dropping the column #)
The columns of the dataset are: Index(['TYPE 1', 'TYPE 2', 'TOTAL', 'HP', 'ATTACK', 'DEFENSE', 'SP. ATK',
'SP. DEF', 'SPEED', 'GENERATION', 'LEGENDARY'],
dtype='object')

# Show the shape of the dataframe
The shape of the dataframe is: (800, 11)
```



```
# Fill missing values in 'TYPE 2' with values from 'TYPE 1'

# Display the first 10 rows of the dataframe after filling missing values
TYPE 1 TYPE 2 TOTAL HP ATTACK DEFENSE SP. ATK SP. DEF SPEED GENERATION LEGENDARY
NAME
bulbasaur Grass Poison 318 45 49 49 65 65 65 45 1 False
Ivysaur Grass Poison 405 69 62 63 89 89 89 69 1 False
Venusaur Grass Poison 525 89 82 83 109 109 109 89 1 False
VenusaurMega Venusaur Grass Poison 625 89 100 123 122 120 89 1 False
Charmander Fire Fire 309 39 32 43 69 50 63 39 1 False
Charmeleon Fire Fire 405 58 64 58 89 63 88 58 1 False
Charizard Fire Flying 534 78 84 78 109 85 100 78 1 False
CharizardMega Charizard X Fire Dragon 634 78 130 111 130 85 100 78 1 False
CharizardMega Charizard Y Fire Flying 634 78 104 78 159 115 100 78 1 False
Squirtle Water Water 314 44 48 65 59 64 43 1 False

# Calculate the number of missing values in each column
TYPE 1 0
TYPE 2 0
TOTAL 0
HP 0
ATTACK 0
DEFENSE 0
SP. ATK 0
SP. DEF 0
SPEED 0
GENERATION 0
LEGENDARY 0
dtype: int64
```

```
# Generate descriptive statistics
TOTAL HP ATTACK DEFENSE SP. ATK SP. DEF SPEED GENERATION
count 800.00000 800.00000 800.00000 800.00000 800.00000 800.00000 800.00000 800.00000
mean 435.10250 69.258750 79.001250 73.842500 72.820000 71.902500 68.277500 3.32375
std 119.96304 25.534669 32.457366 31.183501 32.722294 27.828916 29.060474 1.66129
min 180.00000 1.000000 5.000000 5.000000 10.000000 20.000000 5.000000 1.00000
25% 330.00000 50.000000 55.000000 50.000000 49.750000 50.000000 45.000000 2.00000
50% 450.00000 65.000000 75.000000 70.000000 65.000000 70.000000 65.000000 3.00000
75% 515.00000 80.000000 100.000000 90.000000 95.000000 90.000000 90.000000 5.00000
max 780.00000 255.000000 190.000000 230.000000 194.000000 230.000000 180.000000 6.00000
```

HOME TASK

Home Task1-Helping Kevin Analyze & Visualize

Kevin is a young data enthusiast who loves playing with numbers and visualizing data. He has collected sales data for three products — Product A, Product B and Product C — over the past year and stored it as a csv file with name "monthly sales". However, Kevin needs your help to analyze this data and visualize the sales distribution for the month of June.

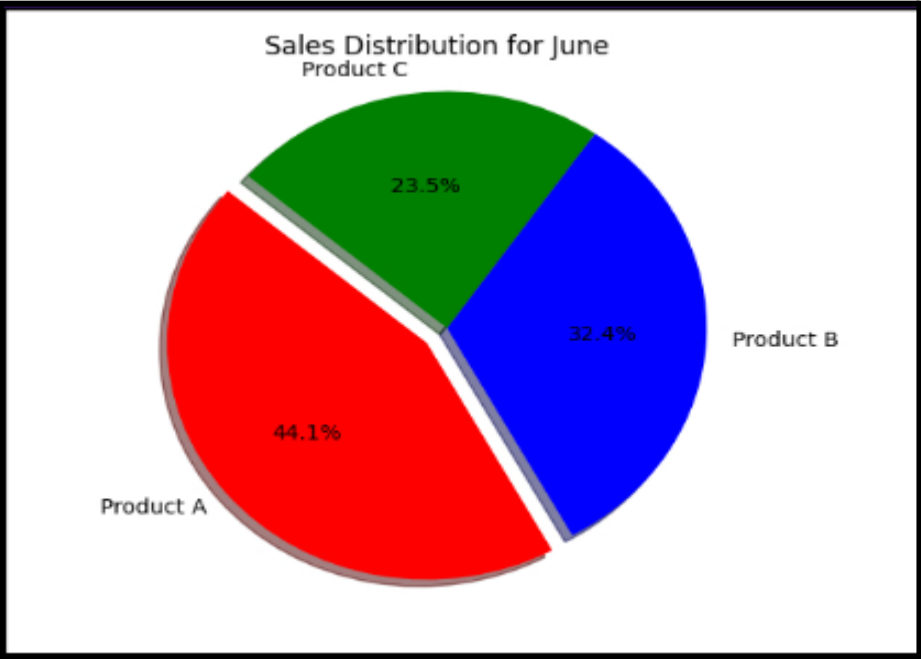
Link of the dataset :

Dataset:				
	Month	Product A	Product B	Product C
0	Jan	200	150	100
1	Feb	220	160	110
2	Mar	230	170	120
3	April	250	190	130
4	May	270	200	150
5	June	300	220	160
6	July	320	230	170
7	Aug	330	240	180
8	Sept	340	250	200
9	Oct	360	260	210
10	Nov	380	270	220
11	Dec	400	280	230

Basic Statistics:			
	Product A	Product B	Product C
count	12.000000	12.000000	12.000000
mean	300.000000	218.333333	165.000000
std	65.782009	44.072942	44.210242
min	200.000000	150.000000	100.000000
25%	245.000000	185.000000	127.500000
50%	310.000000	225.000000	165.000000
75%	345.000000	252.500000	202.500000
max	400.000000	280.000000	230.000000

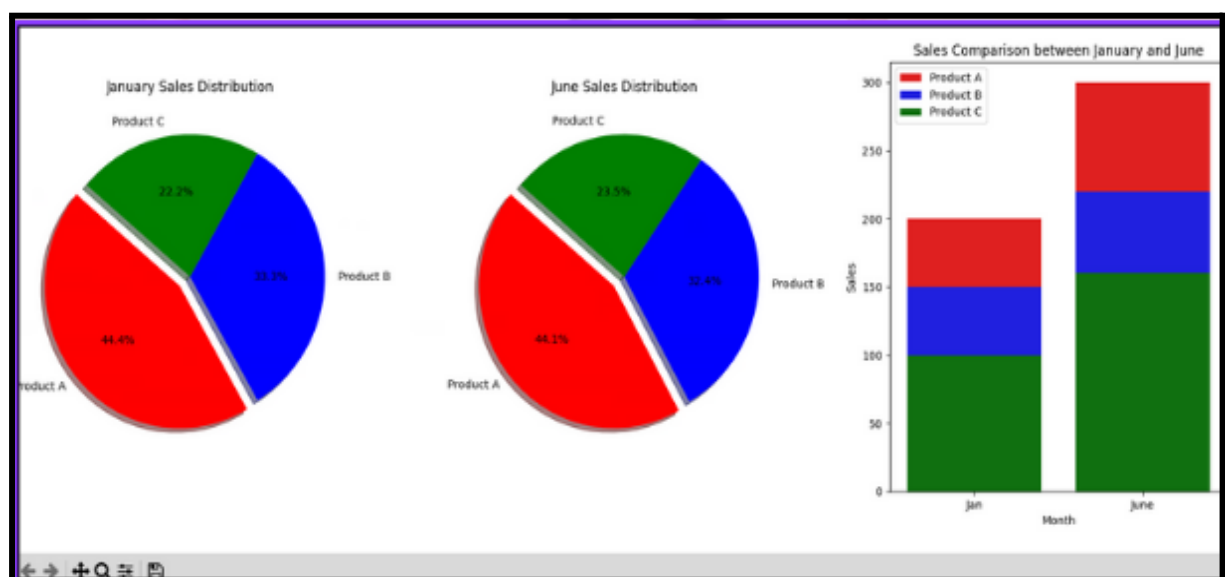
Probability of Sales Distribution per Product:			
	Product A	Product B	Product C
0	0.444444	0.333333	0.222222
1	0.448980	0.326531	0.224490
2	0.442308	0.326923	0.230769
3	0.438596	0.333333	0.228070
4	0.435484	0.322581	0.241935
5	0.441176	0.323529	0.235294
6	0.444444	0.319444	0.236111
7	0.440000	0.320000	0.240000
8	0.430380	0.316456	0.253165
9	0.433735	0.313253	0.253012
10	0.436782	0.310345	0.252874
11	0.439560	0.307692	0.252747

Sales Data for June:	
Month	June
Product A	300
Product B	220
Product C	160
Name: 5, dtype: object	



Home Task 2-Visualizing Kevin's Sales data:

Link of the dataset :



LESSON 48

Enhancing Final Project

Enhancing Final project - Pokemon Stats & Visuals

Link of the dataset :

3. Analyze Generations : Group the Pokémon by their generation and calculate the average values for each stat. How do these averages change from one generation to the next?
4. Count Pokémon : Find out how many Pokémon there are in each generation.
5. Compare Fire and Water Types : Create separate datasets for Fire and Water type Pokémon. Then combine them and add a 'Type' column to distinguish between them.
6. Create Visualizations :
 - Line Plot : Show the average stats by generation.
 - Bar Plot : Display the number of Pokémon in each generation.
 - Scatter Plot : Compare Attack and Defense stats for Fire and Water Pokémon, distinguishing between Legendary and non-Legendary Pokémon.

Using Python, Seaborn, and Matplotlib, write the code to accomplish these tasks and create the visualizations. Your goal is to present your findings to Professor Oak in a clear and insightful way.

SOLUTION:

```

** First few rows of the Data Frame **
#      Name Type 1 Type 2 Total HP Attack Defense Sp. Atk Sp. Def Speed Generation Legendary
0 1      Bulbasaur  Grass  Poison  318  45  49  49  65  65  45  1  False
1 2      Ivysaur  Grass  Poison  405  60  62  63  80  80  60  1  False
2 3      Venusaur  Grass  Poison  525  80  82  83  100  100  80  1  False
3 3  VenusaurMega  Venusaur  Grass  Poison  625  80  100  123  122  120  80  1  False
4 4      Charmander  Fire  NaN  309  39  52  43  60  50  65  1  False

** Extracted columns & re-indexed **
      Total HP Attack Defense Sp. Atk Sp. Def Speed Generation Legendary
0      318  45  49  49  65  65  45  1  False
1      405  60  62  63  80  80  60  1  False
2      525  80  82  83  100  100  80  1  False
3      625  80  100  123  122  120  80  1  False
4      309  39  52  43  60  50  65  1  False
...  ...  ...  ...  ...  ...  ...  ...
795    600  50  100  150  100  150  50  6  True
796    700  50  160  110  160  110  110  6  True
797    600  80  110  60  150  130  70  6  True
798    680  80  160  60  170  130  80  6  True
799    600  80  110  120  130  90  70  6  True

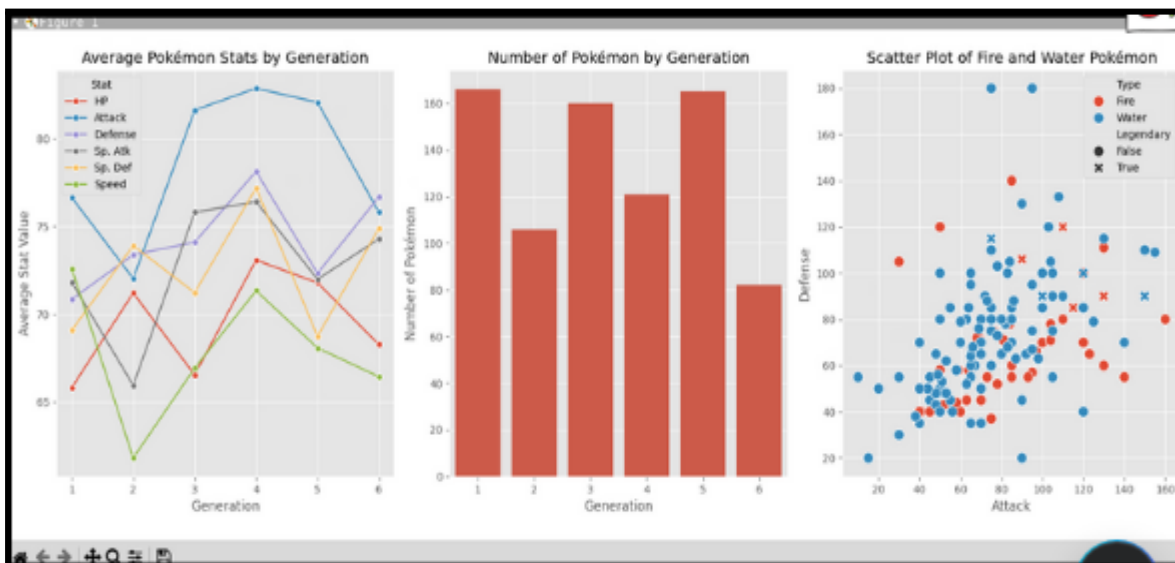
```

```

** GroupBy 'Generation' & Mean for each stat **
      HP      Attack      Defense      Sp. Atk      Sp. Def      Speed
Generation
1      65.819277  76.638554  70.861446  71.819277  69.090361  72.584337
2      71.207547  72.028302  73.386792  65.943396  73.905660  61.811321
3      66.543750  81.625000  74.100000  75.806250  71.225000  66.925000
4      73.082645  82.867769  78.132231  76.404959  77.190083  71.338843
5      71.787879  82.066667  72.327273  71.987879  68.739394  68.078788
6      68.268293  75.804878  76.682927  74.292683  74.890244  66.439024

** Number of Pokémon in each generation **
Generation
1      166
2      106
3      160
4      121
5      165
6       82
Name: count, dtype: int64

```



HOME TASK

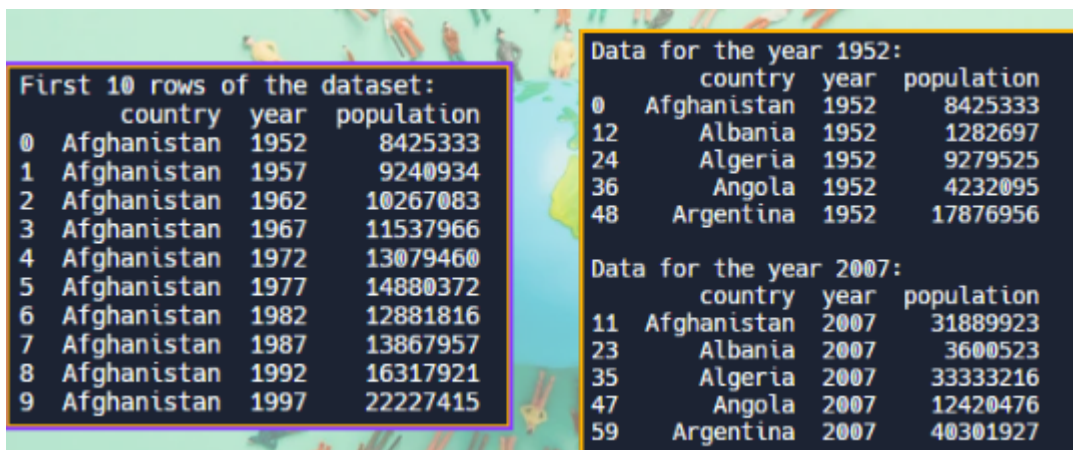
Home Task1 - Analyzing Population Growth:

Your friend Bob is having a csv file named “Countries Data” that contains information about various countries, including their population for different years. The data is so huge due to which Bob is confused and needs your help to analyze population growth across different countries from the year 1952 to 2007.

So, your goal is to identify the top 10 countries that experienced the largest population growth between 1952 and 2007. To achieve this, follow the steps below and write the corresponding code :

- Read & show the dataset : Load the dataset into pandas DataFrame & display the first 10 rows to get an overview of the data.
- Filter & display the data for specific years : Extract & display first 5 rows where the year is 1952 and do the same for the year 2007.
- Merge & display the data : Combine & display the data for 1952 and 2007 into a single DataFrame, ensuring that each country is matched correctly.
- Calculate population growth & display it : Create and print a new column that calculates the difference in population between 2007 and 1952 for each country.
- Sort and display the top 10 : Sort the DataFrame based on the population growth column in descending order and display the top 10 countries with the highest population growth.

Link of the dataset :



First 10 rows of the dataset:			
	country	year	population
0	Afghanistan	1952	8425333
1	Afghanistan	1957	9240934
2	Afghanistan	1962	10267083
3	Afghanistan	1967	11537966
4	Afghanistan	1972	13079460
5	Afghanistan	1977	14880372
6	Afghanistan	1982	12881816
7	Afghanistan	1987	13867957
8	Afghanistan	1992	16317921
9	Afghanistan	1997	22227415

Data for the year 1952:			
	country	year	population
0	Afghanistan	1952	8425333
12	Albania	1952	1282697
24	Algeria	1952	9279525
36	Angola	1952	4232095
48	Argentina	1952	17876956

Data for the year 2007:			
	country	year	population
11	Afghanistan	2007	31889923
23	Albania	2007	3600523
35	Algeria	2007	33333216
47	Angola	2007	12420476
59	Argentina	2007	40301927


```
Merged dataframe:
  country  year_x  population_x  year_y  population_y
0  Afghanistan  1952      8425333  2007      31889923
1    Albania  1952      1282697  2007       3600523
2    Algeria  1952      9279525  2007      33333216
3    Angola  1952      4232095  2007      12420476
4  Argentina  1952      17876956  2007      40301927

Dataframe with population growth column:
  country  year_x  population_x  year_y  population_y  population_growth
0  Afghanistan  1952      8425333  2007      31889923      23464590
1    Albania  1952      1282697  2007       3600523      2317826
2    Algeria  1952      9279525  2007      33333216      24053691
3    Angola  1952      4232095  2007      12420476      8188381
4  Argentina  1952      17876956  2007      40301927      22424971
```

```
Top 10 countries with the biggest population growth:
  country  year_x  population_x  year_y  population_y  population_growth
24    China  1952      556263527  2007      1318683096      762419569
58    India  1952      372000000  2007      1110396331      738396331
134  United States  1952      157553000  2007      301139947      143586947
59  Indonesia  1952      82052000  2007      223547000      141495000
14    Brazil  1952      56602560  2007      190010647      133408087
97    Pakistan  1952      41346560  2007      169270617      127924057
8    Bangladesh  1952      46886859  2007      150448339      103561480
94    Nigeria  1952      33119096  2007      135031164      101912068
82    Mexico  1952      30144317  2007      108700091      78556574
101  Philippines  1952      22438691  2007      91077287      68638596
```

Home Task2 - Analyzing & Visualizing Population Growth:

Link of the dataset :

- Merge the data : Combine the data for 1952 and 2007 using the country as the key.
- Calculate population growth : Create a new column that calculates the difference in population between 2007 and 1952 for each country.
- Sort and display the top 10 : Sort the DataFrame based on the population growth column in descending order and display the top 10 countries with the highest population growth.
- Visualize the Data : Finally, create a seaborn bar plot to visually represent the top 10 countries with the biggest population growth.

SOLUTION :

Top 10 countries with the biggest population growth:

	country	year_x	population_x	year_y	population_y	population_growth
24	China	1952	556263527	2007	1318683096	762419569
58	India	1952	372000000	2007	1110396331	738396331
134	United States	1952	157553000	2007	301139947	143586947
59	Indonesia	1952	82052000	2007	223547000	141495000
14	Brazil	1952	56602560	2007	190010647	133408087
97	Pakistan	1952	41346560	2007	169270617	127924057
8	Bangladesh	1952	46886859	2007	150448339	103561480
94	Nigeria	1952	33119096	2007	135031164	101912068
82	Mexico	1952	30144317	2007	108700891	78556574
101	Philippines	1952	22438691	2007	91077287	68638596

